open rails

Blender Content Creation Notebook

A guide to making content for Open Rails

Pete Willard

Version 2.6.0.2, 2026-10-09

Content Creation Notebook

Colophon . . . 1
Open Rails, Blender Content Creator Notebook . . . 2
About This Document . . . 3
About The Author . . . 3
Contributor and Resources . . . 3
Some notes about some of the References . . . 4
Conventions Used . . . 5
Preface . . . 6
Here is the list of things that I assume about the reader. . . . 7
Organization of This Document . . . 7
Making Content for Open Rails . . . 8
Common Terms and Definitions . . . 9
Content Generation . . . 13
Programs you should have . . . 13
What the author uses . . . 13
Why the Author Uses Blender . . . 14
Why Many Other People Use Blender to Make Content . . . 15
What I Can Recommend . . . 15
How to Install Blender . . . 16
Setting up our Blender Environment . . . 19
Interface . . . 20
Themes-3D Viewport . . . 22
Viewport . . . 23
System . . . 24
Save & Load . . . 26
File Paths . . . 27
Version Compatibility Notice . . . 28
Customizing Asset Library Locations . . . 28
Add-Ons . . . 29
Installing Add-Ons with Version 4.x . . . 30
Enabling Built-In Add-ons . . . 30
Installing Third-Party Add-ons . . . 30
After Installation . . . 30
Add-on File Locations . . . 31
Troubleshooting . . . 31

Also in Blender 4.x - Extensions . . . 31
Enabling and Disabling Add-Ons . . . 33
3rd Party Add-Ons . . . 34
Exit and Save Preferences . . . 36
Other Settings . . . 37
TABS . . . 37
Scene Settings . . . 38
Gizmo settings . . . 38
Overlays . . . 39
Shading Settings . . . 40
Units . . . 41
Scene (Rendering) . . . 42
Shading . . . 43
Done, for now… almost . . . 45
Folder structure . . . 47
Content Creation Overview . . . 48
Orientation . . . 48
Engine/Wagon Model Hierarchy . . . 48
Standard 2 Axle Freight Bogies . . . 50
Isolated Axles . . . 51
OpenRails Animation . . . 51
Configuration Files Overview . . . 53
Include Files in Configurations . . . 53
General INC file structure . . . 54
Items well suited to INC files . . . 54
Specifying relative references . . . 55
Config File Deconstruction . . . 56
Header . . . 56
Body . . . 56
Example ORTS Configuration Layout . . . 58
3rd Party Configuration Tools . . . 59
General Modelling Standards from Erick Cantu . . . 59
Erick Says . . . 60
Additional guidance from Blender Brothers . . . 62
Blender Brother’s Gold Rule . . . 62
General Texture Mapping Guidelines . . . 62
Some additional tools to consider . . . 63
Making a 3D Model . . . 64

Some Basic Things About 3D Models . . . 64
Some Basic Things About Modeling . . . 65
Polygon Count and Vertex Density . . . 66
Preparation of Work . . . 66
Background Images . . . 66
Some things I will miss about using 3D Canvas . . . 70
Where to start? . . . 74
Do we need a quick 3D Modeling primer? . . . 74
The 3D Viewport . . . 75
The Modeling Interface . . . 76
Shortcut Keys . . . 78
View Controls . . . 79
Modeling Modes . . . 81
The four major edit-mode tools you are likely to use the most are listed below. . . . 83
EXTRUDE . . . 83
INSET . . . 85
BEVEL . . . 87
LOOP CUT & SLIDE . . . 88
Additional Tips . . . 90
MIRROR and FLIP . . . 90
Additional Modeling Tips . . . 90
Setting up your Initial Workspace . . . 91
Actually Modeling Something . . . 94
Model Building Exercise #1 . . . 94
Texturing . . . 107
Let’s Begin Texturing . . . 107
Creating Level Of Detail Distance levels . . . 115
Guidelines for creating LOD versions . . . 115
Next Steps . . . 117
Workflow . . . 119
Building a Library of Reusable Parts . . . 121
A Brute-Force Library Solution . . . 121
Saving in alternative export formats . . . 124
Importing Existing Objects . . . 125
Fixing UV MAP Assignment Issues on Imported Objects so they work with the MSTS exporter . . . 126
Model Exercise #2 . . . 129
Intermediate Modeling . . . 129

Building the General Shape . . . 129
Importing the Background Images . . . 130
Setup . . . 130
Making the Interior . . . 135
Making the Doors . . . 137
Making a Vehicle . . . 140
Finding the Right Reference Images . . . 140
Organizing Your Reference Material . . . 141
Using Background Images . . . 141
The Modeling Process . . . 143
Iteration and Refinement . . . 143
Let’s do it. . . . 144
Rolling Stock: Importing Model Objects . . . 144
Making Rolling Stock - Project #4 . . . 146
Using Background Images . . . 146
Starting from the ground up . . . 146
Example: 100-Ton Truck Model . . . 146
Vehicle LOD - Level of Detail Settings . . . 149
Bitmap Editing and Texture Workflow . . . 152
UV Mapping Overview . . . 152
Bitmap Tools Overview . . . 152
Bitmap Tools Overview . . . 152
Why bitmap tools matter for 3D . . . 153
Layered Format Files . . . 153
Using DDS Textures . . . 155
Can Blender Import DDS Files Directly? . . . 156
Using ACE Textures . . . 158
ACE File Types: . . . 158
ACE File Compression . . . 159
Digital Graphics Basics . . . 159
Bit Depth and Color Depth . . . 159
Bit Depth . . . 159
Color Depth . . . 160
Color Modes and Bit Depth . . . 160
Bit Depth and Color Depth Summary . . . 161
JPG Format, lossy compression and UpScaling . . . 161
PNG Format . . . 161
PNG Format Summary . . . 162

Vector Graphics . . . 162
Tweaking Fonts With Affinity . . . 162
Vector Graphics Summary . . . 164
Preparing a texture . . . 165
Which Comes First? . . . 166
UV Mapping Basics . . . 167
Setting up Blender for Texture Mapping and Baking . . . 167
Using SEAMS to Unwrap the Model . . . 174
Purpose of Seams . . . 175
How to Place Seams . . . 175
Packing Islands . . . 179
Windows . . . 181
Texture Nodes . . . 182
Decals . . . 182
Layering Basics . . . 184
Grime Layer . . . 185
Rust Layer . . . 185
Dirt Layer . . . 185
Dust Layer . . . 185
Finalization of effects . . . 185
Applying Fonts and Lettering . . . 186
Layering Tips from Erick . . . 186
Working with Alpha Channels . . . 189
Creating Alpha Channels with Affinity Studio . . . 189
Creating Alpha Channels with GIMP . . . 190
Step 1: Start GIMP . . . 190
Step 2: Open your image . . . 190
Step 3: Add an alpha channel . . . 191
Step 4: Select the image area . . . 191
Step 5: Refine the selection . . . 192
Step 6: Clear or fill the selection . . . 193
Step 7: Save your work . . . 194
Step 8: Optional: Use a layer mask . . . 195
Gimp Alpha Wrap Up . . . 195
Image Tools . . . 195
Measuring Tools . . . 195
Dealing with perspective distortion . . . 196
TextureRipper . . . 198

ACE2BMP . . . 199
ACEIT . . . 199
TGATOOL2 and MAKEACE tools . . . 199
DXTBMP . . . 200
ACE2DDS . . . 202
Alpha materials in Blender . . . 203
The Shader Editor . . . 203
The MSTS Materials Settings . . . 204
Making PBR Textures . . . 205
PBR Workflow . . . 205
How to Make Night Textures . . . 207
How to Make Night Textures with Back Lighting . . . 212
How to Make Snow Textures . . . 213
Highlights and Shadows . . . 216
Generated Shadows - Ambient Occlusion . . . 217
Ambient Occlusion Steps - Summary . . . 224
Ambient Occlusion Baking with GIMP . . . 225
More About Baking Ambient Occlusion . . . 228
Ambient Occlusion Steps . . . 229
Introduction to Blender Scripting . . . 230
Actually Doing It . . . 230
Blender and AI Coding Assistants . . . 232
Running a Blender Python Script Inside Blender . . . 232
A Collection of Scripts . . . 233
Fixing Material Problems . . . 233
3DC Export/Import Issues . . . 235
Making Copies . . . 236
Make Selected Objects Become ASSETS . . . 238
Automating the Boring Stuff with Python . . . 239
So what are we going to do next? . . . 239
Using a script from the console . . . 246
Using a script from the command line . . . 247
A Few Setup Script Examples . . . 248
Creating a basic UV Map layout . . . 249
Creating a DECAL sheet . . . 250
Creating Rolling Stock and Locomotives . . . 251
General Configuration File Format . . . 252
Open Rails "Units of Measure" . . . 253

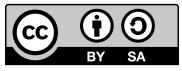
Using *INCLUDE* files . . . 255
Basic ENG and WAG file Details . . . 256
Should there be a ENG/WAG standard layout? . . . 257
Universal Settings . . . 258
Example WAG file with INCLUDE . . . 259
Option 1 - The Benefits of Common Folders for Re-skinners . . . 267
Option 2 - Completely New Content (with possible help from NAVS) . . . 268
Gathering Details . . . 268
Another Example of Rolling Stock . . . 272
Specifying Lights on Locomotives and Wagons . . . 275
Introduction . . . 275
Available Tokens for Lights . . . 275
Setting requirements . . . 276
Defining lights . . . 277
Fade-in and Fade-out times . . . 278
Cycle . . . 278
States . . . 278
Examples . . . 279
Example 1 – the Dash 9's front headlights . . . 279
Example 2 – the Dash 9's front right flashing (ditch) light. . . . 281
Example 3 – the Dash 9's rear red light . . . 282
A brief explanation of hexadecimal numbers . . . 282
Effects . . . 283
Freight Animations . . . 284
Creating 3D Cabs . . . 286
Planning Ahead . . . 286
Appendix A . . . 292
Appendix II . . . 304
Consider Transitioning away from Microsoft Train Simulator to Open Rails . . . 304
The Open Rails Program . . . 304
Index . . . 306

Colophon

Open Rails, Blender Content Creator Notebook

2026-10-09 2.62.6.0.2 Pete Willard

This work is licensed under a creative commons attribution-share-alike 4.0 international license



Open Rails, Blender Content Creator Notebook

About This Document

This guide covers Blender 2.8 and newer, with current notes checked against Blender 5.2. For the most stable workflow, use the latest Long Term Support (LTS) release, currently Blender 5.2 LTS. Versions older than 2.8 are not covered or supported here.

Blender 5 uses a newer Python API than the 4.x LTS series. Core modeling workflows remain familiar, but add-ons, exporters, and screenshots may differ. If an exporter or add-on fails in Blender 5, try Blender 4.5 LTS until that tool is updated.

The Long Term Support (LTS) version of Blender is often the safest choice because it receives bug fixes without major feature or interface changes. Blender 5.2 LTS is the current LTS baseline for this notebook.

About The Author

The author is not an expert in game or simulator content creation, but has created Microsoft Train Simulator content since 2002, mostly with 3D Canvas/3D Crafter through 2016. Earlier Blender versions felt cumbersome, but Blender 2.8 and newer are much easier to use, and Blender is now the author's preferred 3D modeling tool.

Writing in 3rd person is weird.

Contributor and Resources

This document is made possible with the contributions and feedback from people like:

- Wayne Campbell - Original author of the Open Rails Train Simulator Exporter for Blender
- Erick Cantu - Creator of NAVS - North American V-Scale products for Open Rails, available at TigerTrains.com
- Michelle McKell - A well known content re-skinner for Open Rails
- Dave Nelson - Known as Genma Saotome on the Elvas Tower forums that has contributed a LOT to the community
- Peter Newell - Maintainer of "Coals too Newcastle" website, a resource web site for Open Rails
- Curtis Holt - A blender trainer and add-on creator
- Darrin Lile - A Blender trainer, particularly aimed at beginners creating game assets
- Josh Gambrell - A Blender trainer, add-on creator, and member of the Blender Bros. group.
- and big thanks to various Blender add-on authors

Useful resources:

- http://www.elvastower.com
- http://trainsim.com
- http://www.textures.com You need to log in to download.[1]
- https://www.blendersecrets.org/ Author of many tips and tricks videos as well as a book
- https://curtisholt.online/ A really good Blender Trainer and add-on creator
- https://blender.org Where to download Blender from
- https://www.coalstonewcastle.com.au/physics/ ORTS Information
- http://www.nordicfx.net/ HDRI backgrounds
- https://polyhaven.com/ Textures and HDRI Backgrounds

Some sections of the original Microsoft Train Simulator technical documents are included here as reference material because some of these details are difficult to find unless you still have the original Microsoft Train Simulator documentation and CD-ROMs. These sections are outdated, but they remain useful as historical and technical references.

Some notes about some of the References

NAVS

North American V-Scale - Content created by or related to Erick Cantu. Using a set of standards he developed.

Coals to Newcastle

Meaning, "a silly venture", as Newcastle has plenty of coal. Also, a website run by Peter Newell that has very useful Open Rails physics documentation.

MSTS

Microsoft Train Simulator - referring to the original Microsoft Train Simulator CD-ROM distribution of the simulator. To many users, the original Microsoft Train Simulator release felt unfinished: documentation was incomplete, and many bugs were later addressed by community tools such as MSTSBin. There was never an official Microsoft Train Simulator 2.0 release.

[1] The site offers Free Registration and free downloads once signed up

Conventions Used

Regular Note.

Pay attention to these.

You should know this.

With care, you can succeed.

Optional, but good to know.

`Highlighting`

BOLD

Italic

```
Source code
```

`LMB`
: Left Mouse Button

`MMB`
: Middle Mouse Button

`RMB`
: Right Mouse Button

N-Panel
: Number Panel, `N` Hotkey in Main 3D Window

[ENTER]
: Sometimes you will see keyboard entry look like this

Referenced footnotes appear at the **end** of Chapters

Web Links should be active and will open in your web browser if your PDF reader supports it.

Preface

Open Rails is an open-source train simulator platform designed to replicate the experience of operating trains on various railway systems. It was developed as an alternative to Microsoft Train Simulator (MSTS) and is compatible with Microsoft Train Simulator content, allowing users to enjoy a wide range of routes, locomotives, and rolling stock created for Microsoft Train Simulator.

Here are some key points about Open Rails:

Compatibility

Open Rails is built to run on modern Windows systems and offers improved stability and performance compared to Microsoft Train Simulator, especially on newer hardware.

Enhanced Graphics

Open Rails supports a variety of graphical improvements, including higher screen resolutions, improved lighting and shading effects, enhanced textures, and anti-aliasing. These enhancements provide a more immersive visual experience.

Physics and Realism

Open Rails aims to provide realistic train physics and behavior. It features improved simulation of train dynamics, braking, and traction control, making the virtual driving experience more authentic.

Third-Party Content

Open Rails is compatible with a wide range of third-party content created for Microsoft Train Simulator. This includes routes, locomotives, rolling stock, scenery objects, and other add-ons developed by the Microsoft Train Simulator community. Users can install these add-ons in Open Rails and enjoy them with improved performance and visual quality.

Community and Development

Open Rails benefits from an active and passionate community of developers and enthusiasts who contribute to its development. Regular updates and improvements are made to the simulator, ensuring its continued growth and refinement.

Open-Source Nature

Open Rails is an open-source project, which means its source code is freely available for modification and enhancement. This allows developers to contribute to the project and create new features or fix issues.

It’s worth noting that Open Rails is not a standalone game but rather a simulation platform. While it provides a framework for train simulation, it relies on third-party content and addons to provide the routes, trains, and other elements of the virtual railway environment.

Overall, Open Rails offers a robust and customizable train simulation experience for enthusiasts

who enjoy operating trains on various virtual railway systems.

Here is the list of things that I assume about the reader.

- You are new to Blender
- You are reasonably new to content creation for Open Rails
- You have Open Rails installed and working
- You have installed TSRE5 if you plan to inspect routes, consists, or placed objects outside the simulator or need a shape file viewer
- You have a texture file editor that supports PSD, PNG and DDS formats.[1]
- You have some idea about how Open Rails and Microsoft Train Simulator content is created
- You have a project in mind, but you understand that you will not create your Magnum Opus on a first try.

If you have prior experience with older tools such as GMAX, Train Sim Modeler, or 3D Canvas/Crafter, it will help, but it is not required.

Organization of This Document

As the title implies, this document began as a collection of notes made while learning to create content for Open Rails. It is now organized as a practical notebook for filling gaps in the available Open Rails content-creation documentation. This document is free and open source, and it lives in a GitHub repository so others can fork it, correct errors, and add missing sections. I do not claim exclusive ownership of every idea or concept contained here; much of this material has been created, gathered, reorganized, and rephrased from many sources.

Due to the way this document originated, it may seem to jump around and be repetitive at times. My apologies in advance if this bothers you, but if you are so inspired, you are welcome to help fix any errors, problems, or omissions.

[1] Good examples include Paint.net, KRITA, and GIMP and Affinity Studio, all of which are free

Making Content for Open Rails

About ORTS

Open Rails Train Simulator is an open-source simulator created after Microsoft Train Simulator 2.0 was cancelled. Open Rails is designed to remain compatible with most existing Microsoft Train Simulator content while adding new features and fixing many old Microsoft/Kuju limitations. MSTS and ORTS terms are often used interchangeably, but Open Rails can handle more complex models, higher polygon counts, longer viewing distances, 32-bit color, and many simulator features that Microsoft Train Simulator does not support.

OPEN RAILS

Compared with Microsoft Train Simulator, Open Rails can usually keep good frame rates with more polygons. This allows smoother curves and more modeled detail. Open Rails also supports 32-bit color, while Microsoft Train Simulator is limited to 16-bit color. View distance can also be adjusted from about 2 km to 10 km.

from Open Rails.org website

This notebook focuses on using Blender. Other 3D modeling tools are mentioned only when they provide useful contrast.

Common Terms and Definitions

3D modeling and simulator content creation use specialized terms. The following definitions cover the terms used most often in this notebook.

ALPHA

A bitmap channel that makes part of an image transparent or partially transparent.

DISTANCE LEVELS

See Level of Detail (LOD).

EDGE

A line connecting two vertices.

ENG FILE

A simulator configuration file for an engine or locomotive. It defines physical details such as motive power, braking, couplers, and other properties.

FACE

A surface made from three or more vertices.

FOOBIE

A deliberately low-detail model that uses texture detail to create many visual variants from one base shape. The term is a contraction of "fake boobies."

HARD SURFACE MODELING

Modeling rigid objects such as vehicles, buildings, weapons, roads, and machinery. It usually uses standard mesh objects such as cubes, cylinders, planes, cones, and torus shapes.

HDRI

High Dynamic Range Image. In Blender, HDRI environment maps are often used to light a scene with realistic real-world lighting information.

HIERARCHY

The parent-child relationship between objects in a shape. Child objects can move on their own, but they are still affected by the parent object.

LEVEL OF DETAIL

(LOD) A system that swaps the main model for simpler versions at greater distances. Distant models need fewer details because the viewer cannot see them clearly. There are no fixed LOD ranges, but you should use them when they help performance.

MANIFOLD SHAPE

A watertight 3D object. Its vertices, edges, and faces connect so that no water would leak out if the object were filled. A default cube is manifold.

MATERIAL

A surface definition assigned to a model. It may include a texture, lighting behavior, palette information, and shading settings.

MESH

A 3D model shape made from faces, before textures are applied.

NGON

A face with five or more vertices. Many Blender tools work best with quads or ngons and may not work as well after a mesh has been triangulated.

NORMAL

A vector perpendicular to a face. Normals control which way a face points and affect how textures and lighting are displayed. [1]

OBJECT

A 2D object has length and height but no depth, such as a plane, polygon, or line. A 3D object has length, height, and depth, such as a cube or sphere.

ORGANIC MODELING

Sculpted modeling for subjects such as people, animals, plants, trees, and some vehicles. Blender includes sculpting tools and NURBS-related tools, but those workflows are outside the scope of this notebook.

ORIGIN

See Pivot Point.

PIVOT POINT

The position used to align, rotate, and place a shape. It is also called the object's origin.

POLY COUNT

The total number of polygons used in a mesh. It is a common way to describe model complexity.

POLYGON

A flat face of a mesh. Edges define the polygon boundary, and vertices are the points where the edges meet. Blender commonly refers to triangle faces as tris and four-vertex faces as quads.

QUAD

A face made from four vertices.

SHAPE

A finished 3D object that is ready to be placed in the simulator environment.

SMOOTHING

A shading setting that controls how individual polygons are lit by the simulator.

TRI

A face made from three vertices. Also called a triangle.

UV

The mapping system that projects a 2D texture image onto a 3D model. U and V are the texture axes, while X, Y, and Z are already used for 3D space.

VERTEX

A point in 3D space with X, Y, and Z coordinates. Vertices are the points where edges and faces meet. The plural is vertices.

WAG FILE

A simulator configuration file for a wagon or non-powered vehicle. It defines physical details such as braking, couplers, and other properties.

You should also describe the intended compatibility of your finished content in its README file.

Made for Microsoft Train Simulator

The content can run in Open Rails, but it may need WAG or ENG edits because some legacy Microsoft Train Simulator physics values may not perform well in Open Rails.

Made for ORTS - No other files required

The content is configured for Open Rails and requires no additional files.

Made for ORTS - but requires other add-ons

The content is configured for Open Rails, but needs other downloaded files to work. NAVS-based items are a common example.

Made for ORTS but Compatible with Microsoft Train Simulator

The content is built for Open Rails but also configured to remain compatible with Microsoft Train Simulator. This usually takes more work.

Made for ORTS but requires Microsoft Train Simulator

The content runs in Open Rails, remains compatible with Microsoft Train Simulator, and also requires files from a local Microsoft Train Simulator installation.

Made for ORTS but requires Microsoft Train Simulator and other add-ons

The content is configured for Open Rails, but also requires a local Microsoft Train Simulator installation and other default or add-on files.

Choose the option that best matches your goals. Consider these questions:

- How familiar are you with Open Rails and its content options?
- How much effort do you want to put into this project?
- How much 3D modeling experience do you already have in tools such as Train Sim Modeler, GMAX, or 3D Canvas?
- How much Blender experience do you have?
- How much experience do you have creating content for Open Rails compared with Microsoft Train Simulator?
- How flexible does your content need to be?

Author's Advice

```
Start with the option that is easiest to understand, then adjust as you learn. If you
are new to the workflow, a simple "Made for Open Rails" project is a good starting
point.
```

Share your work. Talk with other content developers, ask for feedback, and learn what others are doing. This helps you avoid repeated mistakes and also makes it less likely that projects will be abandoned half-finished.

[1] See https://en.wikipedia.org/wiki/Normal_(geometry)

Content Generation

Creating Open Rails content usually requires several related pieces. The list below is not complete, but it gives the basic shape of a simple train-related model project.

- A mesh, which is the 3D model.
- Reference material and online resources. TrainSim.com and Elvas Tower are useful starting points: https://www.trainsim.com/vbts/forum.php and http://www.elvastower.com/forums/index.php
- A texture, also called a skin: a 2D bitmap image applied to the model.
- A UV map: the mapping that tells Blender how the 2D texture fits onto the 3D mesh. Blender can create UV maps, although tools such as UVLayout or Substance Painter can also be used.
- Configuration files that describe the content to the simulator. For rolling stock, these are usually WAG or ENG files.
- A thumbnail JPG. This is optional, but useful.
- Instructions. Do not leave the project half documented.

Programs you should have

- A 3D program. Blender 5.2 LTS is recommended. The current release, 5.2, should also be fine unless Blender's Python API has changed enough to affect the exporter add-on. Blender 5.0 is an example where exporter updates may be required.
- An exporter add-on for the Microsoft Train Simulator/ORTS format. This notebook uses Wayne Campbell's *S File Exporter* from Elvas Tower. [1]
- A paint program with layers and channels.
- A text editor that handles Unicode files.

If you also make models for Trainz Simulator, current Trainz versions can use files exported by Blender's built-in FBX exporter.

What the author uses

- 3D modeler: 5.2
- Bitmap tools: Canva, Affinity Studio, or Paint.NET
- UV/shader tools: Blender's built-in tools. Third-party tools such as Substance Painter exist, but I do not use them for this workflow.

- Text editor: *Microsoft Visual Studio Code*
- Metric conversion calculator: a hand-held calculator or Google
- Scale calculator: useful for converting dimensions from scale drawings. One is available at http://www.railsimstuff.com
- Texture snipping tool: Windows Snipping Tool or SHOEBOX, an Adobe Air application for game creation

Use metric measurements when possible. Metric is the default in Blender and Open Rails. U.S. customary units may be familiar, but metric is better for worldwide compatibility and is usually simpler once you stay with it.

Why the Author Uses Blender

From Pete Willard

I started using 3D software for game content in 2002, mostly for Microsoft Train Simulator. I have used Abacus Train Sim Modeler, 3D Canvas, Gmax, and SketchUp. 3D Canvas was my favorite for many years, but I now mainly use it only for file conversion. Blender has become my preferred tool because 5.2 includes the features I need, has active support, and includes many tools that 3D Canvas lacks.

[1] Download from https://github.com/pwillard/Blender_Microsoft Train Simulator_ORTS_Exporter

Why Many Other People Use Blender to Make Content

Blender is a powerful, flexible, free, and open-source 3D application. It is widely used for game and simulator assets for several reasons.

Open-Source and Free

Blender is free to use and has an active community of developers and users. It is accessible to hobbyists, indie developers, and professionals.

Versatility

Blender includes tools for modeling, sculpting, texturing, rigging, animation, rendering, and more. It can create characters, environments, props, vehicles, and effects.

Integrated Workflow

Modeling, editing, UV mapping, animation, and rendering can all happen in one application. This reduces the need to move between separate tools.

Asset Optimization

Blender includes tools for LODs, texture baking, and UV map management, all of which help prepare assets for real-time rendering.

Community and Resources

Blender has a large user community with many tutorials, forums, and training resources.

Non-Destructive Workflow

Modifiers and procedural workflows allow changes without immediately destroying the original model data.

Constant Development

Blender is updated often. New features and improvements arrive regularly, sometimes faster than users can keep up with.

Blender is not the only possible tool, but it is a strong choice if you want a free, capable, and actively developed modeling package.

To export a model for Microsoft Train Simulator or Open Rails, use Wayne Campbell's Blender MSTS/Open Rails exporter add-on for Microsoft Train Simulator-format `.s` files.

What I Can Recommend

- 3D modeler: Blender Blender 5.2 LTS or the current stable 5.2 release. Blender 3.6 LTS can still help with older workflows, but Blender 2.79-era workflows are no longer covered here. Blender

is free.

- Bitmap tools: *Affinity Photo*, *Paint.NET* (free), *GIMP*, *Krita*, or *PaintShop Pro* if you already have it.
- UV tool: Blender's built-in UV and shader tools are enough for this notebook. Third-party tools are optional.
- Text editor: *Microsoft Visual Studio Code*, *Context.exe*, or *Sublime*.

Author's Note:

Much of this document was created with Blender 3.6 LTS and updated while using Blender 5.2.

CONTEXT is abandoned, but it had useful syntax highlighting for ENG and WAG files. Visual Studio Code is safer for current work. An ENG/WAG syntax-highlighting project is available at https://github.com/pwillard/engwag

How to Install Blender

Blender.org releases new versions regularly. This notebook assumes a 64-bit Windows installation. The safest choice is the current LTS release. The latest stable release is also suitable when the exporter add-on supports it. This document is currently being checked against Blender 5.2.

Blender can run in portable mode from a USB stick, but the MSI installer is usually the best choice.

Current Blender versions do not install on Windows 7. Upgrade Windows before relying on this notebook. Some concepts still apply to old Blender versions, but the current workflow assumes a supported Blender release.

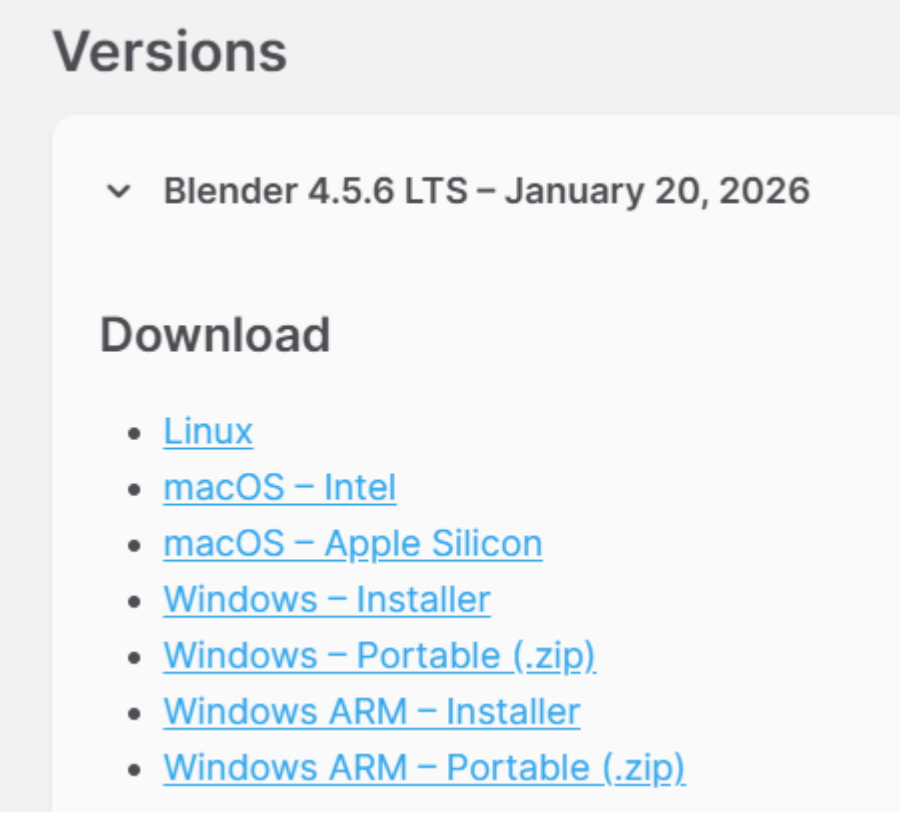
If you install Blender from Steam, Steam may auto-update Blender by default. This can be inconvenient if you need to keep a specific Blender version for exporter compatibility.

Blender's user interface changes between versions. Screens in this notebook may

not exactly match your installed version.

Download Blender from https://Blender.org/download/

The LTS release is linked from the "Looking for Long-Term Support? Get Blender 5.2 LTS" section. It is also available here: https://www.Blender.org/download/lts



Hopefully, Blender.org will continue offering LTS releases.

Choose the Windows `Installer` option, download the `.MSI` file, and double-click it to install Blender.

Blender Defaults

Blender normally uses your `Documents` folder for models and `%APPDATA%` for configuration data and add-ons under the `Blender Foundation` folder. Each Blender version has its own folder, so you can install multiple Blender versions on the same PC.

File associations can be tricky. The Blender version that opens when you double-click a `.blend` file is usually the most recently installed version. You may need to adjust this if you use multiple versions.

The APPDATA folder is normally hidden in Windows File Explorer. You can open it from a command prompt with `cd %appdata%`, or change File Explorer settings

> under **View › Options › View › Show hidden files, folders and drives**.

In PowerShell, `%APPDATA%` can be shown like this:

```
PS C:\Users\willard> $env:APPDATA
C:\Users\willard\AppData\Roaming
```

Blender user data, including preferences, startup files, and third-party add-ons, is stored under `Blender Foundation` inside that folder. For example:

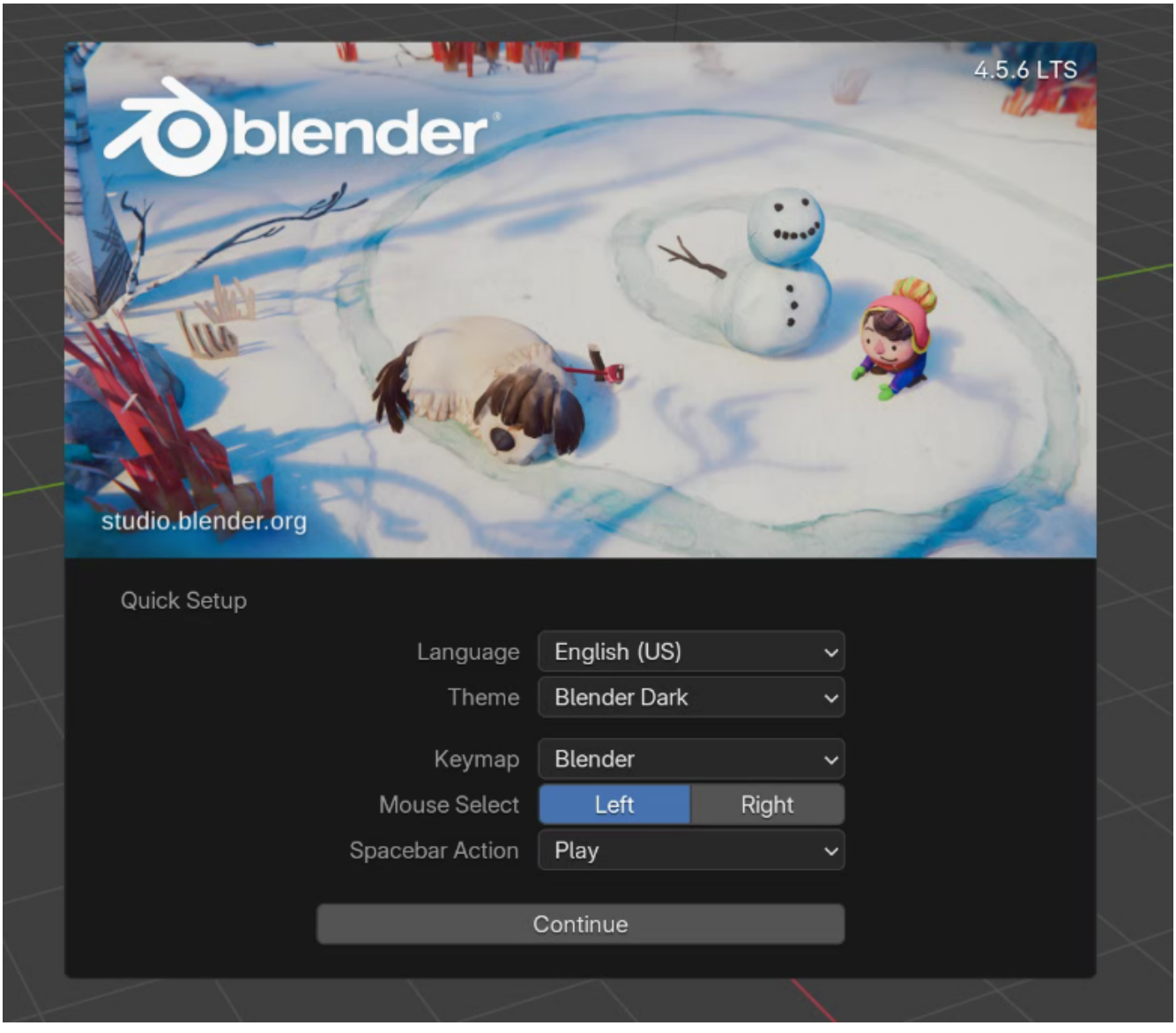
`C:\Users\willard\AppData\Roaming\Blender Foundation\Blender\4.5\config`

The following older video gives a general idea of installing and initially configuring Blender. The same steps are also described below. Video Link: https://youtu.be/ad4vTwCGodo

Setting up our Blender Environment

Most of this setup is optional, but it helps create a comfortable starting environment. Exact settings may vary by Blender version.

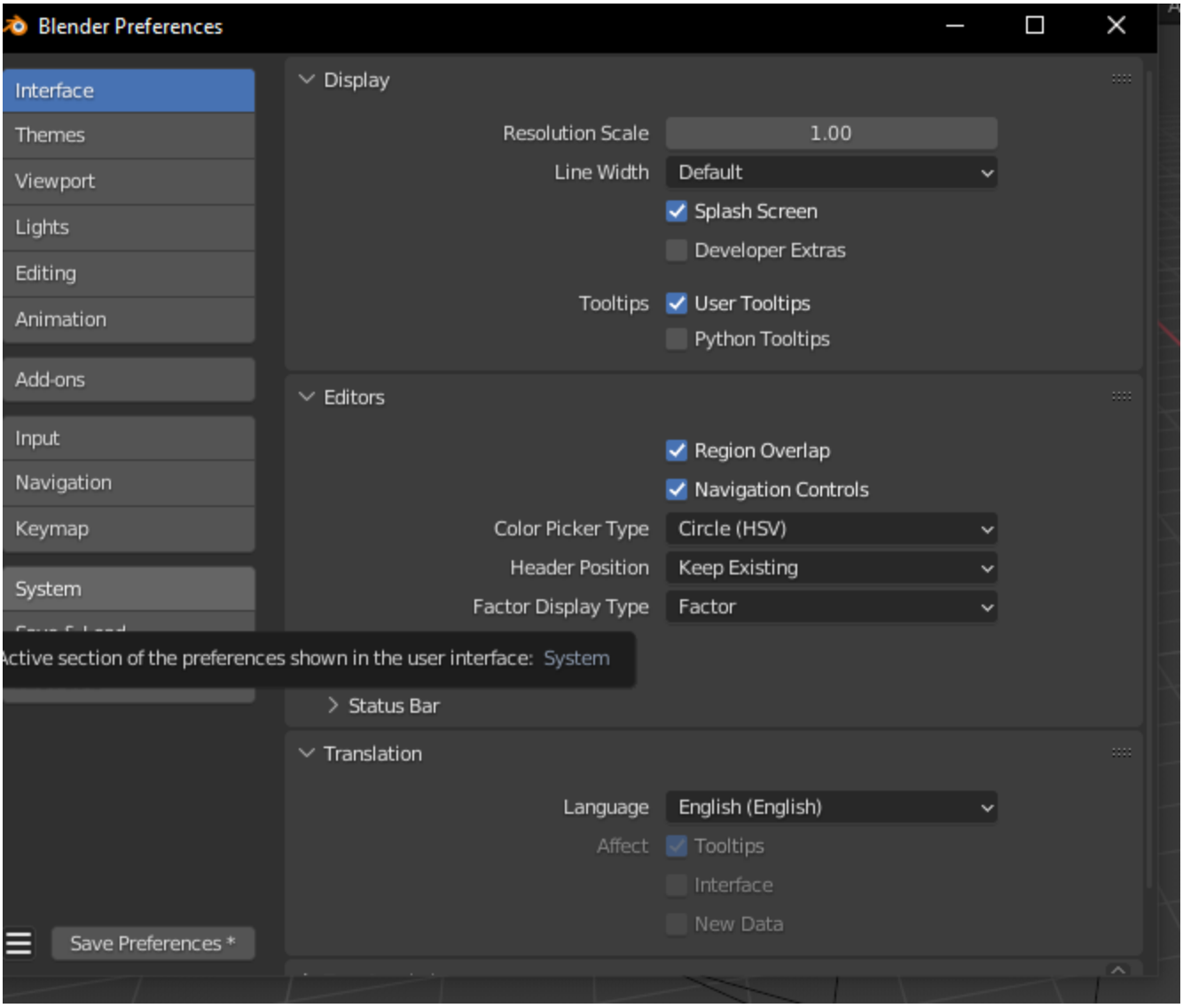
On Blender's first-run splash screen, change `SPACEBAR` so it opens `Search` instead of playing the animation.



You see the first-run screen only when the user preferences file does not exist. You can see it again after a fresh install or after deleting the user preferences file, such as:
`%appdata%\Blender Foundation\Blender\3.3\config\userpref.blend`

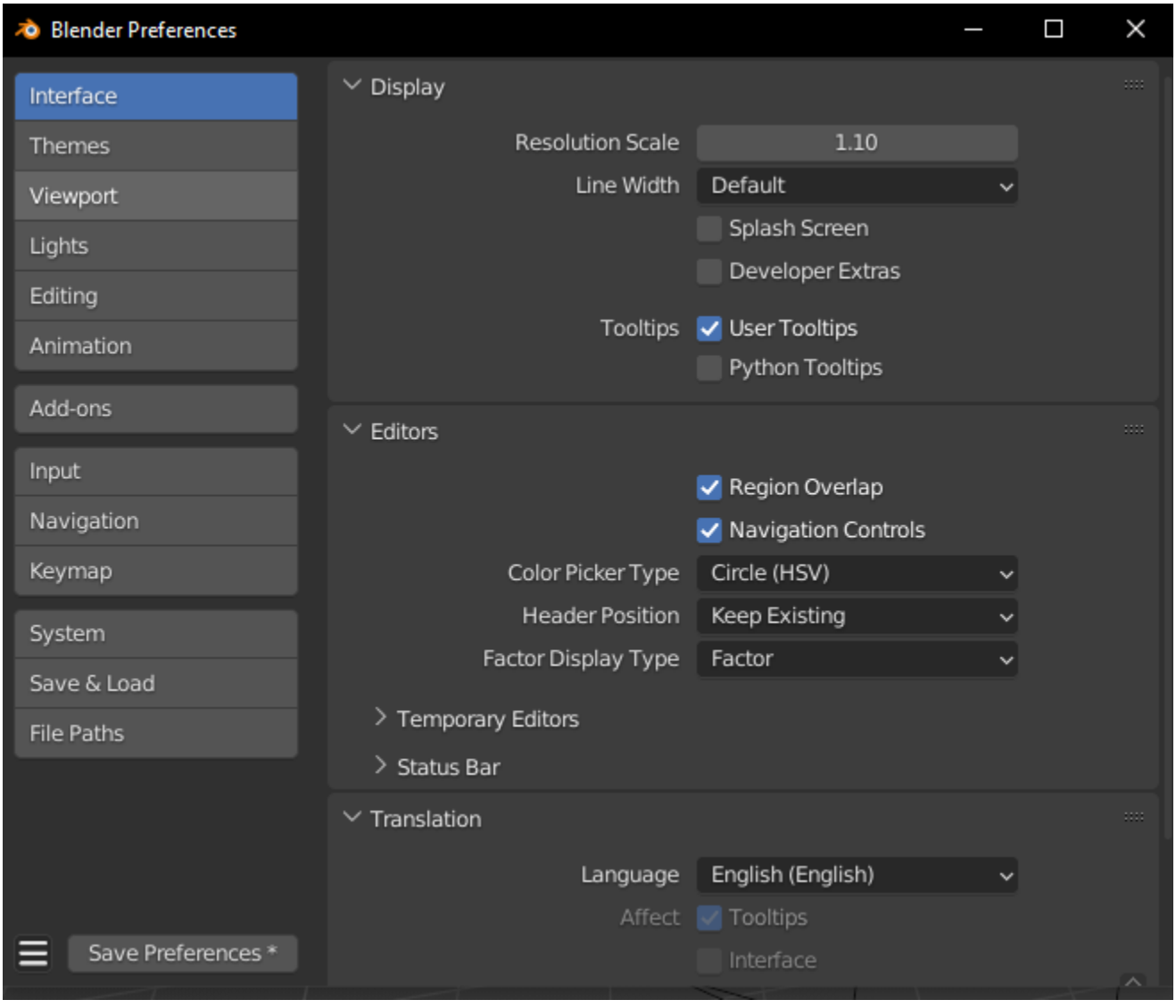
Open preferences from **Edit › Preferences**. The gear icon on the top bar also leads there.

Interface



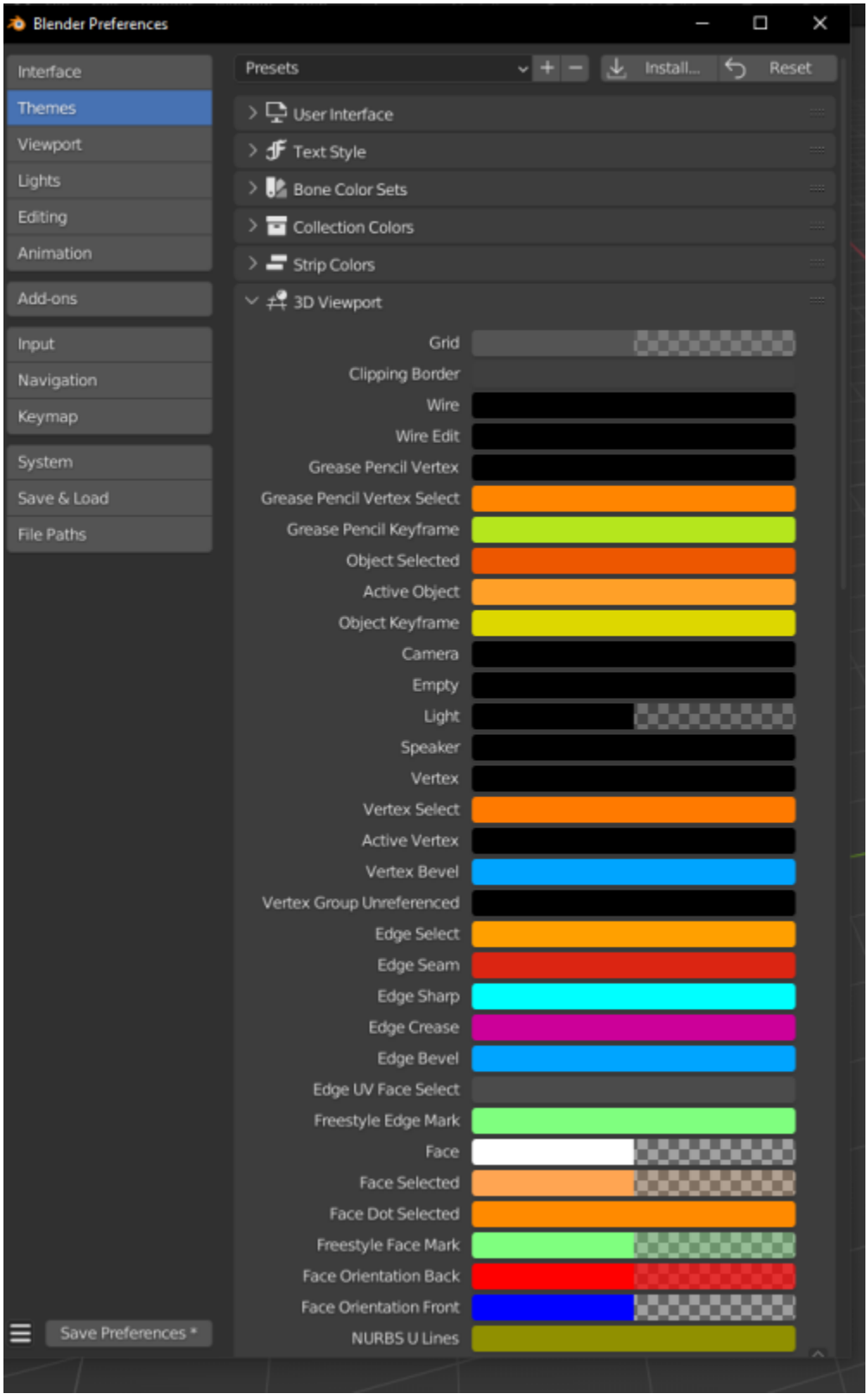
Use `Resolution Scale` to make Blender's interface text comfortable for your monitor. Move the slider left or right until the screen is readable.

Consider disabling the `Splash Screen` option after you are familiar with Blender.



This screen shows the author's preferred settings.

Themes-3D Viewport



In **Themes** › **3D Viewport**, adjust the Face Orientation Alpha setting. The default outside-face blue

can be distracting when Face Orientation is enabled. Setting the blue alpha value to 0 hides the blue while still showing red for inside-facing normals.

At the bottom of this menu, increase the vertex size from 3 to 5 or 7 if vertices are hard to see in Edit Mode.

Viewport

For better viewport quality, adjust these settings.

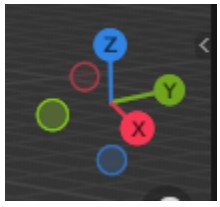
In **Viewport › Quality**:

- Samples = 16

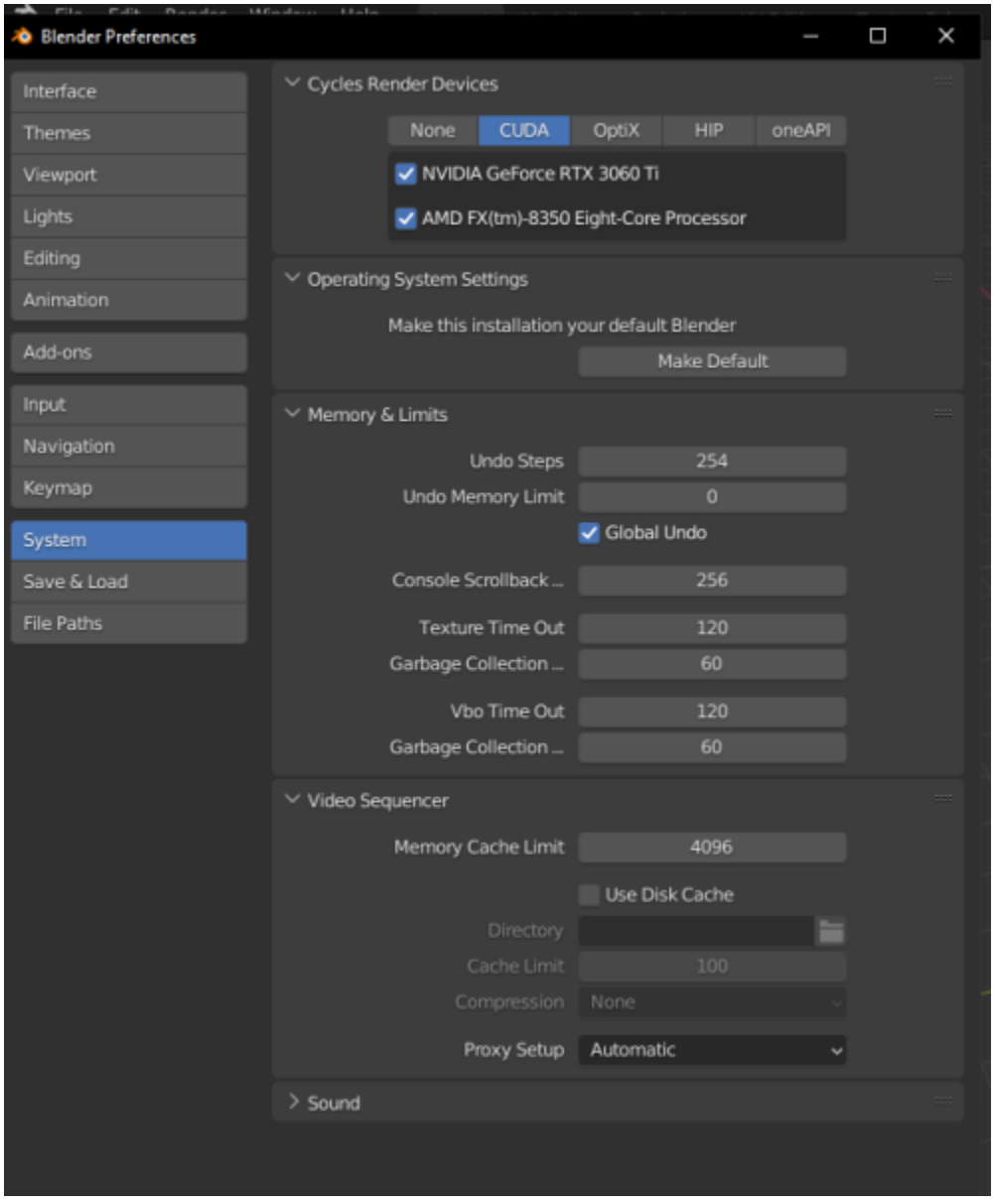
In **Viewport › Textures**:

- Anisotropic = 4x

You can also change the 3D Viewport Axis to a smaller Simple Axis if you do not like the large navigation gizmo.



System



In **System › Cycles Render Tab**:

- Enable both CPU and GPU rendering with CUDA if your system supports it.

In **System › Memory & Limits**:

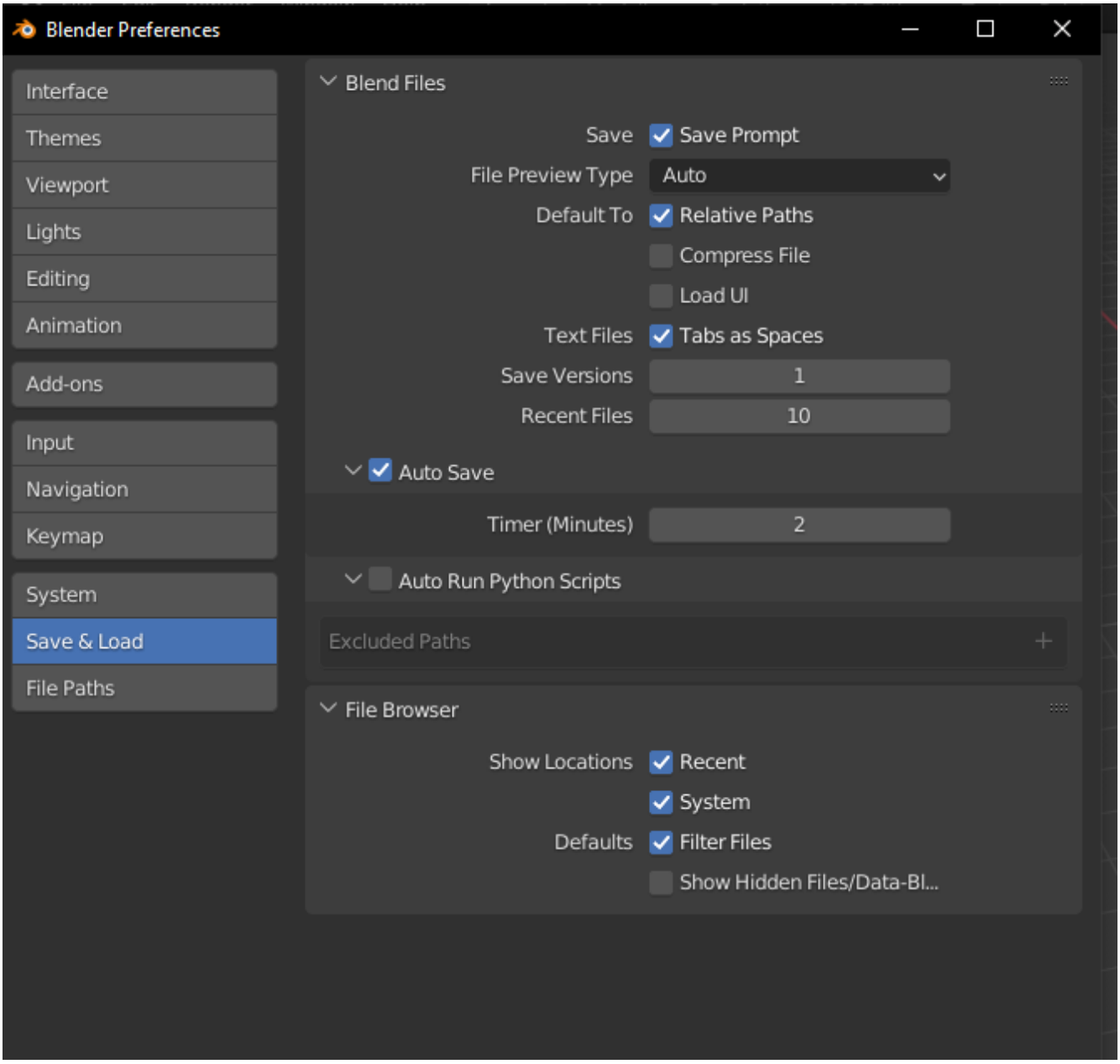
- Increase Undo Steps to 200 or higher. The maximum may be 254. The default of 32 is often too low.

Unless you are using modifiers for non-destructive modeling, most Blender edits are one-way after they are made. You can undo with `Ctrl` + `Z` only during the current editing session. Save versioned files such as `crane_V1.blend`,

`crane_V2.blend`, and so on at the start of each session. This gives you a way back if later edits fail. Blender also has automatic backup options.

If you use Blender 4.3 or newer, open System settings under Network and enable `Allow Online Access`.

Save & Load



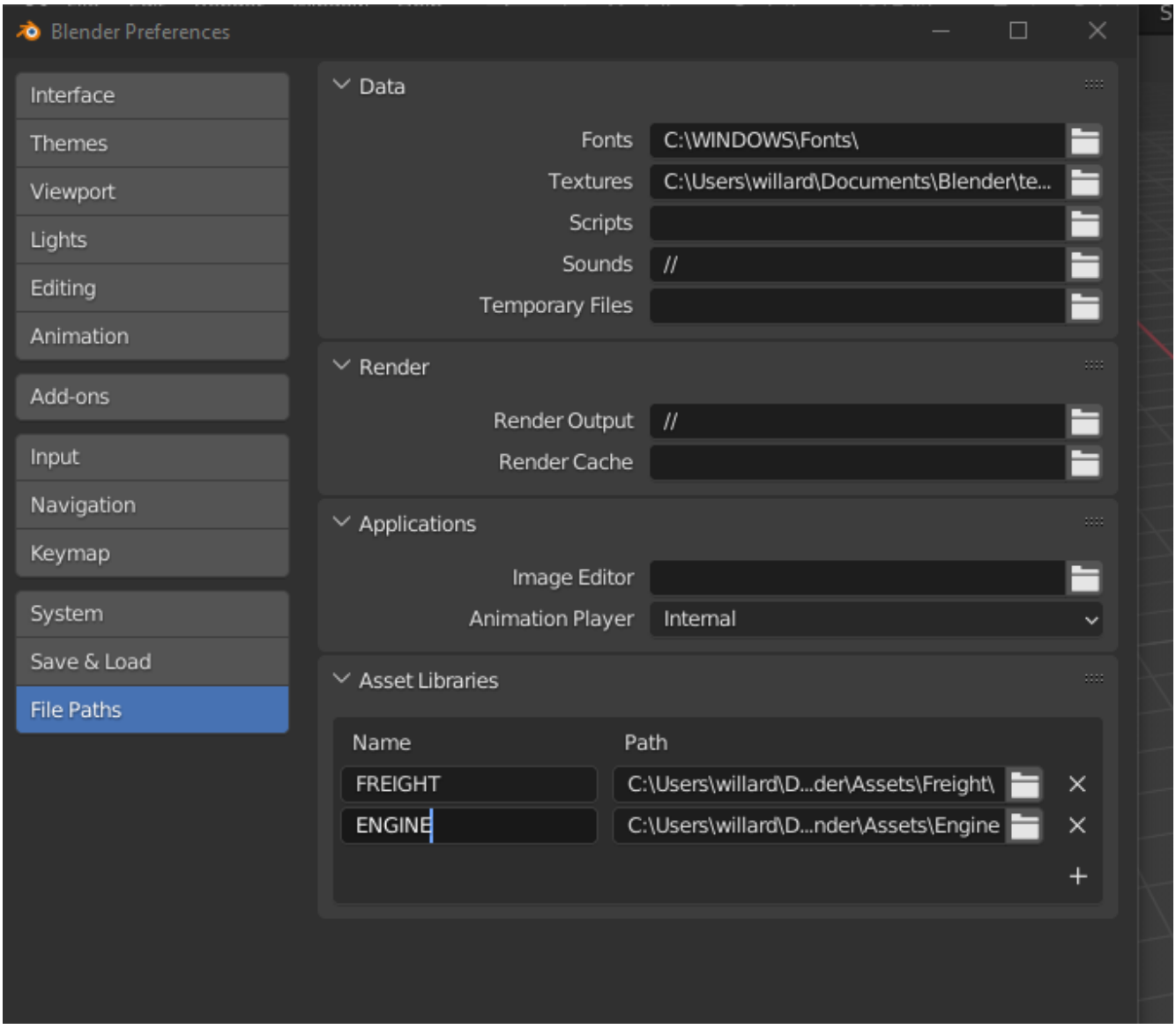
In **Save & Load › Blend Files**, consider disabling `Load UI` when needed. If `Load UI` is enabled, Blender loads the interface layout saved inside the `.blend` file instead of using your normal interface. This can be confusing when opening someone else's file. You may want to turn it on or off depending on the project.

Re-enable `Load UI` when a `.blend` file intentionally stores special settings, such as millimeter units for 3D printing models.

Set `Save Versions` so Blender creates backup copies of the current `.blend` file.

File Paths

Because Open Rails is a Windows program, this notebook uses modern Windows file paths. You can browse to these locations in Windows Explorer.



Blender has default or user-defined locations for fonts, textures, temporary files, scripts, and other resources. Many default to your Windows `Documents` folder.

In **File Paths › Data**:

- I usually set a shared texture library location for objects used across several models, such as `%USERPROFILE%/Documents/Blender/TEXTURES`.
- The Scripts location is where Blender Python scripts can be placed. One script needed later in this notebook may use this location.

Version Compatibility Notice

The following section applies to Blender 3.0 and later. Some Blender 4.0 and 5.0 changes may affect these steps, especially add-on installation.

Starting with Blender 4.0, many add-ons that used to be included directly with Blender are downloaded and installed over the network after initial setup. This can change how add-ons are found, enabled, and managed.

In **File Paths › Asset Libraries**:

- The default location is usually `%USERPROFILE%/Documents/Blender/ASSETS`.
- To add another asset library folder, click the `+` button at the bottom right of the Asset Libraries window.

Customizing Asset Library Locations

In this example setup, the default user library has been renamed `FREIGHT`, and its path has been changed. A second library folder has been added for `ENGINE` assets.

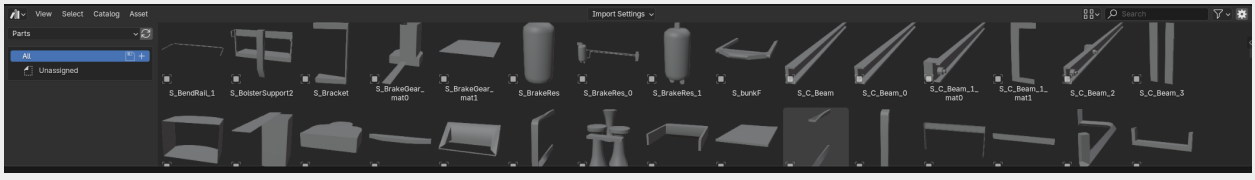
These choices are personal. Name and organize library folders in whatever way fits your workflow. You can add more folders at any time with the `+` icon at the bottom right of the Asset Libraries window.

Asset Library Files

Asset Library files are `.blend` files containing one or more objects marked as assets. Only objects marked as assets appear in the Asset Browser.

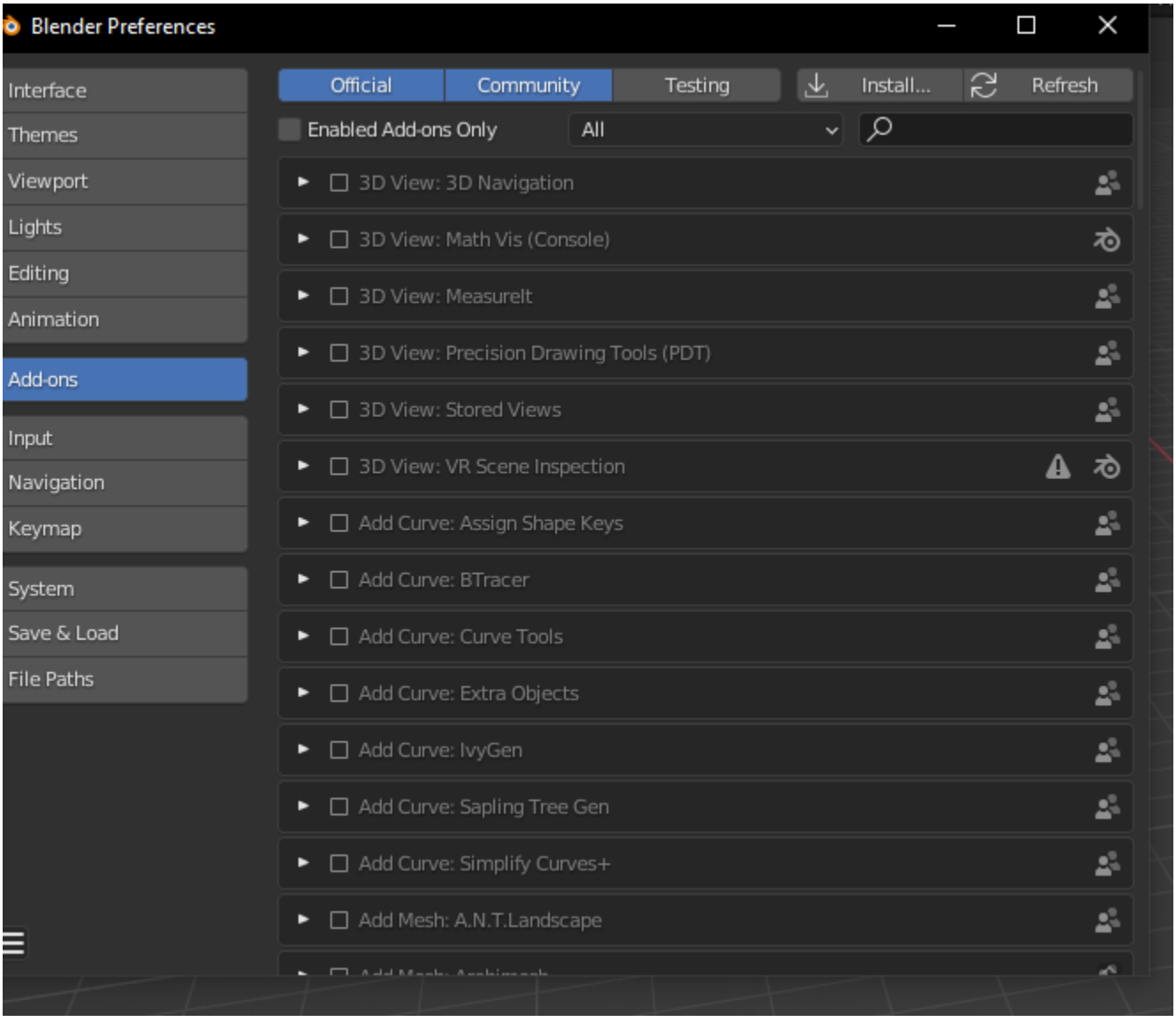
To view the Asset Browser while modeling, use the `Layout` tab. At the bottom of the screen, replace the Animation window with the Asset Browser by clicking the small clock-style editor menu next to the Playback tab and choosing `Asset Browser`.

Resize the Asset Browser by dragging its top border upward when the cursor changes to double arrows.



Select different asset libraries from the menu on the left.

Add-Ons



The Add-ons section manages scripts that extend Blender. Add-ons can add importers, exporters, panels, tools, and automation features. You can search, install, enable, disable, and configure them here.

Official

Add-ons written by Blender developers or distributed through Blender's official channels.

Community

Add-ons written by Blender community members.

Blender 4.x includes an Add-on Manager and Extensions system for installing add-ons from the internet. Enable `Allow Online Access` in System preferences if you

want Blender to download add-ons directly.

In current Blender versions, click the `Install` button beside an add-on to download and install it. You can search by name or category using the search bar.

Installing Add-Ons with Version 4.x

Blender add-ons expand Blender with tools, import/export features, UI panels, and automation. Add-ons may be bundled with Blender, downloaded by Blender, or installed from third-party ZIP packages.

Enabling Built-In Add-ons

1. Open **Edit** → **Preferences…**
2. Select the **Add-ons** tab.
3. Search for the add-on.
4. Enable it with the checkbox beside its name.

Some bundled add-ons may require a small download the first time they are enabled.

Installing Third-Party Add-ons

1. Download the add-on as a `.zip` file.
2. In Blender, open **Edit** → **Preferences** → **Add-ons**.
3. Click **Install…** in the upper right.
4. Select the ZIP file and click **Install Add-on**.
5. Enable the add-on with its checkbox.

Do not extract the ZIP unless the add-on's instructions tell you to. A Blender-installable ZIP normally contains an `init.py` file at its root.

After Installation

Add-ons may:

- Add panels to the Sidebar, opened with `N`.
- Add menu items under **File** → **Import** or **File** → **Export**.
- Add settings in the add-on's Preferences section.

If **Auto Save Preferences** is enabled, Blender remembers enabled add-ons the next time it starts.

Add-on File Locations

Type	Location
Built-in Add-ons	Installed with Blender in the program directory
User-installed Add-ons	Stored in the user profile without requiring administrator access

To reveal an installed add-on's location, use:

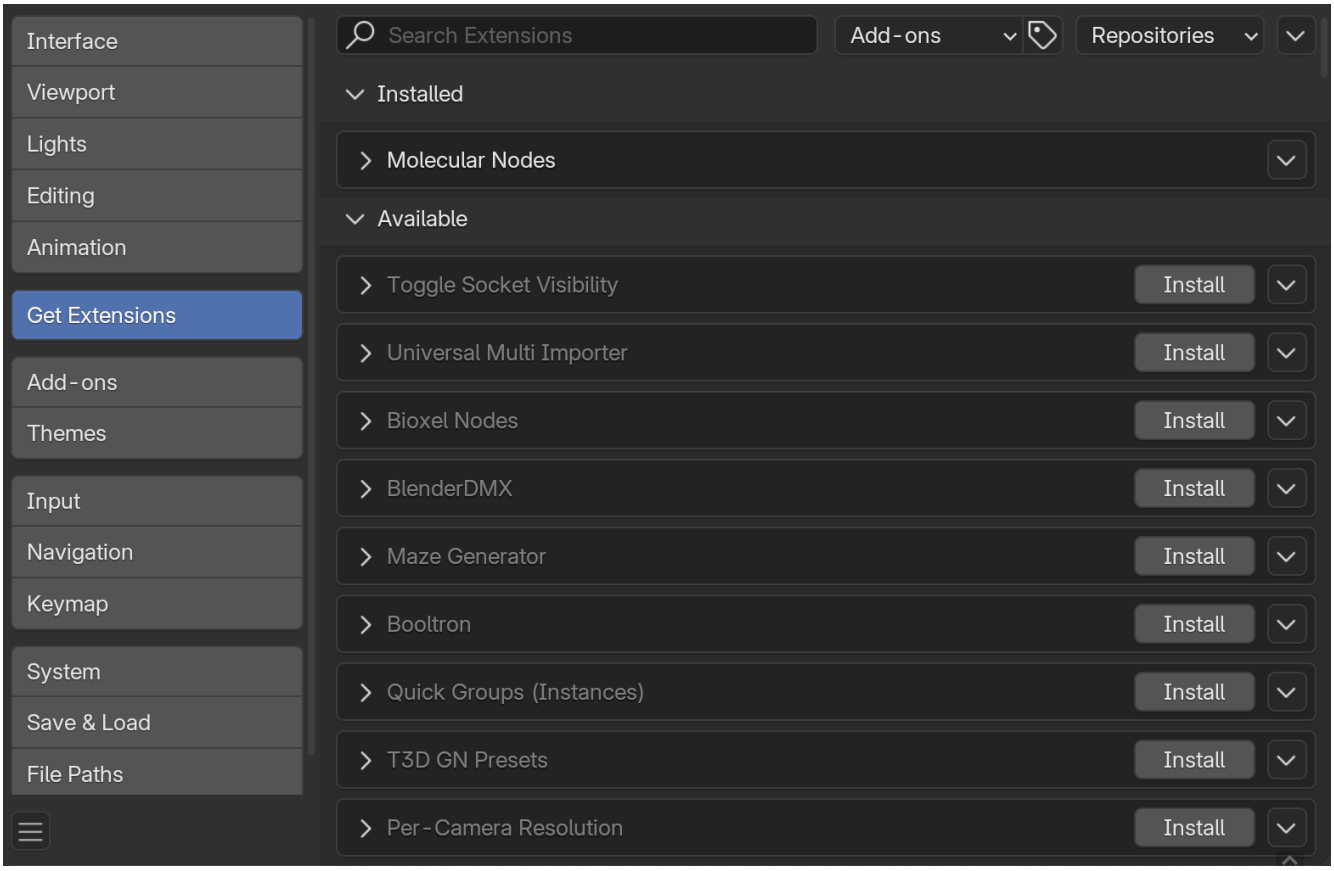
Preferences → **Add-ons** → Gear icon → **Show in File Browser**

Troubleshooting

Issue	Solution
Add-on does not appear after installation	Make sure you selected a ZIP that contains an `init.py` at the expected level, not a ZIP that contains another folder inside it.
Add-on throws errors	Check that it supports your Blender version.
Panel or tool is not visible	Check the Sidebar with `N` or look under **File** → **Import/Export**.
Add-on is disabled after restart	Enable **Auto Save Preferences**.

Download add-ons only from trusted sources such as GitHub, Blender Market, or other reputable sites.

Also in Blender 4.x - Extensions



Blender 4.x introduced the Extensions system. Extensions include add-ons, themes, keymaps, and other modular packages. They are managed through Blender's extension tools and the official Blender Extensions Platform at extensions.blender.org.

Extensions are meant to make finding, installing, enabling, updating, and removing modular features easier from inside Blender.

The term "Add-ons" is still valid. Newer Blender versions use "Extensions" to cover more than Python add-ons.

To enable the feature the first time:

- Open **Edit** → **Preferences** and go to the **Extensions** or **Get Extensions** tab.
- If prompted, allow Online Access.
- Blender will show available extensions from the official platform.

To browse for extensions:

- Click **Get More Extensions** at the bottom of the Extensions tab.
- Blender opens the Extensions Platform website in your browser.
- Search by name, category, or author.

To install, enable, update, or uninstall extensions:

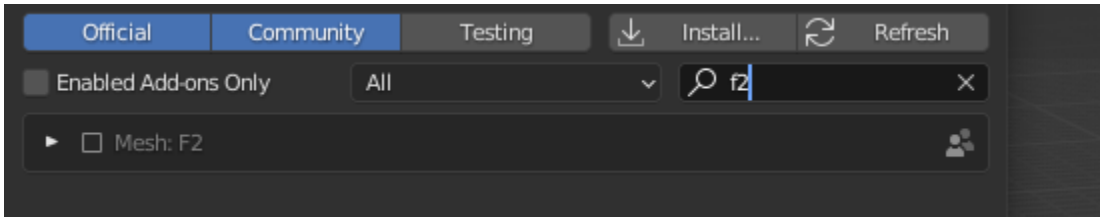
- Click **Install** beside the extension you want.
- Use the checkbox to enable or disable it.
- Click **Update** when an update is available.
- Click **Uninstall** to remove it.

Enabling and Disabling Add-Ons

Blender no longer bundles every built-in add-on directly in the installer. Some must now be downloaded from the internet.

Enable or disable an add-on by checking or unchecking its checkbox. Enabled add-ons are available immediately.

Use the search option to find a built-in add-on, then enable it with the checkbox.



Add-ons are grouped by type, such as Mesh, Import-Export, Curve, and so on.

Recommended built-in add-ons to start with:

- MESH: `LoopTools`
- MESH: `F2`
- MESH: `Edit Mesh Tools`
- NODE: `Node Wrangler`
- ADD CURVE: `Extra Objects`
- ADD MESH: `Extra Objects`
- MESH: `Automirror`

When searching, use the add-on name on the right. For example, type `F2` to find the F2 add-on.

Loop Tools

An Edit Mode add-on with operators for circles, curves, bridges, segments, flattening, twisting, lofting, and spacing vertices. It appears in the `N` panel under the `Edit` tab and in the `RMB` menu.

F2

A face-creation add-on that speeds up filling holes and repetitive face creation. It uses the `F` key.

Edit Mesh Tools

An Edit Mode add-on with tools for vertices, faces, and edges. It appears in the `N` panel and `RMB` menu. It is worth experimenting with, especially for face operations.

Node Wrangler

A node-editing add-on. Useful shortcuts include `Ctrl` + `Space` for the general menu, `Ctrl` + `T` for texture and UV nodes, `Ctrl` + `Shift` + `T` for PBR texture setup, `Ctrl` + `Shift` + `LMB` to isolate a texture, `Alt` + `RMB` for mixed shader nodes, and `Ctrl` + `RMB` for quick links.

ADD CURVE: `Extra Objects`

Adds curve primitives such as arcs, arrows, cogwheels, cycloids, flowers, helices, noise shapes, profiles, rectangles, splats, stars, knots, and other variants.

ADD MESH: `Extra Objects`

Adds mesh primitives such as beams, gears, honeycomb, diamonds, pipe joints, stars, wall builders, and other shapes.

Automirror

A quick mirroring tool with several options. It is simpler than using a Mirror modifier in some situations.

3rd Party Add-Ons

The following third-party add-ons are useful for this workflow. One is optional if you also model for Dovetail Games Train Simulator.

1) Blender28toMSTS

- Available from: https://github.com/pwillard/Blender_MSTS_ORTS_Exporter/blob/main/Blender_MSTS_ORTS_Exporter.zip
- Video: [1]
- Older instructions said to unzip the file before using Blender's install option because support files in the distributed ZIP prevented direct installation.
- Docs: https://github.com/pwillard/Ebook-MSTSORTSExporter/blob/main/MSTSORTSExporter.pdf

2) Edge to Curve

- Available from: https://github.com/Stromberg90/Scripts
- Converts mesh edges into curve shapes that can later be converted back to mesh. Useful for

handrails and pipes.

3) Export To Blend

- Available from: https://blendermarket.com/products/export-to-blend
- Price: $5.00
- Exports selected objects to a new `.blend` file while preserving materials, textures, and texture coordinates. Useful for creating a library of reusable model parts.

4) TrainSimTools

- Available from: https://github.com/pwillard/TrainSimTools/releases/tag/v1.0.0
- Optional add-on for MSTS/ORTS model setup and manipulation.
- It can create required collection structures automatically so you do not need to hard-code them into your startup file.
- This is useful when you work on both train-simulator and non-train-simulator models.
- This add-on is a work in progress.

Most recommended add-ons are free, but some useful add-ons are paid.

Many add-ons are developed outside the Blender project. BlenderMarket, now called Superhive (https://superhivemarket.com/), and Gumroad are common places to find them.

To install third-party add-ons, choose **[Install...]** and select the `.zip` or `.py` file. After installation, enable or disable the add-on with its checkbox.

BLENDERKIT

From Marek at Elvas Tower: I have been using the free `BlenderKit` add-on. It is enabled from **Preferences** > **Add-ons**. `BlenderKit` includes thousands of materials. Rubber, metal, plastic, and checker-plate materials are useful for rolling stock.

Because procedural materials can be scaled without losing image quality, they can help create details such as walkways. I rendered the walkway from the same top-down camera used for the Ambient Occlusion, edited the render in GIMP, and placed it on the texture file. Because the orthographic camera scale did not change, it aligned with the Ambient Occlusion layer. This gave much better results than previous methods.

Blender's render engine and 3D texture-painting tools are worth exploring as additional tools for texture artists.

Exit and Save Preferences

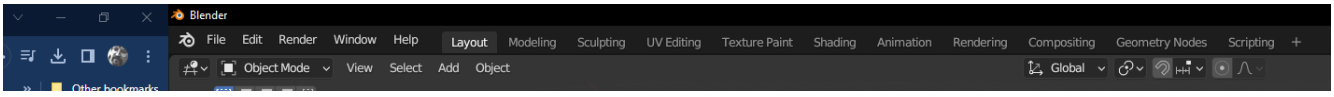
Closing **Edit › Preferences** saves changes automatically unless you disabled `Auto Save Preferences` in the hamburger menu at the bottom of the Preferences window.

Other Settings

Next, adjust useful header and properties-panel settings. These settings are optional, but they are worth reviewing.

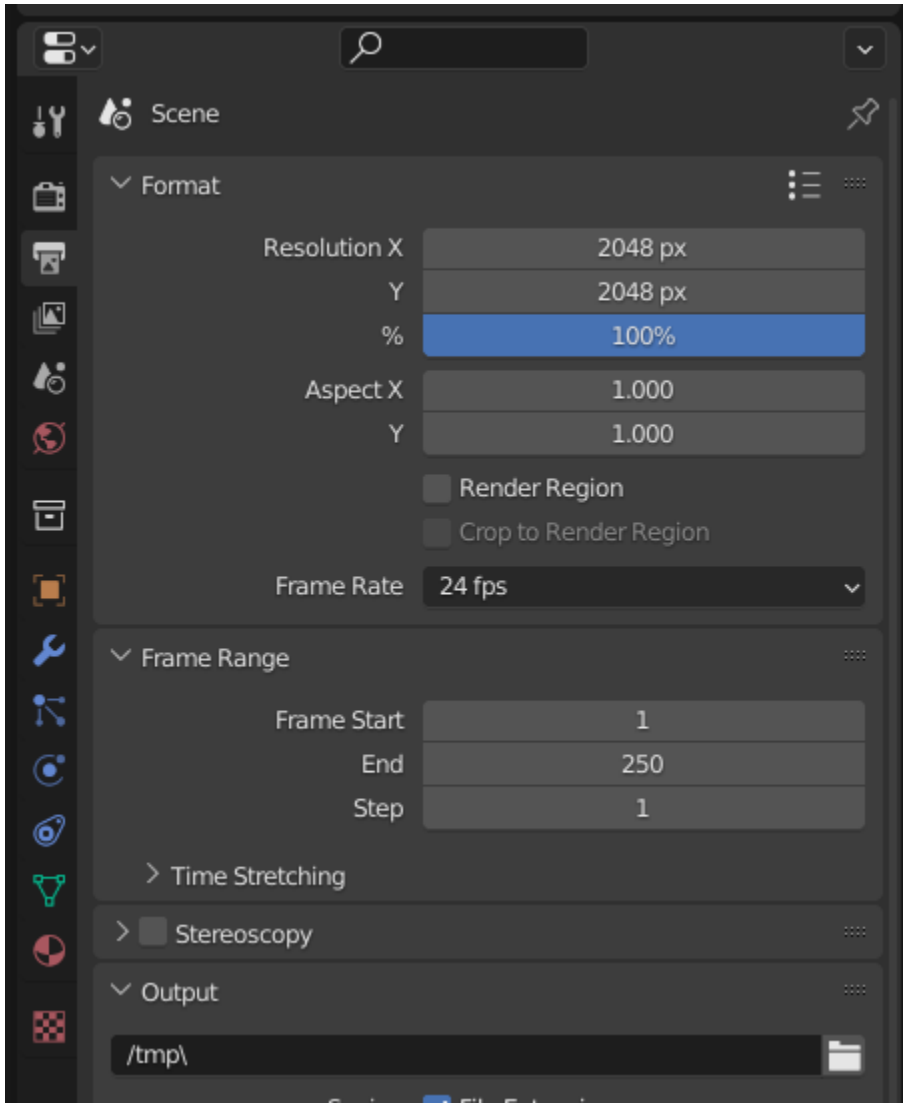
TABS

The top of the Blender window contains workspace tabs. You do not need every tab for Open Rails modeling. The **Modeling** tab is similar to the **Layout** tab and is mostly a holdover from older Blender versions, so you can delete it if you want more room. The Sculpting tab is also rarely needed for hard-surface modeling.



Tabs can be restored with the + button. Save workspace changes with **File › Defaults › Save Startup File**.

Scene Settings



Set the default output resolution to 2048 x 2048 because square textures are common. Open Rails does not require square textures the way Microsoft Train Simulator does, but square textures are still useful for consistency.

Gizmo settings

This is optional and depends on personal preference. Some users find the extra controls cluttered.

In the Layout window, open the Gizmos drop-down menu. It uses an arc-and-arrow icon. Enable the Move option if you want axis-based move arrows for moving objects. You can also move objects with the `G` key.

Overlays

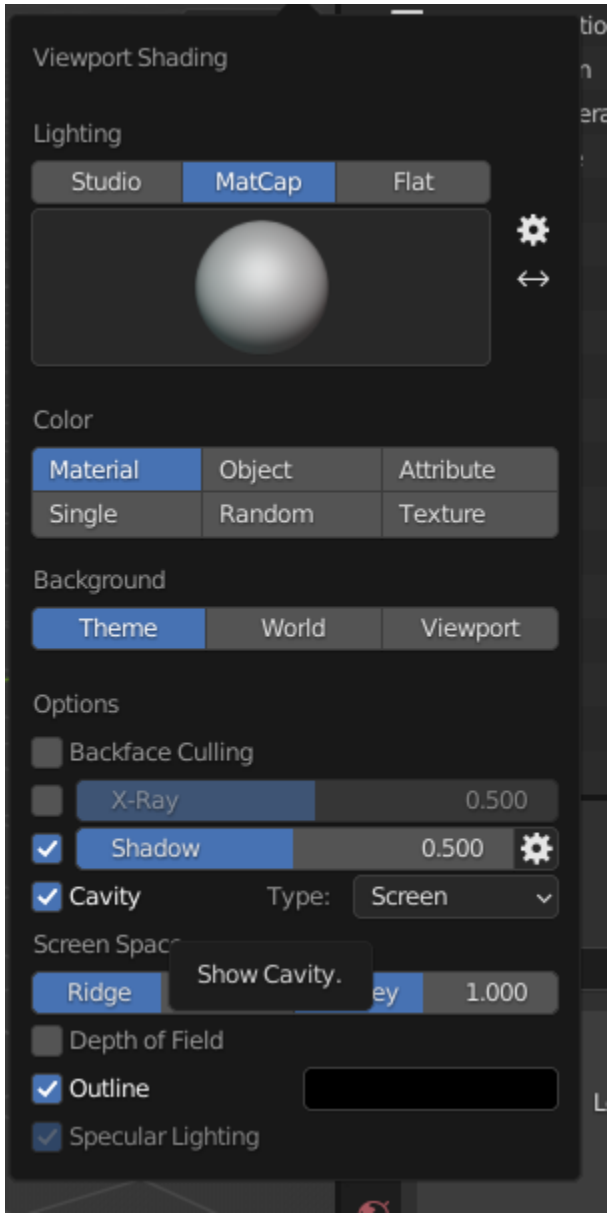
The Overlays icon is next to the Gizmo options.

Enable `Statistics` in Overlays so you can watch polygon count while modeling.

This is also optional and can make complex scenes look cluttered.

To show edge lengths, select the default cube, switch to Edit Mode with `Tab`, open the Overlays drop-down, and enable `Edge Length`. If there is no cube, add one with **Add › Mesh › Cube**. This displays selected edge lengths in Blender units. Add-ons such as Construction Lines can provide more advanced measurement tools.

Shading Settings



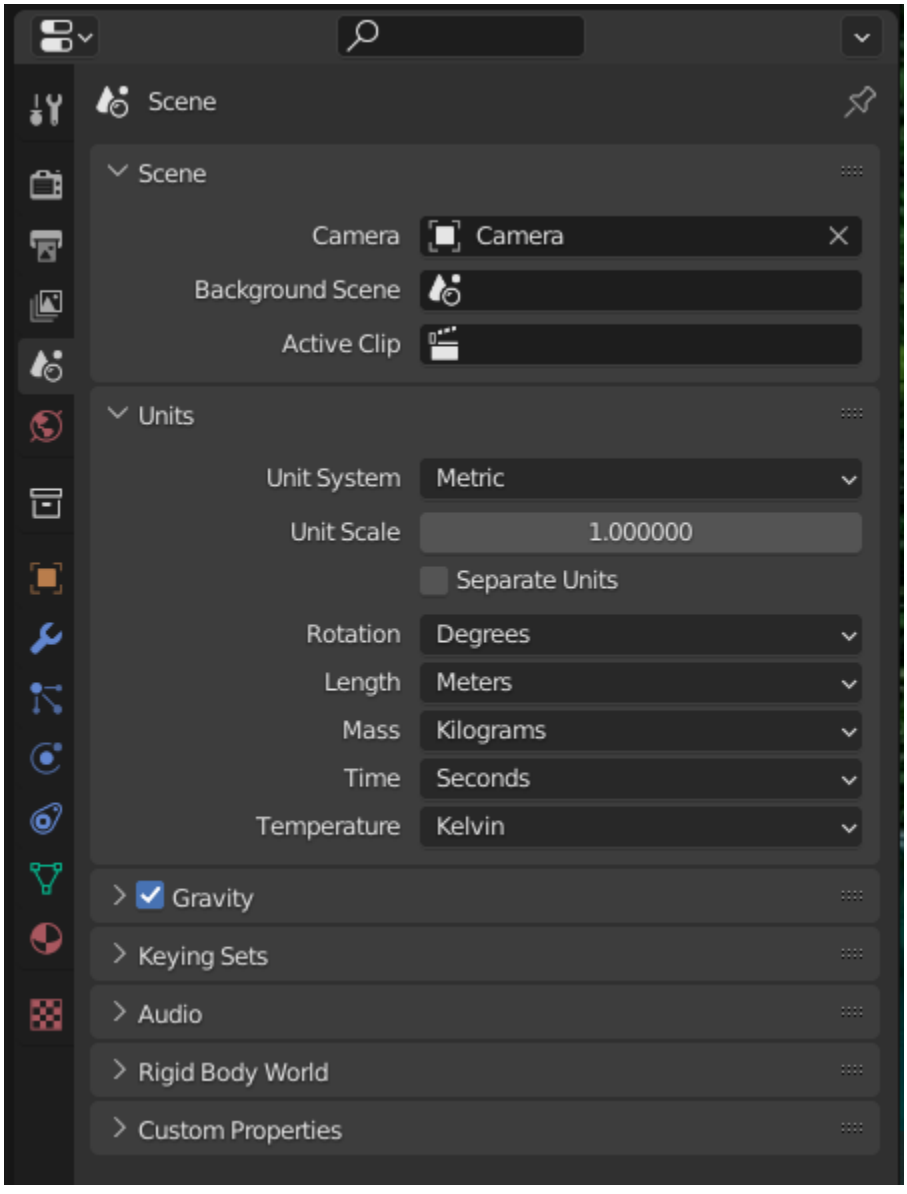
Under Solid Viewport Shading, open the drop-down beside the solid-circle icon and enable:

- Shadow
- Cavity

These settings make shapes easier to see while editing.

MATCAP shading can make shapes easier to read during early modeling. The leftmost MATCAP option is often useful for seeing form and surface errors.

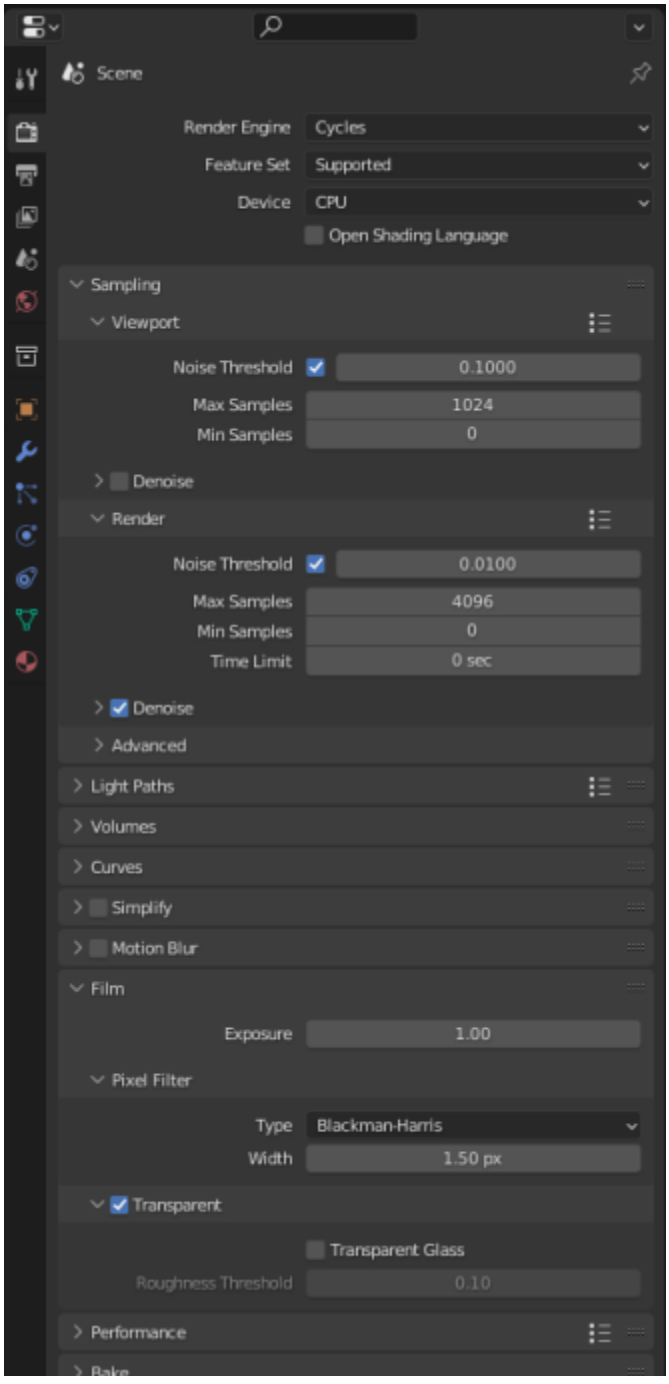
Units



If you need U.S. customary units instead of metric, change the Unit System in Scene Properties.

Changing units can affect Camera View settings such as `Focal Length`, `Clip Start`, and `Clip End`. If part of your model disappears when you zoom out, adjust the clip settings in the *N-Panel* `View` tab.

Scene (Rendering)



These render settings are optional, but they improve output quality.

In **Properties Panels › Scene › Render Engine › Eevee**, set Sampling > Render to 200 samples.

In **Properties Panels › Scene › Render Engine › Cycles**, set Sampling > Render to 200 samples.

In **Properties Panels › Scene › Film**, enable `Transparent`. This removes the render background, which is usually useful for model renders.

Shading

The Shading tab can use HDRI images for general scene lighting instead of relying on light objects.

First, download an HDRI file. Store HDRI files in a folder such as **Documents › Blender › HDRI**.

In the World Properties panel, shown with a globe icon, set the Surface to `Background`. In the Color section, choose `Environment Texture`, then click `Open` and select the HDRI file.

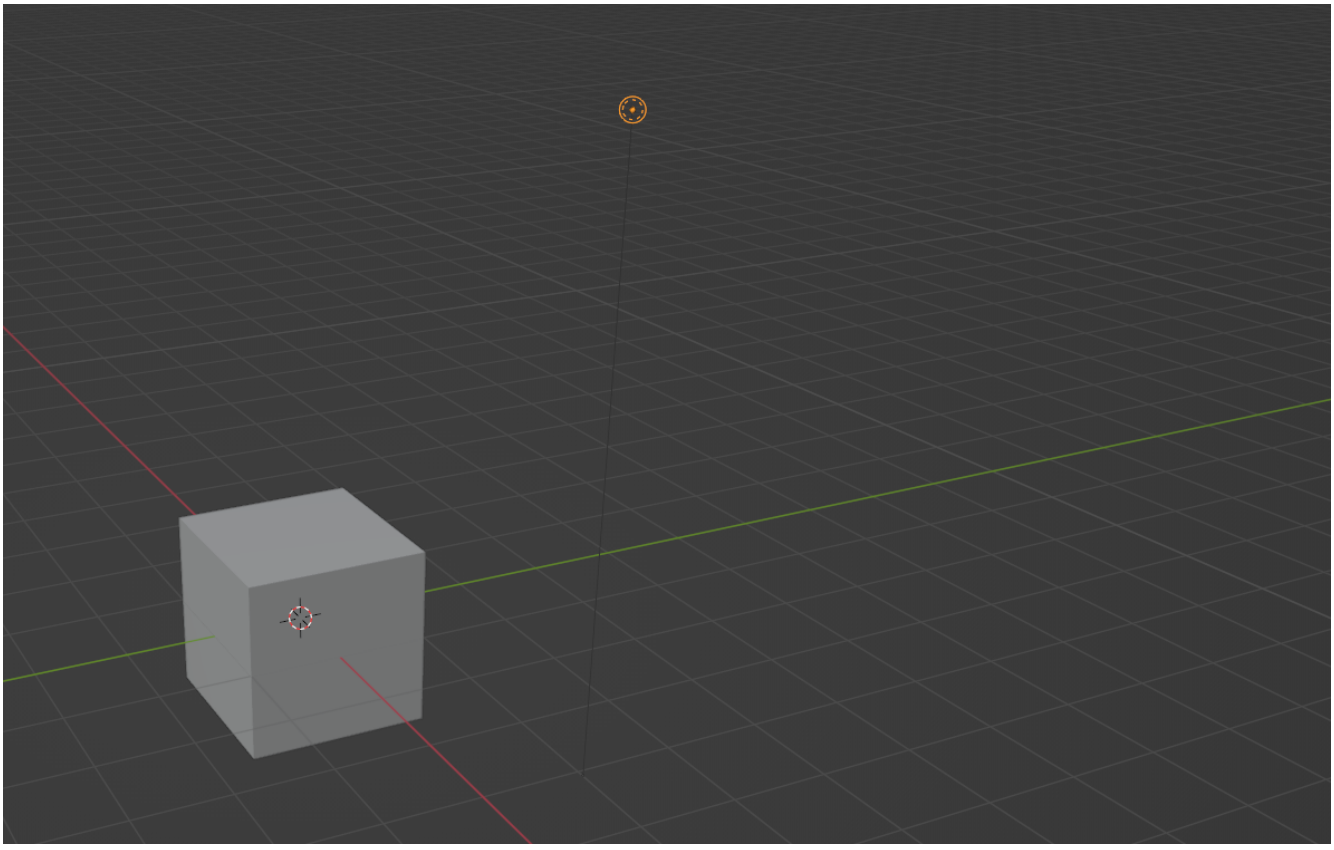
After this, rendered images use the HDRI for lighting.

To see the HDRI in the viewport, open the `Shading` tab, choose the Viewport Shading icon to the right of Solid Shading, open the viewport shading drop-down, and set lighting to `Scene World`.

If you did not enable `Transparent` in Film settings, the HDRI would also become the render background.

The `Abandoned Slipway` HDRI from https://hdrihaven.com has been recommended by Josh Gambrell for neutral outdoor lighting. The `hdri_004_nordicfxnet.hdr`, also called "Railroad Crossing," from http://www.nordicfx.net/ may fit rail projects better. I reduced it from 4K to 2K.

The free "EASY HDRI" add-on can make HDRI world-lighting setup easier. It is optional.

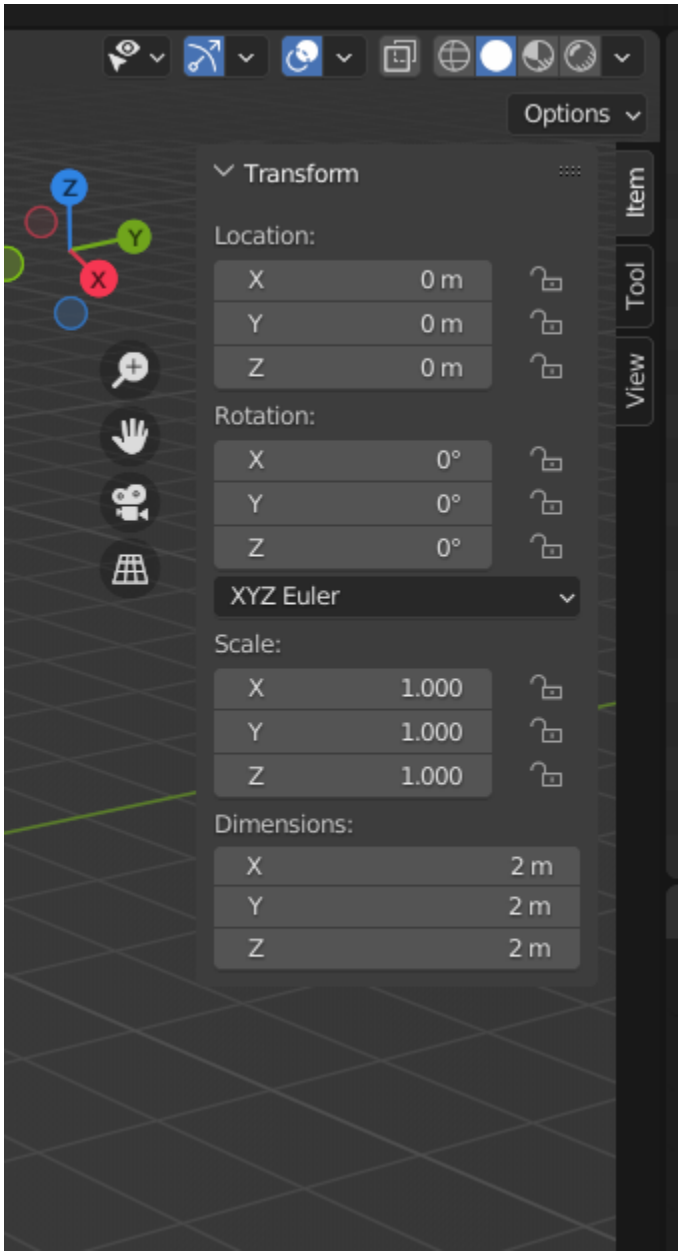


After replacing the default lighting workflow with HDRI lighting, delete the default light object from the Scene Collection or main 3D view.

Done, for now… almost

One last setup step:

Press `N` to open the Number Panel, also called the *N-Panel*. It lets you adjust object settings and locations by entering numbers. Leaving it open in the startup file makes it available for new Blender projects.



Save the setup with:

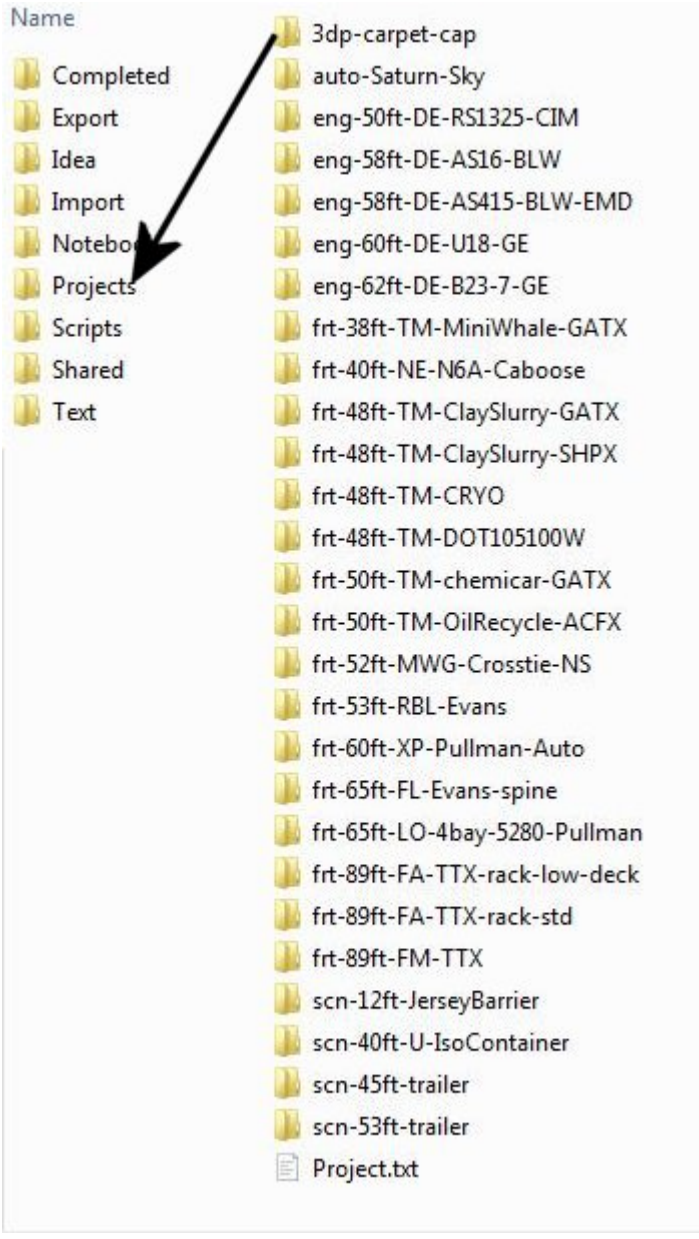
File › Defaults › Save Startup File › Confirm

Your basic preferences and startup-file options are now ready for 3D modeling simulator assets.

[1] Youtube https://youtu.be/j3AVw7s9qoA

Folder structure

Project-folder layout is personal. The following example shows one organization method.



Many details are omitted for brevity. Useful folders include `EXPORT` for renders and exports, `IMPORT` for items converted from 3D Canvas, Train Sim Modeler, or other sources, and `PROJECTS` for active work. A `SHARED` folder can hold common asset libraries and textures. Prefixes can also help organize content, such as `FRT` for freight cars, `SCN` for scenery, and `ENG` for engines.

Content Creation Overview

The next sections are not just about Blender modeling. They describe how to create items that meet simulator requirements and conventions. Some older 3D Crafter notes are included where useful.

This document began as a notebook of content-creation notes. Some topics may feel like gathered notes rather than a finished textbook. Expect updates and revisions over time.

Orientation

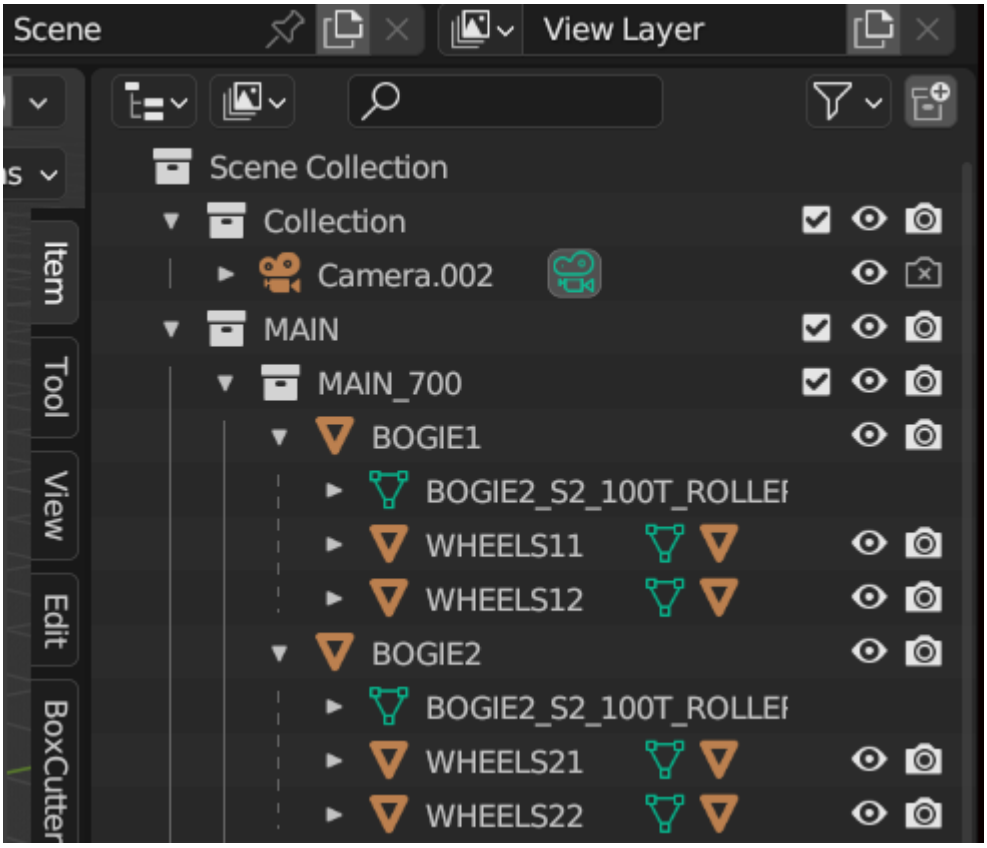
Rolling stock and engines must be oriented correctly. In some older 3D programs, such as 3D Crafter / 3D Canvas or Maya, Z may represent depth, X width, and Y height. Blender is closer to 3ds Max: Z is height, X is width, and Y is depth.

For track vehicles, the front of the model points toward positive Y. The rear points toward negative Y.

Blender's default front view is different. Blender considers the front to face the -Y direction. Pressing `1` on the keypad shows Blender's front view, which is the back end from the Microsoft Train Simulator/Open Rails perspective.

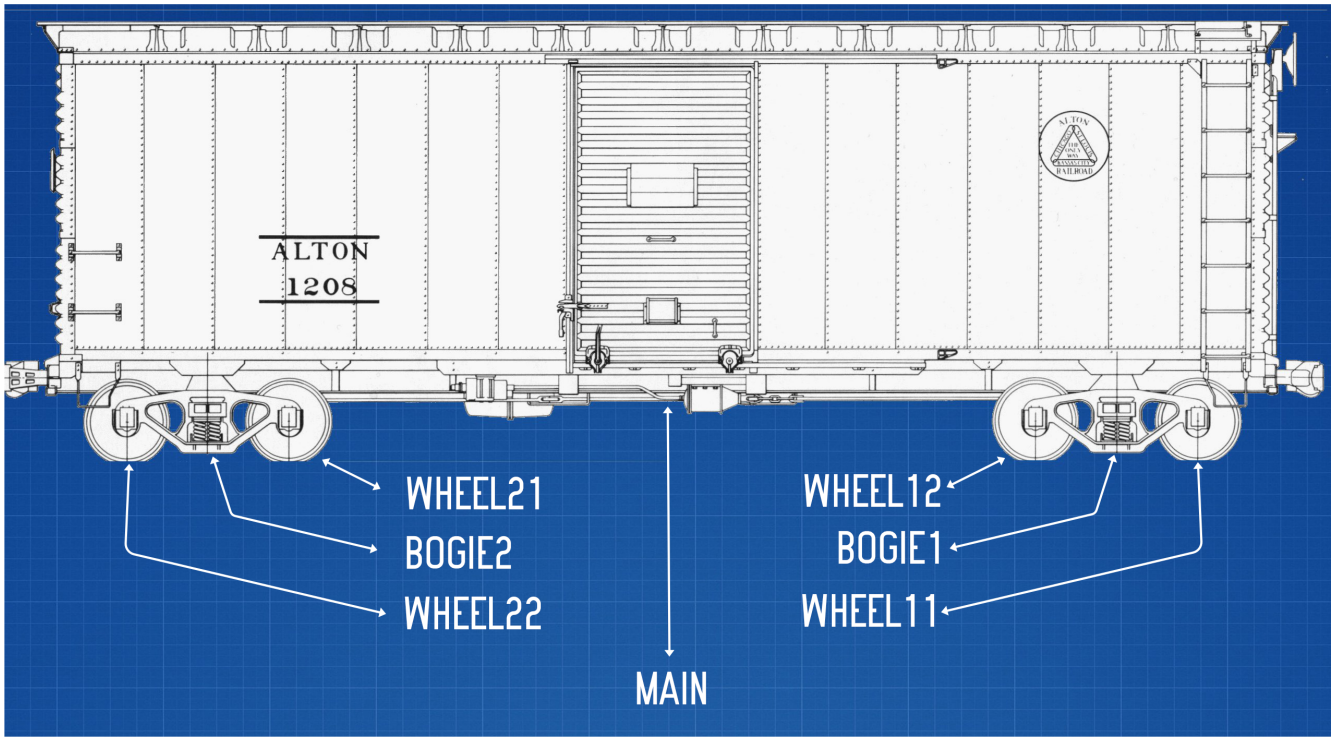
Engine/Wagon Model Hierarchy

The Blender exporter requires a collection named `MAIN`. This is the root of the model hierarchy. The model itself is not placed directly in `MAIN`; instead, model parts are placed in child LOD collections such as `MAIN_700` and `MAIN_2000`.



The Blender exporter expects required collection names to be capitalized, such as `MAIN`, `MAIN_700`, `BOGIE`, and `WHEEL`. Names such as `main` or `main_700` will cause export errors.

For Open Rails to animate wheels and bogies correctly, the model hierarchy, names, parenting, and pivot origins must be correct. In the following example, the A-end of the boxcar is on the right, the B-end with brakes is on the left, and default travel is from left to right. See the Orientation section above.



For a two-bogie, two-axle arrangement, each wheel object should use the center of its axle as its pivot. Each bogie should use its own center of mass as its pivot. Most other parts use world origin as their pivot.

The main body mesh does not need to be named `MAIN`. Modelers have used that name by convention since 2002, but the exporter only requires the `MAIN` collection to exist.

Be careful when selecting all objects and applying all transformations. This can reset all object origins to world origin, which would break bogie and wheel pivot points.

Standard 2 Axle Freight Bogies

From the left-side view, the model faces right. Starting from the right side, name bogies and wheels according to the hierarchy diagram. The correct wheels must be parented to `BOGIE1` or `BOGIE2`.

For an extra axle, use `WHEELS13` or `WHEELS23` behind the bogie pivot relative to forward motion, and shift wheel set 2 to the center of that bogie. Keep the naming sequence from 11 through 23 as shown. Incorrect order can cause wheels to rotate backward or shift incorrectly in Microsoft Train Simulator.

A two-axle bogie would not use `WHEELS13` or `WHEELS23`.

Isolated Axles

The Microsoft Train Simulator naming standard for isolated, non-bogie axles is `WHEEL1`, `WHEEL2`, `WHEEL3`, and so on.

These are mainly used for steam locomotives. Their animation is not automatic and must be created in Blender. Steam wheel and linkage animation uses 16 frames and should be treated as an advanced topic.

In Open Rails, the important parts appear to be the `WHEEL` and `BOGIE` prefixes. `WHEEL` is required for wheel animation and proper bogie parenting. `BOGIE` is required for bogie animation and proper engine or wagon parenting.

Some Microsoft Train Simulator documentation implies that a third bogie is possible in Microsoft Train Simulator. It is not. If you are modeling specifically for Open Rails, the simulator can animate correctly named bogie and wheel objects as long as parenting and local pivot origins are correct.

Microsoft recommended this topology for the Acela model:

```
MAIN ENGINE
PANTOGRAPHTOP1
        PANTOGRAPHBOTTOM1
    PANTOGRAPHTOP2
        PANTOGRAPHBOTTOM2
    BOGIE1
        WHEELS11
        WHEELS12
    BOGIE2
        WHEELS21
        WHEELS22
    MIRRORRIGHT1
    MIRRORLEFT1
    WIPERARMLEFT1
        WIPERBLADELEFT1
    WIPERARMRIGHT1
        WIPERBLADERIGHT1
```

OpenRails Animation

The following names and prefixes identify engine or wagon objects that can be controlled by simulator animations. You may add text after the listed prefix to create multiple matrices using the same trigger. For example, `MIRROR` and `MIRROR_LEFT_1` can both be controlled by the mirror

animation.

Animated object list:

Item / Pattern	Description
WIPER	Wiper animation
DOOR_A / DOOR_B / DOOR_C	Right door animation
DOOR_D / DOOR_E / DOOR_F	Left door animation
PANTOGRAPHBOTTOM1 / PANTOGRAPHMIDDLE1 / PANTOGRAPHTOP1 / PANTO with 1 elsewhere in the name	First pantograph animation
PANTOGRAPHBOTTOM2 / PANTOGRAPHMIDDLE2 / PANTOGRAPHTOP2 / PANTO with 2 elsewhere in the name	Second pantograph animation
PANTO with 3 elsewhere in the name	Third pantograph animation
PANTO with 4 elsewhere in the name	Fourth pantograph animation. Open Rails currently hardcodes support for four pantographs.
MIRROR	Mirror animation
LEFTWINDOWFRONT	Front left window animation
RIGHTWINDOWFRONT	Front right window animation
LEFTWINDOWREAR	Rear left window animation
RIGHTWINDOWREAR	Rear right window animation
UNLOADINGPARTS	Rotary unload animation
ORTSBELL	Bell animation
ORTSITEM1CONTINUOUS	Generic control 1 looping animation
ORTSITEM1TWOSTATE	Generic control 1 toggle animation
ORTSITEM2CONTINUOUS	Generic control 2 looping animation
ORTSITEM2TWOSTATE	Generic control 2 toggle animation. Open Rails currently hardcodes support for two generic controls.
ORTSBRAKECYLINDER	Brake cylinder animation
ORTSHANDBRAKE	Handbrake animation
ORTSBRAKERIGGING	Brake rigging animation

Any animation attached to a wheel without a special name is assumed to be

running-gear animation.

Any engine or wagon can include these animation names, but not every animation makes sense for every vehicle. For example, wipers are not useful on a freight car.

Configuration Files Overview

The Open Rails manual explains many features that are available only in Open Rails. The current manual is limited on content-creation details, but much of the old Microsoft Train Simulator content-creation information still applies.

This notebook is not a complete guide to `sd`, `ref`, `eng`, and `wag` files, but you will need working configuration files to add content to Open Rails. Peter Newell's website has detailed information about good `ENG` and `WAG` files for Open Rails. [1]

Scenery objects normally require an `sd` file when built for Microsoft Train Simulator. Open Rails can use a different scenery definition method, which makes the SD file unnecessary in many cases.

Also see Creating Rolling Stock and Locomotives.

Include Files in Configurations

`Include()` statements are becoming more common in Open Rails SIMIS files, but includes are optional.

In both Microsoft Train Simulator and Open Rails, `.wag` and `.eng` files identify an asset as a locomotive or non-powered vehicle. Historically, these files were large, complicated, and full of repeated data. MSTS files usually had to be self-contained. In Open Rails, `.wag` and `.eng` files can include shared `.inc` files, which makes the main files smaller and easier to maintain.

Include files are especially useful when creating several similar cars that differ only in details such as reporting marks.

An `.inc` file does not have to be in the same directory as the `.wag` or `.eng` file, although that is sometimes easiest. It only needs to be referenced in a way Open Rails can find.

When `.inc` files are used by `.eng` or `.wag` files, they need a blank line at the beginning. For safety, also leave a blank line at the end.

When ORTS loads a locomotive or car, it reads the main `.eng` or `.wag` file until it reaches an `Include` statement. It then locates and reads the `.inc` file, inserts that data into the physics data, and returns

to the main file. It repeats this process for additional include statements.

Common include-file locations:

- A subfolder inside the `.wag` or `.eng` folder.
- Directly in the `.wag` or `.eng` folder. This can become messy, so use caution.
- A special folder for files shared by `.wag` or `.eng` files that use the same `.s` shape.
- A common folder shared by many `.wag` or `.eng` files.

Create a naming standard that supports both common shared files and files specific to a master shape.

General INC file structure

INC files are still standard Open Rails configuration documents, so SIMIS rules still apply. Keep these points in mind:

- The first line of the INC file must be blank.
- Do not include a SIMIS header line such as `SIMISA@@@@@@@@@@JINX0D0t`.
- Use `COMMENT()` sections generously.
- INC files are most useful for shared configuration values.
- INC files are for Open Rails only. MSTS cannot use them.
- Open Rails reads include files from top to bottom. If values are duplicated, the last value read overrides earlier ones.

Items well suited to INC files

- Coupler details shared by many locomotives or cars.
- Wagon details shared by many locomotives or cars, such as sound references and empty/loaded states.
- Brake details shared by many locomotives or cars.
- Lights shared by similar vehicles.
- Davis coefficients shared by similar vehicles.

Use includes thoughtfully. They can simplify a project, but they can also become complicated quickly.

Specifying relative references

Open Rails must be able to locate your INC files. Use references that are relative to the main file whenever possible.

- If the INC file is in the same folder, use `INCLUDE("filename.inc")`.
- If it is in a subfolder, use `INCLUDE("subfolder/filename.inc")`.
- If it is elsewhere, use an absolute path such as `INCLUDE("C:/path/to/filename.inc")`, or a relative path such as `..\common.inc\filename.inc` to refer to a common folder one level up.

There is a way to keep content compatible with legacy MSTS while using this technique, but it is not covered here.

An interesting discussion occurred in 2017 about making these files better and more useful in the post-Microsoft Train Simulator world.

> KUJU defined many of the folder names used in Microsoft Train Simulator. The Include file concept for `.eng` and `.wag` files led me to think that something like KUJU's `\common.cab` directory tree was needed for `.inc` files.
>
> Payware vendors used several folder approaches under `\trains`: sometimes `\common.cab`, sometimes a product-specific folder, and sometimes a vendor folder for an unskinned mesh, such as `3DTrains_FPack`.
>
> The KUJU CAB file provides a useful example for locating include files. After experimentation, I propose a solution that addresses these needs:
>
> - A shared folder for `.inc` files, much like `\common.cab`.
> - Recognition that users may have routes and equipment from many countries, so grouping certain files by country may be useful.
> - Acceptance that payware vendors may sell the same mesh in many railroad skins while distributing each railroad in a unique folder.
> - A simple approach for cases with minimal commonality, where everything belongs in one folder.
>
> — Erick Cantu, 20 November 2017

Config File Deconstruction

A configuration file has two basic parts:

- Header: the first line, identifying the file type and version.
- Body: the actual configuration data.

Header

The first line is always the header. It begins with `SIMISA@@@@@@@@@@JINX0D0t` and is followed by a blank line. It identifies the file type and version; it is not part of the vehicle data. Different file types can use different header lines.

```
SIMISA@@@@@@@@@@JINX0D0t______
```

Body

The body contains the configuration data.

Below is a basic MSTS-compatible wagon configuration example:

```
Wagon ( CP_218041_LD
    Type ( Freight )
    WagonShape ( CP_218041.s )
    Name ( "HiCube 60 CP 218041 LD" )
    Size ( 3.108m 5.211m 20.531m )
    comment ( 38.46t empty, ~115t full )
    Mass ( 120t-us )
    WheelRadius ( 36in/2 )
    InertiaTensor (
        Box ( 3.1m 3.54m 19.931m )
    )
    Comment("==================== Coupling ====================")
    Coupling (
        Type ( Automatic )
        Spring (
            Stiffness ( 8e4N/m 4.5e6N/m )
            Damping ( 2e6N/m/s 2e6N/m/s )
            Break ( 250000lb 250000lb)
            r0 ( 12cm 12cm )
        )
        Velocity ( 0.2m/s )
```

```
    )
    Coupling (
        Type ( Automatic )
        Spring (
            Stiffness ( 8e4N/m 4.5e6N/m )
            Damping ( 2e6N/m/s 2e6N/m/s )
            Break ( 250000lb 250000lb)
            r0 ( 12cm 12cm )
        )
        Velocity ( -0.2m/s )
    )
    Commant("==================== Derail, Davis and Braking ====================")
    DerailRailHeight ( 5cm )
    DerailRailForce ( 2.5*115t )
    DerailBufferForce ( 1123kN )
    NumWheels ( 8 )
    ORTSBearingType ( Roller )
    ORTSDavis_A ( 1168.33 )
    ORTSDavis_B ( 18.9210 )
    ORTSDavis_C ( 1.928619 )
    Commant("==================== Brakes ====================")
    BrakeEquipmentType ( "Handbrake, Triple_valve, Auxilary_reservoir,
Emergency_brake_reservoir" )
    BrakeSystemType ( "Air_single_pipe" )
    comment ( Load/Empty valve? = Yes )
    MaxBrakeForce ( 67.665kN )
    MaxHandbrakeForce ( 112.776kN )
    NumberOfHandbrakeLeverSteps ( 100 )
    BrakePipeVolume ( 0.61ft^3 )
    EmergencyResCapacity ( 0.973ft^3 )
    ORTSBrakeShoeFriction ( 0.0 0.49 8.0 0.436 16.1 0.4 24.1 0.371 32.2 0.35 40.2
0.336 48.3 0.325 56.3 0.318 64.4 0.309 72.2 0.304 80.5 0.298 88.5 0.295 96.6 0.289
104.6 0.288)
    EmergencyBrakeResMaxPressure ( 90 )
    TripleValveRatio ( 2.5 )
    MaxReleaseRate ( 50 )
    MaxApplicationRate ( 50 )
    MaxAuxilaryChargingRate ( 20 )
    EmergencyResVolumeMultiplier ( 1.461 )
    EmergencyResChargingRate ( 20 )
    BrakeCylinderPressureForMaxBrakeBrakeForce ( 77 )

    Commant("==================== Sound ====================")
    Sound ( "..\\..\\TS_COMMON_SOUND\\freight.sms" )

)
```

This file contains the main wagon definition, coupling definitions, derail and braking values, Davis coefficients, and the sound reference. The wagon definition gives the type, shape, name, size, and mass. The coupling sections define coupling type, stiffness, damping, break force, and velocity. The braking section defines brake equipment, brake system, brake force, and related physics values.

The shape file and sound file are external references, and the example uses legacy MSTS-compatible paths.

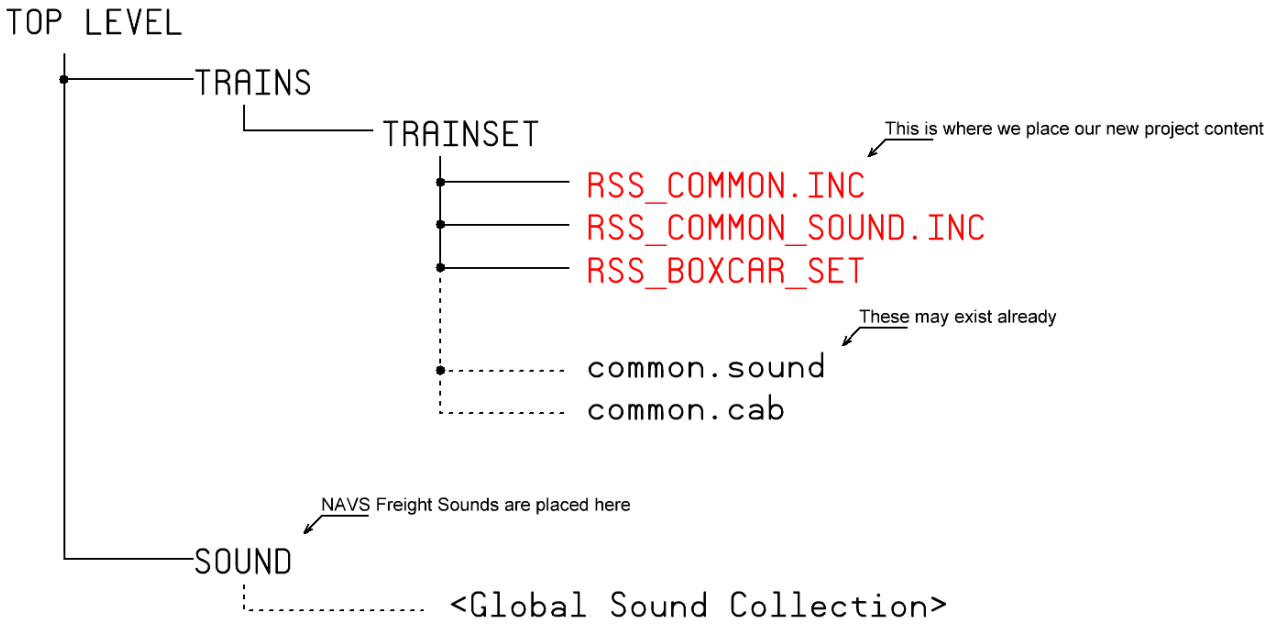
The coupler and braking sections are good candidates for INC files if they are shared by several models. This keeps the main WAG file smaller and easier to maintain.

Example ORTS Configuration Layout

`Include()` statements can be added to SIMIS files, but they are most useful in `.wag` and `.eng` files.

The following example outlines a standard boxcar WAG layout. Because my work focuses on RailSimStuff.com, I use the RSS prefix.

- NAVS Freight Sounds: `NAVSFR10.ZIP`
- Coupler details: `RSS_COMMON.INC`
- Wagon details: `RSS_WAGON.INC`
- The main WAG file contains the freight animation decal definitions.



If you use Visual Studio Code, use `File: Open Folder` to open the folder containing the `.wag` and `.eng` files. This makes it easier to navigate to related `.inc` files. A VS Code add-on for ENG/WAG syntax highlighting is available at https://github.com/matejpacha/ORTSFileSupportForVSC

3rd Party Configuration Tools

There are several tools that can help with configuration file creation or editing. They are not required, but they can make the process easier.

In addition to the VSCODE highlighter tool mentioned above, the following tools could be useful:

Name	Location	Description
WAG Include Editor	https://github.com/pwillard/WagInclude/releases	A Windows tool for editing WAG files and their INC files. It can edit existing WAG files, and manage INC files. It is a work in progress.
Shape File Manager 3	https://github.com/pwillard/SFM3/releases	A Windows tool for managing shape files and their associated textures. It can minimally edit existing ones, it can compress and uncompress files. It is a work in progress. It is a modern update of the original Shape File Manager (SFM) tool by Paul Gausden.
MSTS Engine File Computations	https://trainsim.com/forums/filelib/search-fileid?fid=66546	A Windows tool for computing engine file values.
MSTS ENGMOD	https://trainsim.com/forums/filelib/search-fileid?fid=60461	A Windows tool for editing engine and wagon files.

General Modelling Standards from Erick Cantu

These notes from Erick Cantu are important enough to preserve as a reference. They were extracted from Elvas Tower forum posts and are useful if you want to model with similar standards. They are guidelines, not hard rules.

Erick gives detailed modeling guidance here. Read and keep these points in mind when planning your own models.

Erick Says

- Match the right trucks to the right car. This includes wheel size and truck wheelbase, not just broad truck type. Erick built a library of freight-car trucks for this reason. His truck library is available at Elvas Tower, and importing those trucks into Blender could be very useful.
- Map standard-definition cars at about 1/2 inch per pixel. Align adjoining car surfaces within 1/1000 of a pixel so continuous lines do not break. Map car sides separately. Many cars can use the top or bottom half of a 2048 x 2048 sheet, which creates large continuous surfaces and lets one sheet support two paint schemes.
- A useful setup is four shape files: two mapped to the top and bottom of the first texture sheet, and two mapped to the top and bottom of the second. This supports four paint or weathering patterns per car folder, while numbers can be applied with decals so one shape can represent many cars.
- Texture mapping should be intuitive and paintable. Ask: Would I want to paint this? Could I repaint it for another railroad? Can unneeded parts be hidden with alpha? Could I paint it without constantly checking in Shape Viewer? Avoid awkward angled mappings that make straight lines difficult.
- Use consistent bitmap resolution and mesh resolution across a car and across related parts. Ideally, the top LOD should stay near 12,000 triangles or less, with careful vertex optimization, limited hard edges, and welded UV coordinates where possible.
- Aim for one texture sheet for the car body and one for the trucks. If alpha is needed, the whole car body can often use one BlendATexDiff material with the -Trans flag off. Add extra materials only for a clear reason, because each material adds a draw call. Trucks can use a separate texture because animated parts already create a separate draw call.
- Texture resolution and mesh resolution should match the intended viewing distance. Do not build highly detailed parts with high-resolution textures on a low-resolution model. Detail that greatly exceeds the rest of the model wastes texture space and vertices. There are exceptions, such as number boards with alpha edges, where higher texture resolution may be needed for acceptable results.
- Avoid splitting car sides into multiple sections unless necessary. It adds mesh divisions and UV coordinates. Very long cars may require it, but cars around 60 feet or less usually do not if planned well. Multiple car bodies can share one texture sheet, and this can reduce total train texture counts.
- Place the top node’s pivot about 2 inches below the rail so the wheels sit correctly. You may still need to adjust ride height.
- Consistent detail helps cars look good together. Air hose heights should match across cars. Erick uses brake hose ends at 14.5 inches above the rail and extends them to meet the next car’s hose,

held as in the real world with a chain or cable.

- Couplers passing through other couplers look bad. It is often best to make couplers part of the truck mesh or link them to the trucks. Exceptions include cars where truck centers and couplers are far apart, such as auto-racks or cushioned boxcars.
- Keep draw-call counts low and texture counts even lower. A car body should use one texture sheet and one material unless another material is truly needed. Multiple car bodies can share a single square sheet if needed.
- Avoid unnecessary 1-bit alpha and aliased edges. OR handles 32-bit textures correctly, so separate materials for 1-bit and 8-bit alpha often just add draw calls. Use one material with 8-bit alpha where possible. A 1-bit alpha channel is acceptable only for perfectly square transparent areas.
- Do not leave freight-car undersides empty. The old Microsoft Train Simulator habit of leaving the underside open can show sky through the car when viewed from below, especially on bridges. The underside does not need to be complex, but it should exist.
- Use the units that match your references. Aircraft references may be in decimal inches or meters. U.S. rolling stock references are often in inches. A scale calculator makes conversion manageable, and you may grow to like metric.
- Do not add imagined details. Model as accurately as practical.
- Within your target viewing distance, model visible details at a resolution consistent with the rest of the model. Brake rigging can be complete but simple if the car is normally viewed from far enough away that the whole car is visible.
- Taper wheel treads. It is obvious when they are not tapered.
- Weld UV coordinates wherever possible. Texture vertices matter as much as draw calls.
- Plan hard edges deliberately. If an edge is always in a dark area, smoothing it may be better.
- Cars should look good from all likely viewing angles, including below.
- Blender can export a UV image. Import this image into your paint program as a reference layer so you can see where texture shapes are located.
- Smaller round parts need fewer sides. Wheels may use 24 sides at the rims, but 12 or 10 sides inside the dish. Edges matter, but they can hide a lot.
- Coupler heights may vary. Air hose heights should not. Consistent hose height prevents mismatches between cars.
- If a feature cannot be done well, leave it out. For example, track sounds for switches and crossings can break immersion if they cannot account for truck spacing or wheel count.

The main point is to make deliberate choices. Small inefficiencies add up across a long train, especially with AI traffic. Erick’s move from individual car textures to shared paint and weathering variations with decal numbers reduced texture counts dramatically. For example, 100 BN hoppers can use 13 texture sheets with decals, instead of about 51 sheets without them.

Additional guidance from Blender Brothers

From Pete Willard

Josh from Blender Brothers was one of my mentors when I was learning Blender. I subscribed to their YouTube channel and still receive updates about their ideas and tips.

Josh and Ryuu of Blender Brothers say:

"What's the secret to nailing the detailing stage?"

It comes down to understanding where to add detail, how much to add, and what kind of detail fits. That sounds simple, but it takes practice.

Blender Brother's Gold Rule

Think in terms of visual anchors. Visual anchors are like stars in a constellation. Your eye moves from one anchor to another, while the space between them becomes negative space. These tips help apply that idea:

- **Embrace the 70/30 Rule** Keep about 70% of the design clean and cluster details in the remaining 30%. This creates visual anchors.
- **Keep Details to scale** Oversized details can make a model look cartoonish. Use reference scale when needed.
- **Do your homework** Use reference images. For example, study real or fictional examples before designing a sci-fi room.
- **Less is more** Do not over-detail one area all at once. Move from area to area and add detail gradually.
- **Create your own library** Build a collection of decals, trim sheets, kitbash parts, and other reusable assets. This speeds up work and keeps your style consistent.

The Blender Brothers like making their own decals and have recorded a decal workflow using the Decal Machine Blender add-on. Their other videos are also worth watching.

General Texture Mapping Guidelines

- Microsoft recommended square texture sizes for Microsoft Train Simulator, such as 512 x 512, 256 x 256, and 128 x 128. Open Rails does not have the same limitation, so a 2048 x 1024 texture can be reasonable.
- Use a paint program that supports layers and can save layered documents. PSD is one layered format.

- PNG files support alpha channels, but they do not preserve layers.
- Consider using PNG files while modeling and DDS files for final Open Rails models.
- Consider using modeled decals for reporting marks and car numbers instead of baking them into the main texture. Decals are easier to change and can be reused on multiple models.
- Export and check the model as soon as the UV map exists. It is better to find UV problems early.

Some additional tools to consider

- ORZIP:: A free tool that replaces Microsoft Train Simulator's ffeditc_unicode.exe file for compression and decompression of MSTS files. It is available at https://github.com/pwillard/orzip/releases NOTE: ORSP is a graphical front-end for ORZIP that is also available at https://github.com/pwillard/ORSP/releases
- TextureWorkshop:: A free tool for converting ACE files into multiple other formats. It is available at https://github.com/pwillard/TextureWorkshop/releases
- REPMARK:: Generate transparent PNG decal sheets for railroad reporting marks and numbers. It is available at https://github.com/pwillard/repmark/releases
- Shape Conversion:: A free tool for converting MSTS shape files into Blender-readable formats. It is available at https://github.com/pwillard/ShapeConversion/releases

[1] https://www.coalstonewcastle.com.au/physics/format/

Making a 3D Model

This section will provide an overview of how to create a 3D model in Open Rails Train Simulator using Blender. We will cover some basic concepts and tips for creating a 3D model in Blender.

Some Basic Things About 3D Models

Scale

Open Rails Train Simulator as well as Microsoft Train Simulator each use a scale of 1:1, that is, 1 unit in the 3D drawing area is 1 meter in the simulator and this means all things are in real-world measurements.

3D Space

Comparing **Blender's view of the world** to Open Rails, `-X` value is left, `+Z` value is up and a `-Y` value is the front side of the object. The "ground" level is where `Z`=0

Material

In Blender, a material is a set of properties that defines how a surface appears when it is rendered. Materials can include information about the surface's color, texture, reflectivity, transparency, and other visual characteristics. It's not just the "texture" definition.

When creating rolling stock items for ORTS, pressing `Keypad 1` for front view will actually have you looking at the REAR end of the car or engine. You would press `CTRL` + `Keypad 1` to get a front view of your engine or wagon while drawing it. You have the potential to create backwards rolling stock if you get this wrong.

The **MAIN** object of your modeling collection should always be positioned at world origin, which is `X=0`,`Y=0`,`Z=0`, based on the orientation described above.

There are some general rules that all content creators should follow, regardless of the tools used. Take the next section to heart.

While you *can* have Blender work in Imperial units (feet and inches), it may be far simpler to always work in the metric system since you won't need to change the "UNIT" settings in Blender. You just might need to have a unit converter handy, and the easiest way is to just ask Google.

Some Basic Things About Modeling

Some additional guidelines from Erick Cantu of NAVS

1. In Open Rails, it is important to try to keep the polygon count of your models low, even though it is not as critical as in some other applications. While modern PCs and simulator software have improved in their ability to handle complex models and textures, it is still important to use complexity wisely. In Blender, you can view the polygon count and other details of your model by going to Object EDIT MODE and looking at the lower right corner of the screen. When creating your models, try to avoid including faces and edges that will rarely or never be visible in the final model.
2. Keep the texture files (that will eventually be included with your final model) to a minimum and use as much of the allotted texture as you can. Each texture creates a fresh API call (state change) to a new group versus being able to batch calls together to a single texture.
3. The most important tip. Don't get hung up drawing things that nobody will see. Avoid drawing details that will remain in shadows or small details like nuts and bolts if the camera will never see it. Inversely, now that we have the ability to have more complex models with Open Rails, we **can** add more modeled details where we historically were forced to rely heavily on alpha-channels in our textures.

These comments from Erick should be considered guidelines rather than strict rules. Many creators produce acceptable results while choosing not to follow them. However, the recommendations are intended to help maintain good performance in Open Rails, particularly when running long and complex trains on older hardware or lower-end graphics cards.

Items 1 and 2 should already be familiar concepts dating back to the early Microsoft Train Simulator era. Item 3 is especially important, as the temptation to create highly detailed objects should be balanced against what will actually be visible during normal Open Rails operation.

Additional details can be modeled directly onto the main mesh when the goal is to generate a master texture. This technique allows extra visual detail to be added without increasing the polygon count of the final model.

This process is commonly known as **hi-poly to low-poly baking**. A highly detailed version of the model is created, but it is not used as the final in-game object. Instead, the additional geometry is baked into textures that can represent features such as nuts, bolts, rivets, doors, and other surface details.

Polygon Count and Vertex Density

If you read item #3 in the previous section, you might think you can go crazy with details. However, there is always a balance to be achieved not just between graphical quality and performance, but also in computational performance. Every vertex that has to be mapped to a texture takes computing power. Thus, it's good to save polygons whenever possible and use texture maps instead for details like cutouts, structural members, and bolts. If you really feel like you want to model small details, keep in mind that every detail added might prevent the more generic use of the model for re-skinning later on. A flat sided boxcar is more flexible than one with specifically modeled door layout, latches and modeled roof details.

For large panels, though, a minimal amount of modeling is necessary. A flat plate like a wall or door can be made with only a few polygons. Like every rule, you can break it when you really want to show a particular detail, but also keep in mind that you should make good use of "Level of Detail" ranges (LOD) to reduce computational work when an object is farther away.

Preparation of Work

- Of course, you need to figure out what you want to model
- Find some design references (Blueprints, Scale Drawings). Referring to back issues of model railroading magazines can be helpful. There are also some railroad cyclopedia books for sale that were published by Model Railroader Magazine as well as issues of Railroad Model Craftsman.
- Create a collection of reference images, if doing modern content, grab your camera and snap some of your own references. There are a few railroad picture sites that are highly useful as well. (https://www.railpictures.net/, http://www.rr-fallenflags.org/, https://www.railcarphotos.com/)
- Determine which images you will use for background references in Blender. You should have available at least side and front images. In some situations, you can use the same image for both if the views are present.

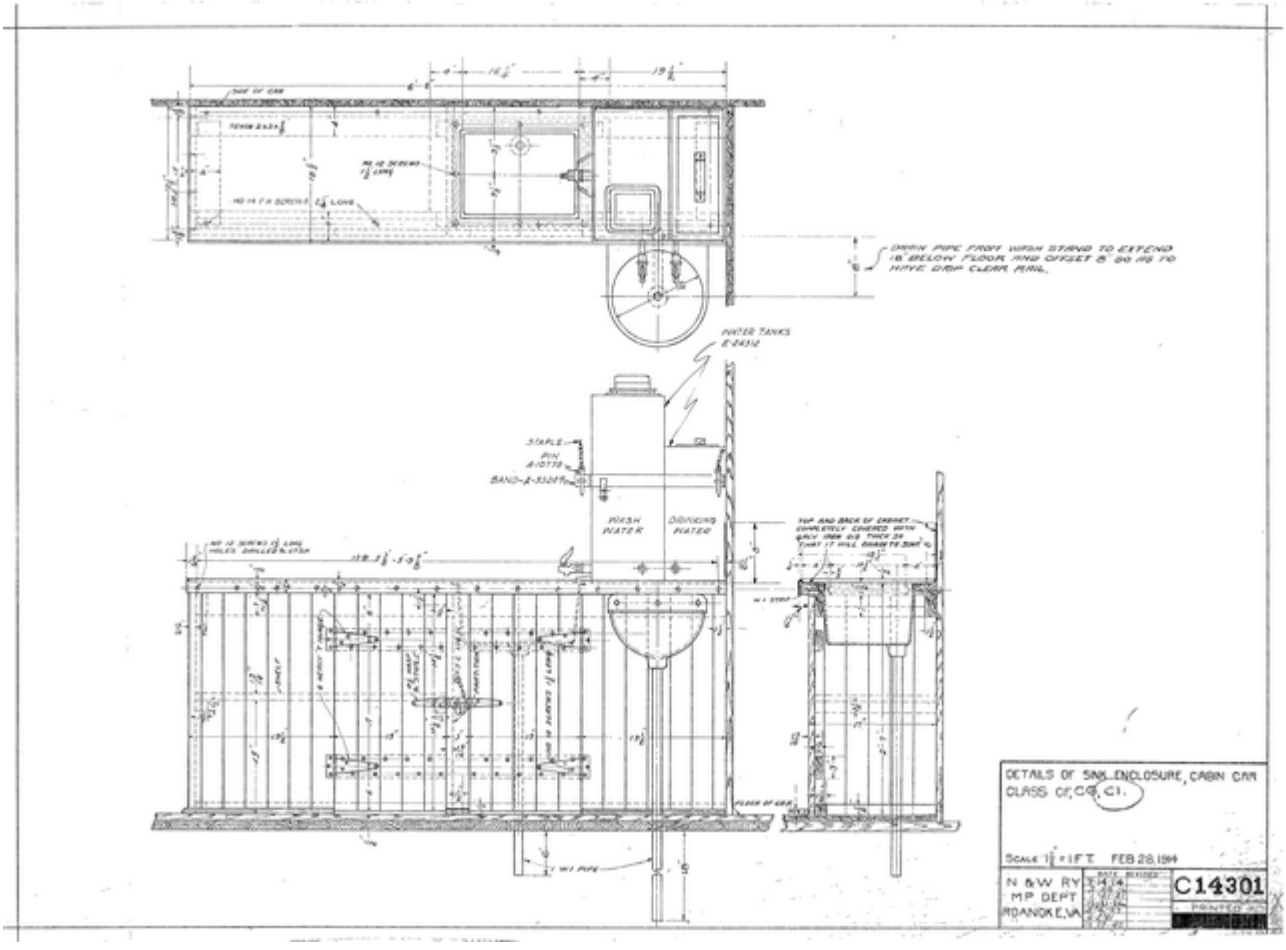
Background Images

It is important to note that many scanned images of drawings and plans that can be found online may contain errors, as they are often reduced in size to fit on scanners and screens. Older plans, which are often very large (nearly one meter wide), are particularly prone to this issue. Plans created using computer-aided design systems tend to be more accurate and smaller in size, but

these are less common and may only be found in hobby magazine articles. Some plans may also be stored as vectors, which allows them to be scaled without loss of quality. Keep these limitations in mind when using background reference images.

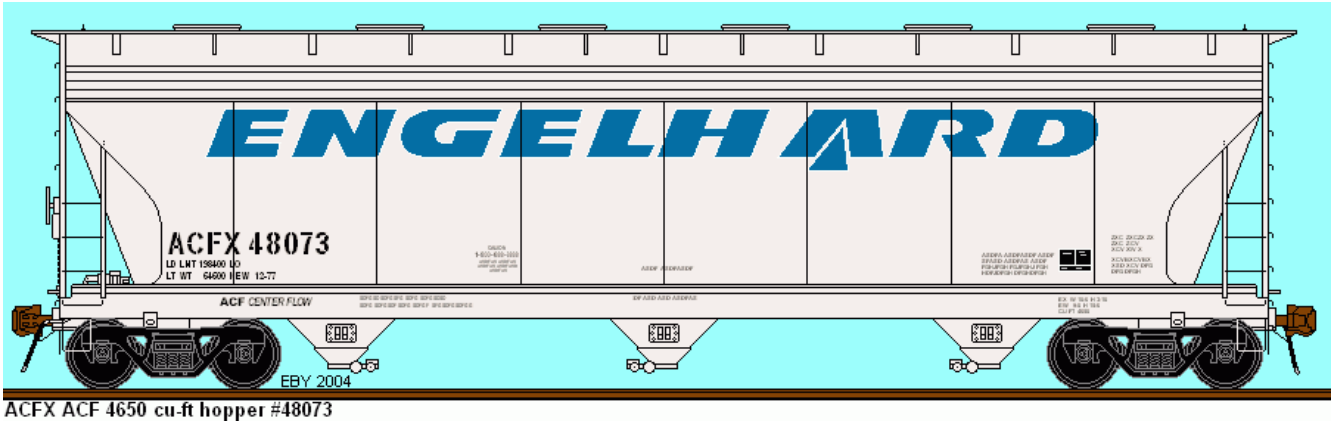
Original Plans

There are web sites online that will sell you scans of original plans. These are often related Railroad Historical Society websites. Unfortunately, there are large images and often quite costly. Example: https://www.nwhs.org/archivesdb/



General Drawings

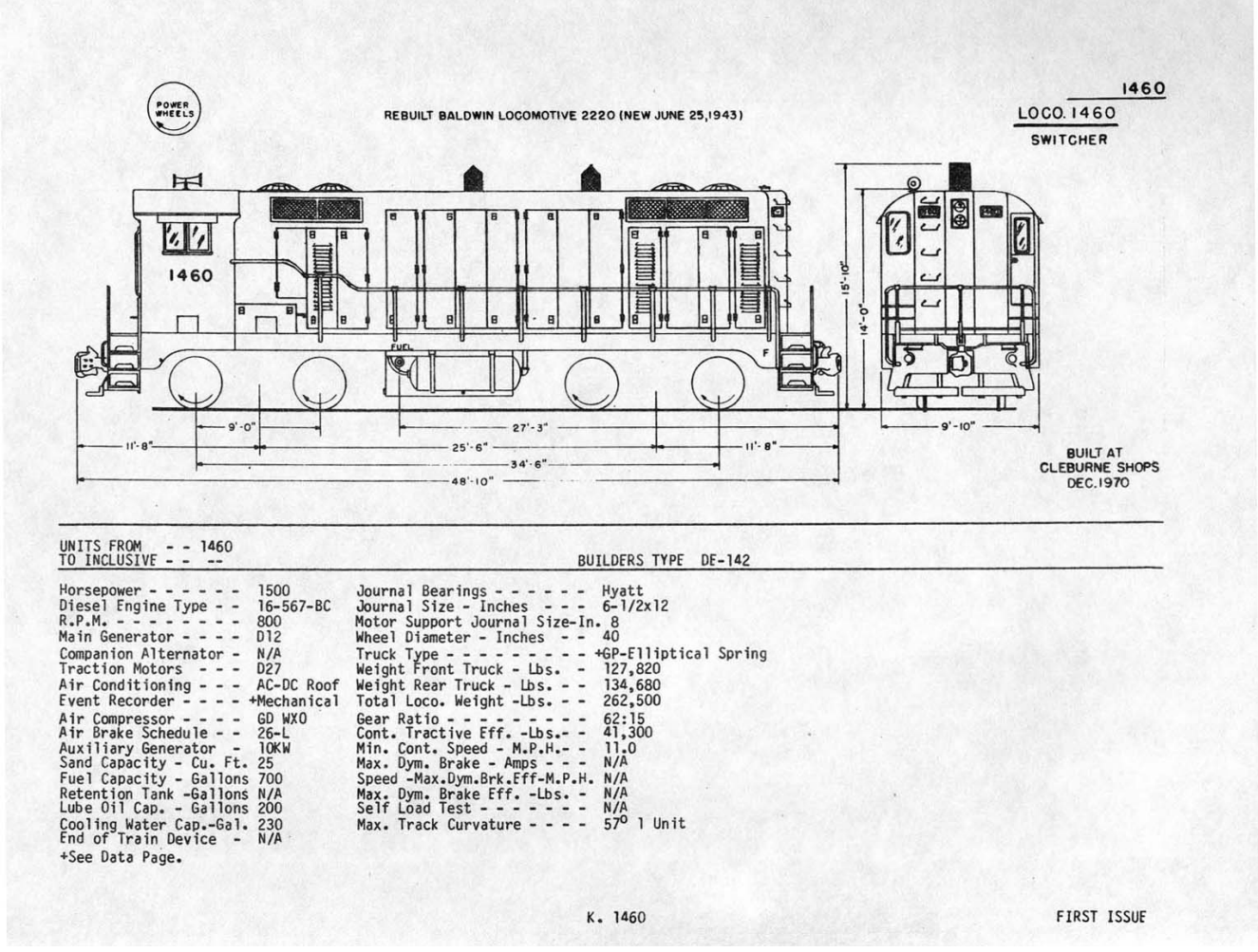
These are often found in back issues of Model Railroader and Railroad Model Craftsman magazines as well as sites like https://www.trainiax.net. These usually contain a good about of Steam Engine, Diesels and Rolling Stock but might lack all the details such as piping and such. These drawings are made based on originals by the staff of the magazines or staff that created the Cyclopedia collections. These have the ability to be a few percentage points off-scale even when they include a scaler reference.



ACFX ACF 4650 cu-ft hopper #48073

Classification and Painting and Lettering references

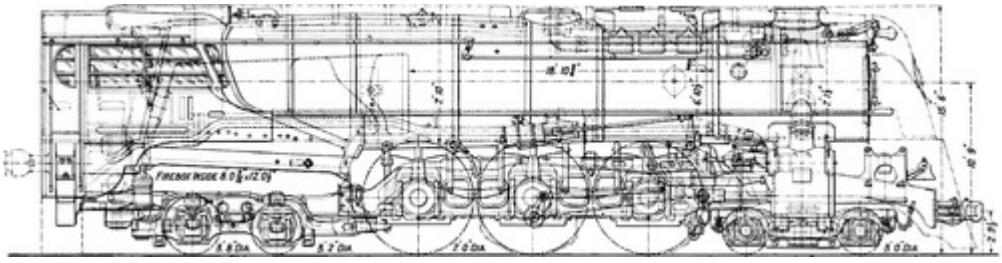
These seem to be rather plentiful and contain lots of useful dimensional data. They are often from discarded rail road documents and might have notes scribbled in them. The drawings are often hand drawn and might be a bit messy. They are not that useful for small details but can get you started on a model if you also have good pictures to refer to.



Blueprint Websites

Sites like http://www.the-blueprints.com offer a variety of drawings to choose from. While this might seem like a good resource, I would use it with caution. The content is all user generated so

there are varied degrees of clarity and accuracy.



Using background images is optional, but many modelers use blueprint backgrounds as modeling aids.

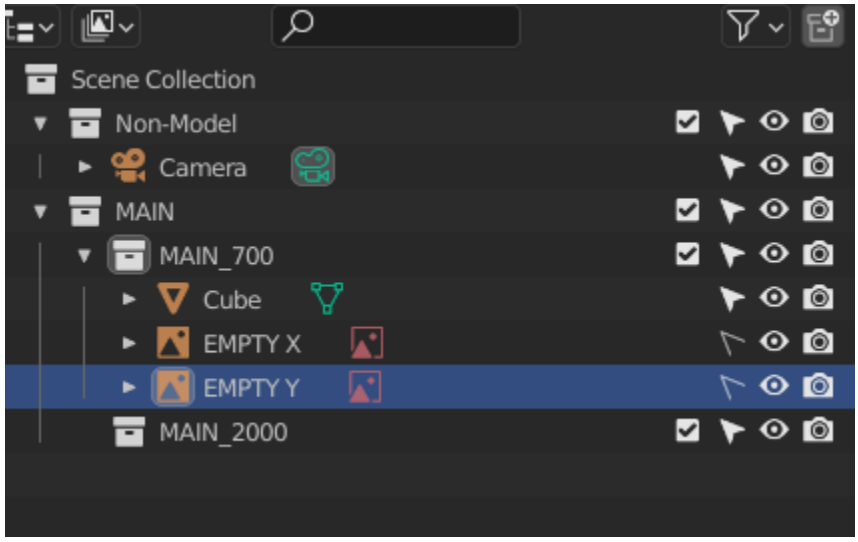
When inserting background reference drawings into Blender, it is not critical that you match up the image size to equal world space dimensions, but it can save a lot of re-scaling steps later. One of the main issues with using backgrounds in Blender is that some methods you might be accustomed to with other modeling tools don't work in Blender, such as applying a background image to a cube and using that as your reference, since in Blender, you only see the texture in a texture viewing mode. It will disappear in Xray/Wireframe and even solid shading modes.

When you insert a background using Blender's **Add › Image › Background**, the image WILL show up while you are in wire-frame modes... but note that by default the image will ONLY be visible from the axis it faces. Be careful when you install a background image as you need to be in the Axis View you want, since the added image will orient itself to your current view. If you want to change this, you can enable the `Perspective View` check-box by clicking on the `EMPTY` and adjusting the setting in its properties window.

Background images are added to our scene as `EMPTIES`, which means that they are not going to be part of an exported model. To avoid accidentally moving or changing the background image, you can use the filter options in the Outliner, first enabling the "mouse cursor" icon and then setting the restriction toggles to disable the "mouse cursor" icon for the empties you added to the list, which will prevent you from accidentally selecting the background images.

The Outliner filter options are pretty handy as you can also SHOW or HIDE the background image empties as needed by clicking the "eye" Icon.

A nice feature of the Image Background is the ability to adjust the ALPHA setting, controlling the with the "transparency" slider to get a see-through effect. First, click the Opacity check-box to enable the feature and then adjust the opacity slider to your liking.



If you want to save the background images **INTO** your `.Blend` file, you can use the **File › External Data › Automatically Pack Resources**, which is OFF by default.

When using background images and modeling a freight car, for example, you would first figure out the distance between truck centers in meters and match the background drawing scale using the grid in the 3D view. Adjust the background image scale using `S` and position it using `G`.

When scaling with the mouse, the cursor position changes how sensitive the movement feels. If the cursor is close to the object when you press `S`, small mouse movements can change the scale quickly. If the cursor is farther away, the same mouse movement changes the scale more slowly, making it easier to fine tune.

When working with images in Blender, you can install the add-on **Insert Images as Planes**. After enabling it, a new option will appear in the **Add** menu.

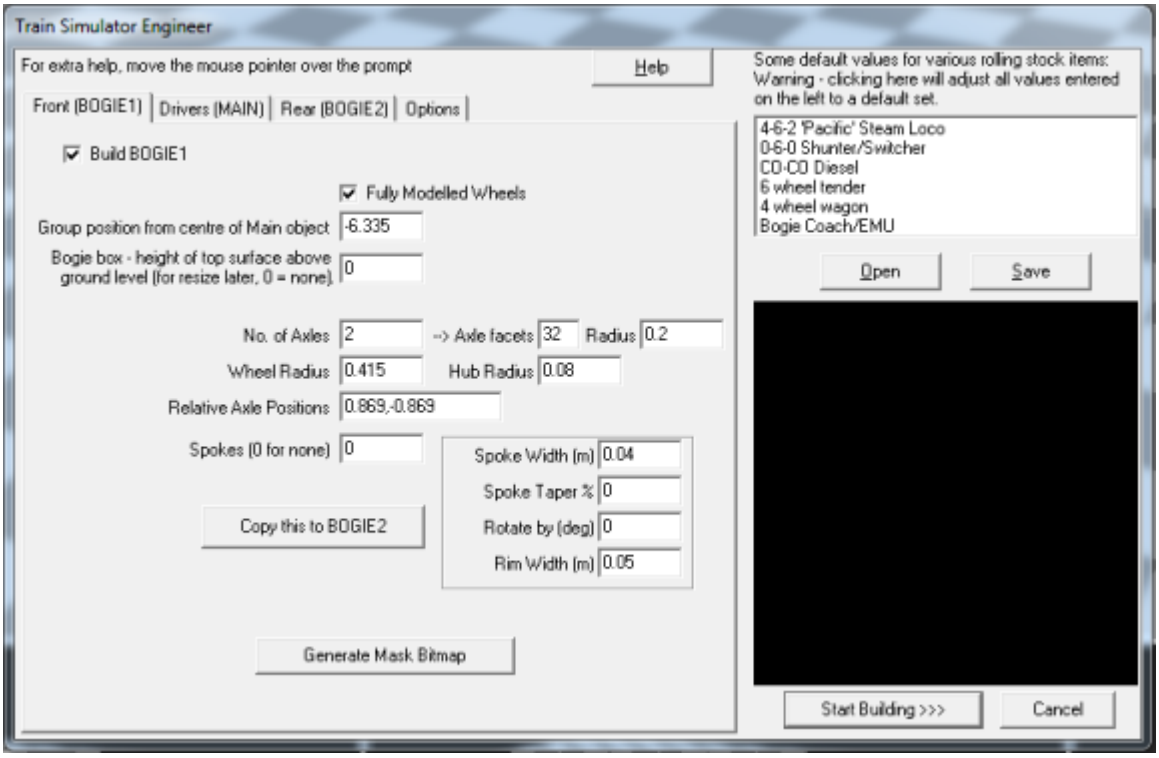
This tool creates a plane and automatically assigns the image as the material texture. It also supports features such as alpha masking and responds to scene lighting.

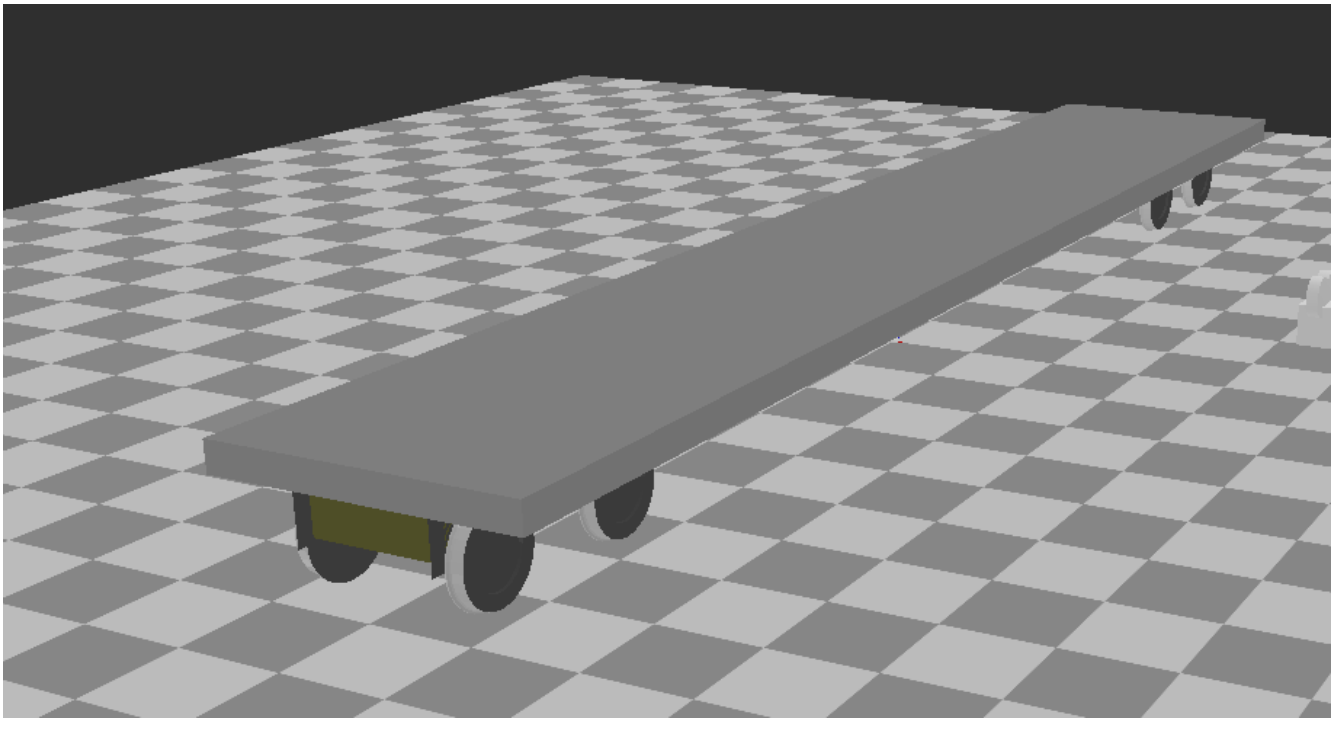
However, the image plane is a real mesh object with a material. It is useful when you want an image in the scene, but a background image empty is usually better for blueprint-style modeling references.

Some things I will miss about using 3D Canvas

We will digress for a moment to share one of the reasons why using 3D CANVAS was considered helpful in the initial stages of making a model.

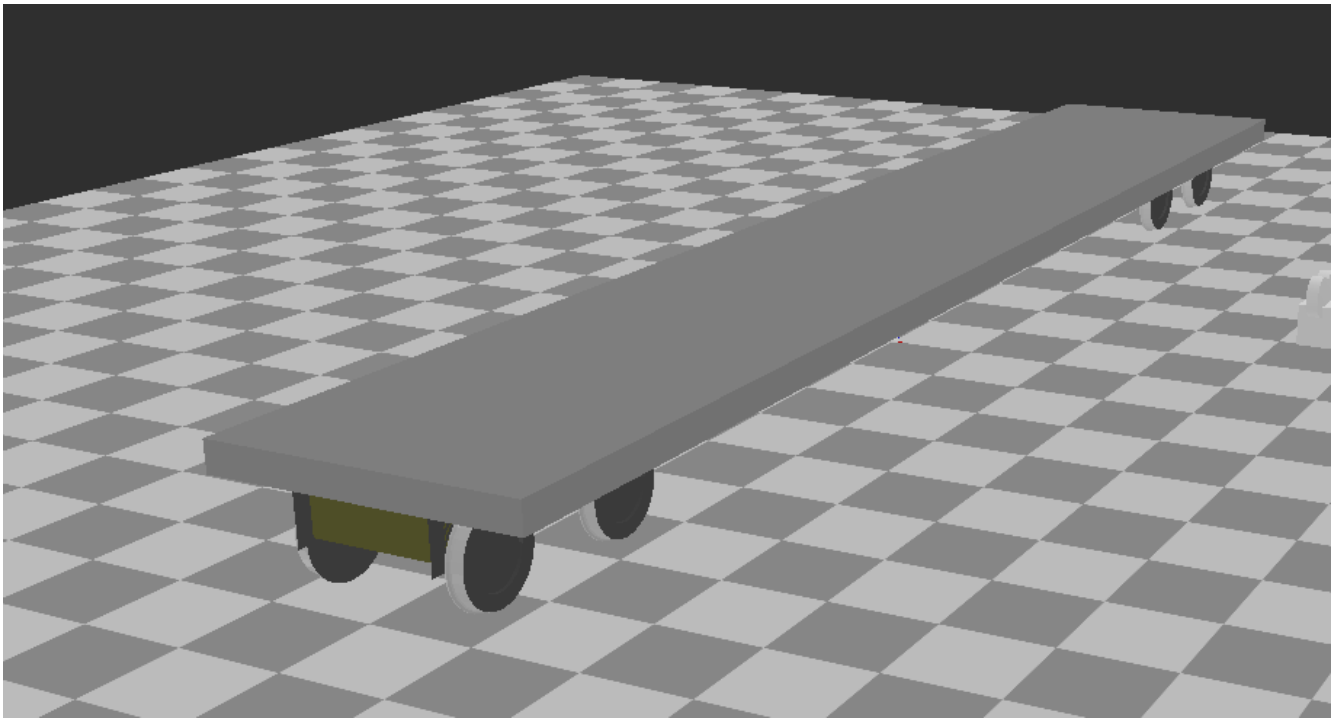
The 3D Canvas/Crafter platform included a train-content ENGINEER add-on — Trainworks → Train Simulator Engineer — that functioned as a quick-start utility for assembling train models. It automatically placed wheels, axles, truck assemblies, and a base body element, all of which could be customized. Blender does not include an equivalent tool by default. In practice, though, the time savings are modest if you already have prepared truck sets and couplers; positioning those parts correctly is straightforward.





Since this is primarily a set of preset shapes with customizable values and

locations, Blender's scripting language could be used to recreate a similar workflow in Python. A tool like this would be a useful companion to the maintained Blender MSTS/Open Rails `.S` file exporter.



Using this plug-in with 3D Canvas gave you a number of starting parts before modeling really began. In Blender, however, you normally create or import these parts yourself.

3D Canvas had a "render 2-side faces" option. Blender's current workflow is different: a plane may appear visible from both sides in Blender, but export formats and simulators may still treat it as a single-sided face depending on material and culling settings.

For reliable Open Rails content, model visible thickness instead of relying on a two-sided display option:

Create a new plane with **Add › Mesh › Plane**, then add a Solidify modifier to give the plane real thickness:

1) Go to the "Modifiers" tab in the Properties panel.

2) Click the "Add Modifier" button and choose "Solidify".

3) Adjust the thickness in the Solidify settings.

If you want to make the thickness permanent, click **Apply** on the Solidify modifier.

Solidify adds geometry. That is usually better for export than relying on viewport-only double-sided display, but it also adds

faces you must UV-map and manage.

Where to start?

A good initial project is to create a simple scenery object by modifying the default cube.

Do we need a quick 3D Modeling primer?

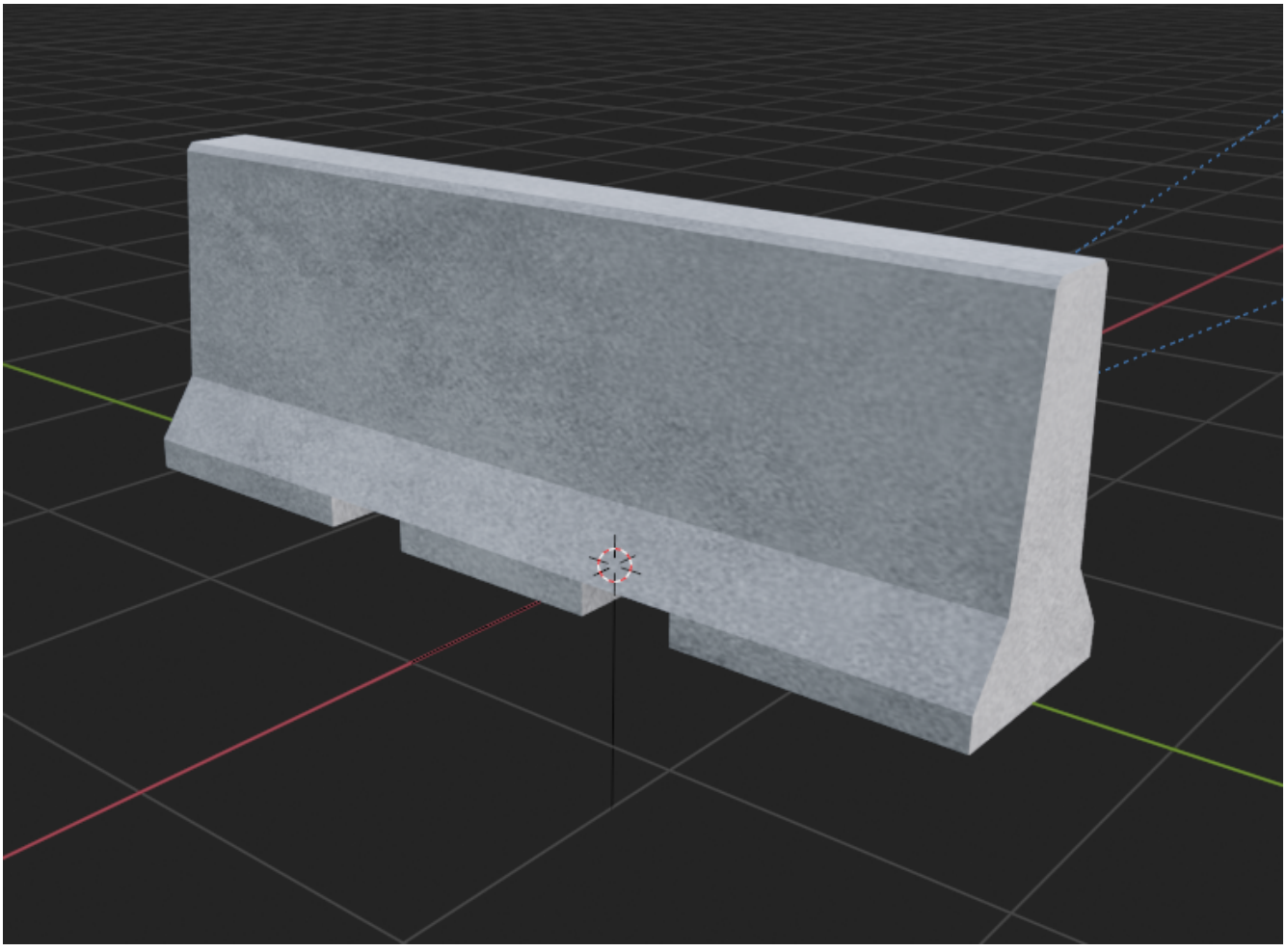
A fundamental concept in Blender is the close interaction between mouse and keyboard. The right hand is typically used for mouse navigation, while the left hand handles keyboard shortcuts. Although Blender has introduced optional pie menus and on-screen widgets in recent releases, it remains largely a key-driven program. Many operations can still be executed more efficiently through shortcut keys combined with precise mouse actions.

Like most 3D software, Blender is built around a set of core elements known as primitives. These are the foundational building blocks for creating models. Among them, the most commonly used for hard-surface modeling are the cube, followed by the cylinder and plane. Blender refers to these items collectively as mesh objects.

By adjusting the scale, rotation, and position of these primitives—or by manipulating their faces, vertices, and edges—it becomes possible to construct and refine virtually any shape. The process can feel overwhelming at first, but proficiency comes through hands-on practice. Observation and tutorials are helpful, yet true understanding develops only through active experimentation within Blender itself.

It is important to watch and read tutorials to become familiar with Blender's features and workflow. However, practical experience is essential. Experimenting and creating a few simple models is the best way to build confidence and skill.

For example, an excellent beginner project might be something straightforward, such as a 12-foot Jersey-style concrete barrier. It is simple, structured, and easy to refine.



When starting a new project, approach it with the understanding that some or all of the work may need to be redone. As new and more efficient techniques are discovered, revising or discarding earlier attempts is a natural part of the learning and creative process.

Although Blender has a steep learning curve, even a beginner can produce a simple model like the one shown above by applying a few basic commands.

The 3D Viewport

The 3D Viewport is the main workspace in Blender. It is where you will see your models, textures, and other objects. It is divided into two sections: the 3D View and the Properties panel.

Adding a new MESH object to the scene can be done by clicking on the "Add" menu and selecting "Mesh" and then "Cube". This will create a new cube object with default dimensions of 1x1x1. You can adjust the dimensions by clicking on the cube and dragging the edges or corners.

KEY	ACTION
G	Grab - To Move things. Can be followed by an Axis option and distance option
R	Rotate - Can be followed by Axis and Angle options
S	Scale - Can be included with other keys to limit scaling to desired axis

Pressing GG as in pressing the G key twice, will allow you to SLIDE vertices along an edge.

When scaling with the mouse, the closer your mouse is to the object, the more impact the mouse has on scale movement. If you want more control, begin the S operation with the mouse cursor further way from the selected item's origin.

These keys can manipulate basic objects. By using these keys and following them with additional commands like X, Y, or Z you can constrain actions to the specified axis. By pressing SHIFT prior to an axis key, you remove it from the list.

Example1

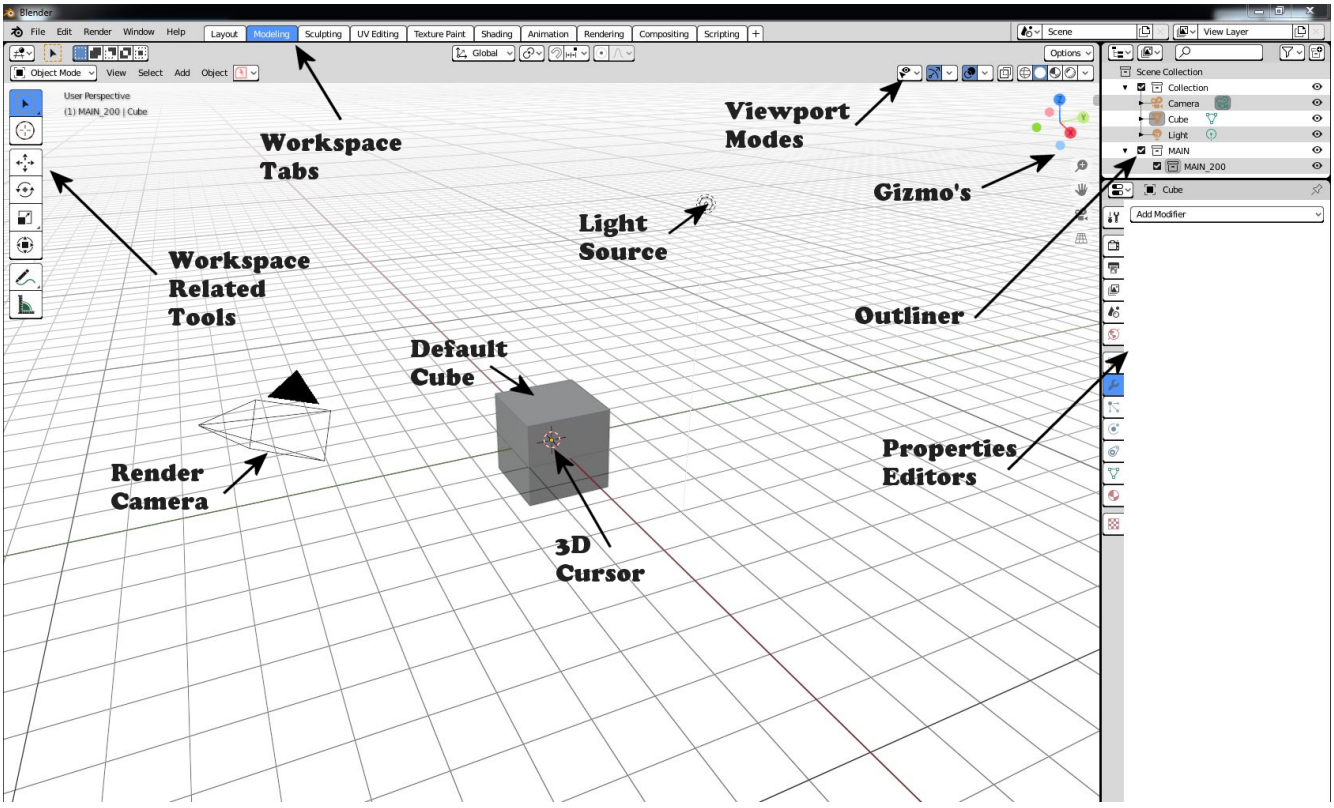
By pressing the sequence: G SHIFT X you will be able to move the selected item in the Y and Z axis, but not X.

The Modeling Interface

This is the initial default screen layout you will see when you start up Blender.

Don't get too attached to the default layout. It seems that the Blender developers like to change things up from time to time. Some of the examples shown here may not match the latest version of Blender.

For added clarity on printed copies of this document, I've switched to a high contrast theme in Blender Preferences. (Preferences-→Themes-→Presets[Pulldown for options])

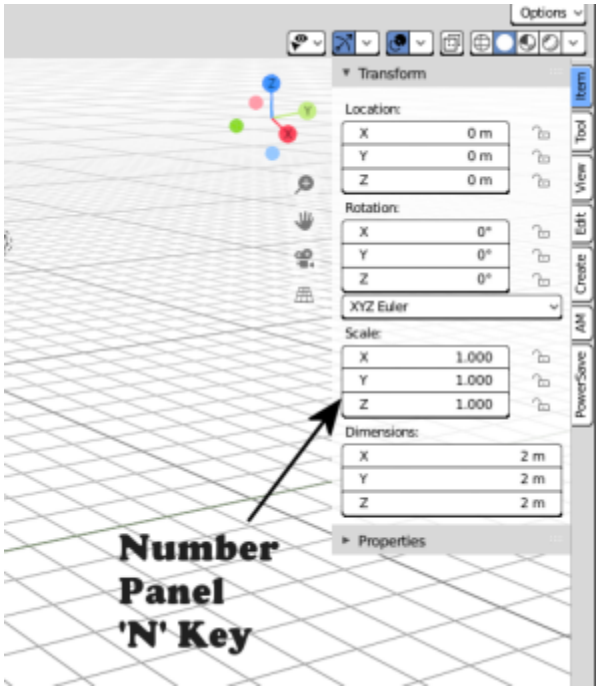


The 3D cursor is where any new object will be placed. By default it is at world origin `0,0,0` but it can easily be moved to new locations. The position of the 3D Cursor can also be manipulated using the cursor widget on the left side panel. It is available in both OBJECT and EDIT MODE. You can also place the 3d cursor position using the `Shift` + `S` `SNAP` Pie Menu.

Shortcut Keys

MENUS

To hide or reveal the Number Panel on the right side use `N` key (Not shown above)
To hide or reveal the Tools Panel on the left side use `T` key
To see your "Quick Favorites" menu, use the `Q` key
Use `F3` for access to the search feature
Use `CTRL` + `TAB` combination for access to the `MODE` pie menu
Use `~` for access to the `VIEWPORT` pie menu
Use `.` for access to the `PIVOT POINT` pie menu
Use `,` for access to the `AXIS ORIENTATION` pie menu
Use `SHIFT` + `S` to access the `SNAPPING` pie menu
Use `Z` to access the `VIEWPORT SHADING OPTIONS` pie menu



The above image shows what it looks like when rolled out. It is also where some installed add-ons will show up.

VIEWS

Use `HOME` to show all objects
Use `H` to hide a selected object
Use `ALT` + `H` to un-hide all objects
Use `SHIFT` + `H` to show all objects
Use `ALT` + `Z` to toggle X-Ray mode

The Number Pad Keys are also used to select various views.

Use of Blender's Pie Menus is totally optional. Some people find them to be a time saving device, others disagree.

View Controls

There is a check-box option in **Preferences › Input › Emulate 3-Button Mouse** for people who do not have a Middle Mouse Button. To create a `MMB` press while this mode is enabled, you hold `ALT` while pressing the `LMB`. There is also a checkbox to EMULATE the number pad for systems that lack a keypad. This option will enable the number pad key mapping on the TOP ROW number keys instead. (This will affect some options, like switching edit modes for vertex, line and face)

Orbiting

Select the default cube and press and hold your middle mouse button, `MMB`. Moving the mouse right and left, you will orbit your view around the selected object.

Panning

Select the default cube and press and hold your `MMB` and then press the `SHIFT` key. Moving left and right will PAN left and right on the screen view.

Snapping

Select the default cube and press and hold `MMB`, then press the `ALT` key and by moving left, right, up, down you will snap the screen to various orthographic views.

Zoom

Select the default cube and press and hold `MMB` and then press the `CTRL` key. Moving up, down will zoom the views in and out. You can also use the mouse `SCROLL WHEEL`

Camera View

Pressing the `INS` key on your keypad will toggle the Camera View. In this view, you will see what the camera sees of your screen which is also what your render will output.

Zoom to Selected

Pressing the `DEL` key on your keypad will zoom in and give preference to the selected object.

Front/Rear View

`Keypad 1` / `CTRL` + `Keypad 1`

Side View Left/Right

`Keypad 3` / `CTRL` + `Keypad 3`

Top/Bottom View

`Keypad 7` / `Ctrl` + `Keypad 7`

Rotate Z axis

`Keypad 4` & `Keypad 6`

Rotate X axis

`Keypad 2` & `Keypad 8`

Toggle Perspective and Orthographic view modes

`Keypad 5`

Most of the time, you will be using the default `BOX SELECT` mode of the Arrow (Select) tool. Use the `B` to switch back to `BOX SELECT` if you end up changing it from default. Other modes include `CIRCLE SELECT` `C` (Note, use `RMB` to exit `CIRCLE SELECT` mode), and `LASSO SELECT` `CTRL` + `RMB`. The `W` key will cycle between modes sequentially. You can also **INVERT** selections by using `CTRL` + `I` and add to an existing selection by holding `SHIFT` and selecting with `LMB`.

Figure 1. Box Select icon

You can move around using the mouse with the screen gizmos (X,Y,Z)in the upper right. The multi-color Axis tool will allow you to drag to a new orientation, the Magnifier is for Zoom, the Hand is for panning , the Camera icon will toggle the Camera view and the Plane icon will toggle between Perspective and Orthographic. These on screen items are helpful when using a laptop that does not have a keypad.

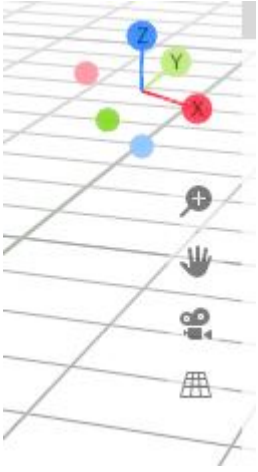


Figure 2. On-screen navigation gizmo

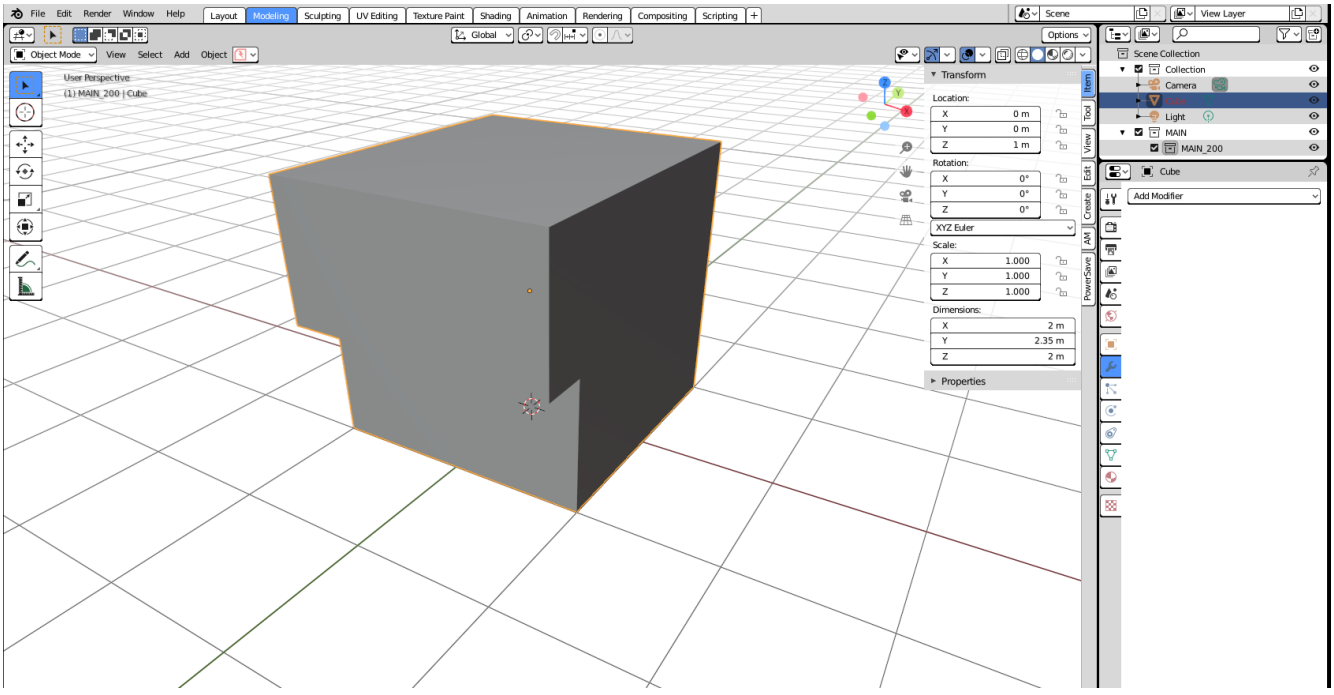
Modeling Modes

In Blender, there are 2 primary 3D model manipulation modes. These are `OBJECT MODE` and `EDIT MODE`.

Use the `TAB` key to switch modes.

OBJECT MODE

With `OBJECT MODE`, which is the default mode in Blender, actions are available for all object types since this mode is dedicated to Object data-block editing (e.g. position, rotation, size) as well as Modifiers. Edges, Faces and Vertices cannot be modified in this mode.



In this mode, you can select individual objects that make up your design so they can be further manipulated in `EDIT MODE`.

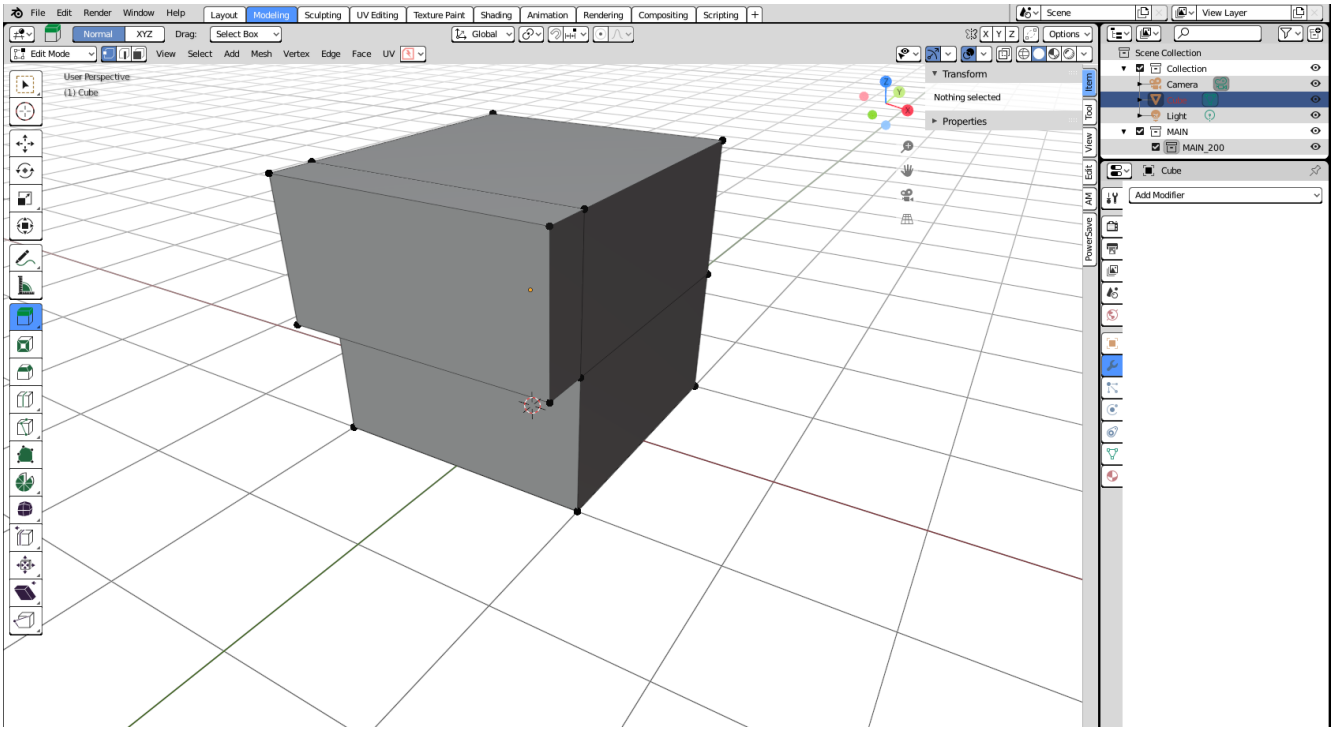
In object mode, the following shortcut keys are useful:

Use `SHIFT` + `A` to Add a new object
Use `SHIFT` + `TAB` to toggle `SNAPPING` modes
Use `CTRL` + `A` to Apply transformations
Use `CTRL` + `J` to Join objects together
Use `SHIFT` + `C` to reset cursor to center
Use `SHIFT` + `D` to Duplicate the selected object

EDIT MODE

The **selected** item in `OBJECT MODE` becomes the focused object when moving to `EDIT MODE`. `EDIT MODE` is a focused mode and you will not accidentally select other parts of the model in this mode. This mode available for all object types that can be rendered, as it is dedicated to manipulating their "shape". The `EDIT MODE` allows adjustment of Vertices, Edges and Faces for mesh object types as well as the control points for curves, surfaces and points.

In `EDIT MODE`, the object selected will show Vertices, Edges and Faces. Selected edges, vertices or faces will adopt a highlight color when selected. Also notice the larger tool set on the left compared to `OBJECT MODE`.



In Edit mode, the following shortcut keys are useful:

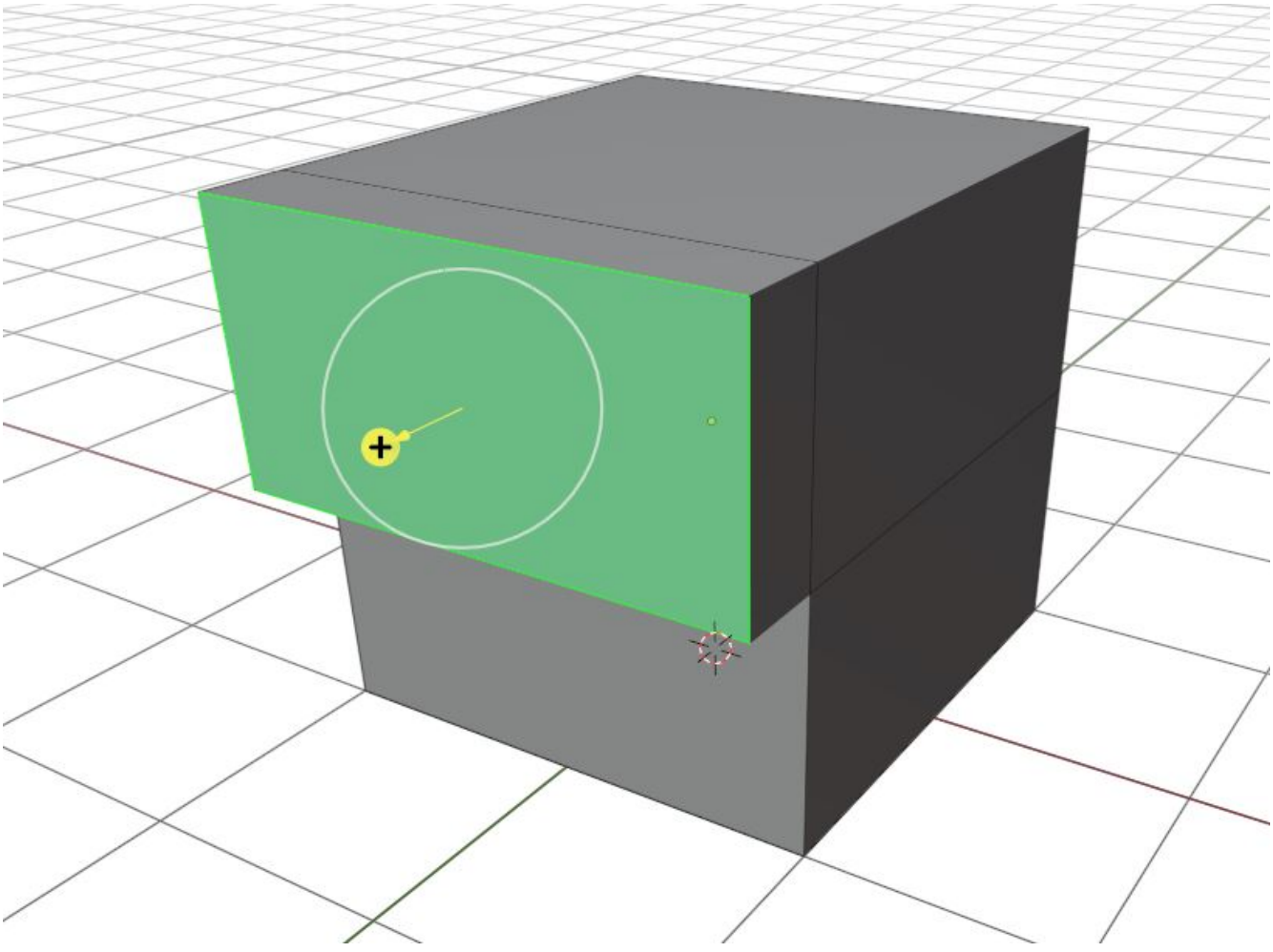
Use 1 Vertex Mode
Use 2 Edge Mode
Use 3 Face Mode
Use P Create Separate objects from the selection in various ways
Use F Fill Face

There are three sub-modes in `EDIT MODE`; 1 Vertex Edit, 2 Edge Edit and 3 Face Edit.

The four major edit-mode tools you are likely to use the most are listed below.

There are now thousands of YouTube videos about modeling in Blender if any of these concepts mentioned here are not clear.

EXTRUDE



E Key

The official definition of the extrusion operation is: *The* `extrusion` *operation duplicates vertices, while keeping the new geometry connected with the original vertices. Vertices are turned into edges and edges will form faces.*

There are various options with extrusion operations that define how the extrusion will behave, these include "Extrude Region", "Extrude Individual", "Extrude Edge". Extrude works by shifting position along "Normals".

When using EXTRUDE and you don’t get the behavior you wanted, try using `ALT` + `E` to get the Extrude Options pop-up menu. Try the other EXTRUDE options.

When using EXTRUDE, you can extrude to the mouse gizmo location (MOVE MODE) in by using `CTRL` + `E` and right click. This object tool is directly under the ARROW SELECT tool.

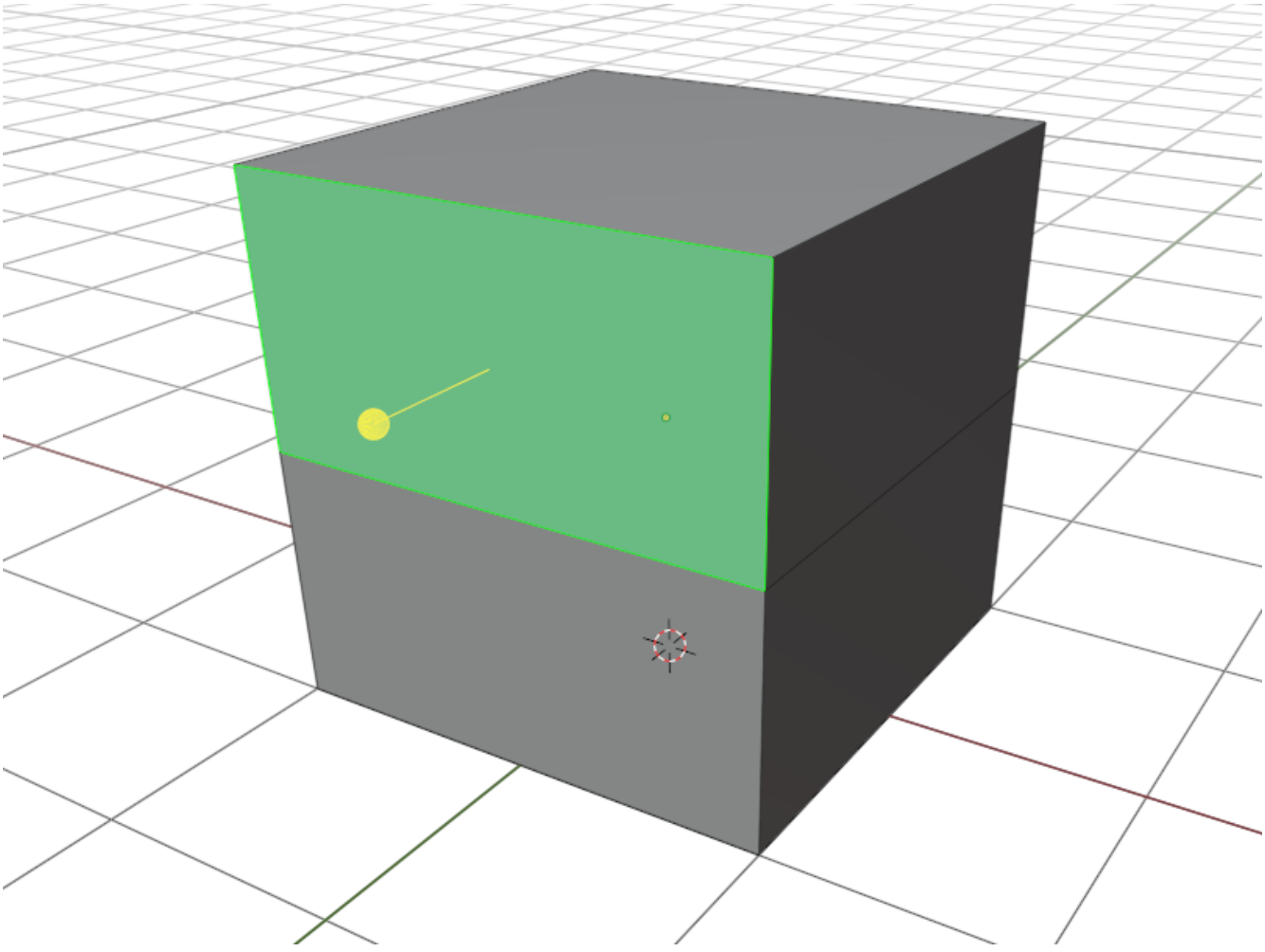
If you select extrude and accept the extrude without dragging a distance away from the selection, the extrude "still happened" even though you might not be able to see it.

INSET

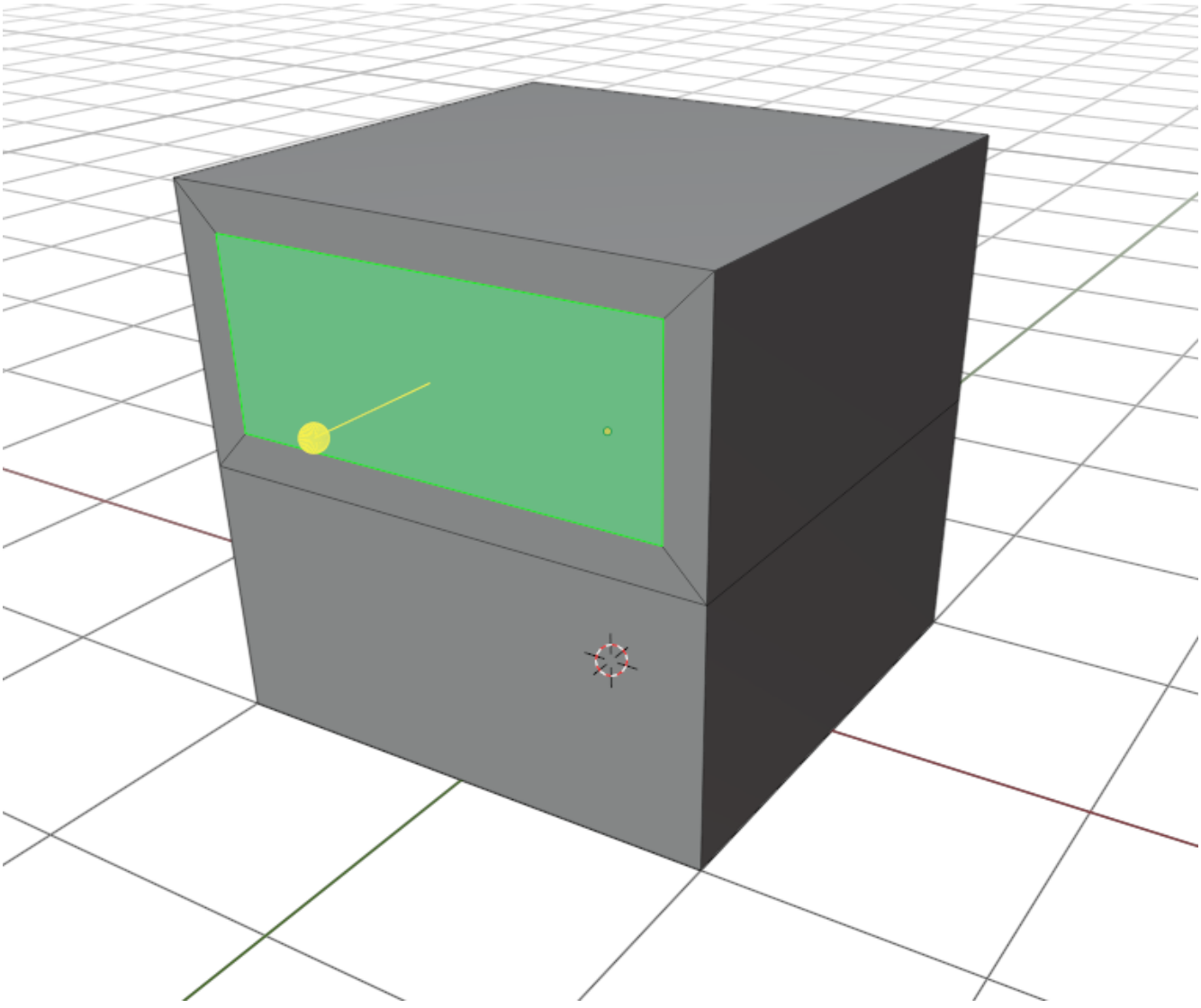
I Key

This tool takes the currently selected faces and creates an inset of them, with adjustable thickness and depth. (For clarity, we are referring to the letter i on the keyboard)

- Select the faces to inset:



- Press I to inset:



When you use inset and the inset amount appears to be unevenly applied, you likely have not applied your scale transformations to the underlying object ahead of time. In other words, your scale values in the object transformation numbers panel are not all set to 1.00. You correct this in `OBJECT MODE` by selecting `OBJECT→APPLY→SCALE` from the top menu.

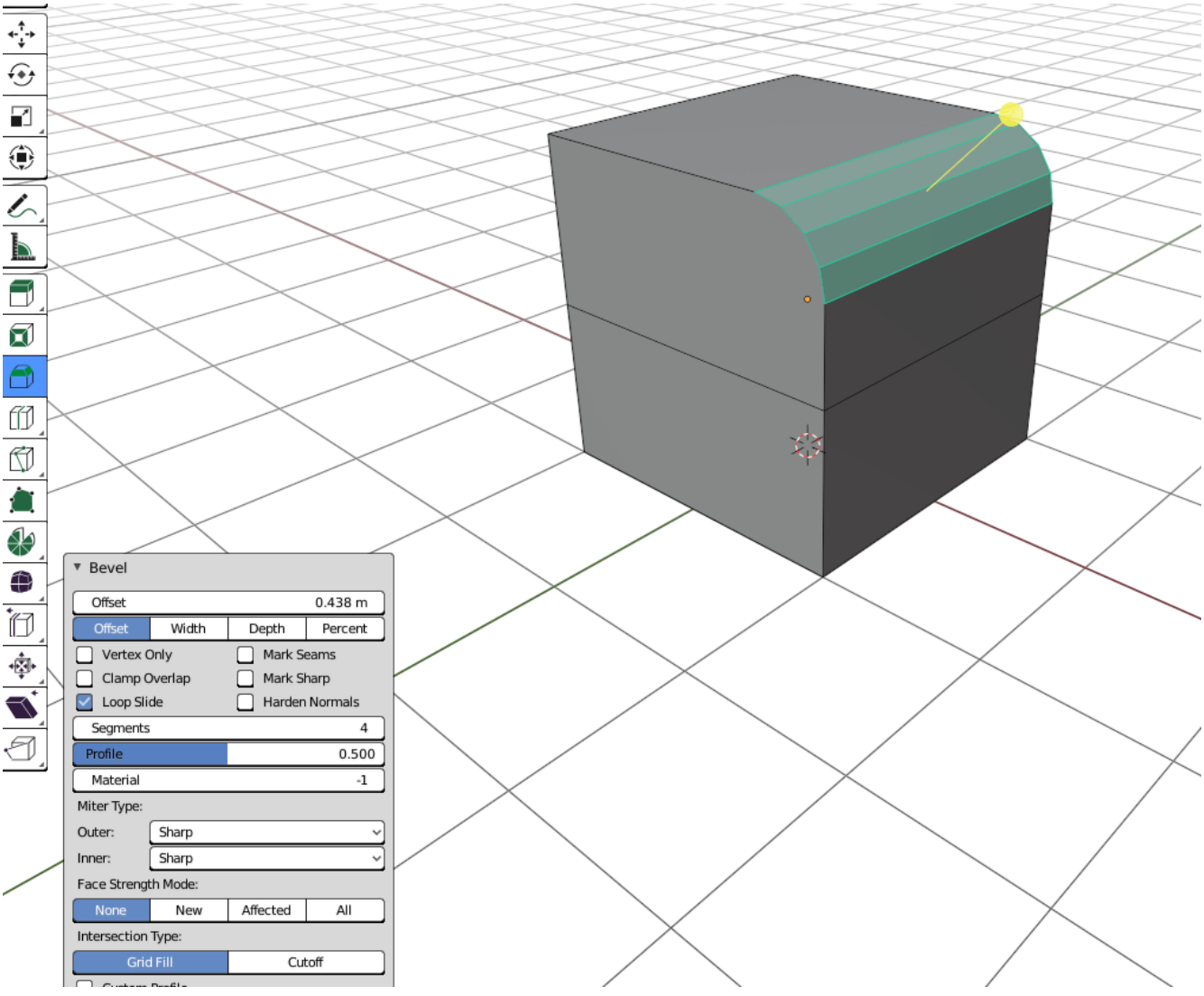
You might need to press `I` again to enable INDIVIDUAL ORIGINS if you notice that inset is not doing what you expect.

BEVEL

`CTRL` + `B` Key Combination

The Bevel Edges tool works only on selected edges with exactly two adjacent faces. It will recognize any edges included in a vertex or face selection as well, and perform the bevel the same as if those edges were explicitly selected. In “vertex only” mode, the Bevel Vertices tool works on selected vertices instead of edges, and there is no requirement about having any adjacent faces.

The Bevel tool smooths the edges and/or “corners” (vertices) by replacing them with faces making smooth profiles with a specified number of segments.



A Bevel on a FLAT PLANE will create an INSET.

There are multiple bevel options in Blender version 5. Worth looking into if thats

the version you are using.

When using BEVEL with a individual vertices instead of objects, try using `CTRL` + `SHIFT` + `B` to get the proper bevel operations. You will notice this when working with selected vertices and the standard bevel operation and seeing nothing happen. <<<

LOOP CUT & SLIDE

`CTRL` + `R` **Key Combination**

This tool splits a set of faces by inserting new edge loops intersecting the chosen edge(s). It will preview the loop cut as you move the mouse cursor around the object, snapping from horizontal to vertical based on position. The Loop-Cut will stop at `NGON` Intersections so it is better to use this tool early before you add a lot of geometry changes or booleans. If your mesh has non-NGON shapes, IE; it has been triangulated already… then this and other some other tools won't work as expected.

Press `E` if you want it to evenly match one of the adjacent edges.

Press `F` will flip the sdge it matches

Use the Scroll Wheel on your mouse (before pressing `E`) to add or remove additional cuts.

After **[Left-Clicking]** the mouse, you can slide the loop cuts up or down and **[Left-Click]** again to confirm. `Right-Click` center the Loop Cut.

The operation has an "Adjust Last Operation" panel where you can still adjust some values. Use **[F9]** to open this option if you don't see it on the screen.

For more precise adjustments, you can turn on the "Edge Length" feature in "Viewport Overlays" to visually see the edge lenths.

The "smoothness" value will scale the selected loop cuts using a fall-off profile.

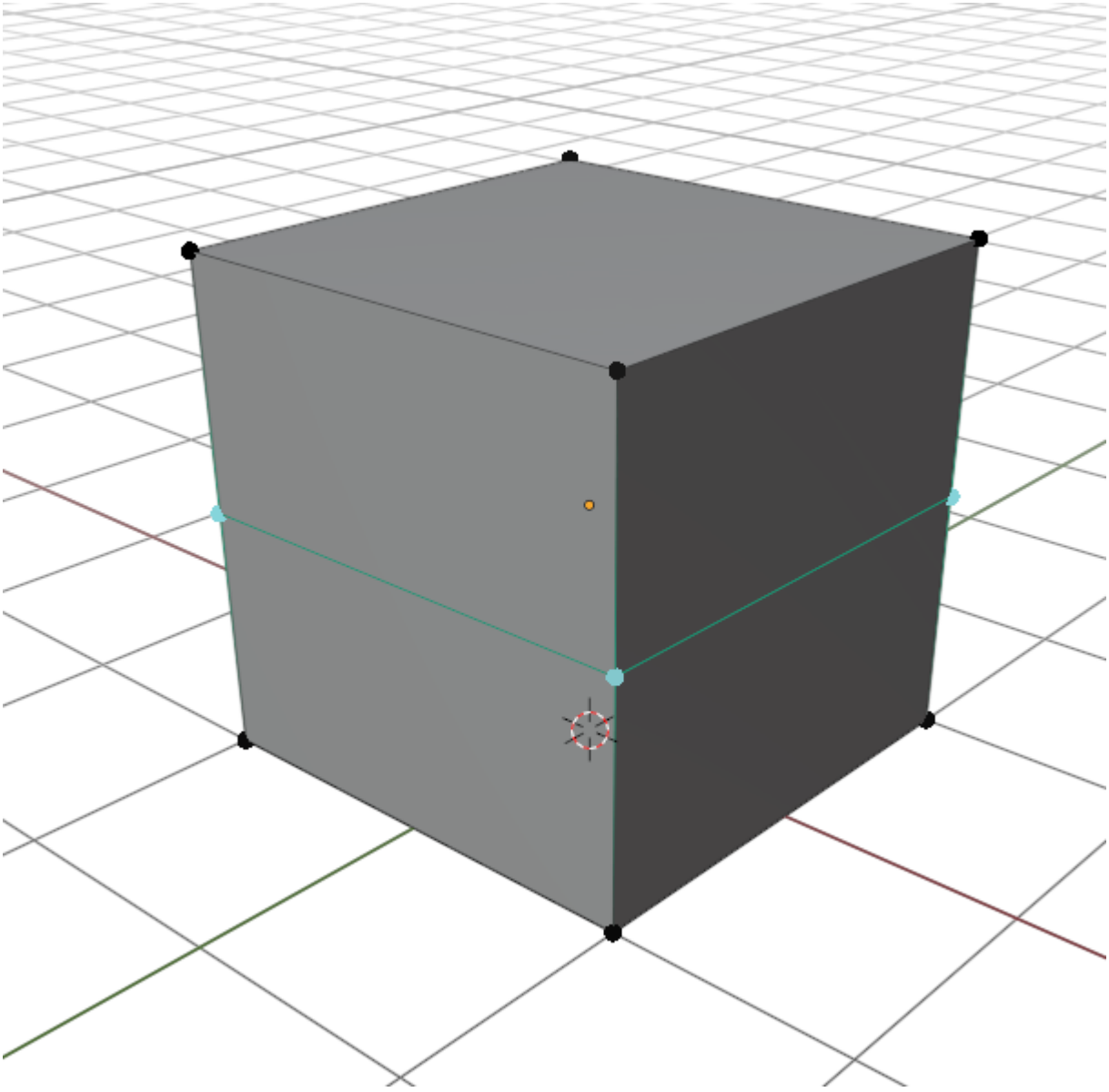
You can use the function `CTRL-F` or Search for "Tris to Quads" to un-triangulate a shape to possibly regain the ability to use the Loop Cut options again.

Later on… you can also shift selected vertices or edges with `GG` (G twice) to slide the selected vertices along edges… (You can do this with faces as well, but, well, its usefulness is limited.)

You can use the mouse to SLIDE the selected edge loop(s) into position before

confirming. You can add multiple edge loops at once by using the scroll wheel on the mouse or by entering a value with the keyboard before confirming.

You can also use the options box that shows up in the bottom left of the screen to adjust parameters manually.



Additional Tips

Here are some tips from some lessons I've learned while using Blender.

MIRROR and FLIP

Sometimes, when working on a vehicle model particularly, you might find that using the MIRROR option is not enough. You might also need to FLIP the item along an axis. For example, lets say you are adding coupler lift bar to your model and you have finished adding it to the rear end of your model. You now want to do the logical thing and mirror it to the opposite end of your model. But, mirror alone will have the part end up on the opposite end of the model, but on the same side of the model. It needs to also be flipped.

STEPS:

- On your active part (coupler lift bar), `CTRL` + `A` and choose, "Apply Rotation and Scale"
- `RMB`, from pop-up menu. choose "Set Origin to 3D Cursor", assuming 3D Cursor is current World Origin. If not, it should be for this.
- `Shift` + `D` to duplicate the current object, then choose **Object › Mirror › Global Y**
- And finally, flip it. `S X -1` which translates to `SCALE`, in the `X` axis, amount `-1`, which will FLIP the object in the X-AXIS.

You might be tempted to try to use the Mirror Modifier, however, that operation will ultimately LINK the objects and the option within the modifier that says "FLIP X" actually does not do what you might think. The FLIP option here changes the active side of the Mirror.

Additional Modeling Tips

- Model one side of the mesh and mirror it to the other size using the Mirror Modifier. This helps to make sure you get symmetrical results
- If you have an item that will be replicated many times, like a handrail stanchion, create one, UV Map it and then replicate it. You can shift the UV coordinates later, but you only need to uv map it once
- Add MARK SEAM and MARK SHARP settings while you work instead of saving these steps for later
- Export your model as soon as you get the initial UV mapping completed so you can find issues earlier rather than later as a time saver
- Remember that you can `hide` and `un-hide` collections and objects in the Outliner to help with focused modeling

- Remember to use `Smooth Edges` and `Auto Smooth` settings
- When you want to select a loop, you can use the `ALT` key to select all the edges that make up the loop.
- When you wan to select the shortest path of edges between to selected edges, use `CTRL` + `SHIFT`
- You can change the viewport shading options from studio to matcap and then select a lighting setup option to get different shading effects on your work. Some options really do make it easier to see what you are working on, such as "Clay" or "Metal Car Paint".
- When you create a new object from a primitive, it will retain that name, IE; CUBE, so it is a good practice to name the item as soon as you create it, otherwise you can end up with dozens of CUBE items in your collection, making it harder to find what you are searching for.

Setting up your Initial Workspace

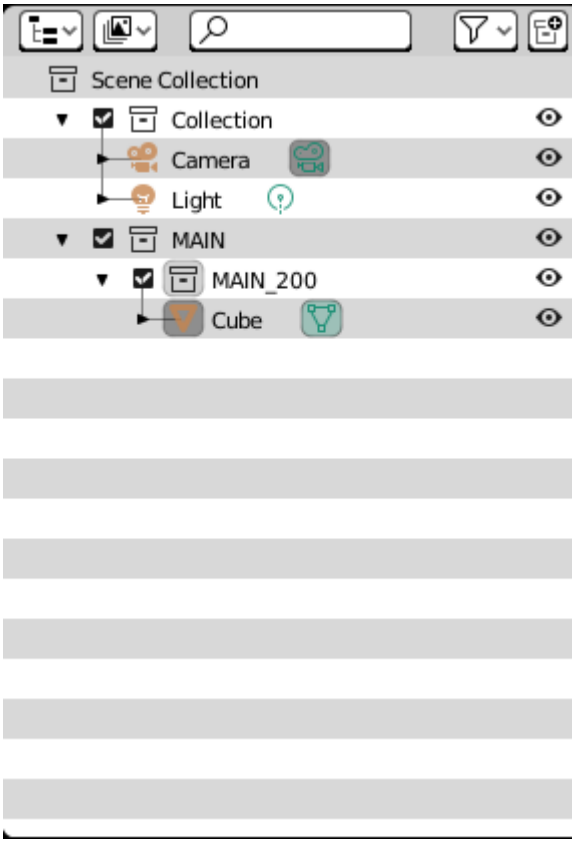
Default workspace setup

You have a way to set some custom options that you always need to change and then save that as default to avoid having to perform those changes when you start a new project.

The default workspace in modern Blender versions is something you will see in many YouTube tutorial videos. The first thing they will often do is select and then delete the "default cube". Rather than go through this every time, it is possible for you to delete it, and then save your current cube-less `.blend` file as your new startup file. (I don't actually recommend it though)

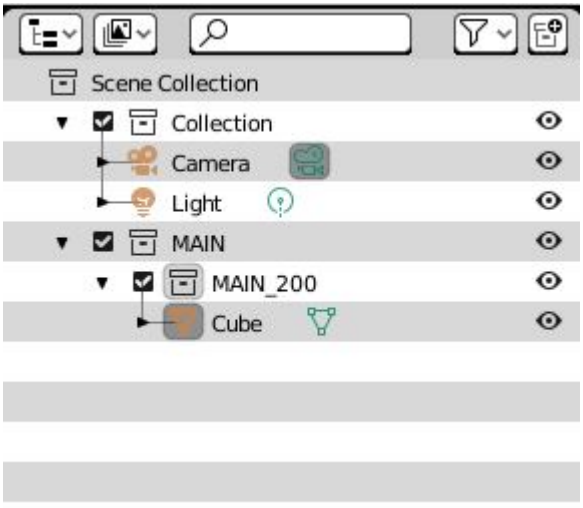
`FILE→DEFAULT→Save Startup File` will save your current workspace as your default startup file. This will help you get started faster and with a consistent layout.

Before you do that just yet, go over to your `Outliner` window... and do the following:



1. Create a new collection by clicking the `box icon` in the upper right `Collections` window with a plus sign on it. Create a new collection called `MAIN`. (All uppercase)
2. Click the new collection `MAIN` and then create a new `Collection` again so it becomes a child collection under `MAIN` and call it `MAIN_0700` for LOD distance, or use any LOD distance value that makes good sense to you.
3. Now click on `SCENE COLLECTION` at the top and rename the original main collection to `Camera`.
4. Drag and Drop the Camera object into `Camera`.
5. Now you can save your `.blend` file as your default startup file and you will have the scene outline setup that will work with the MSTS exporter.
6. Optional: If you are going to continue to use the default cube, drag it from where it is to the MAIN_0700 collection.

You would later create as many LOD based MAIN_xxxx collections as needed for your model.



Exporter

You will want to download and install the "MSTS Exporter". https://github.com/pwillard/Blender_MSTS_ORTS_Exporter/releases

The exporter documentation is maintained with the project on GitHub: https://github.com/pwillard/Blender_MSTS_ORTS_Exporter

Helper

There is also a "Work In Progress" helper script located at https://github.com/pwillard/Blender_trainsimstools This add-on can generate the required Exporter Collection layout for you.

Actually Modeling Something

This is a quick tutorial on general model building. To get started, we are making a very simple scenery item.

Model Building Exercise #1

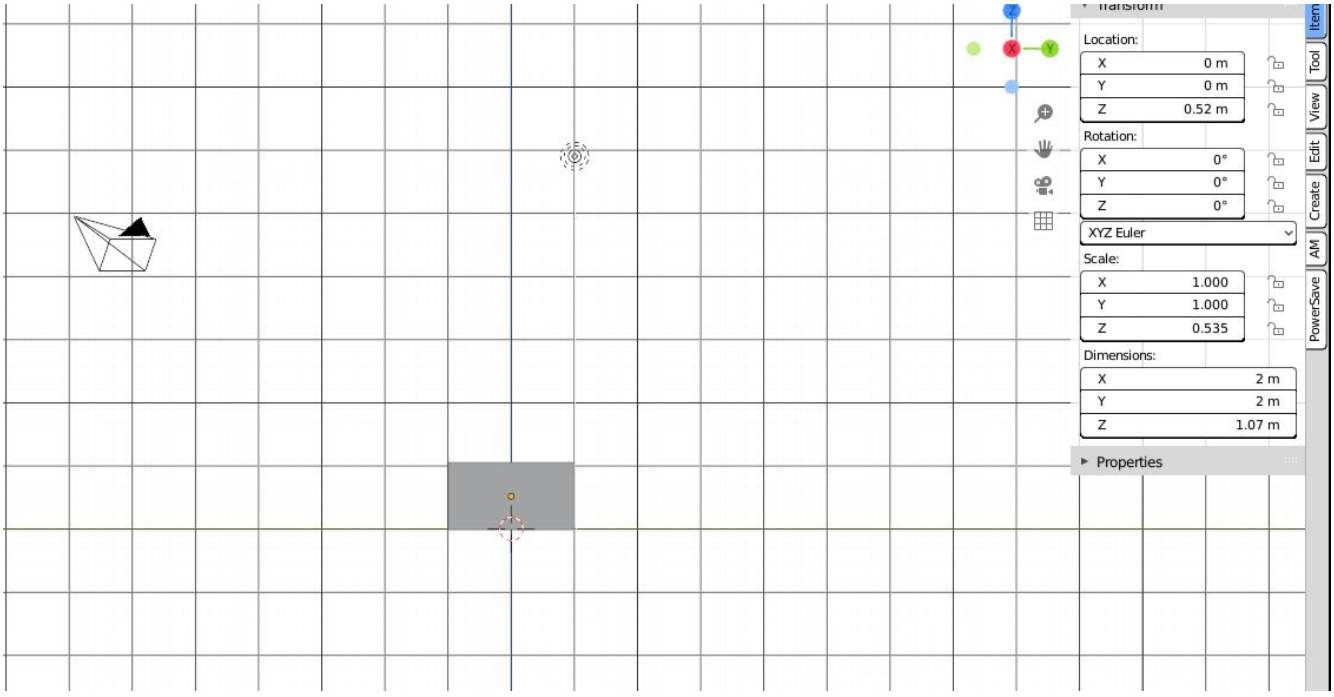
- We are building a very simple shape without using a background image.
- We only need a few general dimensions.
- We will re-make my first Blender project; The Jersey Barrier, 12', by 42" by 32".

Key items to take away from exercise #1

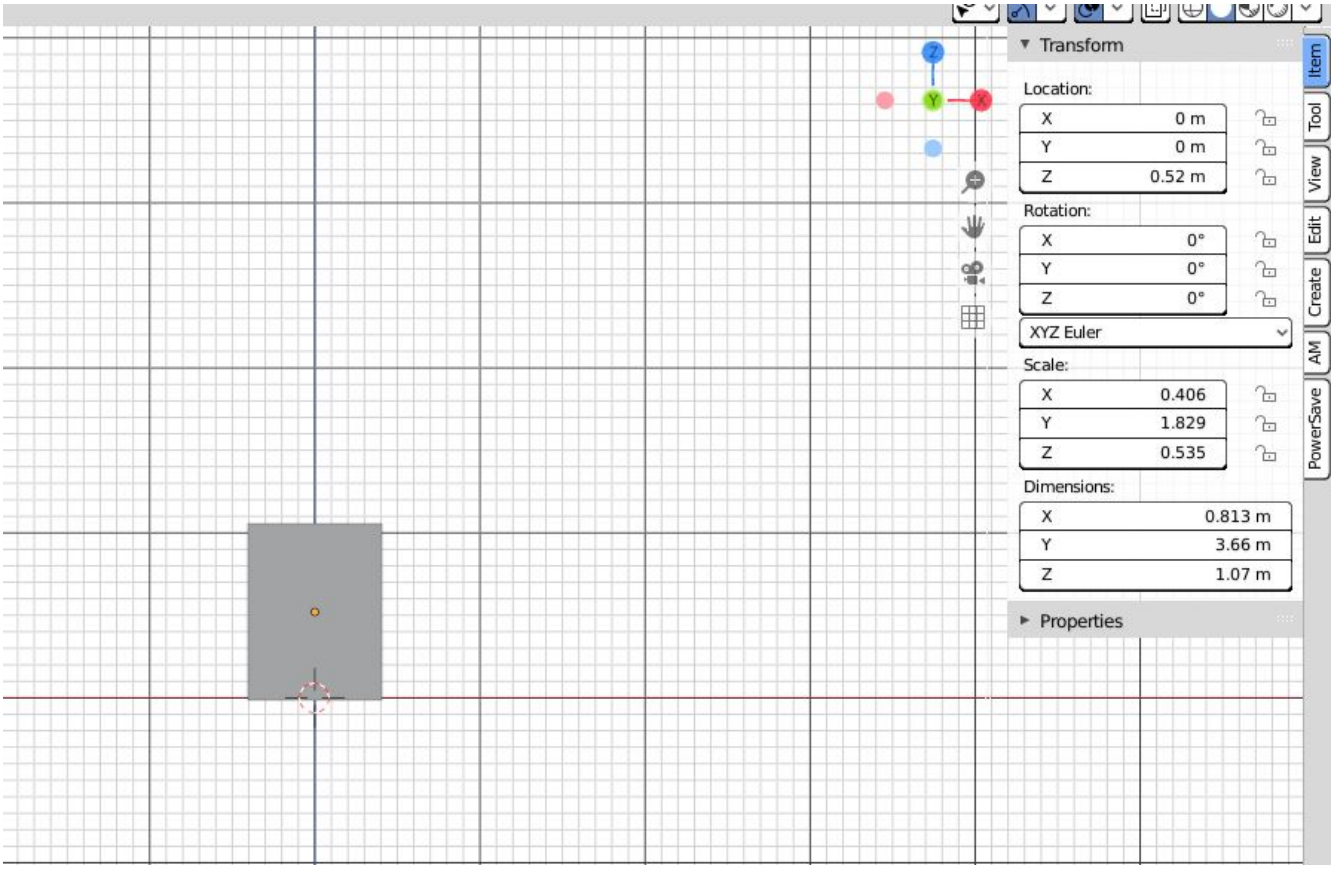
- Object and View Manipulation
- Basic Edit Tools
- Quick UV Texture Mapping

For added clarity on printed copies of this document, I've switched to a high contrast theme found in Blender Preferences.

- Start with the default cube, and select it with `LMB`.
- Shift it up 1 meter. `G` `Z` `1` `ENTER`
- As an aid to modeling, pull out the right side number panel by pressing `N`
- Scale to 42" (Z Height = 1.077m) Manually enter 1.07 in the Z dimension field.
- Shift it back to ground level. `Keypad 3` on keypad for side view, `G` key `Z`, drag down to about ground level (Roughly: Locaton 0.52m in Z axis if you manually enter the `Z` position)



- Adjust your view so you can see the Y axis `MMB` + `DRAG` left to right (You can press "Y" in the axis gizmo on the upper right or use `KEYPAD 3`)
- Adjust cube length to 12', (3.6576m) by using `S` `Y` and dragging, or by manually entering 3.6576 in the Y Dimension field
- Switch to front view and adjust X width to 32" (0.8128m) `Keypad 1` then `S` `X` and `DRAG` or enter .8128 in the X Dimension field.

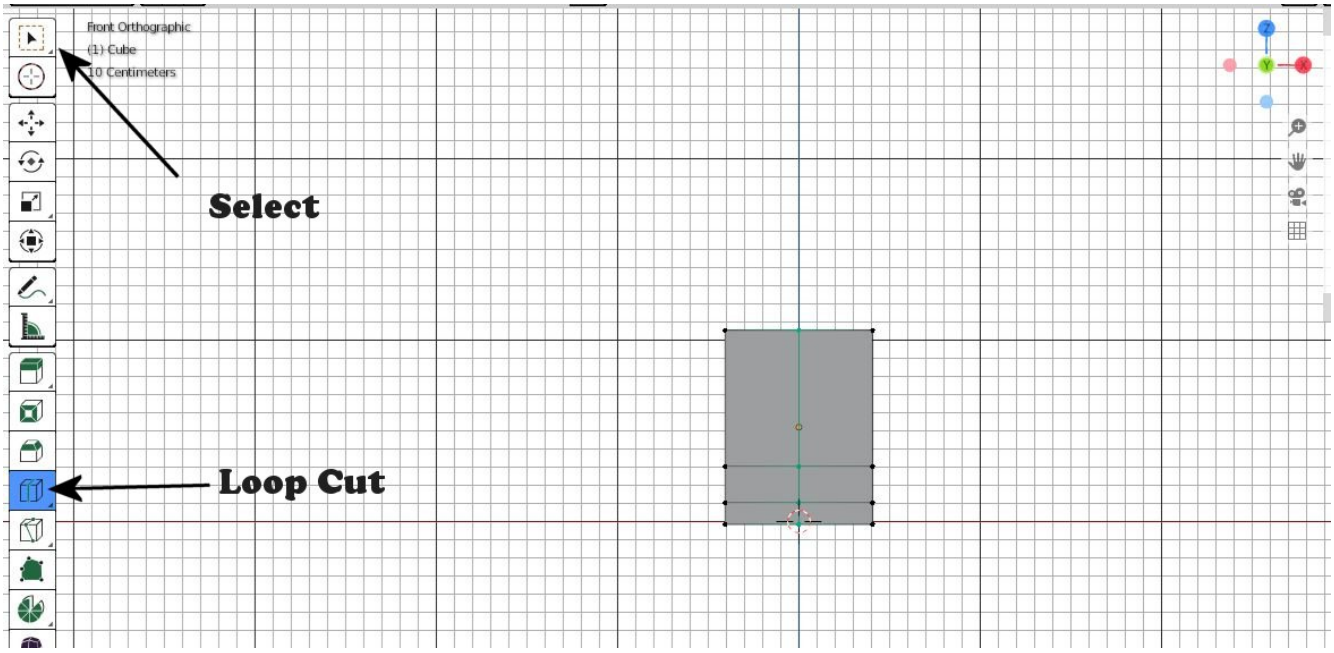


- Choose **Object › Apply Scale**
- With the CUBE still selected, switch to EDIT MODE using the TAB KEY.

We are doing this without worrying about EXACT dimensions just to keep things simple.

- Add a LOOP CUT CTRL + R and slide it down (Z axis) to the grid line closest to the bottom, and add another LOOP CUT CTRL + R and drag it down to be 2 grid lines above the first one.
- Add a final LOOP CUT CTRL + R but this time, add it vertically. It should end up dead-center by default.

Drag the mouse around to get it to snap to a vertical loop.

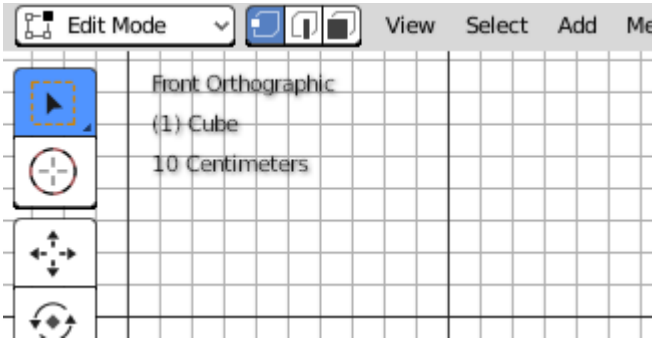


- Go back to SELECT Mode by clicking the Arrow icon if its not already selected since we are done with LOOP CUTS for now.

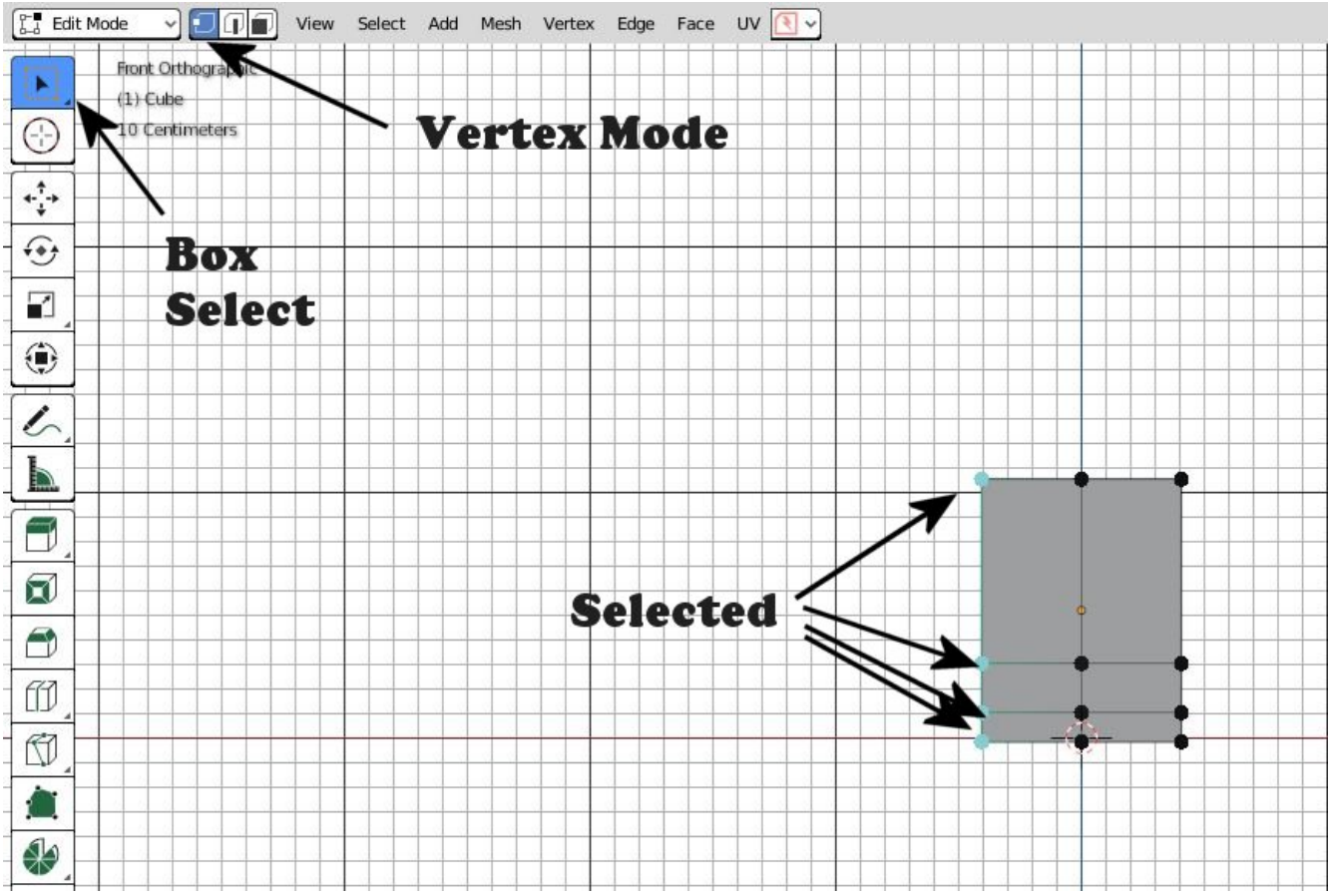
In these next steps, we will be using Vertex EDIT MODE. From the front view, we will DELETE the vertices on the left side of the object because we are going to use the Mirror Modifier to create a symmetrical object.

- Press KEYPAD 1 for front view, and then 1 on the keyboard to select Vertex EDIT MODE. You should see the vertex dots on the selected object.

You can also select the vertex mode with the screen menu. Its the small square icon with a dot on one side next to the view tab.

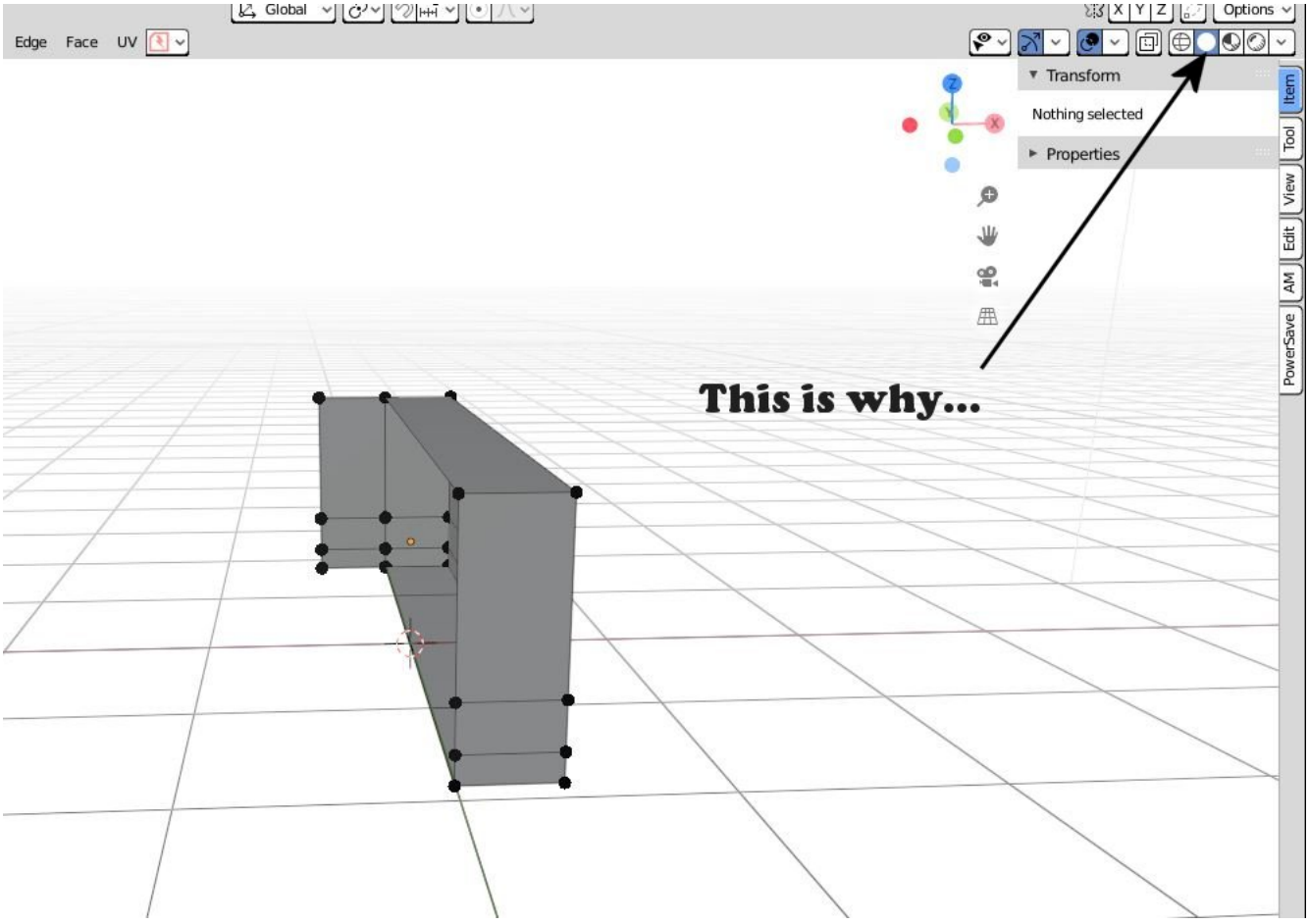


- Now use BOX select on the vertices on the left side with your mouse. (They will change to the SELECTED color)



- Press `X` for Delete and in the pop-up window, choose to `Delete - Vertices`. Blender will delete the selected vertices. Wait… What just happened?

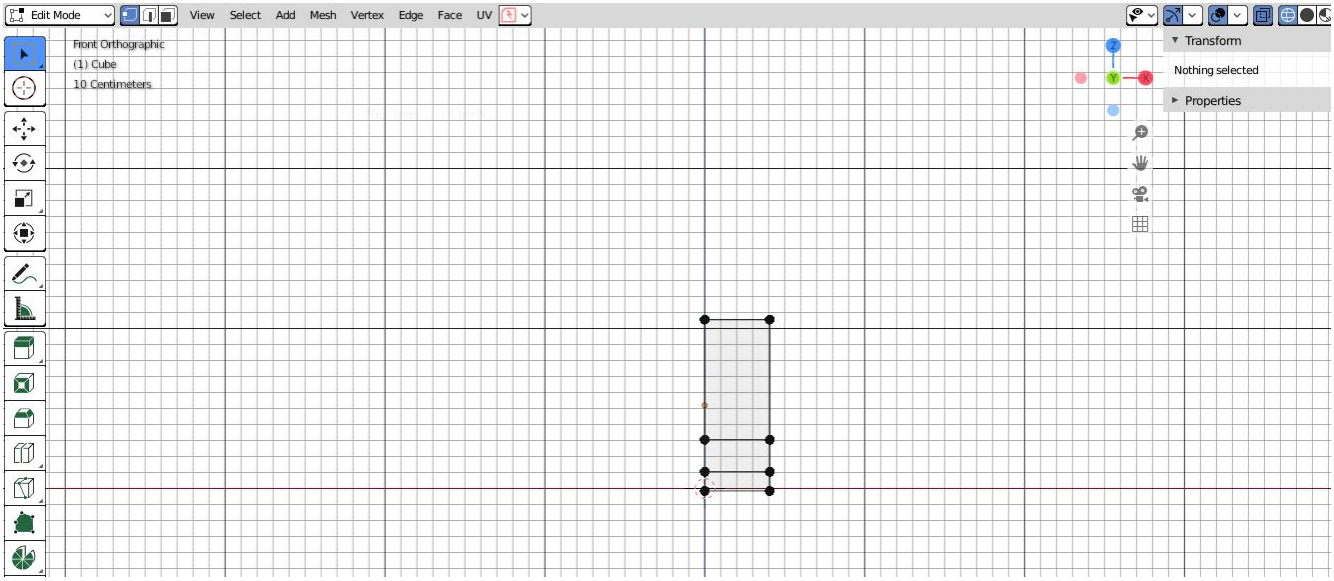
It didn't perform a delete? Oh my, yes it did, but not what we wanted!



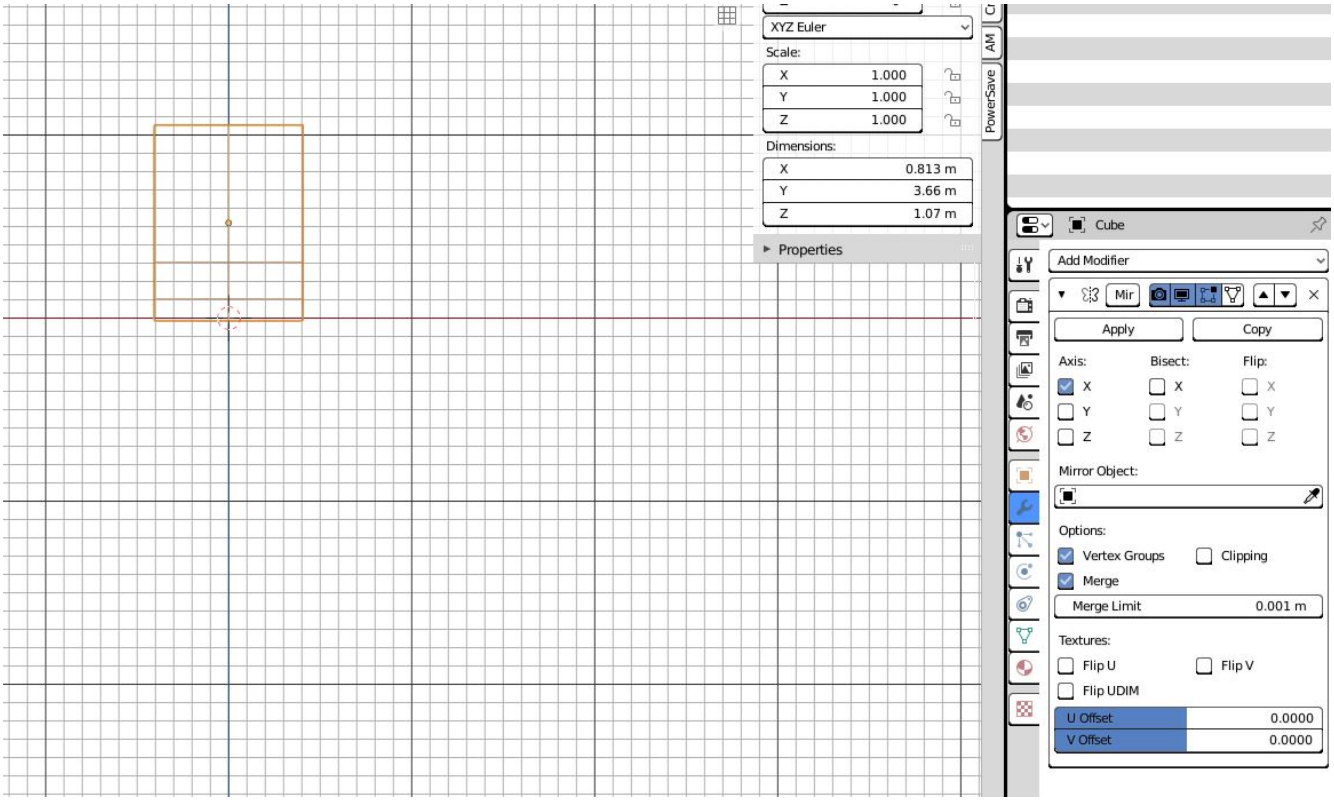
We are in ViewPort "Shading Solid" Mode.

Do you understand what happened? We only selected the FRONT facing vertices! We didn't touch the ones in the back. Press `CTRL` + `Z` to undo if you completed the above step. To select ALL of the vertices that we really want to select, we need to be in `XRAY/Wireframe` mode. To choose this mode, press `Z` and choose `WIREFRAME`, making sure that the viewport mode on the top right of the screen agrees. You can also toggle Wire-frame mode by using `ALT` `Z`. The Circle with LINES in it and the `X-RAY` icon next to it should also be highlighted.

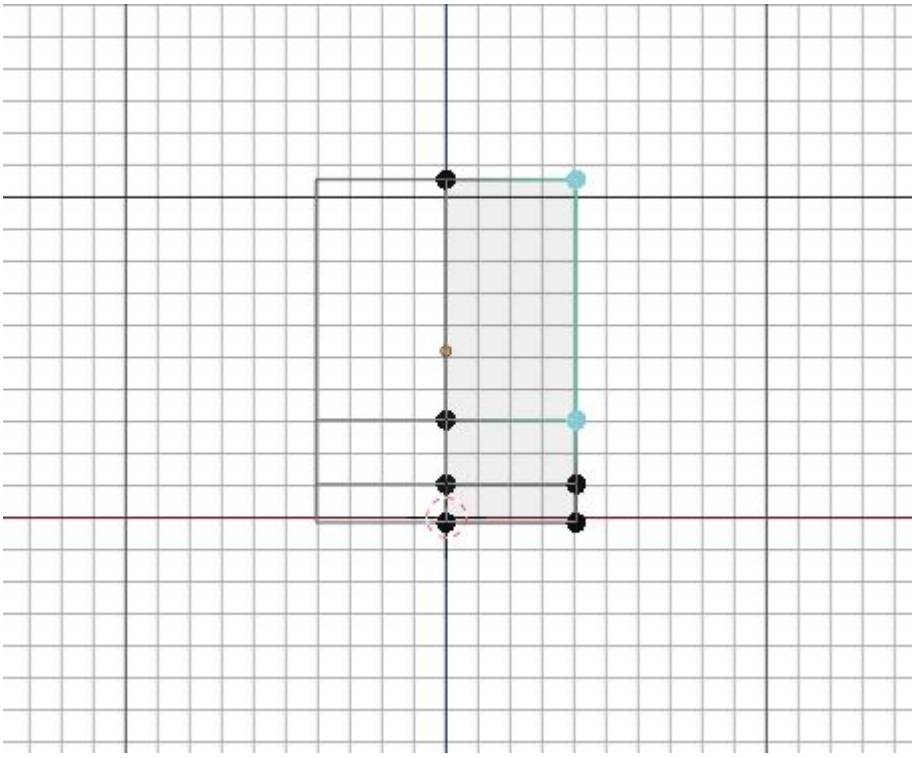
- Make sure you are in vertex select mode `1`, and also in the front view `1` we will remove the left side vertices. Press the `Z` key and select `WIREFRAME`, Press `Keypad 1`, and then `1` on the keyboard to select front view & vertex mode. You should see your vertex dots and the model will look transparent now and not solid.
- Now, `BOX` select the vertices on the left side, like before. (They will change to the SELECTED color) and press `X` and choose to Delete Vertices. Blender will delete the selected vertices. Now, you will finally only see 1/2 of your object remaining.



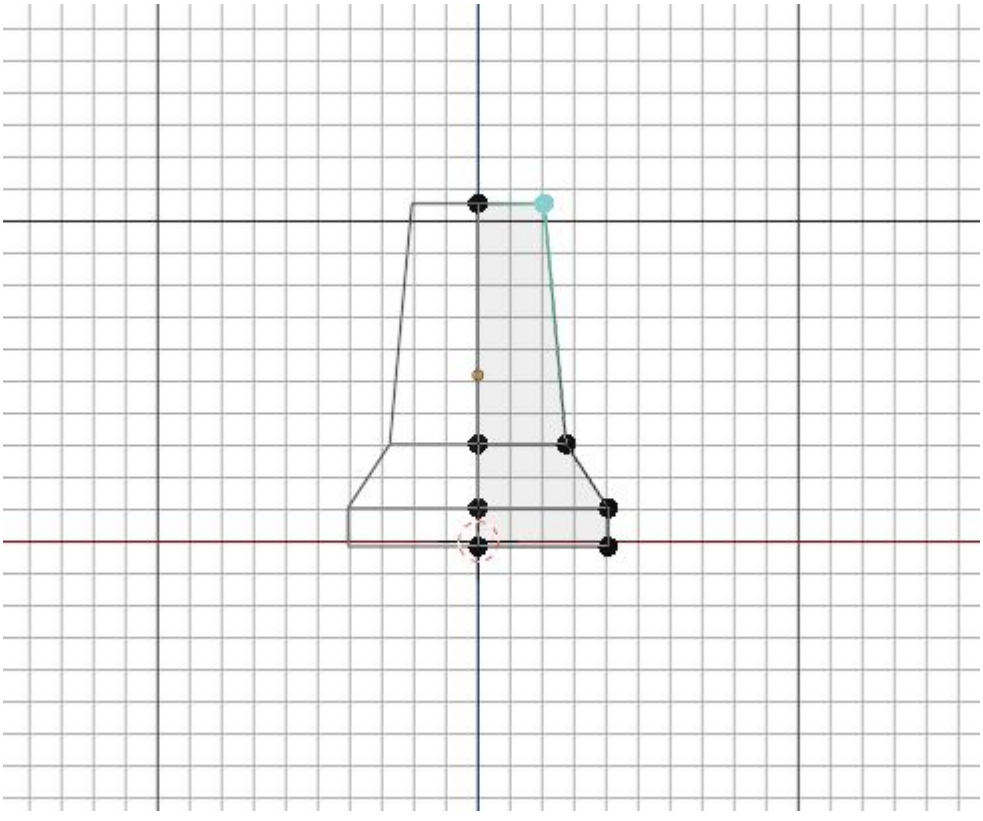
- `TAB` back to `OBJECT MODE` and with the current OBJECT selected, locate the `WRENCH` icon on the right panel on the screen.
- From the `Add Modifier` dropdown menu, select `MIRROR` Modifier. You should see the section we deleted above come back into view since the `MIRROR` is using the `X` axis to mirror of the original object by default. (See the Check Box that is already checked)



- `TAB` back to `EDIT MODE`. Note that you should now only see Vertex dots on the right side of the object, but you will see the full shape with the mirrored side visible. It's mirrored now, and whatever you do on the right side gets mirrored to the left side.



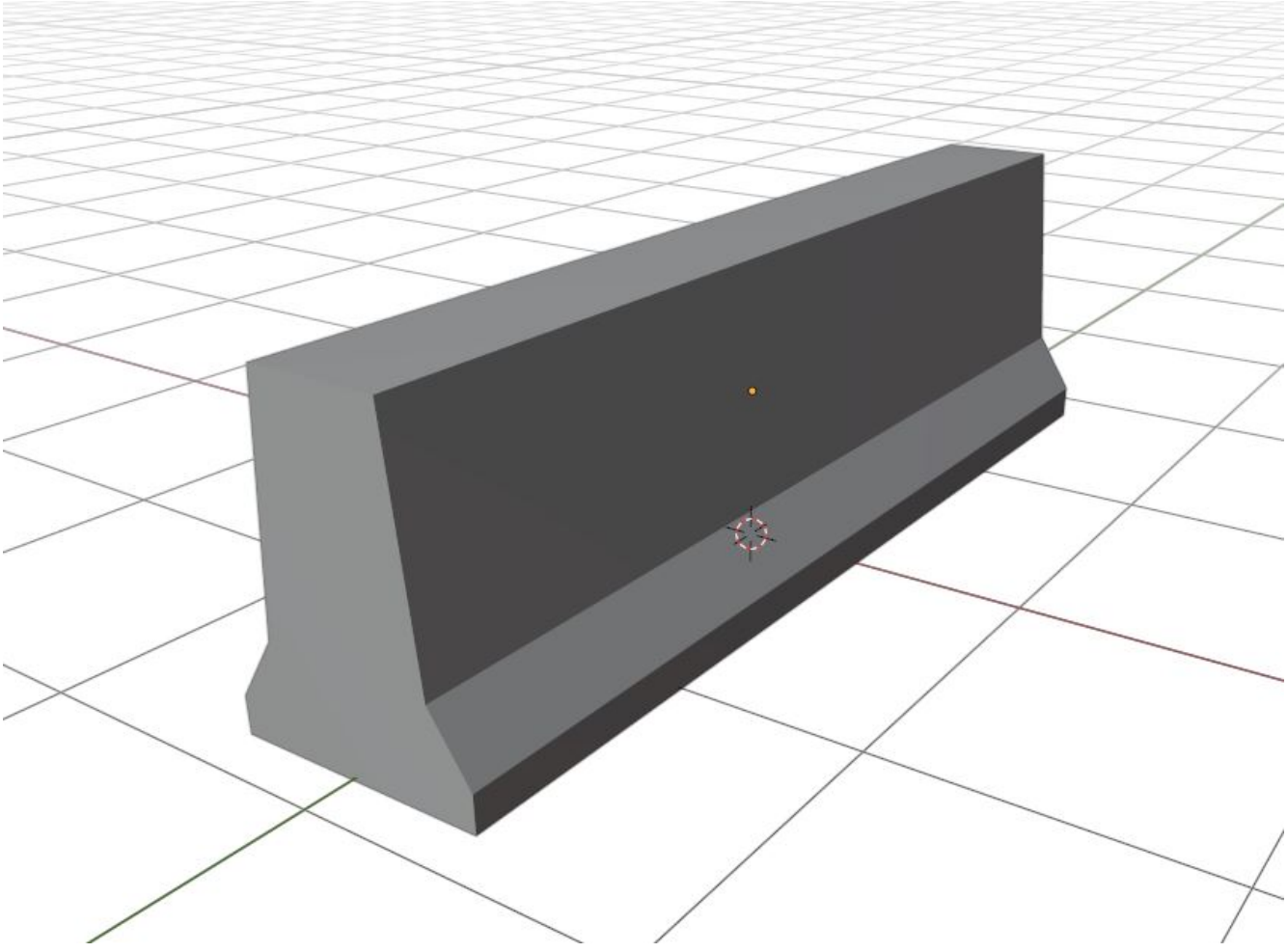
- Select the top 2 right side vertices and press `G` `X` to shift them inward until you get about a 55 degree angle.
- Repeat the same process with just the top right vertices until you get about an 85 degree angle.



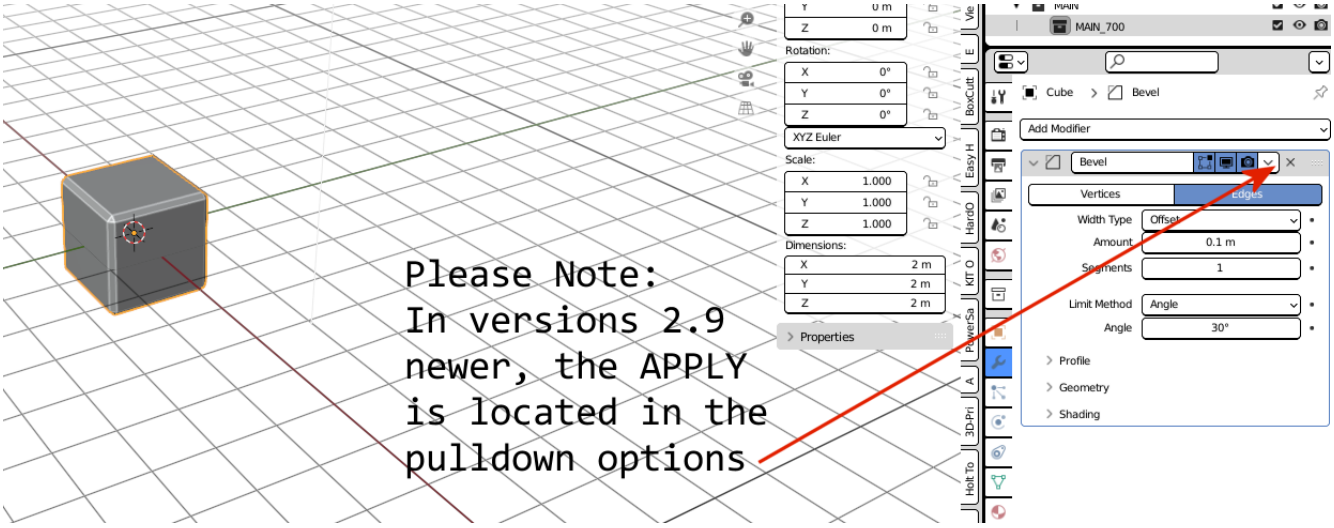
So now we have a basic shape of the concrete barrier. The next steps will complete the shape.

We can go back to `ViewPort SOLID` mode now.

If we look at the barrier closely, we will see that the edges are not sharp. They are beveled. So now we will use the Bevel tool. For this next operation, we no longer need the Mirror Modifier so we can Apply it.



The Apply button for modifiers was MOVED into the pulldown options in Blender version 2.9 and newer.



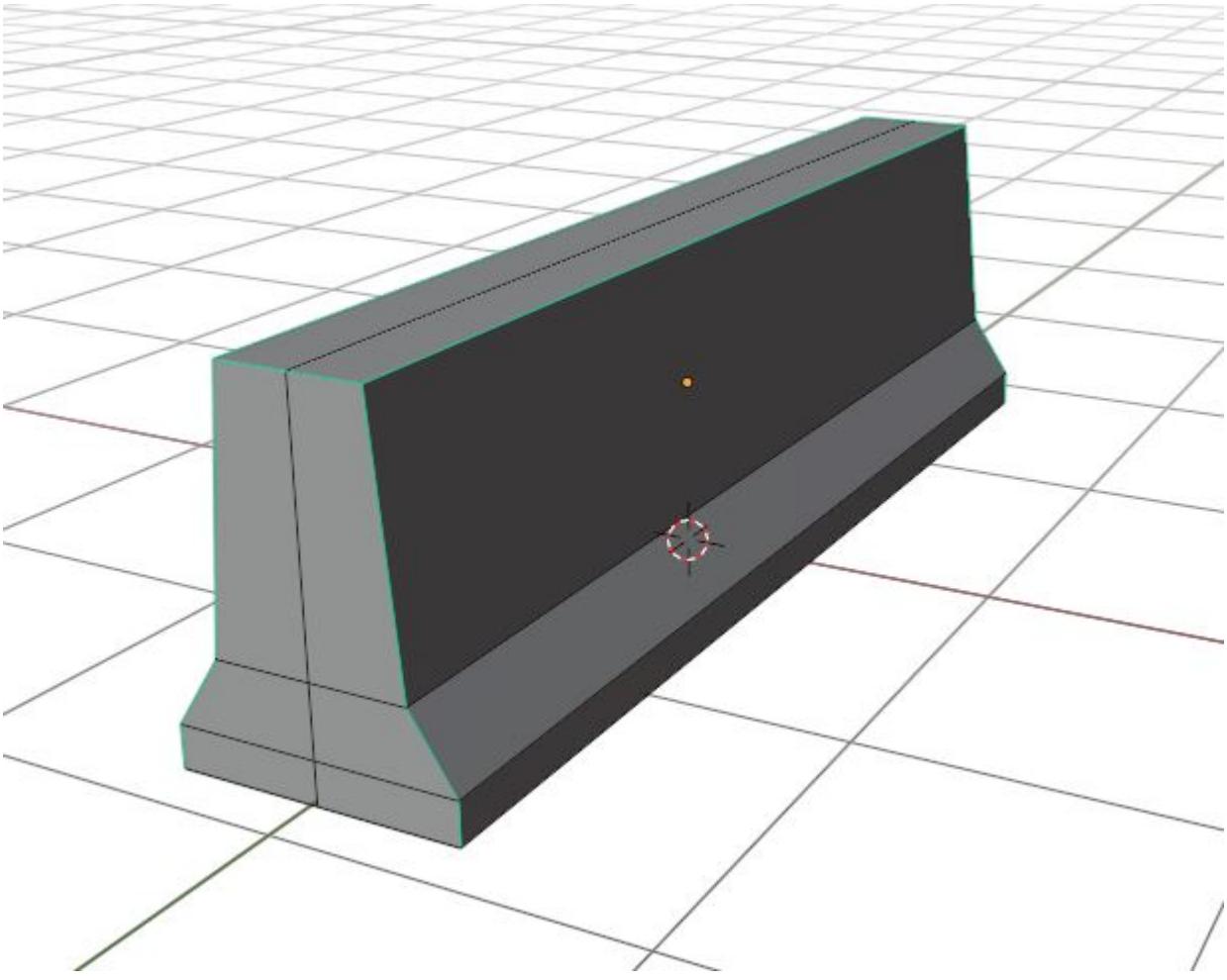
- Go back to `OBJECT` mode, select the `WRENCH` Icon and with our object selected, click `APPLY`. The modifier will apply and go away.

When you apply a modifier, you lose the ability to adjust it further. Prior to applying it, you can still make adjustments to the shape. In our case, we were done making a symmetrical object, so it was `OK` to apply the modifier.

- Now, `TAB` back to EDIT MODE and select the EDGE select mode with `2` key.
- We will select the visible edges of the shape. You will need to `SELECT` multiple `EDGES` so here is what we will do. Hold `SHIFT` then select the TOP LEFT Edge, you will need to shift your view with the `MMB` to get a good viewpoint for selection. The TOP EDGE will be selected... Now click the remaining "outside" edges while still holding `SHIFT`.

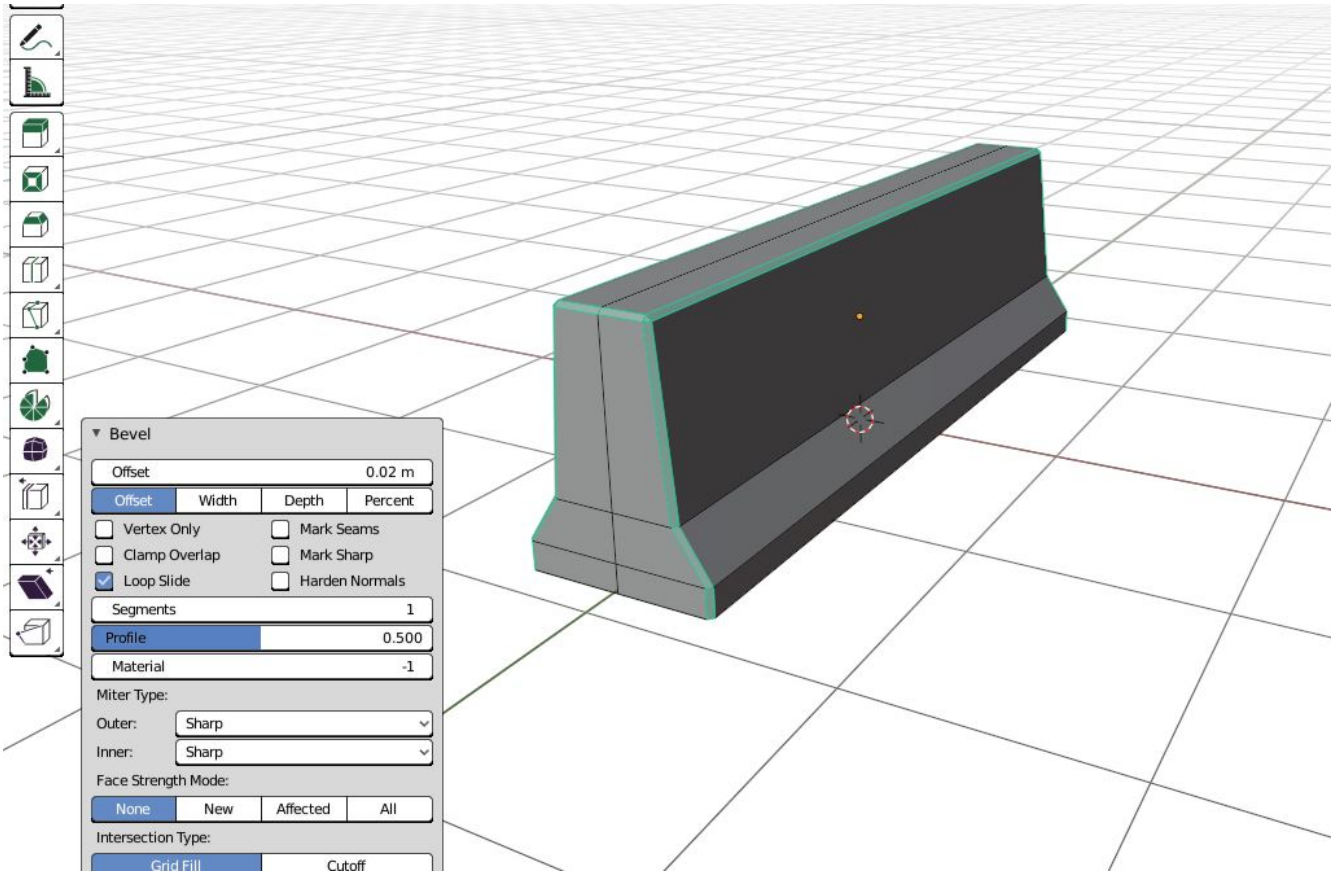
This creates a selection group. If you left click an edge again without holding `SHIFT`, you will lose the selection group and will need to reselect all the desired edges again.

If you hold `ALT` + `SHIFT` you will select all connected `edges` at once. It often will select more than you actually need, so you might need additional `LMB` to unselect unwanted edges.

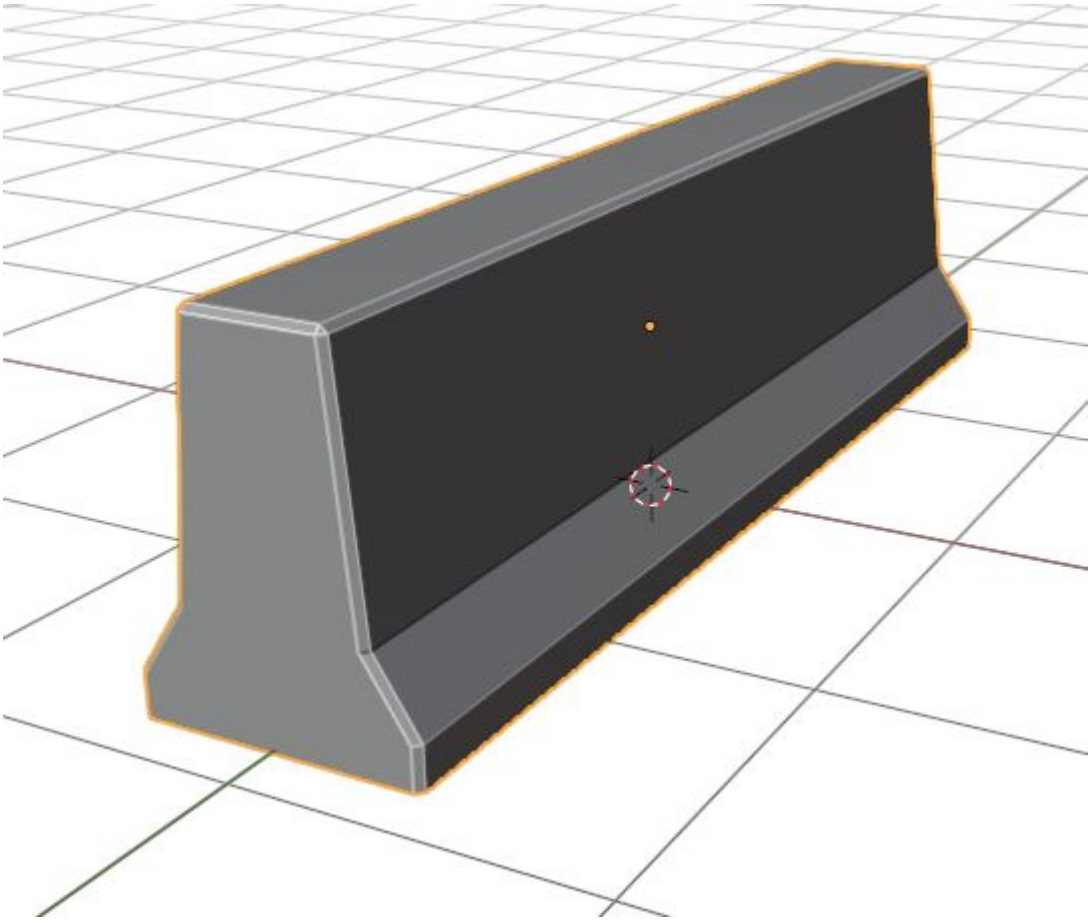


- Press `CTRL` + `B` to use the BEVEL TOOL. and adjust the offset to be about 0.02 and Left Click the mouse to accept.

I am aware that I could have left the modifier on during the BEVEL operation. I did not this time because it is good practice to rotate around a model and select specific edges manually.



Here is what we have now after the bevel operation.



If we look at the object back in OBJECT MODE with the Solid Viewport Shader, we see this.

Texturing

The goal with texturing is to be able to apply a 2 dimensional bitmap to a 3 Dimensional object. Its rather tricky and there are multiple ways to do it. The easiest is to just `Mark Seams` and then UNWRAP the object, then moving the resulting `UV Islands` into position on your bitmap.

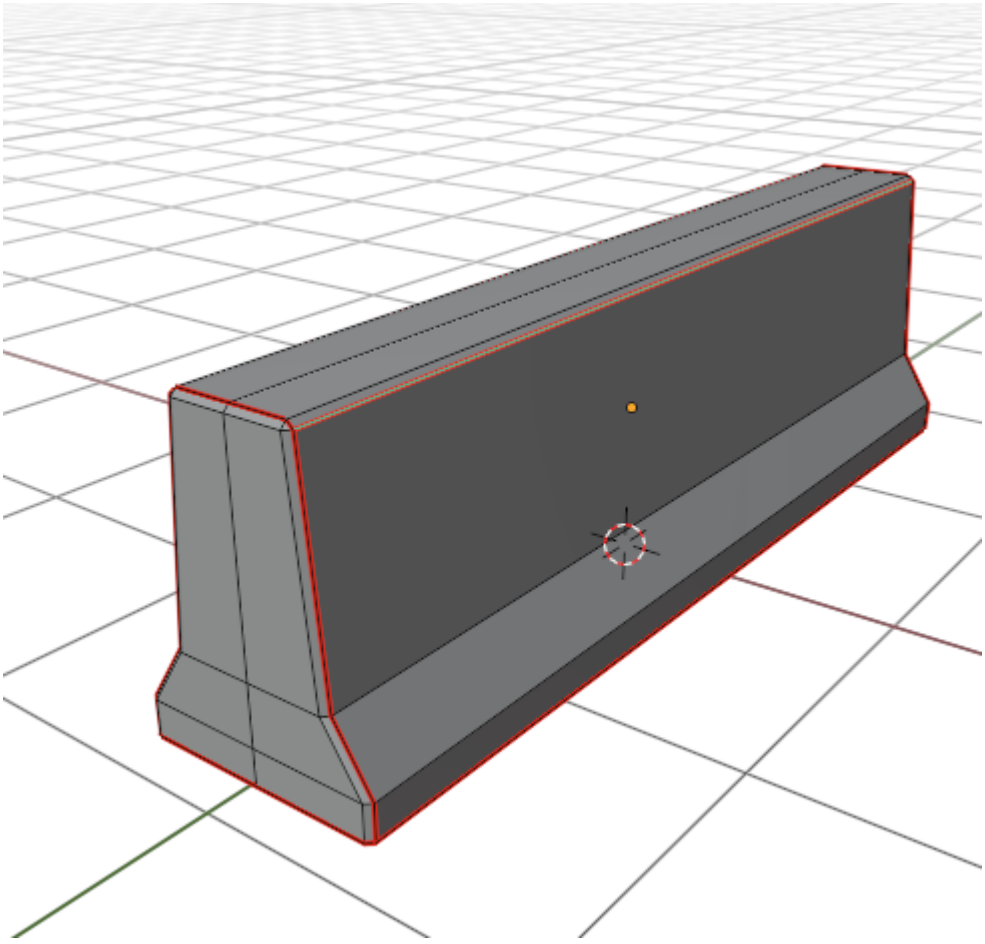
While I say it is the easiest way here, that is a bit misleading. Sometimes, using the `Mark Seams` and `Unwrap` steps create more work for texture creation than is reasonable. Especially if the object is complex but non-organic, like a Boxcar, Engine or Building.

For the texturing steps, we will use a 512x512 texture that looks like this:

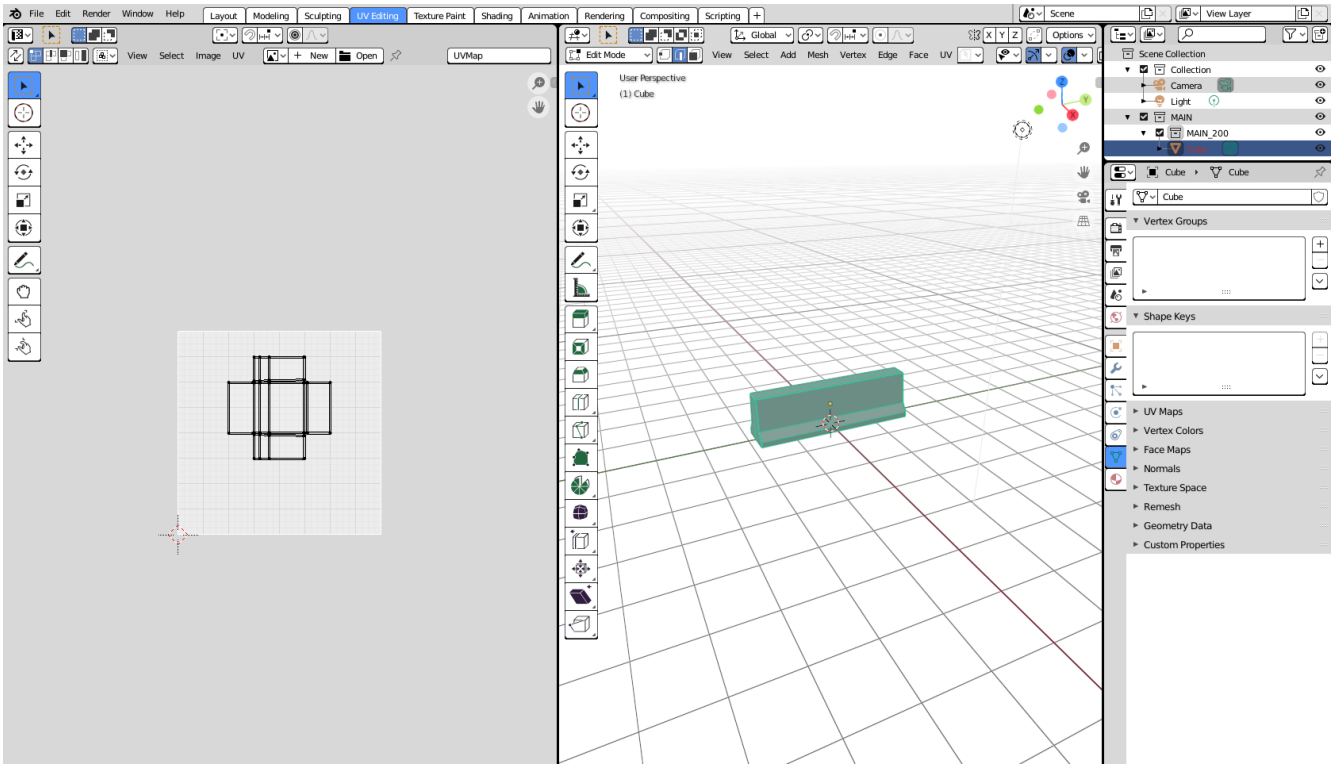
Let's Begin Texturing

Marking Seams

You would Mark Seams (Edges) where the flat edges stop and in the case of our Jersey Barrier model, we mark TOP, BOTTOM, FRONT, BACK and both SIDES by selecting all relevant edges and then use EDGE→MARK SEAM to define seams. These will now highlight in the SEAM COLOR.



- Make sure you are in EDIT MODE and select ALL parts of the object by pressing `A` and then change your WORKSPACE tab to UV EDITING. (Top of the screen)
- You will now have your EDIT window and the UV EDIT window on your screen. Also, you might see the UV UNWRAP of the default cube... which is not what we want.



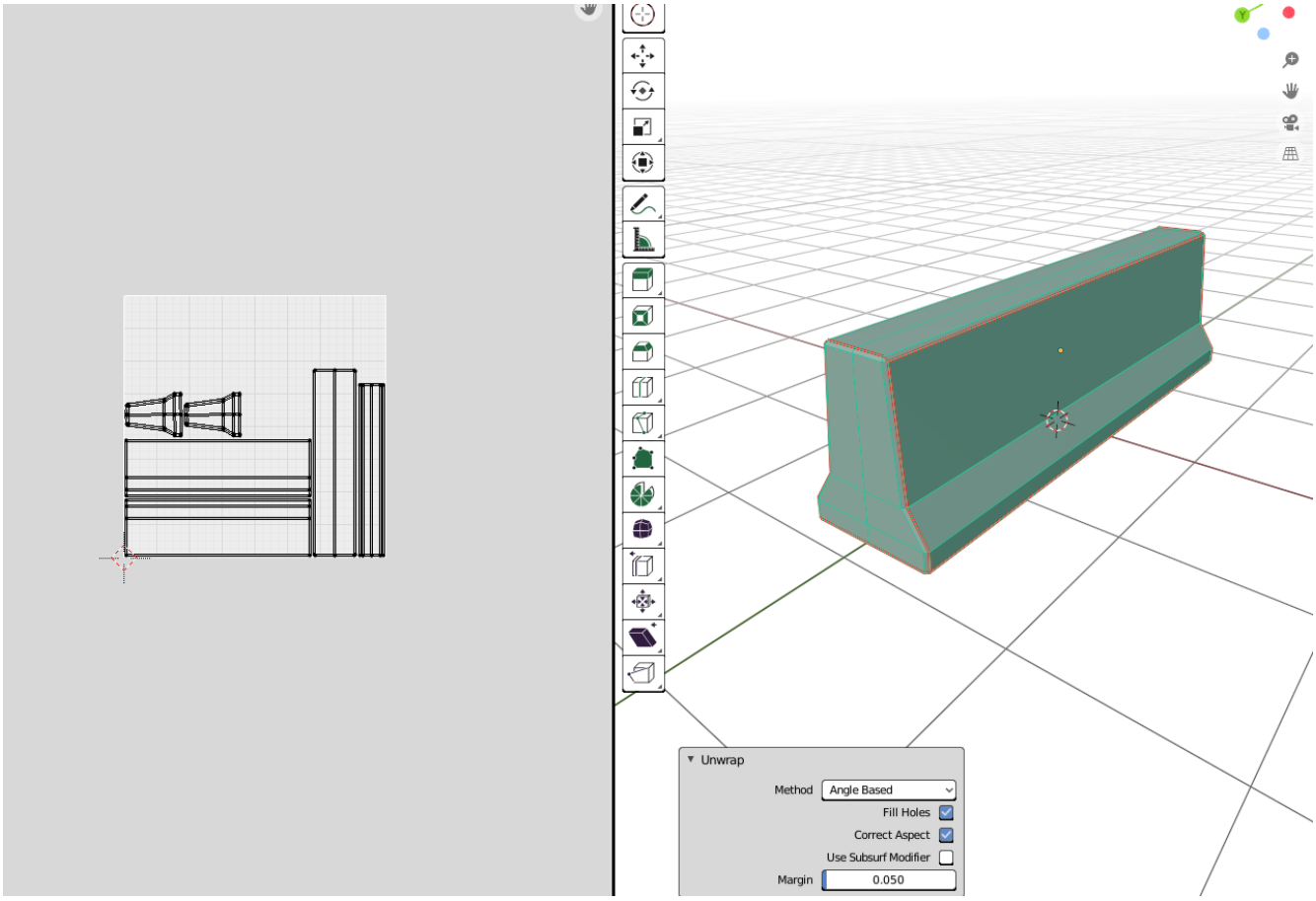
- In the EDIT MODE window select the UV tab , or press `U` and then choose UNWRAP.

If you want the **EASY BUTTON** for Mark Seams, try this: menu:Select→Select Sharp Edges and then press `CTRL` + `E` and Mark Seam.

When unwrapping the kind of models we are making, it's usually a good choice to use the "conformal" method. It is a drop down option under unwrap.

If nothing happens in the left UV EDIT window, you probably didn't have everything selected.

- Now, before you do anything else, locate the UNWRAP tab that showed up at the bottom of the screen and adjust the Margin to be a value of 0.05 or 0.08. The default value 0.001 is just too small for our needs. This will give a greater separation between te generated UV Islands. we will want to move them around.



- Give your model a new material. In the `PROPERTIES` window on the right side, locate the sphere with a grid inside it near the bottom of the Properties window. That is the material panel.

Blender Materials

To create a material in Blender, you can use the Material tab in the Properties panel. In the Material tab, you can specify the material's color and texture, as well as adjust its properties such as specular intensity, roughness, and transparency. You can also use the Shader editor to create more complex materials using nodes.

Once you have created a material, you can apply it to an object in your scene by selecting the object and then clicking the "Assign" button in the Material tab. The material will be applied to the surface of the object, and will be used to determine how the object appears when it is rendered.

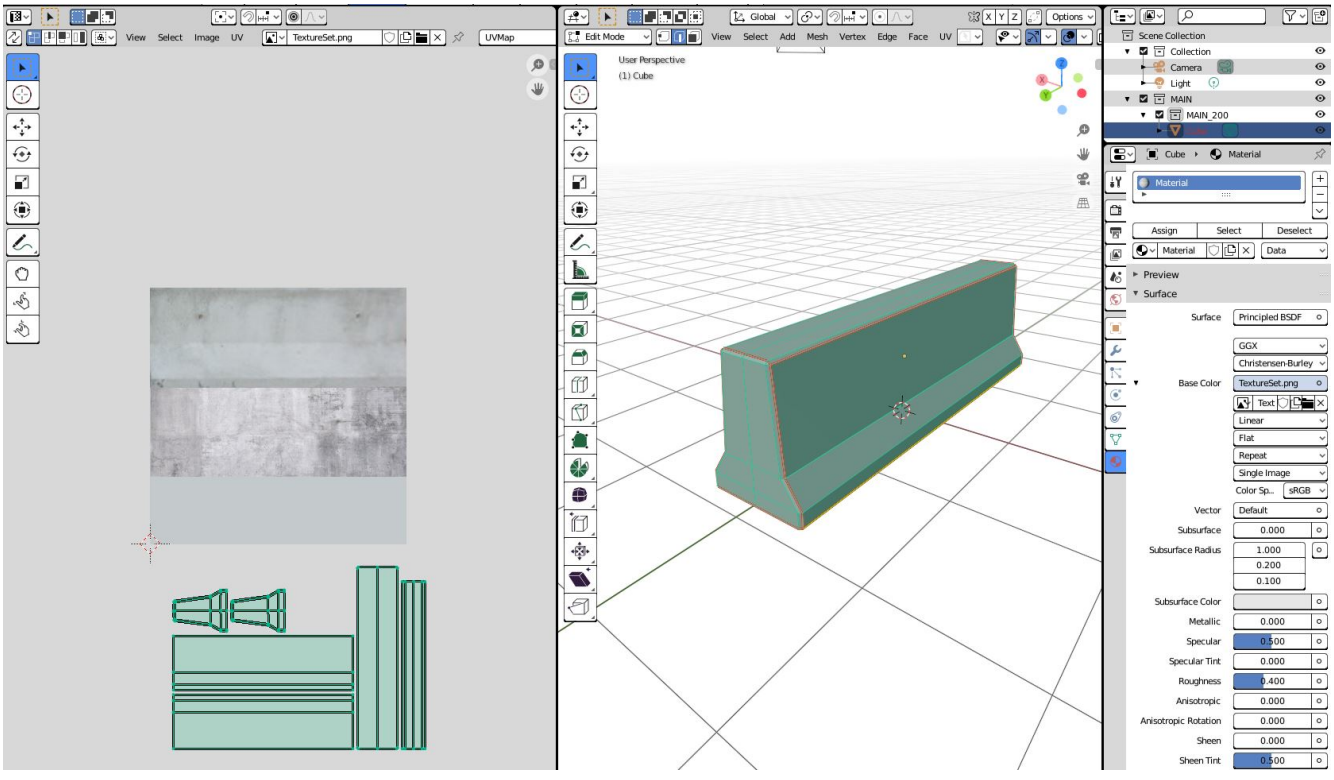
Materials are an important part of creating realistic and visually appealing 3D scenes in Blender, as they allow you to control the appearance of objects and surfaces in your scene

- Your object likely received a default empty material. Let's update it. Under `SURFACE PROPERTIES`, Click the small circle on the left side of the `BASE COLOR` field. You will get a list of options.
- Choose `Image Texture` and we will then locate the Concrete texture we will use. (You can create

your own or use the one I created for this) You should have some texture file ready in advance.

- Under BASE COLOR you now see +NEW and OPEN icons. Click OPEN and choose your existing texture file. The UV SQUARE background image in the UV Window should now display your image behind your UV ISLANDS. If you choose NEW, you will have a blank image assigned to the material, which in this case is not helpful.

The GITHUB page for this document has a DEMO1.ZIP file that contains the concrete texture I used.



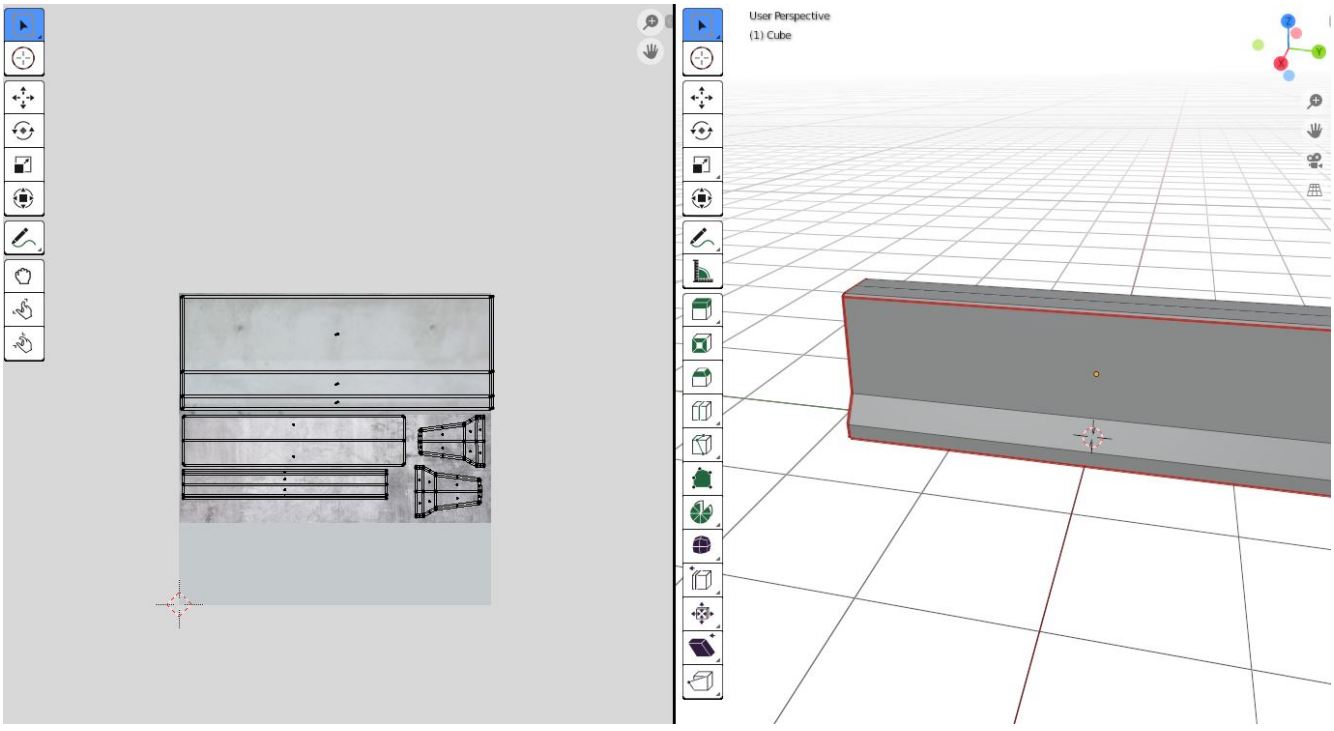
- The task now is to arrange the UV ISLANDS (Using the standard tools G X, G Y keys as well as resizing with S and rotate with R) You can temporarily shift islands outside the 1x1 UV SQUARE, but by the end of this process, all of the islands will need to be back within the 1x1 texture space.

When editing UV Coordinates, the X axis is left-right and the Y axis is up-down.

- Because we added some space between islands, they should be much easier to grab, rotate and place on the background. I recommend moving ALL UV islands outside the space and moving them back in 1 at a time. If you enable the Double Arrow icon in the UV WORKSPACE, updates will be reflected in both workspaces so you can see what you are doing in real-time.
- You can select faces in the EDIT window to isolate them in the UV window or use the L key to select linked faces.

- In this specific case, we will overlap the Left and the Right Side on the same texture space. When appearance is not critical, this works out just fine. Unlike 3DC, there is no "paint" with defined UV coordinates option.

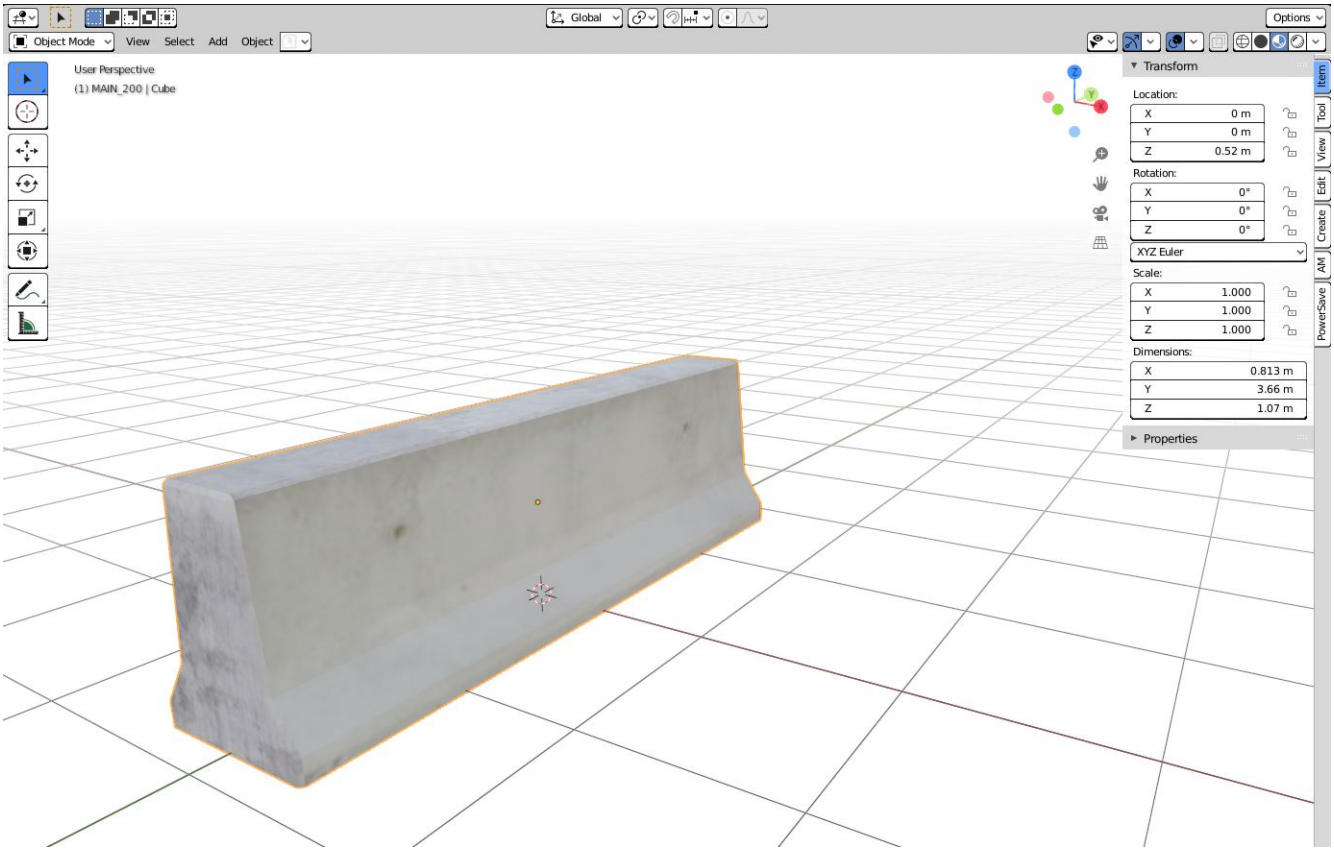
What I ended up with is shown below. Yours will likely be different based on what you marked as seams and how you laid out the islands. Again, its not critical with this type of model.



- Now we can return to the main modeling workspace and then change the viewport shader to Material Preview mode.

MATERIAL PREVIEW image::images/lookdev.jpg[]

We now see the object with the material applied.



This model is now ready for the MSTS exporter.

The MSTS Materials shader properties provided by the exporter follow the standard material options available in MSTS:

- Solid - A material that is using an opaque texture only.
- Trans - A material that is using a transparent texture only.
- Alpha - A material that is using a semi-transparent texture only.
- Specularity - A lighting highlight effect that gives the illusion of shine.
- Gloss - A material that is using a glossmap, which is an artificially created reflection.
- Cruciform - A tree shape that requires it's own material so that it can prioritize with the terrain and the surrounding objects.
- FullBright - A material usually assigned to the inside of trains so they never get dark.
- HalfBright - The same as full bright but at half the intensity.
- Dark Shade - The opposite to full bright, it reduces the overall intensity of light.

In `OBJECT MODE`, under the Materials property panel, scroll down to the bottom to where the MSTS exporter section is, titled: `MSTS Materials`.

1. Update the BaseColorFilePath (using the Folder Button) to select the texture of the highlighted object we have been working on.
2. Now switch to the SHADING window using the top menu bar. Note: that the NODES window now has a UVMap node and the text field says "UVMap". This is needed for the MSTS S file export.
3. With this verified, you can now use the MSTS exporter.
    - Choose **FILE › EXPORT › OPENRAILS/MSTS(s)** file, and choose where to save. Use the browser to select your project's \final\ folder and modify the filename as needed... I chose barrier.s for example. The bottom of the screen will say "Finished OK" when done.

The exporter will ONLY create the .S file and by default assume your texture is .ace. Converting your TGA, .JPG or .PNG textures to .ACE (or .DDS) remains for you do as you will need to do this manually outside of Blender.

For a sanity check... edit your exported .S file with a unicode editor to make sure your texture reference is correct.

When adding lighting/illumination effects to your model, you will need to separate the object that is the source of illumination from the rest of the model since the lighting effects are applied to the whole object. They can share the same Image texture but not the **same** material. This is important. You will need to create a new material with the same image texture and THAT is the one that you set to full-bright to be illuminated. Otherwise, the whole object will emit light, which is seldom desirable unless it matches your modeling intentions.

Version 4.4 and newer of the MSTS/Open Rails exporter includes a button to define the textures used in the .S file as .DDS instead of .ACE. Also, if Open Rails does not find an ACE file, it will attempt to use a DDS file of the same name.

```
images ( 1
       image ( TextureSet.dds )
  )
```

ABOUT DDS SUPPORT

With a layered master texture file, such as a PSD, you can keep the editable source file intact and export separate runtime textures from it. For Blender, export a PNG or TGA copy for material setup. For Open Rails, export a DDS copy for the final simulator texture.

Also note that even if an S file references a `.ace`, Open Rails will attempt to find a `.dds` with the same name, and if found, it will use the `.dds` instead of the `.ace` file. This is a nice feature. To me, relying on it does seem to be a bit lazy so it's best to always specify what your intended format is.

Again, the Exporter script will **only** create the model `.S` file. You still need to create all the final textures and the ENG or WAG or similar related files that define your model to the simulator.

The final step is to copy the contents of the project's `FINAL` folder into the Open Rails `trains` content folder for testing.

Creating Level Of Detail Distance levels

Once you have your main model created, you should consider creating Level of Detail or "LOD" entries. These are easy to add or change by editing LOD collection name, eg create `MAIN_0500` or rename `MAIN_0700` to `MAIN_0500`. You add content to a new LOD level collections by dragging parts around or using the `M` key to assign collections. With collections, all your LOD assignments show in the outline panel and you can hide or un-hide whole collections as needed since your LOD assignments are also clearly shown in the object's 'Collection' panel. Making use of the filter options and checkboxes of the Outliner helps to focus your work.

Blender WILL rename your parts when you add them to a new LOD, but the MSTS exporter is looking at the COLLECTION and BASENAMES of the objects, like WHEELS, which means you won't need to rename entries that suddenly have 001 appended to their name.

You can select a group of objects in the Outliner by using the `SHIFT-LMB` to copy multiple items at once to a new COLLECTION. It would be good to experiment with the Outliner filter options so you are aware of what is available.

Guidelines for creating LOD versions

Level of Detail (LOD) versions of a base model are determined by how much detail is visible based on the distance from the camera or player view. When your camera is very close to an object in Open Rails, then you want to see the maximum detail available. As your view of an object becomes far away, there is no reason to display the full detail of the object as much of it will be hidden or too small to see. The reduced-polygon models (LODs) improve graphics performance by reducing the amount of drawing that needs to be performed by the graphics card. For example, your main model might have 2000 polygons, but at 750 meters, you might only need to display 200 polygons and at 3000 meters, you might only need to display 50 or so polygons to get the same overall effect.

You probably won't need much more than 2 or 3 LOD options to get good results.

You define your intentions for each level of detail in your outliner. As explained earlier, you define your LODS using a naming convention under the MAIN collection where each lod distance setting is part of the sub-collection name, for example:

NAME	DEFINITION
MAIN	MAIN COLLECTION
MAIN_0500	0 - 500 meters
MAIN_1000	500 - 1000 meters
MAIN_2000	1000 - 2000 meters

```
MAIN
    MAIN_0500
         BODY (20,000 polys)
         BOGIE1
            WHEELS11
            WHEELS12
         BOGIE2
            WHEELS21
            WHEELS22
         etc

    MAIN_1000
      BODY (3000 polys)

    MAIN_2000
      BODY(200 polys)
```

You can use the DECIMATE operation in Blender to quickly reduce the polycount of a shape while maintaining most of the topology of the original shape. This can give you a head start on reducing what gets drawn on each LOD level in addition to removing the finer details that won't be seen. Remember, though, reducing draw calls is usually more important than reducing polygons.

ACE files are no longer mandatory for Open Rails and if you do not plan to make your work compatible with Microsoft Train Simulator, the DDS format is a good choice, provided you can get your PNG or TGA files converted to DDS format.

Pete Willard's ACE2DDS tools are still under development. The project can batch-convert ACE files to PNG files, and the version 1.2 start-over release changed the

> PNG to DDS workflow. The older Microsoft DirectX `tex2dds.exe` / `texconv` path is no longer used because of problems with the DDS files it generated. The current `png2dds.ps1` PowerShell script relies on ImageMagick to generate DDS output from PNG sources. You can find the tools at: https://github.com/pwillard/Ace2DDS

Next Steps

Now that we have covered the basics of creating a model in Blender, we can move on to preparing the solution for use within Open Rails.

1. Export your model as an `.S` file using the MSTS exporter.
2. Create all the textures and related files for your model. This means converting your textures to the proper format for Open Rails: DDS or ACE. For DDS, you can use PAINT.NET to save the PNG files as DDS
3. If you had collected all of your work into a folder named "final" in your work area, copy the contents of the project's `FINAL` folder into the `route` or `trains` folder you will use with Open Rails.
4. Start Open Rails or TSRE and review your model.

   For Scenery

   The FINAL folder would contain the `s` file, the DDS or ACE textures, and an SD file if special lighting or seasons are involved.

   - The .s (shape) and .sd (shape definition} files go in the shapes folder.
   - The .ace files (textures) go in the textures folder. Open Rails can also use .dds textures and they go in the textures folder.
   - If the .sd (shape definition) calls for night or snow or spring textures they will go in their respective folders within the textures folder. You don't need to have these textures if they aren't called for.

Day/Night texturing is handled by adjusting the texture properties in the .s file. Illuminated items would use an emission related texture setting, while other components would use the normal texture settings.

Traditionally, scenery is added to a route by adding a reference entry in the routes `REF` file. *<routename>.ref* This is also control how TSRE will display the scenery item by name.

For the Open Rails to know that a Scenery item exists, it would to be added to the existing route `REF` file. The `REF` file is a text file that contains a list of all the Scenery items that are available to the route. The `REF` file is located in the root of the `ROUTE` folder.

Sample REF file format, use this format to add items to the existing REF file:

```
Static (
```

```
    Filename ( "dealership.s" )
    Class ( "MyBuildings" )
    Description ( "Car Dealership" )
)
```

IN TSRE v8.004 and newer, the program will automatically list the contents of the routes `SHAPES` folder, sort of bypassing the need for REF file edits.. This is a nice feature.

For Trains

The FINAL folder would contain the `s` file, the DDS or ACE textures, and an ENG or WAG file.

Vehicle related content resides in the `trains` folder. The ENG or WAG, Shape and Texture files reside in the same folder unless making use of Open Rails's `INCLUDE` feature. Rolling stock no longer needs `SD` files in Open Rails.

Higher resolution models should have more LOD levels than lower resolution models. Building the LODS is best done just before the model is exported and before separate pieces of the model are merged together, allowing you to isolate sections that will still be visible at a distance. You really should save object merges until the very end.

Surely you can export test versions of the model before any merging of objects to see if the model is working as expected. The goal in the end though is to simplify the final hierarchy in the model.

While Blender does have a method of saving incremental edits and numbered saves, you will probably be better served if you make use of the "SAVE AS" option and create your own WIP VERSION saves to numbered files... (IE; `ProjectA_01.blend`, `ProjectA_02.blend`, etc). When my project working folder gets crowded, I usually move all of the opder saves into a folder named "OLDER".

Initially, you will save the main model as the top Level of Detail version or LOD0. You would then Clone this into a new collection, following the LOD NAMING convention of the MSTS/OR Exporter. You would then hide the contents of the LOD0 collection and work on the cloned version, renaming the parts to match the new LOD LEVEL. You would then start removing or simplifying objects that would not be seen at the new LOD distance. IE; Interiors that won't be seen are a good candidate at this stage. Gladhands, stirrups and any indide facing polygons are candidates as well.

On a 3rd LOD setting, you might want to just remake shapes from cube and cylinder primitives to keep the polycount low.

Workflow

Everyone has a way of working creative projects that suits their comfort level. The sequence of steps you become comfortable with becomes your workflow. This is based on the tools you use, the processes you trust, and any scripts you write or acquire to help with your projects.

As a quick discussion of workflow, the author will share their workflow as a reference.

Once you start a project, it will go through many revisions as you make progress on it. Save a fresh numbered copy after milestones are reached or when decisions are made about how best to proceed. You can fall back to older revisions if the chosen fork does not work out.

Once the model is underway, the initial stages of texturing begin. One way to start is to generate a blank PNG file at the size you need in the UV Editing tab, then save it. In the Materials tab, create a new BSDF material with an Image Texture node and use that newly created PNG file for the texture. It will still be default black. Assign that material to the model without worrying about the current UV mapping yet. Back in the UV Editing tab, map your UVs using the method that makes the most sense for the object. With the mapping done, use Export UV Layout to create an outline `.png` file that you can import into a program like Affinity. From there, you can map proper texture imagery to the UV map outlines you exported.

With Affinity, when importing the UV map layout `.png` file, use the Place option and paste anywhere on the screen. Then, in the Transform panel, set the location to X=0 and Y=0. The placement will be aligned correctly.

Create an export folder where you assemble all of the required files for Open Rails. This is where you export the `.S` file from Blender, and it is also where you place the `.DDS` files, `.wag` or `.eng` files, and any other items needed by Open Rails to make the content usable.

Since you are likely using PNG files for textures inside Blender, you need a conversion process from PNG to ACE or PNG to DDS. DDS is recommended because the conversion is simpler and is not based on a proprietary format. A script was developed to handle this process for a folder or an individual file. The script can be found in the `SCRIPTS` folder of the GitHub project.

In the final stages of the project, generate the Ambient Occlusion map. Refer to the ambient occlusion section for details. When the AO bake image has been generated, place the AO layer into Affinity at location 0,0. Put the layer at the top of the stack, set opacity to around 30%, and change the layer mode from "Normal" to "Multiply".

With this bake layer added, export to PNG again, convert to DDS, and place the result in the export folder. Test the content at this point using the TSRE shape viewer option.

Look for mistakes or mapping issues and repeat the process until pleased with the results.

The TSRE shapeviewer can read `WAG` files, so if your model relies on FreightAnims(), you will be able to see them by specifying the `WAG` file to the viewer.

Building a Library of Reusable Parts

This section mainly applies to users who still use a version of Blender prior to version 3.0.

Blender 3.0 and newer include the Asset Library / Asset Browser workflow, which is a better way to store and reuse pre-made objects, textures, bogies, couplers, and other common parts across projects. If you are using a modern version of Blender, prefer the Asset Library workflow instead of the older manual library method described below.

It is common to see regular parts used multiple times in a single design. Handgrips, railings, wheels, trucks etc are often regularly available parts purchased by Rail Vehicle companies as commodity items. In the same way, we should not have to keep making the same parts over and over again.

One way to accomplish the re-use of the common parts is to `duplicate` one with `SHIFT` + `D`. A selected part will be duplicated and ready for a location transform to move it to a new location. This is easier to manage after you have completed the UV-Unwrapping of the object. All duplicated parts will share the same UV MAP locations so if you need them to have different mapping, its a matter of shifting them around in the UV Editor.

You should not use `CTRL` + `C` and `CTRL` + `V` to duplicate objects as an alternative to `SHIFT` + `D` because the standard copy/paste operation will make new copies of everything, including Materials. So your MAIN texture would become MAIN001, MAIN002 etc on each copy/paste operation.

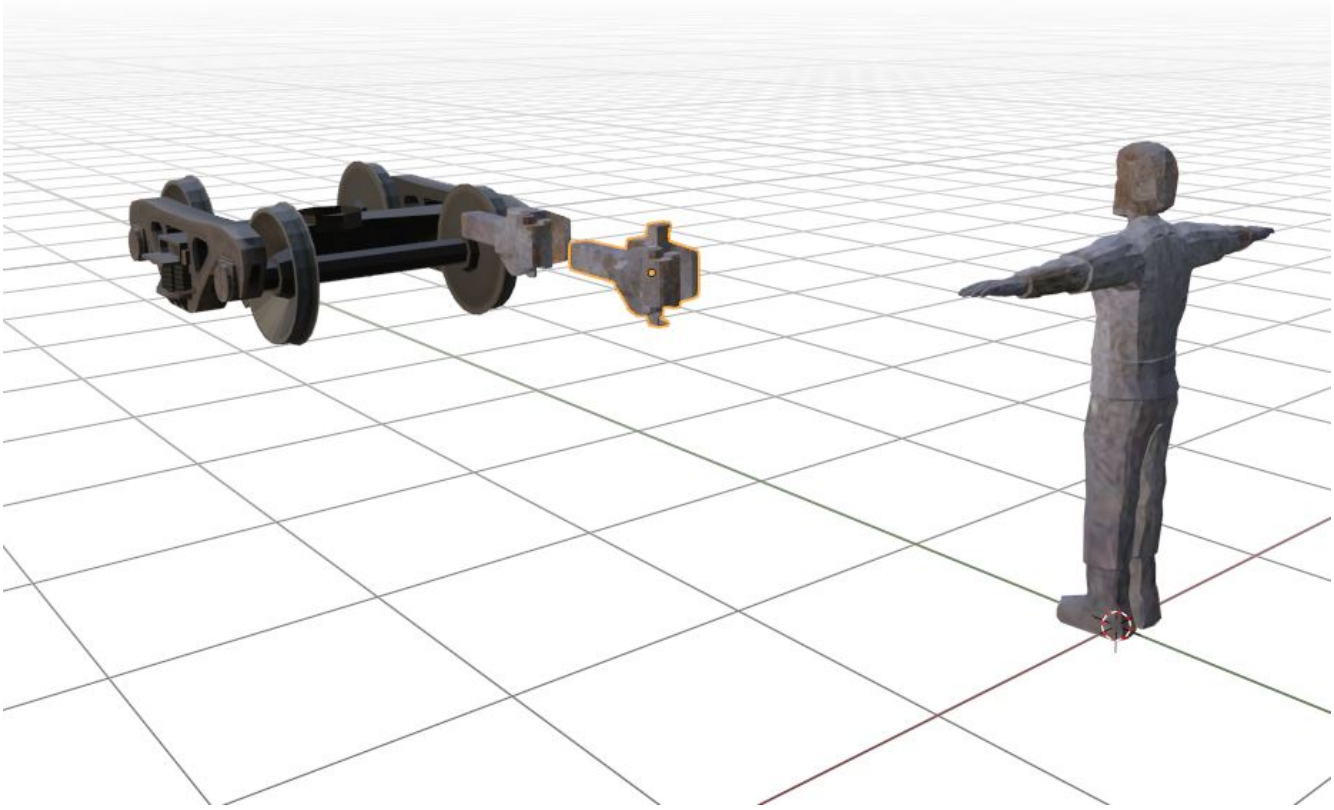
A Brute-Force Library Solution

This is an optional legacy workaround for older Blender versions that do not have Asset Library support. It is not the recommended workflow for modern Blender versions. If your version of Blender includes the Asset Browser, use that instead.

A simple way to create a collection of reusable parts in older Blender versions is to **COPY** and **PASTE** specific parts from your current file into a separate library `.blend` file. This method requires two copies of Blender running at the same time.

- Save your current project, just in case.
- Highlight the object you want to export to a library file, but first make sure you set the origin to object geometry. **View › OBJECT MODE › OBJECT › SET ORIGIN › ORIGIN to GEOMETRY**
- To copy, press `CTRL` + `C`. The object will be copied to a system buffer.
- Open a *fresh instance of Blender* so you have 2 blender sessions running, clear out all unwanted default items from your new scene and press `CTRL` + `V`

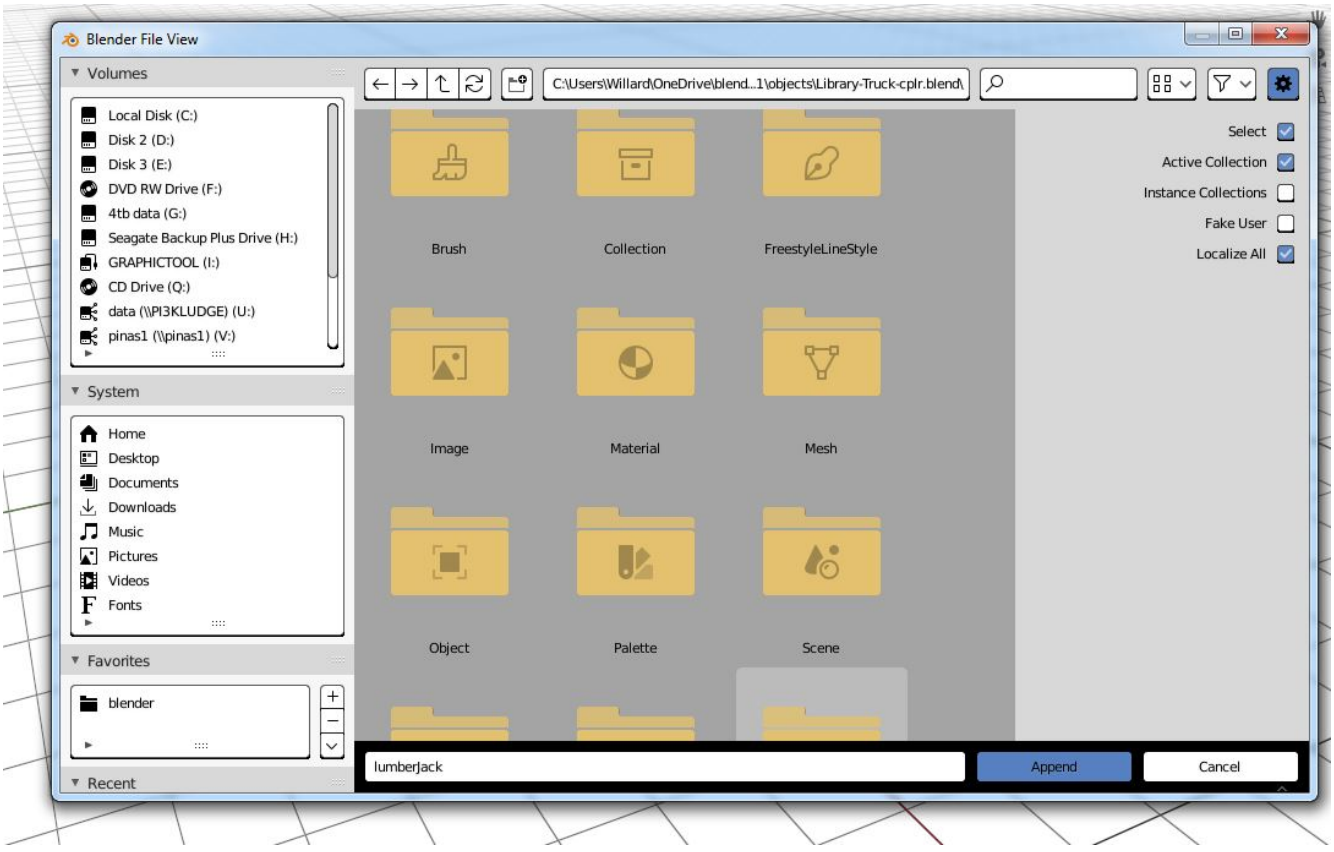
- The copied object(s) from your first Blender session will be placed in your new blend file. Copy as many objects into this file as you need. It is best to avoid overlapping them.



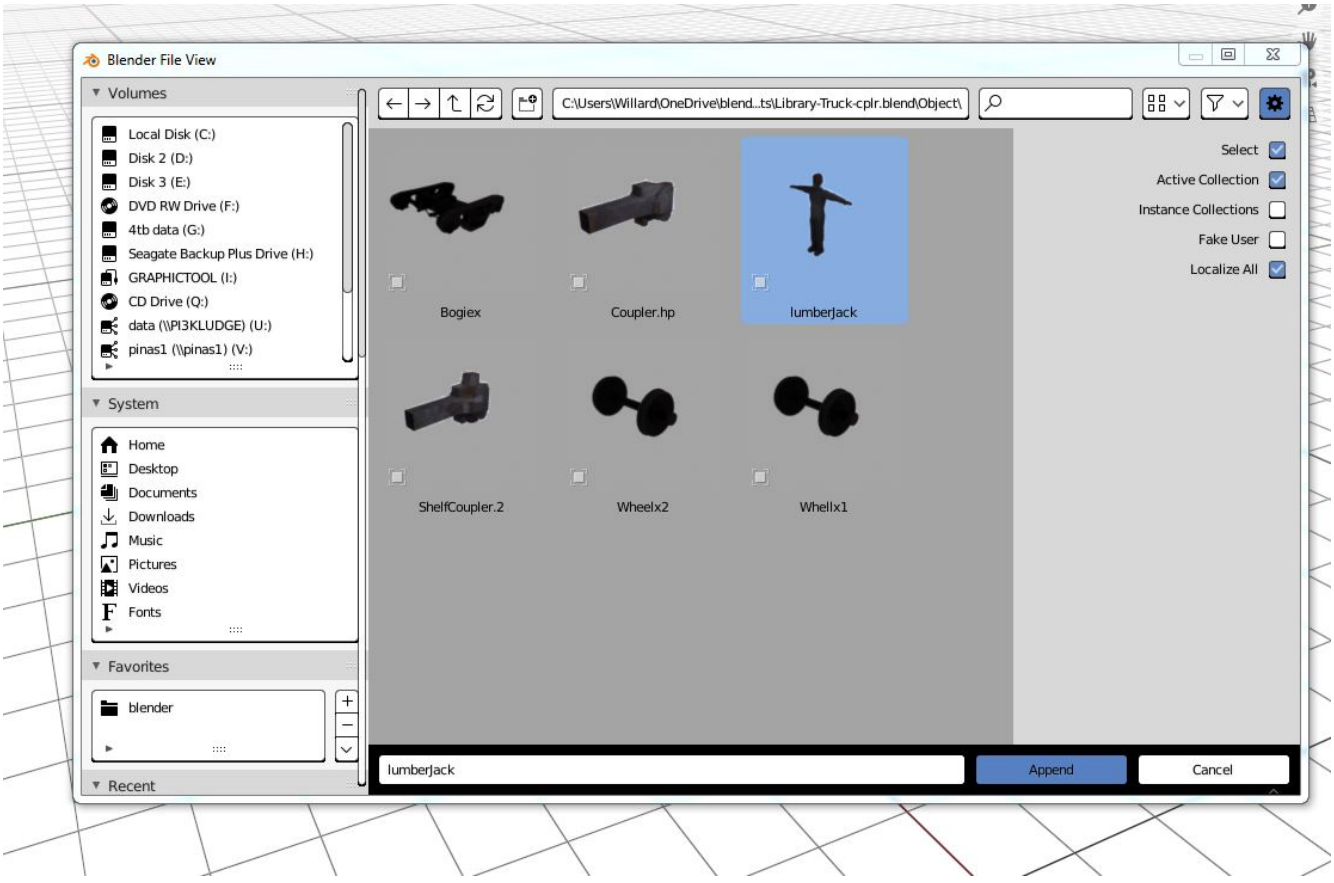
- Save it now with a name that refers to the new object library you are making. Example: `Library_Freight.blend` (Don't skip this step)
- Now the tricky bit, open a **NEW GENERAL Blender FILE**
- Choose **FILE › DATA PREVIEWS › BATCH GENERATE PREVIEWS** and choose the file name you used in the prior steps. Example: `Library_Freight.blend`

This will take a few moments and will eventually present you with a console screen when complete. By doing this, we are telling Blender to create Object Icons in the saved file.

- In the future, you just need to use **FILE › APPEND** option to insert the object into your current projects, but, the bonus is that you can select them using the `THUMBNAIL` view option in the `FILE APPEND` menu, so now you will know which part you are appending to your current `blend` file in a visual way.
- Choose append ans select `THUMBNAIL` view you will see file menu.
- Choose the library file you saved



- Choose OBJECT folder and you will see ICONS of the parts in the file. Select one and it will be appended to your current file.



When using this technique for an object library or multiple object libraries, you should to consider your file structure because when you build up a repository of `.blend` files it can become confusing when searching for a specific item later on. Using a well planned and organized file structure is better than having files all over the place or all in a single bucket.

It might mean that there are some redundancies in your files in the long run, but consider populating the OBJECTS and TEXTURE folders of each project local to that project versus using a master folder for all projects. This way you can make folder-relative reference to files and you have the ability to move folder as well as share with others without breaking Texture file references, for example.

Saving in alternative export formats

You can also export selected parts as FBX or Collada (`.dae`) files. These formats can be imported later with **File › Import**, and they can also be opened in other applications. The main drawback is that they are no longer native `.blend` files, so some object, material, or texture data may be changed or lost when imported back into Blender.

For the curious,

A file structure that I often use for individual project folders is shown below:

```
<project>
<project>\final
<project>\mesh
<project>\objects
<project>\textures
<project>\tmp
<project>\reference
```

I would store any `local` object library collections created with the COPY/PASTE method under `\objects` folder. There is a handy tool available https://www.dcmembers.com/skwire/download/text-2-folders/ "Text2folders", that makes creating this consistent folder structure easy. Just replace the text "<projects>" with the name of your current project and pass this text file to the "Text2folders" application and it will create all the folders for you. It can even do sub-folders.

Keeping the Part Library and Texture files available under the current project structure is helpful if you share your modeling content with others, provided you have the available disk space to allow duplicates in multiple project folders. It also allows for small tweaks that are specific to a project.

If using the Asset Browser feature of Blender 3.0, you would use the **3D View › Object › Asset › Mark as Asset**. Then, Save the file in your ASSET folder, otherwise your asset edits won't be available to other files. This is why it might be better to create specific library collection .blend files to be stored in the asset folders.

Importing Existing Objects

Importing shapes from other 3D Software can be a bit tricky and there are numerous reasons why. I will try to outline a few basic steps and possible pitfalls that can be avoided.

- Imported shapes often get renamed with a S_ prefix when using Blender importers which might require you to rename your objects in the Outliner.
- When your source objects are in an Autodesk 3DS file format, you need to account for Blender version differences. Some Blender versions dropped direct 3DS support, while later versions restored it. If your Blender version cannot import 3DS directly, use Blender 2.79, another converter, or a current Blender version that supports 3DS import.

You can also run multiple versions of Blender side by side. If you need Blender 2.79 for a legacy conversion, import the .3DS file there, perform a Select-All, COPY, and then PASTE into your current Blender session.

Blender 3.6.x LTS and newer versions once again support importing 3DS files.

- UVMapping might survive an import operation… but material and texture selection probably didn't. You most likely need to manually assign materials and textures to your imported objects.
- The import step has also probably has assigned a new material names for each object and to be honest, it's easier to manage if you have all objects that use the same texture also use the same material (unless you need special alpha or reflective properties for something like metal or glass).
- You also will probably need to manually tell the S file exporter (under MSTS Materials) where to find the texture file under the BaseColorFilePath input field in addition to assigning the texture to the material.
- Lastly, from personal experience, if you are importing a DAE (Collada format) file from say 3DCanvas, you might encounter an issue with UVMAP assignment in SHADER EDITOR where you are not getting the right UV map assigned and you see no texture in the Material Preview mode (Commonly called LOOKDEV mode).

This is the second tab from the right in the viewport selections, by the way. Everything will LOOK right in the Material Preview, but exports will fail with a "Missing UVMAP in:" error when you try to

use the S file exporter. *See the next section for how to deal with this problem.*

Material Preview image::images/lookdev.jpg[]

Fixing UV MAP Assignment Issues on Imported Objects so they work with the MSTS exporter

Each mesh that you export must have a uv map named UVMap.

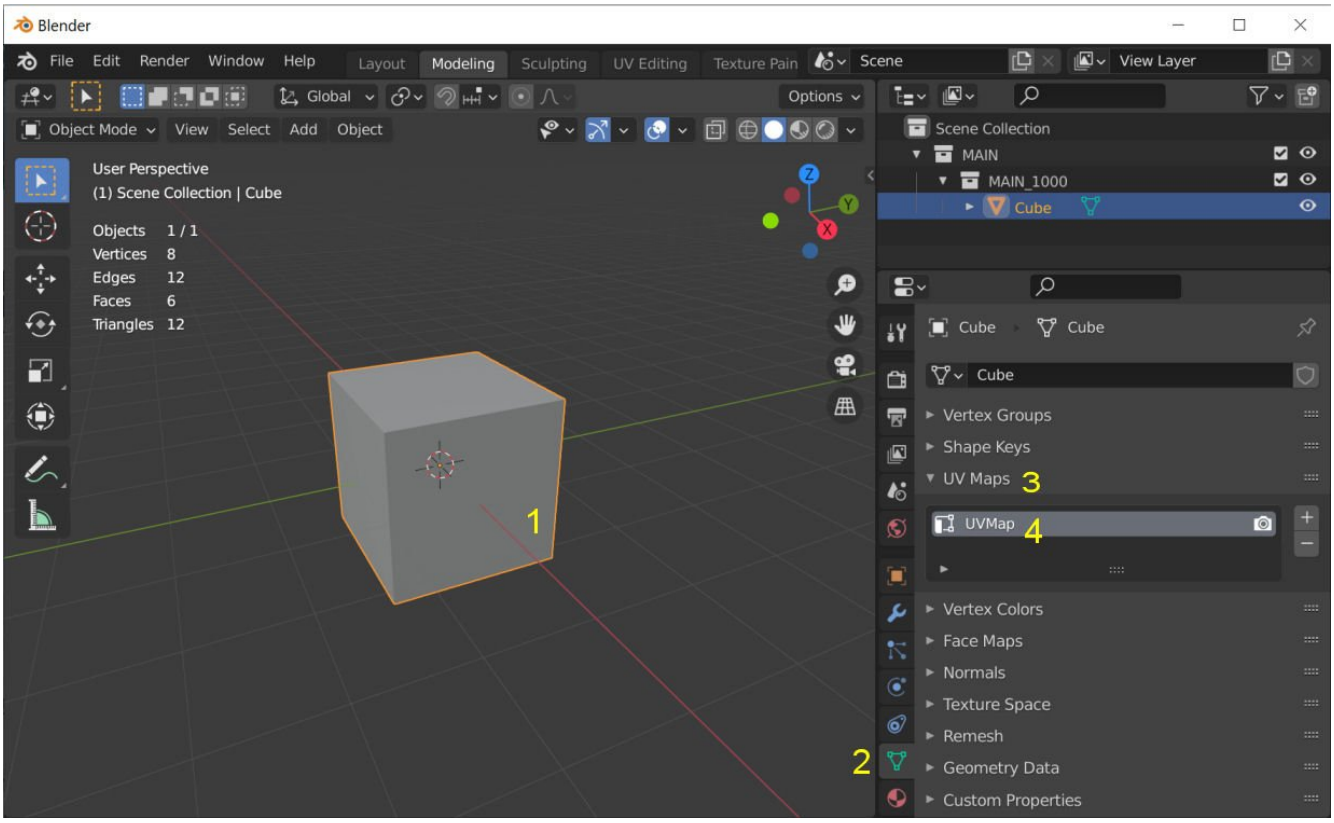
Since UVMap is the **default** name assigned by Blender for new UV MApping data, if you create everything from scratch, you won't encounter the problem when you export.

The mapping reference of UVMap is the only map that will get exported to an S file by the MSTS exporter. When using the DAE importer, for example, you might end up with a UVMap that is referenced using an object name from the time of importing.

There are some other cases where this could be an issue, for example:

- An advanced user may have multiple uv maps with other names assigned. (eg layered shaders, baking, etc)
- When importing a mesh from some other program it may come with different uv map names
- Some non-english versions of Blender may use a different default name.

You could check or try modify the name of a uv map like this:



1. `LMB` Click on your mesh object.
2. Select the `MESH PROPERTIES` tab.
3. Open the `UV Maps` panel.
4. Double click on a `map name` to change it to 'UVMap'

There is a tested script that will do this for you.

```
import bpy

for mesh in bpy.data.meshes :           # for every mesh in the .blend file
    if len( mesh.uv_layers ) == 0:      #   if it doesn't have any uv maps
        mesh.uv_layers.new()            #   create one, if it doesn't have one
                                        #   with the default name
    if not mesh.uv_layers.get('UVMap'):
        firstmap = mesh.uv_layers[0]    #     rename the first map
        firstmap.name = 'UVMap'         #     to the default name
```

To use this code:

- You would select the `Scripting` Tab and `LMB` + `CLICK` the NEW option to get a new edit window.
- You would copy and paste this code snippet into the text editor window.

- Last, you would press the "RUN" arrow icon in the file window menu to run it.

It will fix the UVMAP naming on all objects for you.

This code is also included in the add-on: https://github.com/pwillard/Blender_trainsimstools

Model Exercise #2

Intermediate Modeling

Key items to take away from exercise #2

- Interiors
- Glass
- Texture preparation
- Back Ground Images

Building the General Shape

Let's say we are modeling a building. I'm thinking… 2 stall engine house 40' x 80', and 20' tall with two 12'x18' garage doors and standard metal door with an office window.

I will use this for a background image. .Example 40 x 80 x 20 background reference image::images/40x80x20.jpg[]

This reference is low-resolution, but it is adequate for blocking out the overall proportions.

Importing the Background Images

Much of my previous content was created using Amabilis 3D Canvas modeling software. It was rather difficult to create and use good background images, so I seldom used this method. With Blender, the process is much easier and I highly recommend it.

You should pre-determine the center-point of each view and align them with the axis lines in Blender.

If you can find blueprints of at least the front/back and side view, you are all almost all set to use them as backgrounds.

If your blueprint is black and white, you might consider modifying it to be a negative image with the background as black and the convert the background to clear alpha.

In the "LAYOUT" workspace (in the top menu) in `OBJECT MODE` select SIDE VIEW (KEYPAD 3). Use the ADD → Image → Background menu tree to load in your side view image. Select FRONT VIEW (KEYPAD 1) and then ADD → Image → Background menu tree to load in your front view image.

By default in modern Blender versions, you may only see side or front background images when using the matching orthographic view. The images may disappear when you pan around your model. This behavior is controlled in the Data Properties of the background image object.

Setup

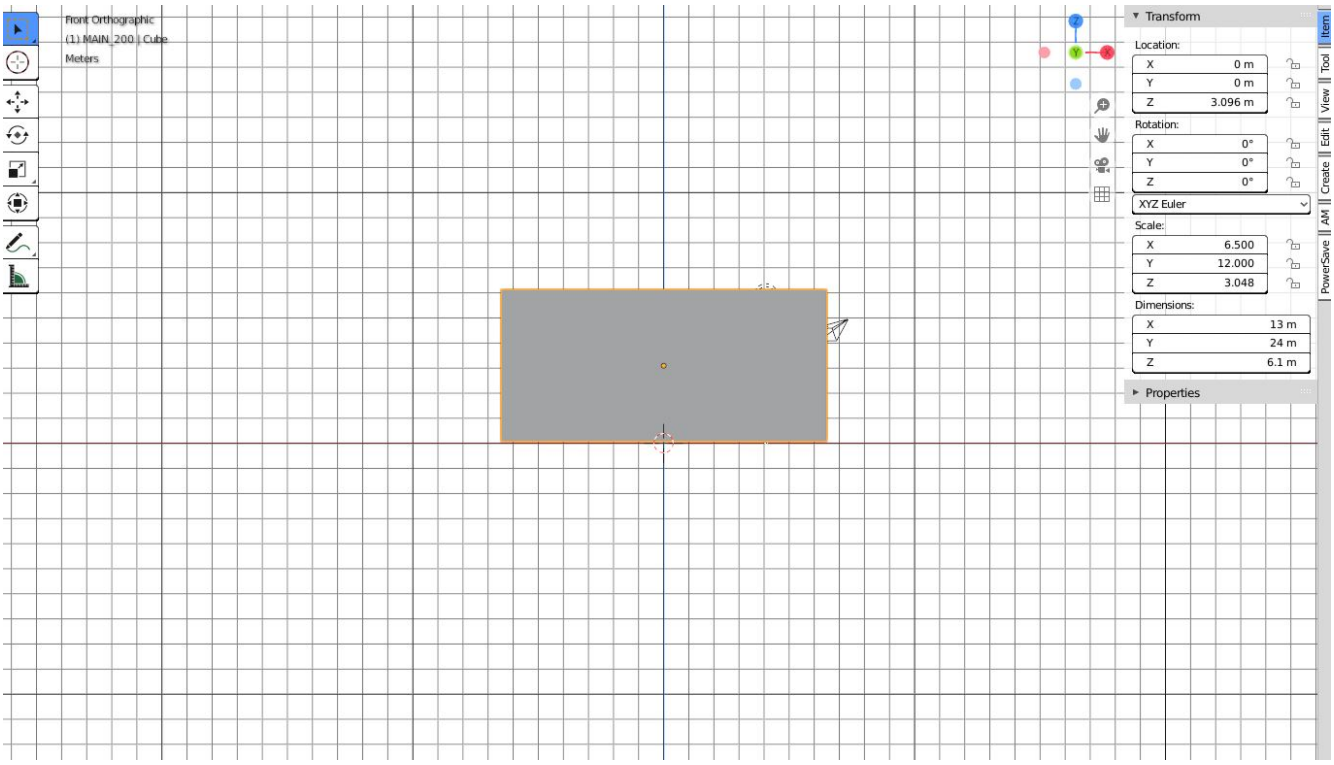
Use the file structure that was outlined earlier.

```
twostall
twostall\final
twostall\mesh
twostall\objects
twostall\textures
twostall\tmp
twostall\reference
```

- Add the Background image to `reference` folder
- Adjust the default cube Dimensions to be 40x80x20 (X=12.192, Y=24.384, Z=6.096). This will be

the basic shape.

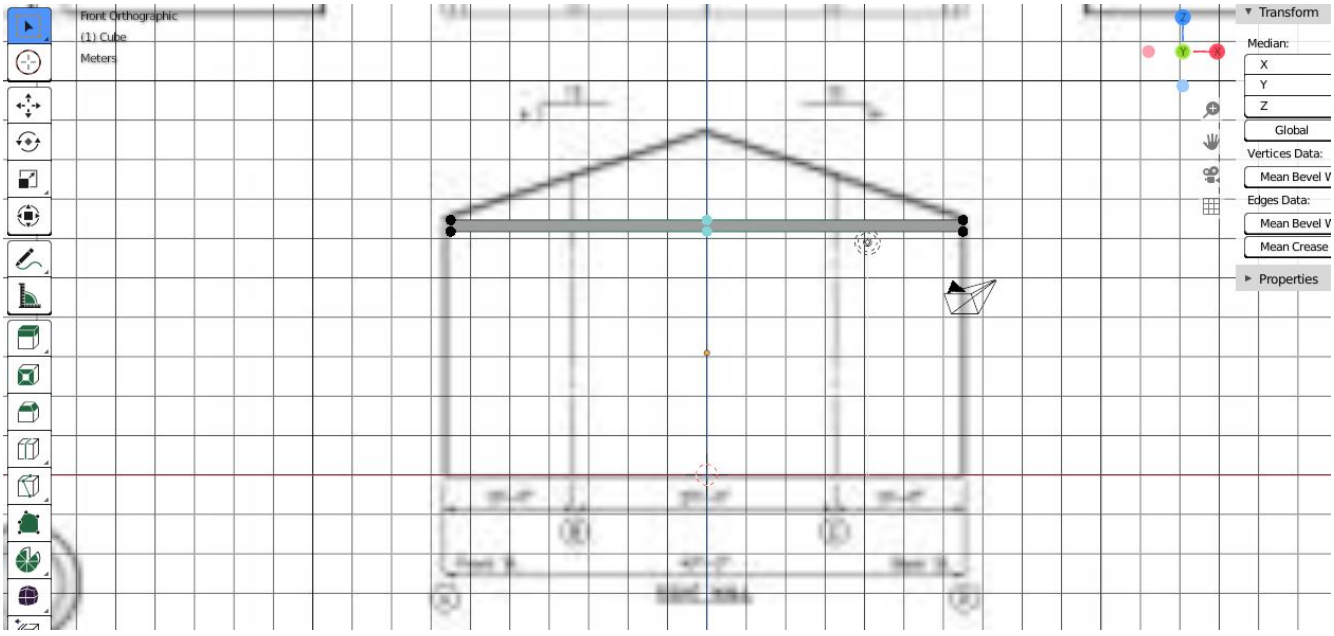
- Apply the scale with `OBJECT→APPLY→SCALE`



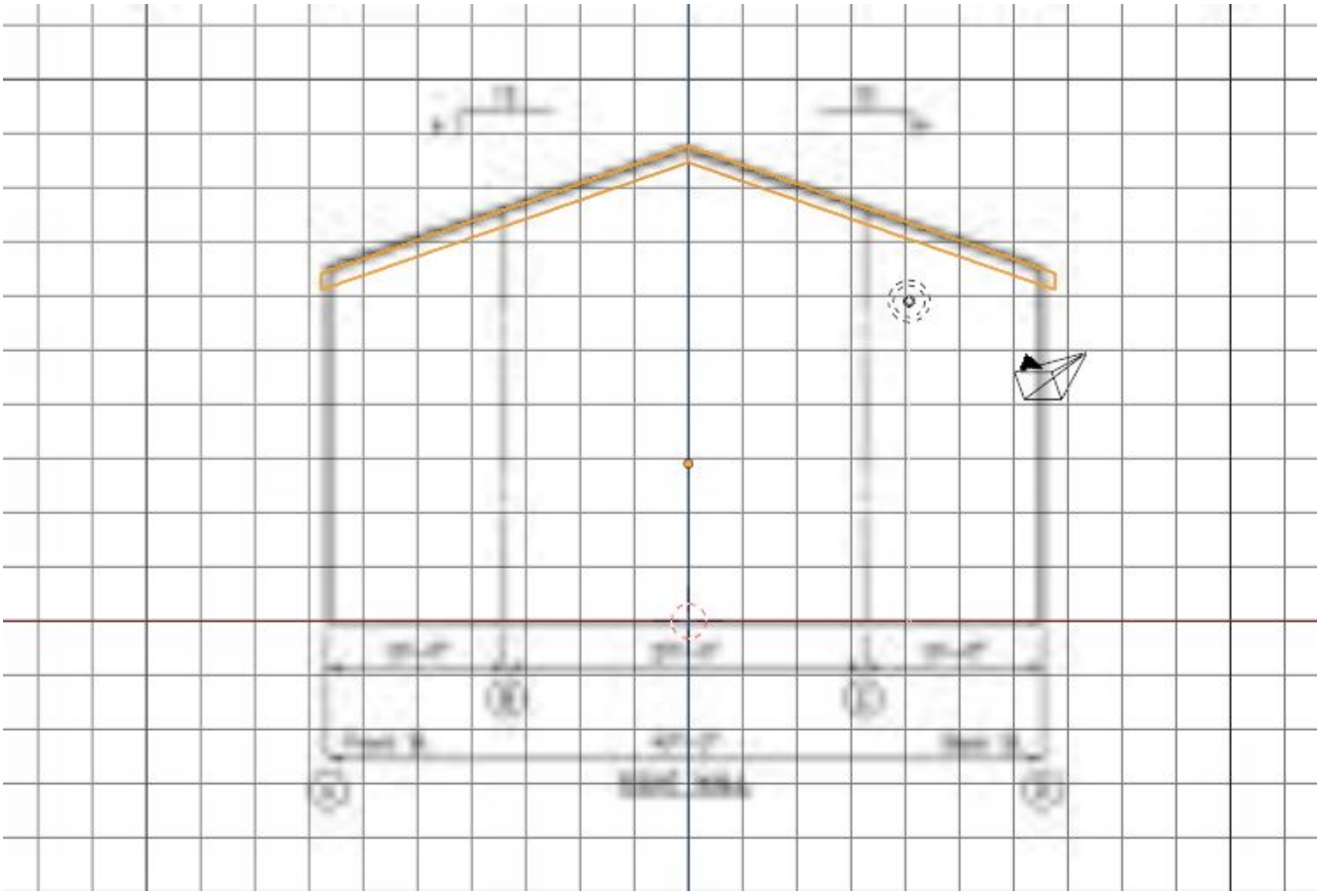
- Save the `twostall.blend` file under `twostall\mesh` folder.
- Switch to front view `KEYPAD 1` and select `ADD→IMAGE→BACKGROUND` and choose the image from `twostall\reference` we saved there earlier.
- Position and scale it to match the basic shape, but not the roof.

Now, it might seem intuitive to just use the basic outline we have now, insert a `loop cut` on the top and shift the top vertices upward to make the roof shape. Here is why that is the hard way. You won't have a roof overhang and all roof edges have an overhang. We will instead shift the current shape so it is only as tall as the roof overhang.

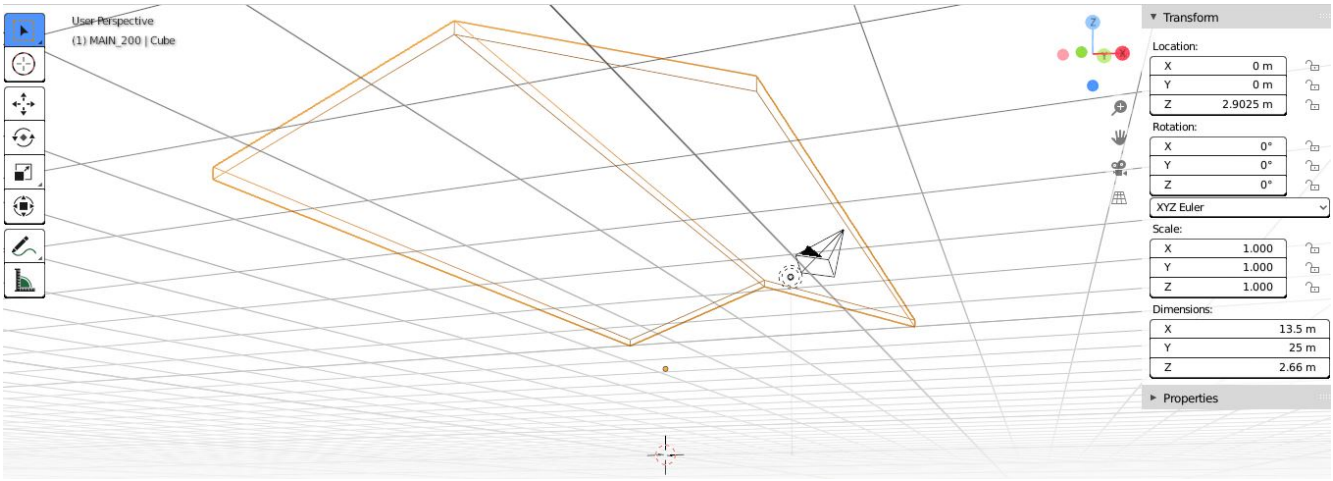
- Press `TAB` to reach `EDIT MODE` and `1` for vertex select and `Z` and select (WIREFRAME).
- Add a vertical `Loop Cut` `CTRL` + `R` in the center.
- `BOX` select the bottom vertices and shift `G` `Z` to be about the right height for the roof overhang.



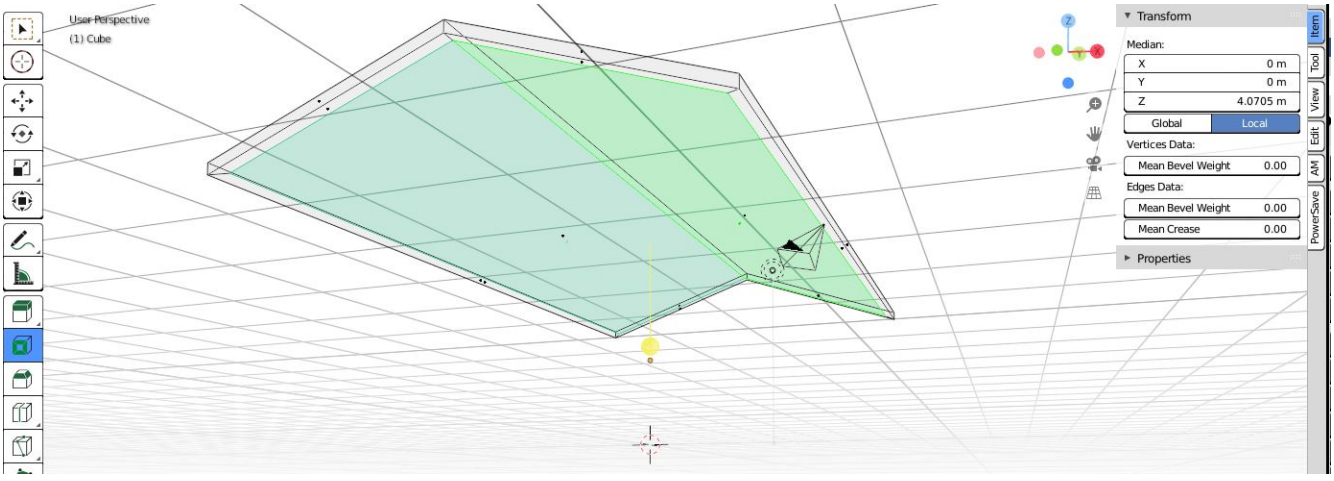
- Use BOX select to choose the central vertices and shift them upward to match the roof angle, G Z
- Switch to Face mode, 3



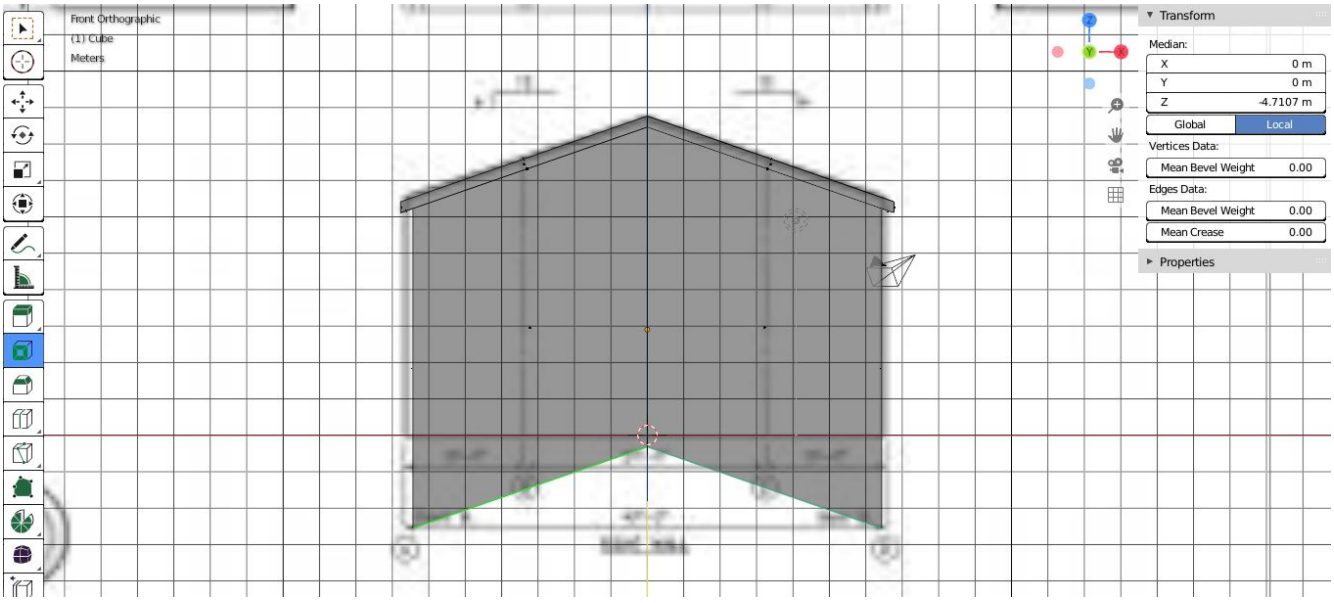
- Shift your view to be looking to the underside of the roof shape.



- Select the underside faces and use `I` to inset the selected faces to the amount of overhang that looks right.



- With these inset faces still selected switch to front view `KEYPAD 1` use the Extrude option `E` `Z` and drag the inset faces to ground level using the central vertices as a guide. Note: I used `solid view` mode for clarity.



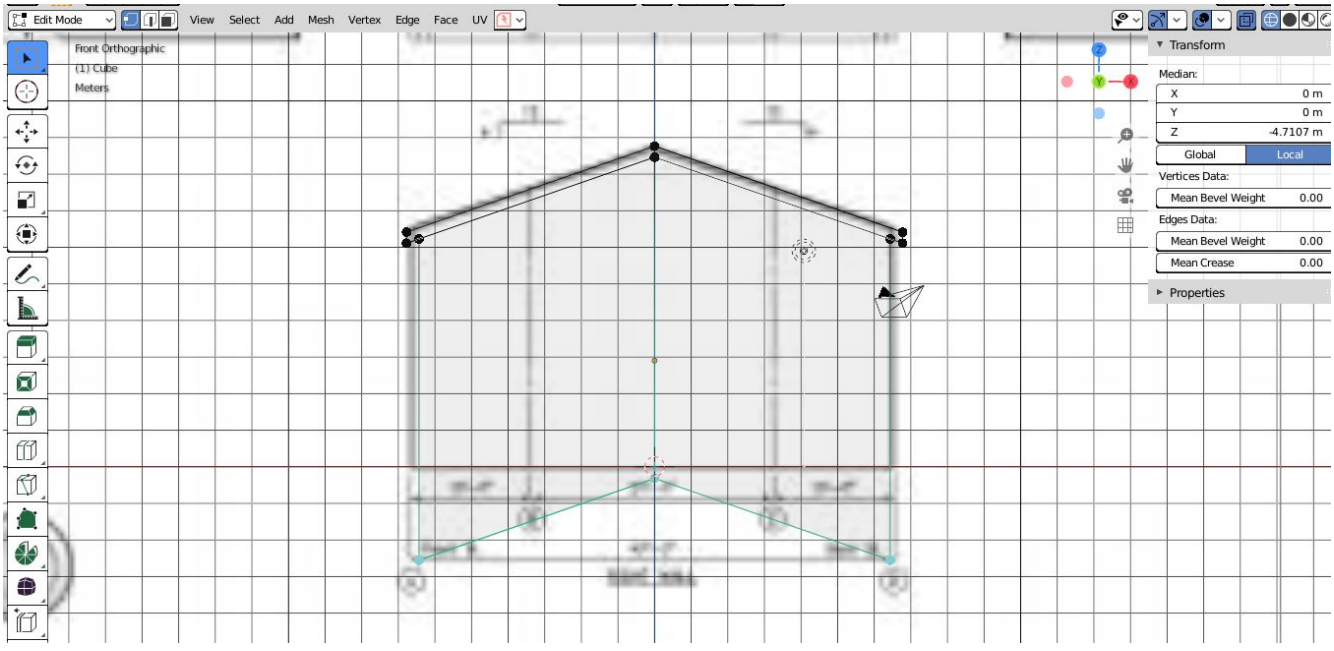


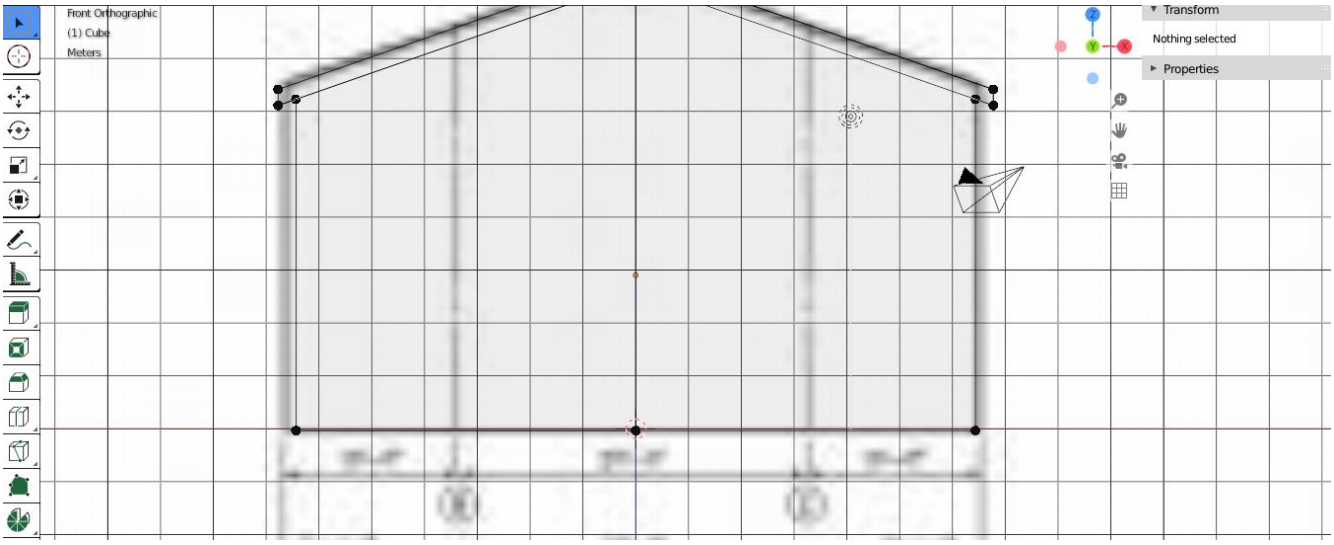
Figure 3. Extruded roof faces before flattening the bottom vertices

The temporary over-extension is subtle in the screenshot; focus on the lower edge before it is flattened in the next step.

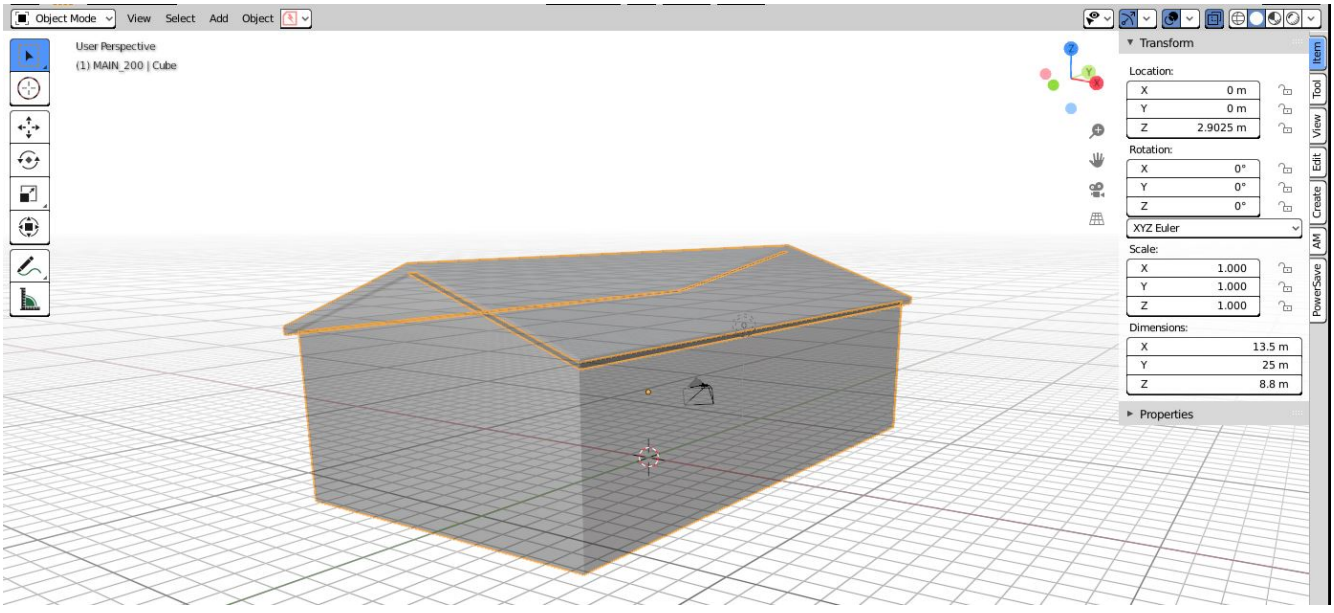
- `ALT` `Z` to toggle WIREFRAME mode on. Using `Wireframe/X-Ray mode`, switch to **vertex edit** mode with `1` and select all of the bottom vertices.
- Type: `S` `Z` `0` to flatten out the vertices on the bottom.

`S` `X`,`Y` or `Z` `0` is a nice magical incantation to remember to align vertices to a

specified axis

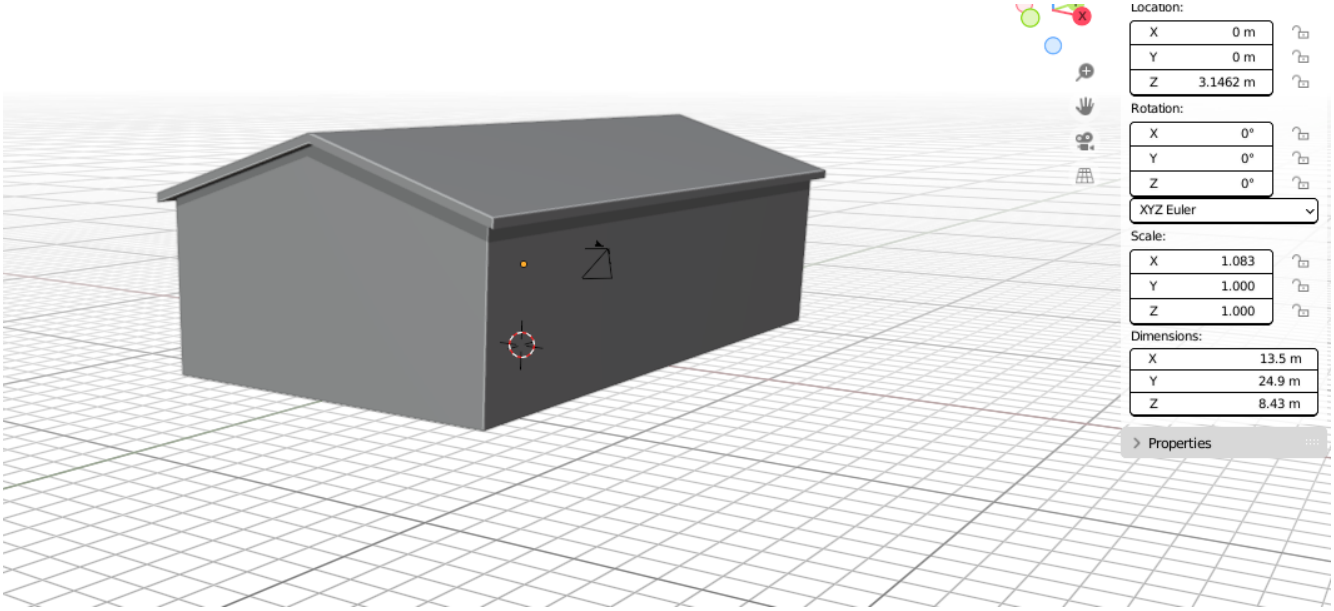


- While still in front view (`KEYPAD 1`), Shift all of the bottom vertices back to ground level with `G` `Z` as needed



- `TAB` back to `OBJECT MODE` and use Solid View, `Z` and select (SOLID), to view the result in greater detail.

Making the Interior



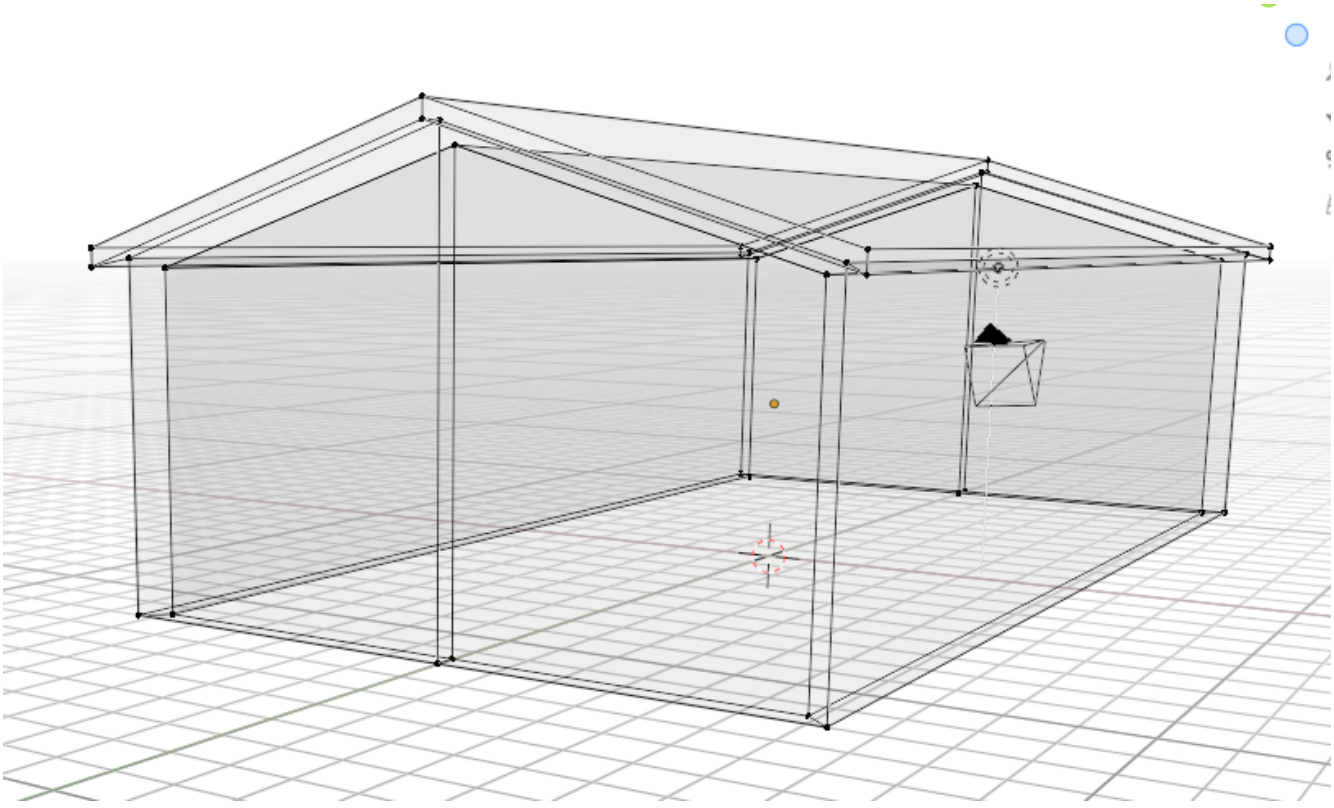
Now we need to give the building an interior.

- Shift your view to be below the model.
- Select the building and press `TAB` to enter edit mode (Unless you were already in edit mode)
- Choose X-RAY mode by clicking the X-RAY button on the top right or `ALT` `Z`
- Press `3` to use face selection mode and choose the bottom faces.
- Press `i` to Inset and adjust until you get a realistic wall thickness and press [ENTER] to accept.
- Press `keypad 1` for front view and then `E` `Z` to extrude upward into (but not through) the model.

Note: The top of the inside of the model will be flat like the floor.

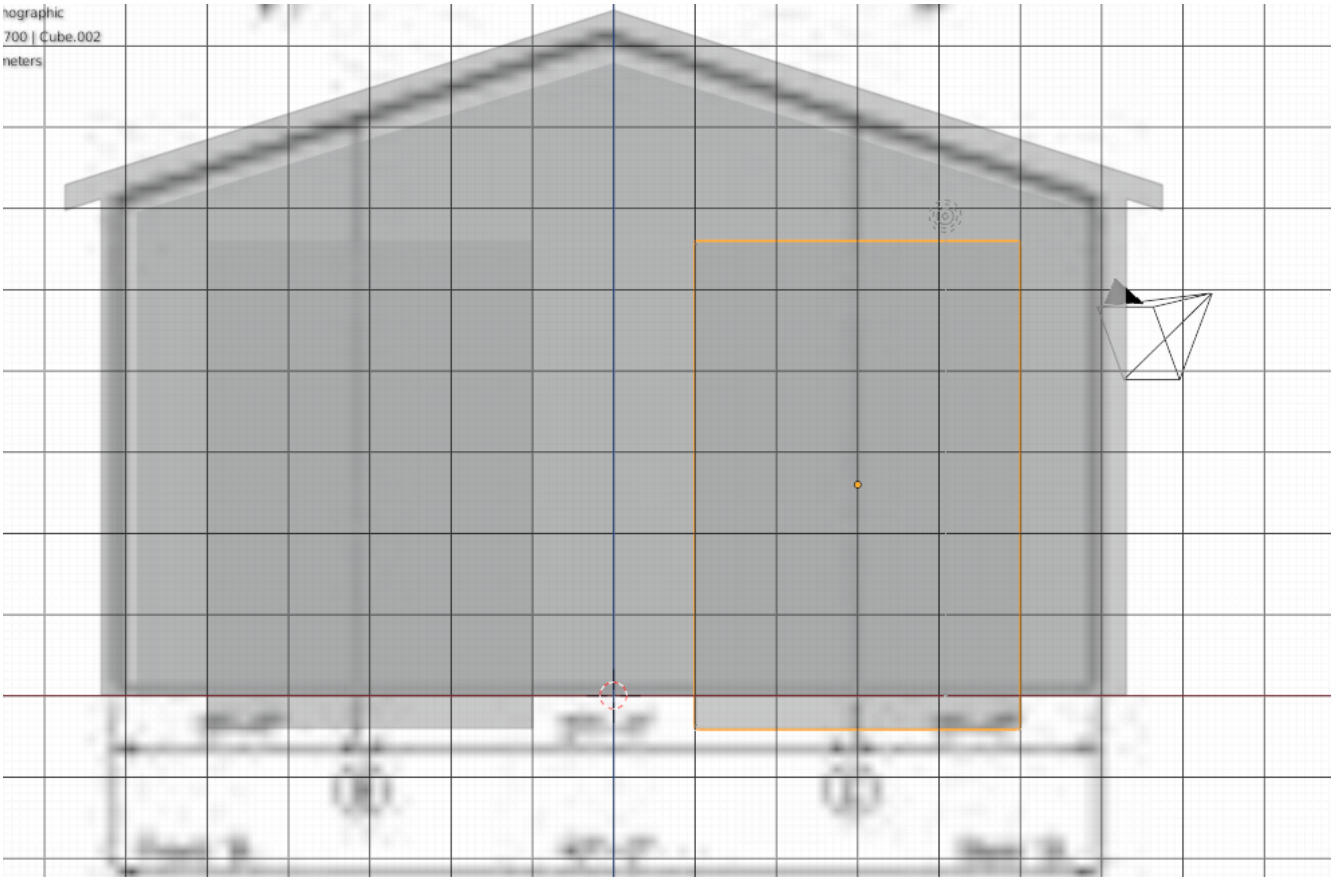
To make the roof match the top roofline and staying with the front view mode…

- Press `1` for vertex selection mode and then box select the middle vertices on top of the inside being cut out.
- Press `G` then `Z` and drag the vertices up to match the angle of the roofline.

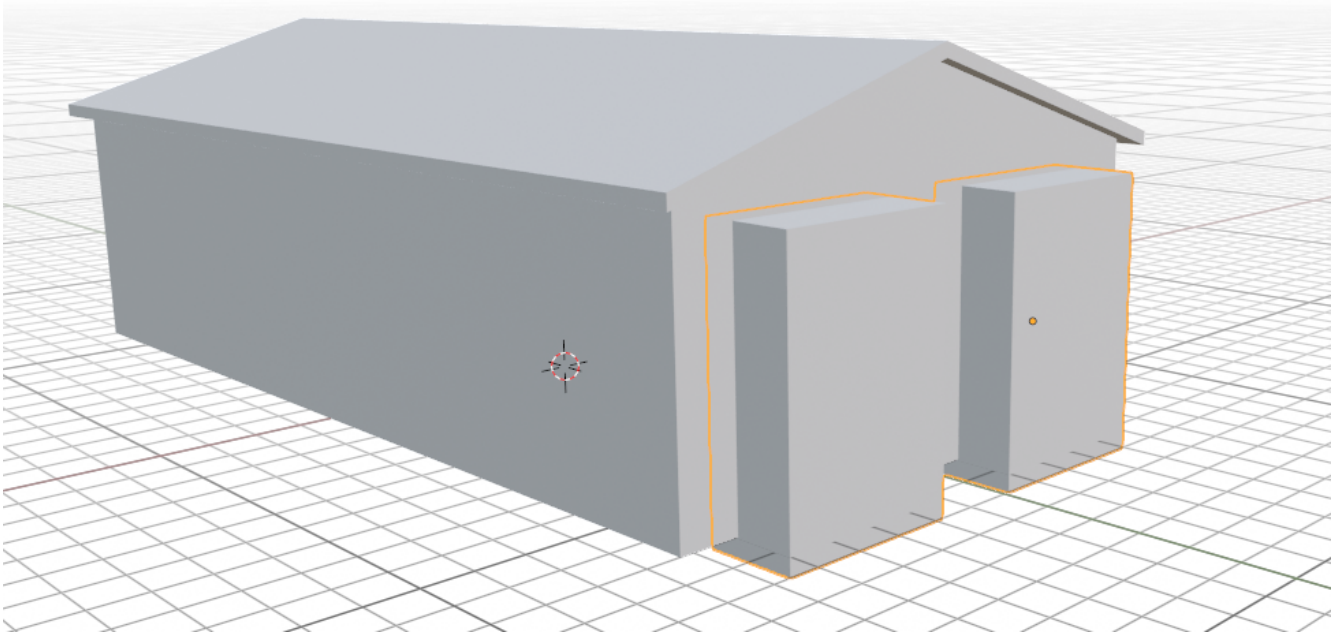


Making the Doors

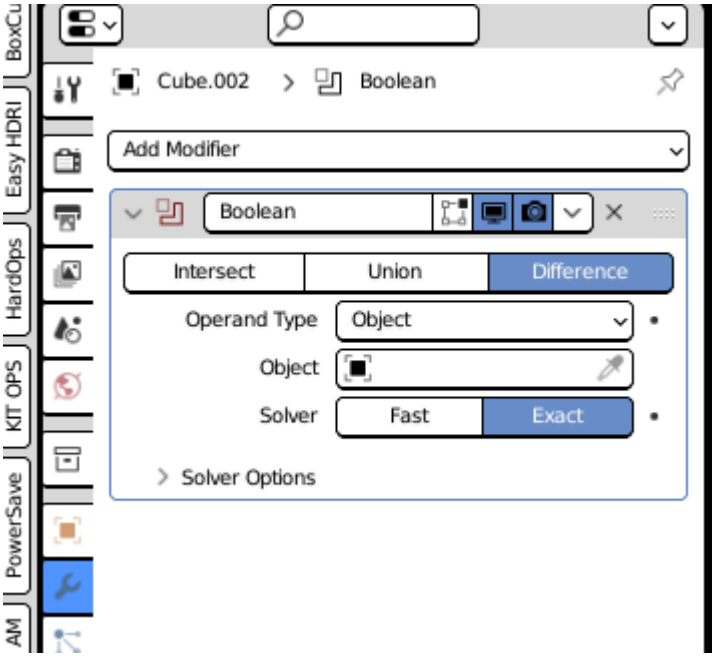
To make the main doors, we will use a boolean cutout method to remove the shape needed for door access.



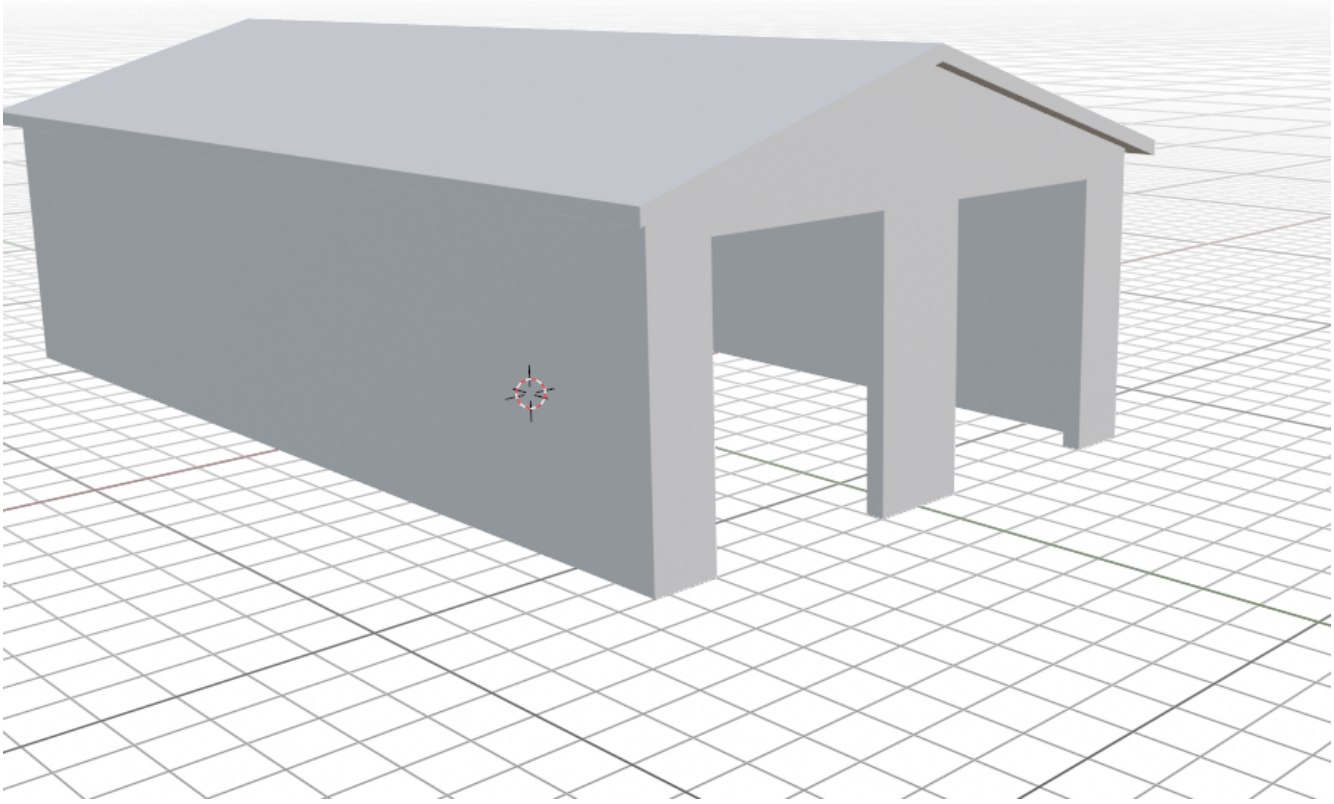
- Create a cube and resize it to: `X = 4m Z = 6m` using the dimensions fields in the N-Panel (press `N`)
- Move the cube so it's X position is -3m. `G` `X` `-3m`
- Move the cube so it's Z position is 2.6m. `G` `Z` `2.5m`
- Duplicate the cube: `SHIFT` `D` and set its X location to `3m`
- Select BOTH cubes and press `CTRL` `J` to join them
- Make sure you are in Xray mode: `ALT` `Z` and top view `Keypad 7`
- Move the joined cubes along the Y axis until it intersects the front wall from both sides. `G` `Y` or set Y position to about `-12.3xx`
- Select the Building object and use the modifier panel (wrench) on the right side of the screen.



- Choose `ADD MODIFIER` and select Boolean
- Make sure boolean option is set as `Difference` and click the EYEDROPPER icon



- With the EYEDROPPER, select the cubes we just created to make the door shapes. Since we won't be adjusting the door frame further, you **could** apply the modifier and we will see the new cutouts for the doors.



The completed version of the model can be found in the Trainsim.com file download section. Title: 2-Stall Reparations Depot

Making a Vehicle

Creating a road vehicle is a challenge as many road vehicles involve complex shapes and curves. In this section, we are going to focus on making a vehicle used as a load on an open rack auto carrier.

As we delve deeper into the world of 3D modeling, we often find ourselves needing inspiration and reference materials to guide our creative processes. One powerful tool in the arsenal of a 3D artist is the use of background images to model a specific object or scene. In this section, we will explore how to use background images to model a classic 1970s Ford van, capturing the essence of that era's automotive design

Finding the Right Reference Images

Before we begin modeling, we need to gather a collection of reference images that showcase the van from various angles and perspectives. Luckily, the internet is a treasure trove of information and images from the 1970s, including photographs and advertisements for the Ford van. You might even be able to find an online model that you can rotate and position to take snapshots from

various angles.

Online Searches

Start with a simple online image search using keywords like "1987s Ford van," "Ford Econoline 1970s," or "1970s van advertisements." This will yield a variety of images that can serve as your references.

Classic Car Websites

Explore classic car websites and forums. Enthusiasts often share high-quality images of vintage vehicles, which can be invaluable for your modeling project.

Archives and Magazines: Check out digital archives of automotive magazines from the 1970s. These publications often contain advertisements and feature articles that can provide detailed reference material.

Organizing Your Reference Material

Once you have collected a sufficient number of reference images, it's essential to organize them for easy access during the modeling process. Create a folder on your computer dedicated to this project and categorize your images based on different aspects of the van:

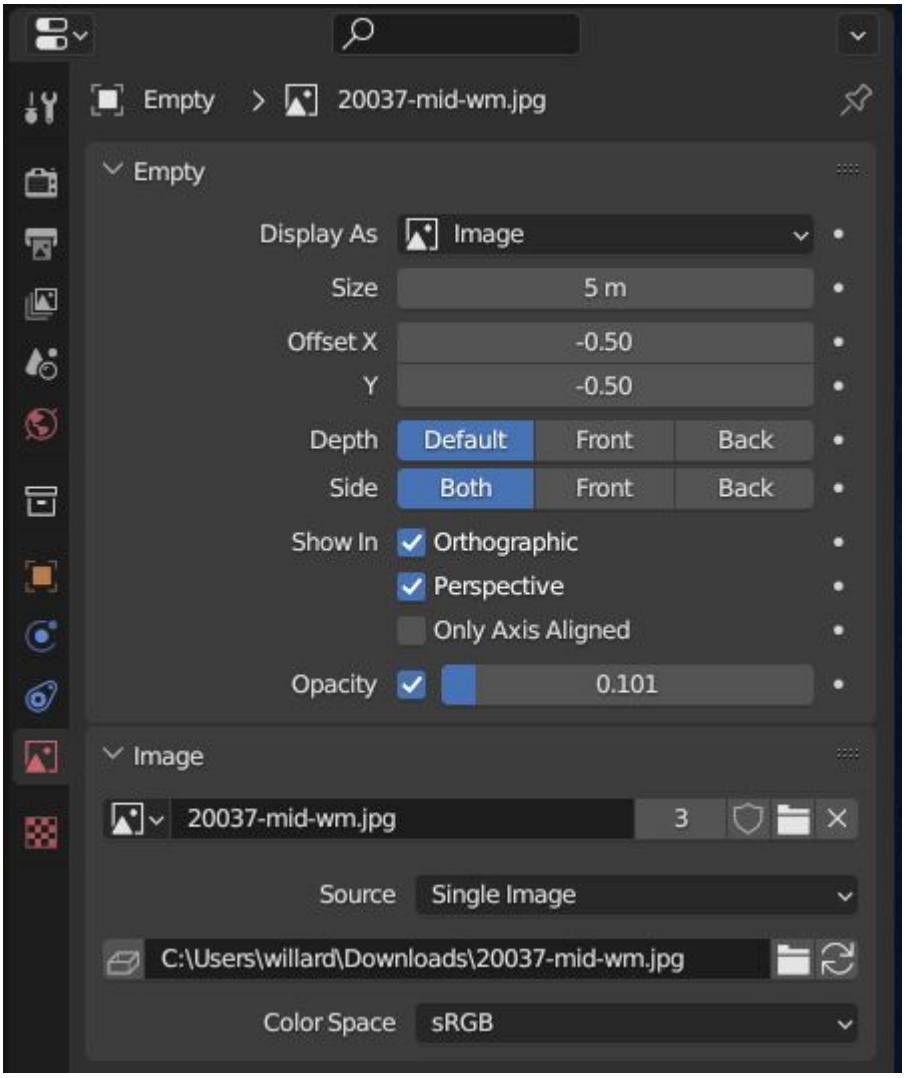
- Exterior: Include photos of the van from various angles, showing details like headlights, taillights, grille, and body lines.
- Color and Materials: Pay attention to the color schemes and materials used in the van's design. This will help you accurately recreate textures and finishes.

Using Background Images

Now that you have a well-organized collection of reference images, it's time to use them as background references in Blender. If possible, you modify the images to contain an Alpha channel so there will be less "white" making the drawing reference less distracting.

I often find that images are not 100% accurate, this one is no exception.

- Import images: In object mode, adjust your view to one of the primary viewpoints available in the reference images `Keypad 1`, `Keypad 3`, `Keypad 7`, for example. Using **Mesh › Add Image › Reference or Background)** to add an "empty" reference image to the scene as background images. Create separate views for the front, side, top, and rear of the van, and import the corresponding reference images for each view.
- Adjust Opacity: Reduce the opacity of the background images so that they don't obstruct your 3D model. You want them to serve as a visual guide rather than a distraction.



- Align and Scale: Ensure that the images are aligned correctly with your modeling workspace. You may need to adjust their scale and position to match the proportions of your model.

The Modeling Process

With your background images in place, you can now start the 3D modeling process. Use the images as a guide to create the van's exterior and interior components. Pay attention to the finer details, such as the curvature of the body panels, the shape of the windows, and the design of the wheels. You might want to break up the complex shapes as smaller faces that you can merge later.

The easiest way to start is to use a MESH PLANE, go to edit mode and use `CTRL-R` add then enough subdivisions `mousewheel` to match the edges in the drawings.

`To do: Add example`

Iteration and Refinement

Modeling a complex object like a Ford van from the 1980s may require several iterations and refinements. Continuously refer to your background images to ensure accuracy, and don't be afraid

to make adjustments as needed. The goal is to capture the spirit of the era in your 3D model.

Let’s do it.

Example Session

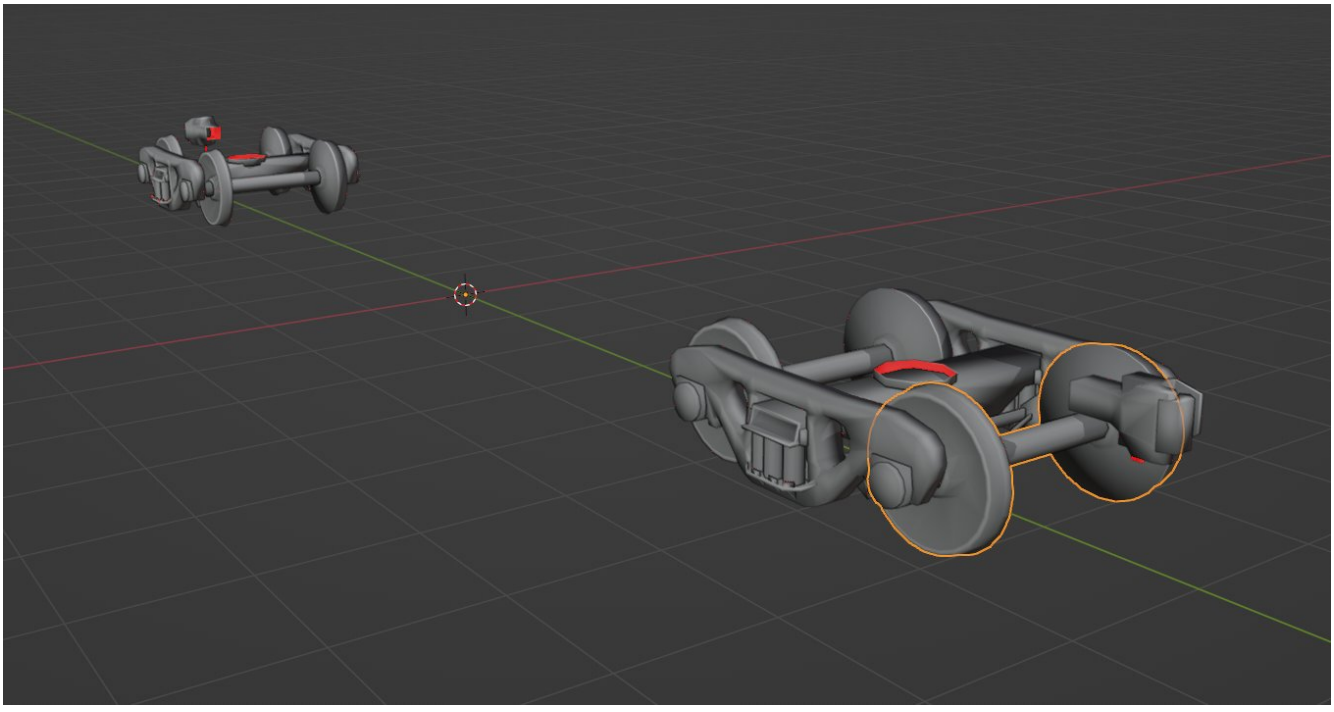
Rolling Stock: Importing Model Objects

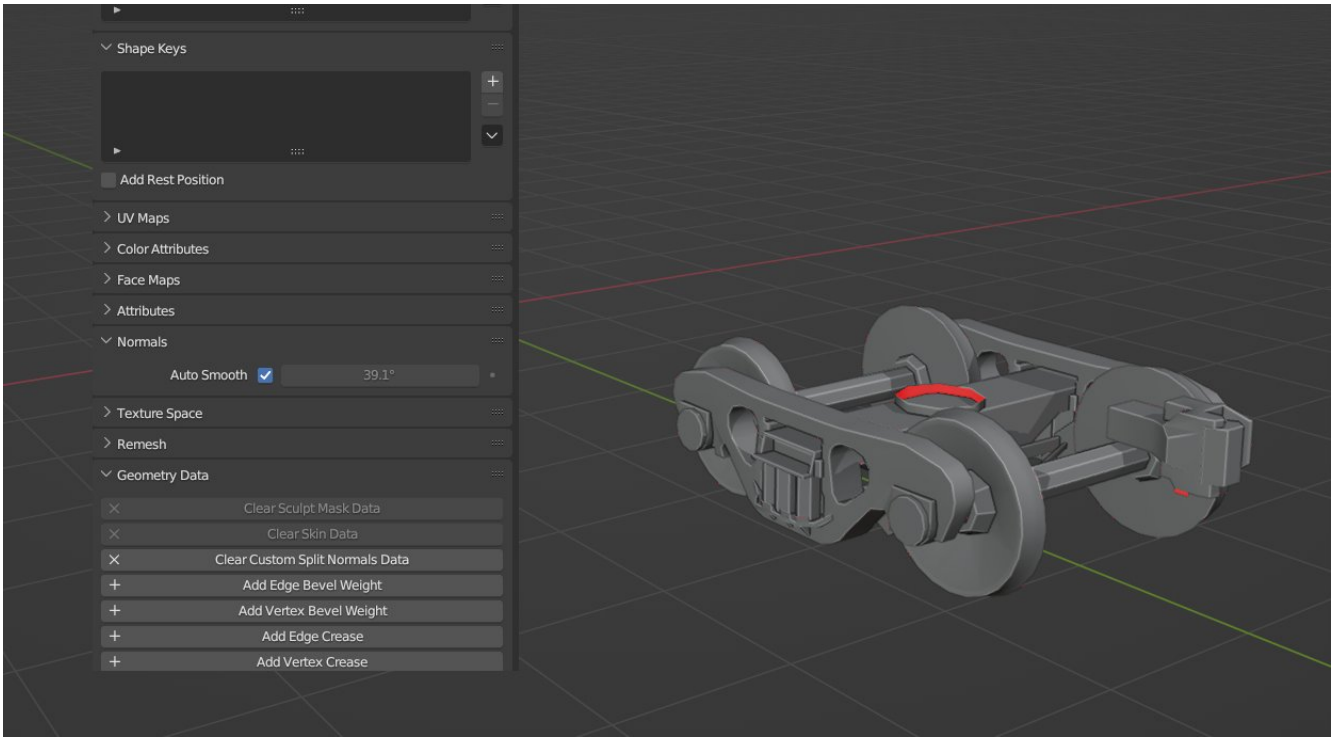
There are going to be times when you have the ability to save time by importing previous work or importing model objects that people have shared.

For example, on Elvas Tower, you will find that Erick Cantu has shared some of his modeled bogeys and couplers. Making use of these a little tricky if you are a Blender user because Erick doesn’t model in Blender. Here is how we would go about importing these models and making use of them.

- Download NAVS-N Series trucks from the Elvas Tower - Elvas Tower> Downloads> Source Files & Utilities> 3D Modeling Utilites and Source Files
- Unzip them to your hard drive in a convenient spot for importing.
- Open Blender to a NEW drawing file
- File-→Import-→Wavefront(.OBJ) and select the 100Ton Trucks, for example.
- The import will place 2 sets of bogies and couplers. however, the S2 and the A3 variants will be stacked on each other.
- Use the Scene Collection interface to **HIDE** the A3 Series, leaving behind the S2 Series as "visible"
- Now, to address how the imported model "looks". It arrived SMOOTHED, but that does not mean it looks good. Normally, you would perform the 2-step process OBJECT-→ SHADE SMOOTH followed by the Properties Panel DATA-→NORMALS-→Auto Smooth and tweak the Auto Smooth slider for best effect BUT!… the slider doesn’t work! OUCH.
- To fix the issue with Auto Smooth… in the same DATA PANEL, move down to GEOMETRY DATA and expand it… then choose "Clear Custom Split Normals Data"
- You should now be able to use the Auto Smooth Slider to fix the smoothing.

After this, you can proceed with verifying the UV mapping and tweak the model as needed. (I prefer to use my own coupler, for example)





Back in 2004, when I started modeling, I was frustrated by 2 things 1) Everyone's Couplers looked different 2) I could not make a coupler look good on my own (yet). My solution was that I reached out to Larry Goss who was kind enough to willingly share his coupler source file so more of us could make models where the couplers matched each other versus being do disparate when coupled, causing less jarring couplings.

Making Rolling Stock - Project #4

Starting of this section with some tips related to creating rolling stock.

Remember these tips?

- Rolling stock is typically symmetrical so making use of the `Mirror` modifier to create the opposite side of a shape is a timesaver
- Consider "MARKING SEAMS" while you are modeling instead of saving it until you are ready to UV map your textures.
- Recycle as much as possible from existing models (Bogies, Couplers, Coupler Lift Bars, etc) Hint: Create a collection of re-usable parts.

Let's Say we are was making a simple flatcar. Nothing too complicated.

Using Background Images

As with the previous project, we load in our reference background images. I picked a random image from the internet using a Google search because… Why not?

Starting from the ground up

I like to start constructing vehicles starting with the trucks, and for this I'm going to recommend downloading a *.blend file to simplify this. Erick Cantu has graciously provided various trucks that we can include in our own models. They can be downloaded from Elvas Tower.

You really should not need to make your own truck and coupler models. These are pretty standardized objects with a minimal number of variations. Chances are good that you can find available models to import instead of making them yourself. In the long run, this will lead to consistency between modelers.

Example: 100-Ton Truck Model

A usable Blender example of a 100-ton freight truck is provided, based on Erick's original model. Available resources include the following:

- **Source files for NAVS N-Series freight car trucks**
- **Source files for NAVS Z-Series freight car trucks**
- **100T Roller Bearing Truck Blender Object**

Some of these items are also available through **Trainsim.com**.

Erick's truck models include attached couplers by default. In practice, couplers can be separated if desired, depending on the needs of the project. Keeping them attached allows the couplers to follow

truck rotations accurately, which is the reasoning behind Erick's design choice.

Using these existing objects, we can Append the ready-to-use model into our initial blend file.

I have been able to adjust the wheel radius as needed, though it's probably not entirely prototypical to do so on the trucks provided so far.

If you feel so inclined, you could create your own wheel sets from scratch, but I leave that up to you. Also note, since it is very likely we will be using LOD ranges with this model, understand that we will make the highest detail model first. It is much easier to take away than add objects later.

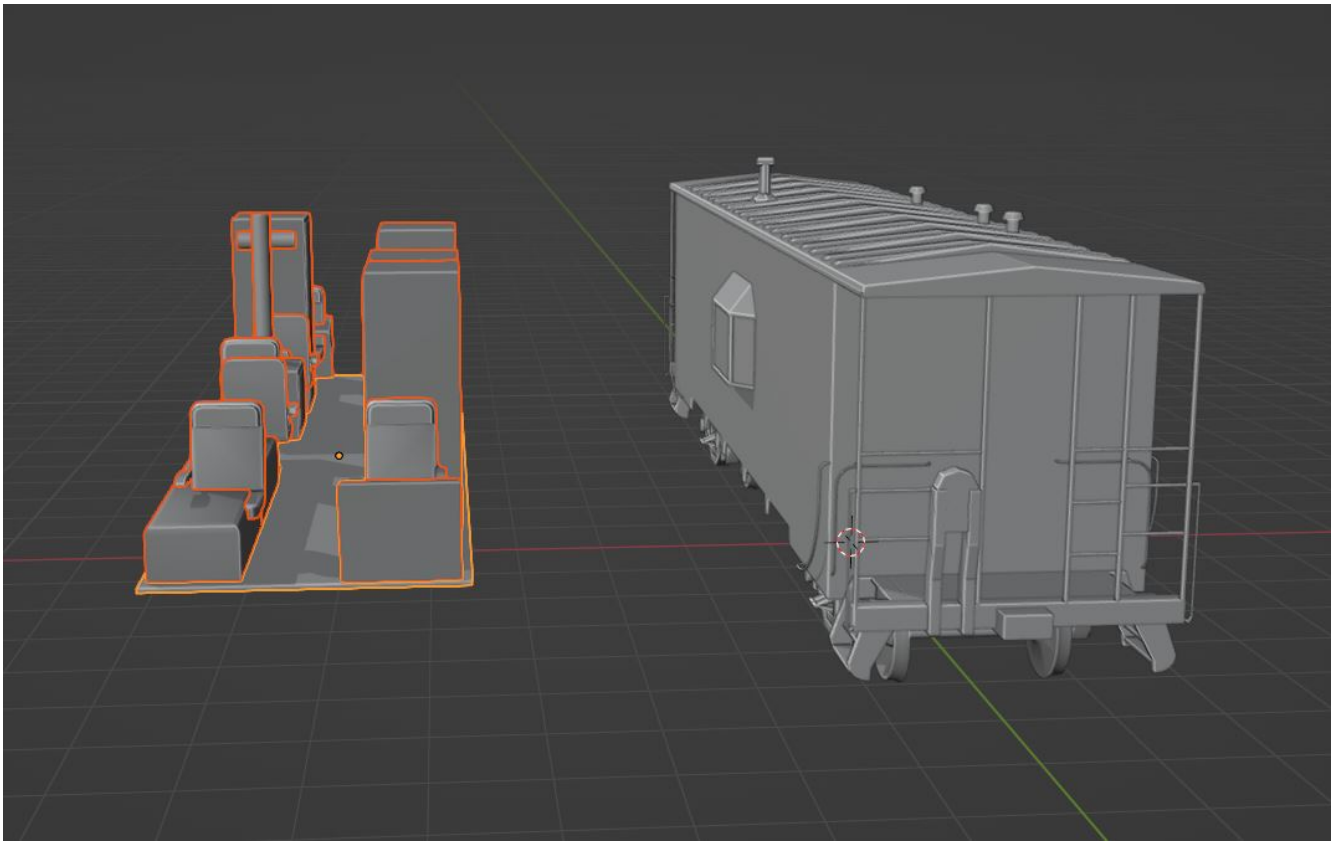
Moving multiple objects at once using the transform panel makes use of the `ALT` otherwise you are only able to move objects individually.

Example

Moving (transform location) ALL SELECTED objects at once.

Setup:

- Select a primary object in the group of objects that you want to move as a group.
- Press `SHIFT` `S` for the PIE menu and "Set Cursor to Active"
- Select ALL of your objects you want to move together and use **Object › Origin › Set Origin to 3D Cursor**



- Press `SHIFT` `S` for the PIE menu and "Set Cursor to World Origin"
- Now, lets say you wanted to move all of the objects to 0 on the X axis. Using the Transform > Location panel for X value input, HOLD `ALT` and key on "0" in the field.
- All of the objects should have moved to X = 0 location in the 3D View together.

Needs to be finished

Vehicle LOD - Level of Detail Settings

In realtime computer graphics, a technique called "level of detail" (LOD) is commonly used in various applications ranging from commercial simulators to PC and console games.

When 3D games are running on a graphics card, there is a limit to the number of surfaces (polygons and triangles) and textures that can be drawn. If this limit is exceeded, the game or simulation starts to slow down and the movements become jerky.

If the graphics engine is asked to do too much then the program’s framerate drops. This drop in framerate results in jerky movements and slower update rates, which can become very noticeable

and take away from the sense of immersion.

Lets also look at an excerpt from the Open Rails Manual:

```
Many visual objects are modelled at more than one level of detail (LOD) so, when they
are seen at a distance, Open Rails can switch to the lesser level of detail without
compromising the view. This use of multiple LODs reduces the processing load and so
may increase frame rates.
```

Always apply textures before defining any LOD settings.

Referring to some example snippets from the Outliner window in a Blender Working session... Watch out for the "GOTCHA's"...

```
Scene Collection
        MAIN
              MAIN_1000
                     BodyLOD1
                           Bogie1
```

Copying a part and adding it to the collection BodyLOD2. Outliner Result...

```
Scene Collection
        MAIN
             MAIN_1000
                    BodyLOD1
                         Bogie1



        MAIN_2000
             BodyLOD2
                   Bogie1.001
```

Looking at it now, shouldn't MAIN_2000 be directly under MAIN_1000 like this

```
Scene Collection
   MAIN
         MAIN_1000
                BodyLOD1
                      Bogie1
```

```
          MAIN_2000
                  BodyLOD2
                          Bogie1.001
```

Answer: YES, What happened is that MAIN_2000 was created at the same level MAIN not MAIN_1000. Only what resides below the MAIN collection will be considered for EXPORT.

Let's Look at some other issues here with what is shown above… Don't forget capitalization.

For Example: All cases of **Bogie1** should be **BOGIE1**.

With the above example setup, the high resolution BOGIE1 part will disappear at 1000 meters. Then the low res BOGIE1.001 will take over. That low res bogie won't be animated, which is fine, since its too far away to see anyway.

Here's how to do it if you need the lower resolution bogie to be animated:

- Use an Empty for BOGIE1
- Link it to both MAIN_1000 and MAIN_2000 using drag and CTRL drop in the Outliner
- Attach the low and high resolution meshes to it in their respective LOD collections

```
Scene Collection
   MAIN
         MAIN_1000
                BodyLOD1
                BOGIE1
                    BogieLOD1


          MAIN_2000
                BodyLOD2
                BOGIE1
                    BogieLOD2
```

Don't forget to check out the sample locomotive file that is provided with the MSTS exporter download.

Once you copy the completed model to the Alternate LOD Collection, you can begin deleting details that won't normally be visible at longer distances.

Bitmap Editing and Texture Workflow

This chapter describes practical methods for creating, editing, and exporting textures for 3D models. It focuses on clear, repeatable workflows and avoids tool-specific bias. Texturing, like modeling, improves with regular practice.

UV Mapping Overview

UV mapping (also called unwrapping) defines how a 2D image is applied to a 3D surface. The process involves laying out a model's polygons on a flat 2D plane, forming a UV map. During rendering, the GPU uses this map to wrap the texture image onto the model's geometry.

UV mapping and 3d object unwrapping are the same process. The term simply refers to creating a 2D layout of a 3D mesh for texture application.

Author's Note

```
I'm not an Adobe evangelist. My goal is to help you get great results with the tools
you prefer—whether they are free/open-source or commercial. If you're new to
texturing, starting with a free tool is perfectly fine. If you're already comfortable
with a particular bitmap tool, the techniques in this guide focus on workflows that
use layers, which most modern tools support.

My concern with Adobe products is the subscription model, you pay monthly just to keep
access to the software. If you already subscribe, it makes sense to continue using the
tools you know. But I do not recommend purchasing Adobe Photoshop solely for
occasional 3D texturing work.
```

Bitmap Tools Overview

In this section, we will provide an overview of the various bitmap tools available and why they are important for creating textures for 3D models. We will also discuss the features and capabilities of bitmap tools that make them essential for texture creation and editing.

Bitmap Tools Overview

Bitmap (raster) editors let you paint and manipulate pixels on a grid. They're essential for photo editing, digital painting, and texture authoring. Typical features include layers, blend modes, masks, brushes, filters, color correction, and precise per-pixel control. Popular choices include GIMP, Krita, Paint.NET, Affinity Photo/Designer, and Photoshop. Raster images are resolution-dependent—scaling them up can reduce quality—so work at an appropriate resolution from the

start.

Why bitmap tools matter for 3D

Proper bitmap tools or pixel based editors are crucial for creating and refining textures for 3D models because they allow artists to craft the detailed surface details that make a model appear realistic and visually appealing. A pixel editor enables artists to create intricate textures, such as fabric patterns, skin details, metallic finishes, or weathering effects. These details are mapped onto the surface of 3D models, bringing them to life by simulating real-world materials. Bitmap tools bridge the gap between 2D artistry and 3D modeling, enabling the creation of rich, lifelike textures that elevate the visual quality of 3D assets. Finding the right tool for the job is essential for creating high-quality, realistic 3D assets but in the end, its up to what fits your workflow and preferences.

Layered Format Files

For serious texture work you should be using layers (with opacity and blend modes). Save a master file that preserves layers; export flattened copies for your game/sim.

To recommend a few tools and formats, the following list should be a guide on what you can use (These all support layers):

Tool	File Type	Comment
Affinity Studio	.af	Proprietary, Native
Affinity Studio	.psd	An Import/Export Option
Affinity Studio	.tiff	An Import/Export Option
Gimp	.xcf	Native
Gimp	.psd	An Import/Export Option
Gimp	.ora	An Import/Export Option (plugin)
Krita	.krt	Native
Krita	.ora	An Import/Export option
Krita	.psd	An Import/Export Option
Krita	.tiff	An Import/Export Option
Paint.Net	.pdn	Native
Paint.Net	.psd	An Import/Export Option
Paint.Net	.ora	An Import/Export Option (plugin)

Tool	File Type	Comment
Paint.Net	.dds	An Import/Export Option, Alpha Only
PaintShopPro	.psp	Proprietary, Native
PaintShopPro	.psd	An Import/Export Option

You will not find Adobe Photoshop listed here. Photoshop continues to rely on its native PSD file format, and if you already own (or subscribe to) it, it is certainly capable of producing layered textures. I no longer use Photoshop and cannot speak confidently about its current import/export capabilities, but it is reasonable to assume that PNG and DDS are supported.

- All of the above programs support the PNG image export format, used for final textures.
- Some of the above support DDS image format, eg. Paint.Net and GIMP through the use of external plug-ins
- The PSD format was originally a proprietary format used by PhotoShop and some software has trouble opening all PSD format variations seen in the wild
- TIFF is also a layered format, though not as common and in some cases prone to some tools not fully understanding file contents. Testing is needed.
- Open Raster, ORA, is a newer file format that is supposed to replace PSD as a standard solution for exchanging layered images between graphics editors. It's still not widely used.

One additional tool to mention is InkScape, which like Affinity Studio, is a Vector Graphics program that contains a "make a bitmap copy" option. Vector graphics are scalable without creating "jaggies" like you see with bitmaps. Like Affinity Studio, Inkscape is free.

DDS uses lossy compression. Treat DDS like a final delivery format (similar to PDF for documents), not a master format for repeated edits. Always keep a lossless layered master.

Very simple editors (e.g., legacy MS Paint) are poor choices for layered texture work.

While the list above does not attempt to catalogue every bitmap editor capable of using layers, it highlights the most common options in regular use.

In summary, choose a tool that has a native layered file format that can also support PNG and possibly DDS. For maximum compatibility, choose a free tool, like GIMP or Paint.net, otherwise, save your master documents in PSD or ORA format

You can do some rudimentary texture painting inside blender. Many improvements have been made in recent years, but it's still a bit clunky. You can use the paint tools to create some basic textures, but it's not as good as using a dedicated tool. You can also use the texture editor in blender to create more complex textures, but it's still not as good as using a dedicated tool.

The best approach is to use a dedicated tool for texture creation and editing. There is also a free add-on called UCUPAINT (from Blender Extensions) that can be used to create textures in Blender. It's not as good as a dedicated tool, but it's a decent starting point if you want to try it. It has a number of options though that are probably overkill for Open Rails texture creation.

While some tutorials mention `Substance Painter`, it also falls into the category of overkill for Open Rails texture creation. It's a great tool for creating complex materials, but it's not necessary for Open Rails texture creation. If you already have it and want to use it, it's a good idea to use it for creating the base textures and then export them as PNG or DDS for use in Open Rails. There is pretty much no reason to get it if you are only going to use it for Open Rails texture creation.

Using DDS Textures

DDS (DirectDraw Surface) stores GPU-friendly compressed textures (with optional mipmaps). Benefits include smaller memory footprint and fast GPU upload. Ideal for large scenes or hardware with limited VRAM. Export from modern editors (Paint.NET, GIMP+plugin) rather than legacy tools.

Using `.DDS` textures for your materials has the advantage of reducing the memory usage of textures by slightly decreasing their quality (Though honestly, not as bad as the ACE file format). This is helpful if you have a limited amount of graphics card and system memory and want to include as many textures as possible. Additionally, this method is particularly suitable for organic/nature textures where precision is not critical.

Always maintain a layered, lossless master. Do not round-trip edit DDS; repeated saves degrade quality.

For working with DDS, the easiest approach is to utilize Paint.net or GIMP as both can export DDS textures directly. Exporting large 32-bit ACE textures using the original MSTS tools is often not feasible and some people won't even have access to MSTS tools if they only use Open Rails. The most significant advantage of using GIMP or Paint.net is that exporting is much more straightforward and faster compared to outdated programs like TgaTool2.

DDS is a useful format but many of the export options are not suitable for best performance.

The short explanation is:

Always use DXT1 compression with full MIPs, except… you can use DXT5 compression when you need alpha translucency (ie; alpha values other than on/off)

One of the most significant performance issues with current GPUs is related to the texture size. All textures used on loaded tiles in a scene must fit into the GPU at once. Although modern GPUs have 2G, 4G or more, adding up the texture file sizes for all buildings, terrain, and rolling stock will quickly reveal that the GPU's capacity can be reached. This is especially true when using 2K and 4K textures. When the GPU reaches its limit, performance suffers because additional textures must be swapped out to the CPU every frame.

So maximum compression is the key to good performance. Just compare the size of uncompressed textures and you will see how bloated they are.

TYPE	USAGE	Comment
DXT1 no alpha	Textures without transparency	Normal maps without shine, All glow maps
DXT3	Menu icons / UI elements	No mipmaps but has transparency
DXT5	Textures with transparency	Mip Maps and Normal maps with shine (if we ever get that ability)

An alpha channel increases the file size so it should be left out unless it is needed. On color maps, the alpha channel is used for transparency, on normal maps for glossiness. If the texture has no transparency or the normal map has no glossiness saving them as DXT1 (no alpha) instead of DXT5 instantly saves on the file size for no loss.

Can Blender Import DDS Files Directly?

Yes, Blender can now import DDS files directly. Blender has built-in support for reading and using DDS textures. However, the quality of the import and how you use the textures depends on the specific DDS file and your project's needs. In modern releases, you don't need any additional add-ons to import DDS files in Blender. Just like importing any other image format. Refer to the COMPRESSION TYPE table above.

Authors Note

```
You know how sometimes people report that their icons or textures become a rainbow
pixel mess? That is related to Gimp saving the "no mipmap" textures incorrectly, it
writes in the header that the image has 1 mipmap, but it has none and the game gets
confused. I never used Gimp and I can't find where I read about this right now, but if
I'll do, I'll update this comment.
```

Ensure you are keeping uncompressed versions of the source files in the editing tool's native format that you can load for editing so you don't accumulate compression artifacts.

- With **GIMP**, you would use `export as` and then choose `select file type` and set the options for DDS such as Compression, mipmap, etc. Latest versions of GIMP seem to come with DDS support so no post-install plugin is needed.
- With **Paint.net** versions newer than 4.2.2, DDS support comes bundled with the program. You would use the `save as` option and `save as type: DDS`. In the Save Settings window, you would select **DXT1** and under **Error Metric**, and check **Generate Mip Maps** and **Use Best Quality**.
- With Affinity tools, you just save your master file since saving to a "common format" like PNG is done using an EXPORT option.

Using ACE Textures

Microsoft Train Simulator uses a proprietary Kuju image file format known as ACE (extension `.ACE`). ACE files are used for textures applied to shapes and the bitmaps used for the cab panels and controls aka. bitmap ACE's.

A conversion utility called `MAKEACE.exe` was supplied with Microsoft Train Simulator. Newer tools have been created that eliminate some of the limitations that come with the original and updated `MAKEACE` program.

Open Rails has the ability to automatically use DDS files if they are present, even if the model's `S` file defines an ACE file. This is a great way to use DDS files without having to convert them to ACE files. The downside is that the DDS files are really not editable as they will lose detail on every export. In other words, the DDS files use a lossy compression method, which means you will lose detail and color information compared to the original texture.

ACE File Types:

There are three types of ACE files:

Solid

these are created by converting 24-bit BMP or TGA files and the resulting ACE file contains only solid pixels. You might use such ACE files for the general texture detail on buildings and trains you are creating.

Trans

these are created by converting 32-bit TGA files and specifying the transparency mask option. The resulting ACE file will contain both solid pixels and transparent pixels. You can still use solid areas of this ACE type for the general texture detail on your object but in addition you can use areas containing transparent pixels for details such as railings, ladders and other areas that require the appearance of a "hole" in your object. This is also a good choice for simple cab backgrounds and controls.

Alpha

these are creating by converting 32-bit TGA files and not specifying the transparency mask option. The resulting ACE files will contain solid and both partially and completely transparent pixels. This texture type can be used in a similar way to the Trans type, except it also allows you to create effects such as tinted windows.

ACE File Compression

There are two types of compression that can be applied to ACE files:

No compression

the image is stored uncompressed.

ZLib compression

this is a lossless compression technique similar to that used by programs such as PKZIP and WinZip.

DXT compression

this is a lossy technique that can only be used for Solid and Trans type texture ACE files. It may create unwanted artifacts.

DXT compression cannot be used for bitmap ACE files.

For Alpha type ACE files you cannot use DXT, so the recommended option is ZLib.

I'm relatively sure that even the updated MAKEACE only supports textures up to 1024 x 1024 in size.

Digital Graphics Basics

The following is a brief introduction to some of the basic concepts of digital graphics.

Bit Depth and Color Depth

Color depth describes the maximum number of colors an image can contain. This is dependent on the bit depth of its pixels. The higher the bit depth value, the more color information each pixel can display and the greater the number of colors the image can contain.

Bit Depth

Bit depth is a value that describes the number of colors that an individual pixel can display. A bit can either be on or off. Therefore, a 1-bit pixel can display two colors: black and white. An 8-bit pixel displays 256 colors. Each bit can be on or off (2 states). When combined in eight different ways (2x2x2x2x2x2x2x2 or 28) it can display up to 256 colors.

Pixels are usually one of five standard bit-depths. Rarely is anything above 8 bits necessary.

Bit Depth	No. Colors	Calculation
1	2	2^1
8	256	2^8
16	65,536	2^16
24	16,777,216	2^24
32	16,777,216 plus 8-bit alpha channel	24 + 8 bits

Color Depth

Sometimes the terms bit depth and color depth are used interchangeably. We're going to use the term color depth to describe the overall ability of an image to display colors. The distinction lies in the fact that some color modes use more than one color 'channel'. For example, The RGB color mode contains 8 bits in each of its three color channels (red, green, blue) giving an RGB image a color depth of 24 bits (8 bits per channel x 3 channels). Below are examples of an image displayed in various color modes to demonstrate how image quality and file size are affected.

Color Modes and Bit Depth

1-bit bitmap image	Color Mode: Bitmap, Color Depth: 1-bit (1 channel x 21), Colors: 2, File Size: 4 kb (as GIF)
4-bit indexed color image	Color Mode: Indexed Color, Color Depth: 4-bit (1 channel x 24), Colors: 16, File Size: 8 kb (as GIF)
8-bit gray-scale image	Color Mode: gray-scale, Color Depth: 8-bit (1 channel x 28), Colors: 256, File Size: 24 kb (as GIF)
8-bit indexed color image	Color Mode: Indexed Color, Color Depth: 8-bit (1 channel x 28), Colors: 256, File Size: 16 kb (as GIF)
24-bit RGB image	Color Mode: RGB, Color Depth: 24-bit (3 channels x 8 bits per channel), Colors: 16,777,216, File Size: 12 kb (as JPEG)

The increase in file sizes theoretically should be proportional to the number of bits used in an image; however, the difference in GIF and JPEG compression resulted in a smaller-than-expected 24-bit image file size.

Photoshop supports 16 bits per channel. This provides for slightly better color rendering (if your monitor supports it) but increases the file size dramatically. A 16-bit pixel will display 65,536 shades of color. For Web graphics, this would be overkill.

Bit Depth and Color Depth Summary

- Bit depth is a value that describes the number of colors that an individual pixel can display.
- The higher the bit depth, the more color information it can display.
- Color depth describes the maximum number of colors an image can display.
- An image's color depth is dependent on the bit depth of the image's pixels and the number of channels it contains.
- The file size of an image is proportional to its color depth.

JPG Format, lossy compression and UpScaling

JPG is a lossy compression format. It is a compression format that is designed to reduce the file size of an image while maintaining the quality of the image. It should be avoided for images that contain text or graphics with fine detail and is really not suitable as a general-purpose "saved" image format for the textures used in the simulator.

A new thing to try though is Artificial Intelligence (AI) tools that offer image upscaling. Many have tried this and have shown that can offer some improvement to the master textures that needed to be a bit larger and sharper. One example is https://www.topazlabs.com/gigapixel from Topaz Labs. It is not a free tool but it does offer a free trial. It is a bit expensive but it does offer some improvement to the textures.

Another option is to try https://github.com/upscayl . Upscayl is a free and open source desktop application that lets you upscale your low resolution images using advanced AI Models. It is a bit slow but it does offer some improvement to the textures.

PNG Format

Portable Network Graphic is a new bitmap file type developed in part with the Internet in mind. Its development was instigated by the GIF licensing issue and was intended as a replacement for GIF.

There are two PNG sub-types, **PNG-8**, and **PNG-24**. Both use the same compression method. PNG-8 supports a single alpha channel and PNG-24 supports variable transparency (covered later). PNG-8 is limited to 8-bit color depth (256 colors), and PNG-24 can have millions of colors (24-bit color depth). Both are lossless compression formats

PNG Format Summary

- The Portable Network Graphic format was designed to replace GIF.
- PNG-8 supports 256 colors and a single alpha channel.
- PNG-24 supports 16.8 million colors and variable transparency.

Vector Graphics

Vector graphics are created by a computer program using mathematical formulas. The program draws lines and curves using mathematical formulas. The program then stores the formulas in a file. The file contains the information needed to recreate the image. Programs that create vector graphics are called vector graphics editors. Examples are Adobe Illustrator, Affinity Studio and Corel Draw and Inkscape.

Vector graphics are referred to as resolution-independent because they do not plot images on a pixel-by-pixel basis and so are not tied into a monitor's resolution. Vector images describe images in terms of shapes, lines, curves, points, colors, length, etc. These images rely only on the resolution of an output device (like a printer for example) to determine their final resolution. Examples of images suitable for vector graphics are logos and type.

Vector graphics are not suitable as a replacement for bitmap images, but they are suitable for logos and typed lettering inside bitmap images that then get rasterized.

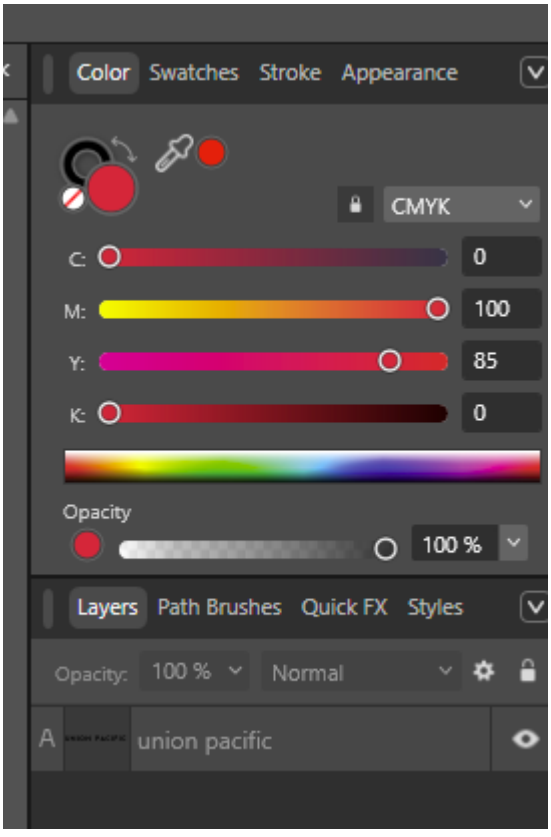
Fonts used by windows programs are vector graphics. This is why RailSimStuff creates their own fonts with railroad symbols and logo's for others to use.

Tweaking Fonts With Affinity

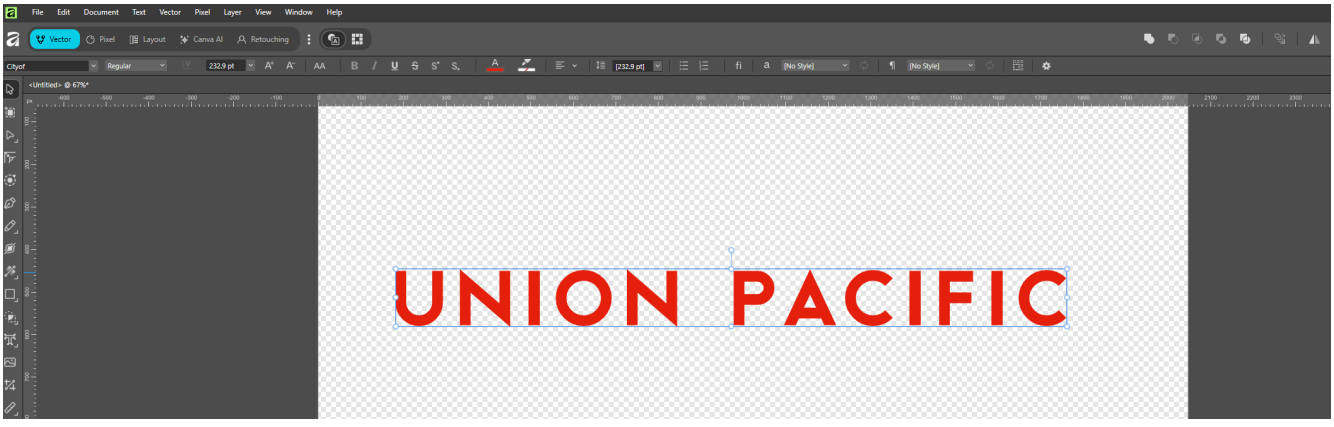
Lets say you want to create the Union Pacific distinctive outline lettering. You may have noticed a font available all over the web called "City Of" by Ben Coifman of RailFonts.com. Ben released this lettering back when he was still in college and the Internet never forgets. You can probably get a much improved version direct from their web site but that I leave up to you conscience regarding "supporting the little guys".

Anyway, you have the font and you have Affinity Studio or the older Affinity Designer installed. So lets enter some text with the "CITYOF" font. Make sure you are in VECTOR mode.

Change it to RED…

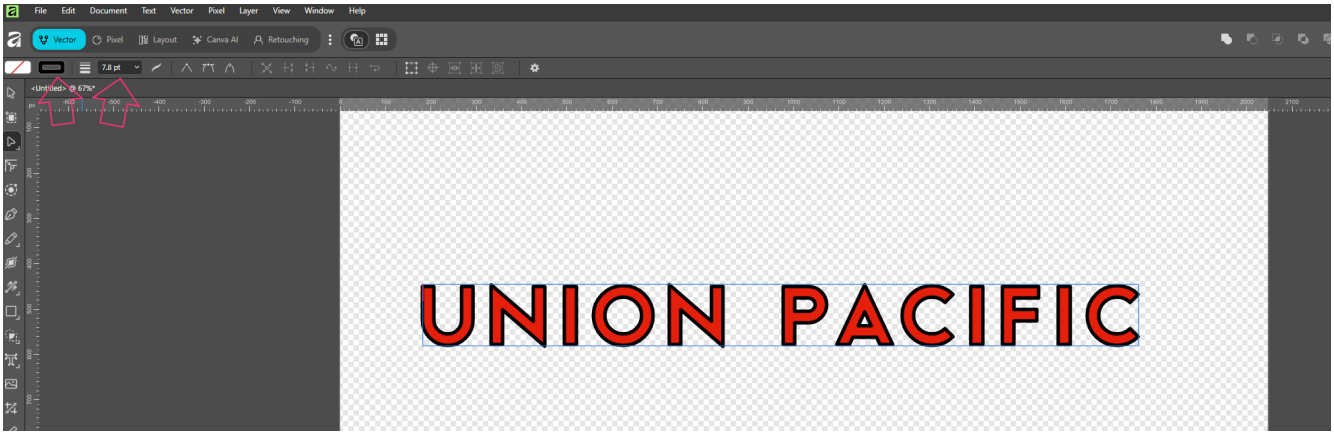


So you get:



The next step is to choose the VECTOR *TAB*; with the TEXT selected, choose "convert to curves" in the pulldown menu.

And finally, choose the outline thickness; here I chose a value of 7.8. And it's done.



Vector Graphics Summary

- There are two main types of digital graphic files: bitmap and vector.
- Vector files are resolution-independent.
- Vector graphics use mathematical formulae to create the image, not pixels on a grid like bit-mapped images.
- Simple vector images can be quite small compared to bitmap; complex images like photographs are difficult and inefficient with vectors.
- Bitmap is still more common than vector.
- Vector images can scale without loss of detail or a change in file size.
- Vector graphic images must be rasterized into a bitmap format for use as a 3D model texture image.

Preparing a texture

When creating textures. remember

> Texture mapping needs to be intuitive and functional.
>
> — Erick Cantu

Probably the best way to start a texture file is to create a background layer that contains the primary colors of your final model. If your base model is primarily "Tuscan Red", then fill your background with "Tuscan Red". Variants for Pennsylvania, for example, would be RED rgb(121,68,59), BROWN rgb(111,78,55), TAN rgb(166,123,91). Many tools allow you to enter a RGB color value into your editor.

You don’t need to complete your texture before you apply it to your 3D Model. Even starting with the single base color would be OK.

For texture size, consider working with 2048x2048 textures. Try to avoid creating multiple smaller texture files versus one large file. You can always shrink your texture to 1024x1024 when all your work is done as the coordinate mapping will remain relative as long as the proportions remain the same. You could not, however, adjust 2048x2048 to 2048x1024 though, keep that in mind.

If your model is wide but not tall, as many vehicle models are, then you might consider starting with a 2048x1024 texture size, provided that you are only creating content for Open Rails. Open Rails will not have issues with textures that are not square, unlike Microsoft Train Simulator.

Once you have created your base texture for your model, you should a) Save it in the native format of your editor or in one of the Layered formats like `PSD` or `ORA`. b) Save a copy in `PNG` format for use with Blender. Copy the `PNG` file to your project folder for your current model so it is easily available during a Blender working session.

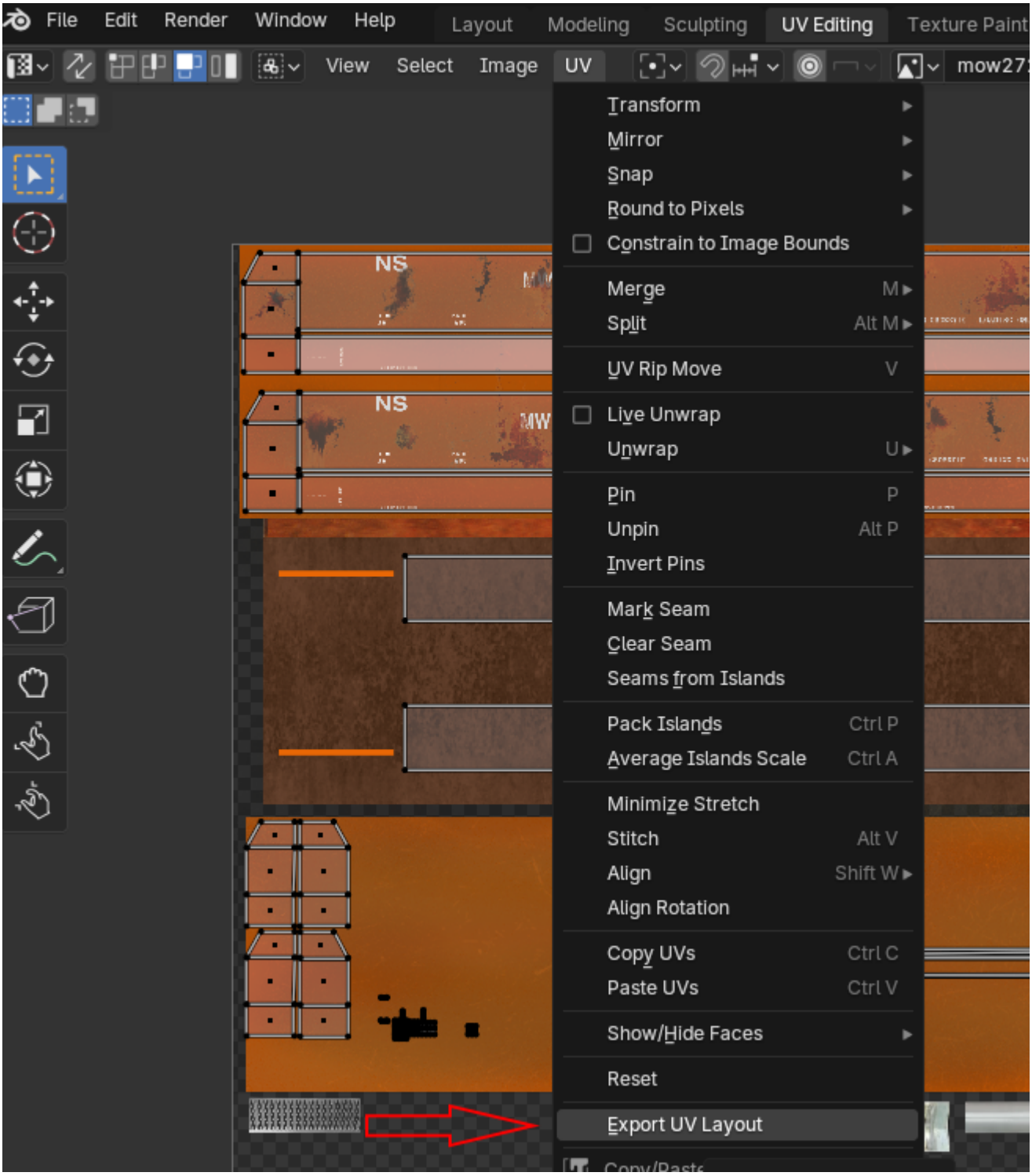
Keep in mind that some people who might consider repainting your model will want to make use of their own photographs of actual vehicles or buildings. This means that it would be "unkind" to these "re-skinners" if you were to split up the sides of your model into multiple sections as they would have difficulty getting the sections to rejoin cleanly from their photographs. Try to keep the side and top views as continuous shapes in your model and your textures. However, if the model is unlikely to be re-skinned by 3rd parties, then you can conceivably create a separate texture for each section of the model. This will allow you to rejoin the sections as needed.

For scenery like brick buildings, use a seamless base layer (e.g., brick) below windows/doors layers.

Which Comes First?

The model or the texture? It depends on your references and goals.

- Texture first: works when starting from photos and rough dimensions.
- Model first: is best with drawings/specs; unwrap, UV → Export UV layout, paint against the guide, then replace the blank texture in Blender.



Above you can see how the image was painted to match the UV layout that was exported from the 3D model. The texture in this case has already been created and updated in Blender with the painted image.

UV Mapping Basics

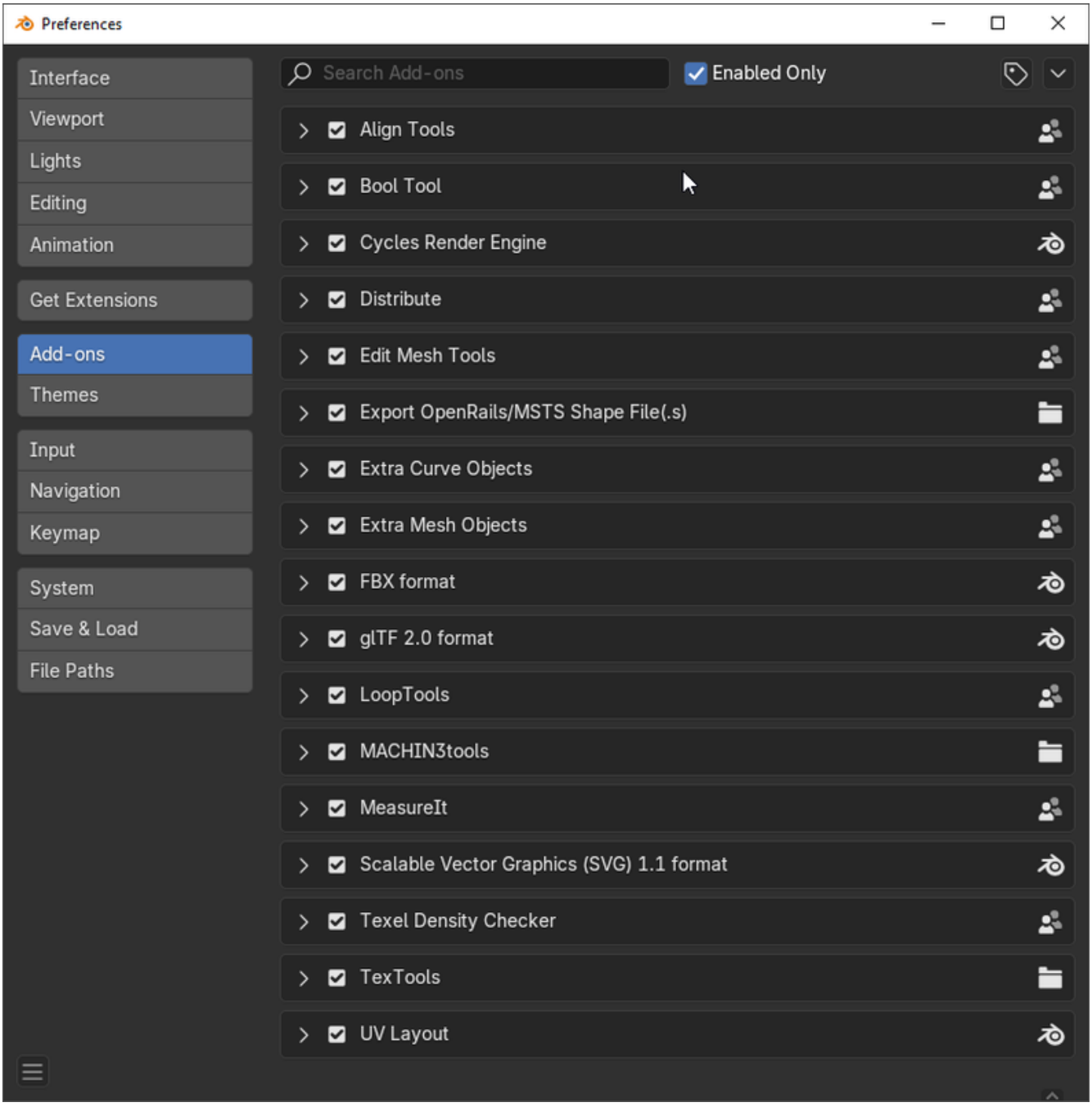
Setting up Blender for Texture Mapping and Baking

This section contributed by Scott Brunner.

This is intended as a primer, not a tutorial, so we'll be addressing the concepts involved that need to be considered for Mapping and Baking operations in Blender.

Author Tips

```
Blender supports many third-party add-ons. This section only covers a few free tools
that are useful for Open Rails work. Other modelers may prefer different add-ons, so
experimentation is encouraged.
```



In Blender 4.2 and newer, many add-ons are managed through **Preferences** > **Get Extensions**. **Texel Density Checker** is available there, while **UV Layout** remains an official add-on under **Preferences** > **Add-ons**. **TexTools** is installed separately from its GitHub ZIP file.

Texel Density Checker

shows up the **N Side Menu** when in the **UV Mapping** window. It allows you to get or set the bitmap density of a UV island. For the best-looking models, you want a relatively consistent texel density, and this add-on makes it extremely easy to set on each island as you map it.

UV Layout

adds mapping functions to the **Top Menu** in the **UV Mapping** window. Some of the most

important features are the ability to export your **UV Map** to a texture file so you can load it into your paint program, and the **Pack Islands** function help to pack your islands in the most efficient manner possible.

TexTools

shows up the **N Side Menu** when in the **UV Mapping** window. This adds a host of operations for manipulating your **UV Map** and single click **Baking**. If you've tried the default Blender **Baking** operation - this add-on makes your life much easier.

Textools Download: https://github.com/franMarz/TexTools-Blender

Textools Addon Video: https://www.youtube.com/watch?v=rEcJQ6Jdue4

Let's do a quick review of terms again...

Texel Density

- refers to the number of texture pixels (texels) per unit of 3D surface area. It's an important concept for ensuring that textures appear consistent and detailed across different parts of a model. Maintaining a consistent texel density is crucial for achieving a uniform look, especially when combining multiple objects in a scene. For isolated objects, a higher texel density can be used to create a more detailed look. Example: Air line hoses might benefit from a higher texel resolution than a typical car body.

Seams

- are the edges where a 3D model is "cut" or "split" during the unwrapping process. These cuts allow the 3D surface to be laid out flat in 2D space with minimal distortion. **Seams** only work under the following **Unwrap** operations: **Angle Based**, **Conformal**, and **Minimal Stretch**. *Regarding **Minimal Stretch**, this might have been added in Blender 4.3.*

UV Mapping or Unwrapping

- is the process of projecting a 2D image texture onto a 3D model's surface. It involves unwrapping the 3D model into a flat 2D plane, much like peeling an orange and laying its skin flat. This allows textures, such as images or patterns, to be precisely applied to the model, ensuring that each part of the texture aligns correctly with the corresponding part of the model's surface.

UV Vertex or UV

- is a point in the 2D space of a UV map that corresponds to a vertex on a 3D model. These UV vertices are used to define the position of the texture coordinates, determining how a 2D texture is applied to the surface of the 3D model. Essentially, they serve as the anchors for mapping the texture accurately onto the model's surface, ensuring that the details of the texture align correctly with the geometry of the model.

UV Island

- is a contiguous group of UV vertices and edges that form a separate, unbroken piece of the UV map. It represents a section of the 3D model's surface that has been unwrapped and flattened for texturing purposes. Each **UV Island** typically corresponds to a distinct part of the 3D model, making it easier to apply detailed textures accurately.

Pack Islands

- in **UV mapping** is the process of organizing and arranging **UV Islands** efficiently within the UV space to maximize the use of texture space. You can do this with entire texture sheets or any subset therein. You can define a Margin to set how much space is kept between Islands.

Tiling Texture

- is a small, seamless image or pattern that is repeated, or "tiled," across a surface to create a continuous, uniform appearance. This technique is particularly useful for covering large areas without visible seams or repetitions, like walls, floors, or any large surfaces in 3D environments. By seamlessly repeating the texture, it allows for efficient use of memory and resources, as a single small texture can cover extensive areas without noticeable patterns or disruptions.

Swatches

- I don't think this is an official name - just what I call it. It's using small little swatches of material - that won't have any detail or baking applied - used on small parts, edges, or parts that aren't seen often.

Baking

- refers to the process of capturing detailed lighting, shading, and other surface information, such as ambient occlusion and reflections, and storing it in a texture map. These effects are created with ray tracing and add a great deal of "pop" to any model.

Ray Tracing

- is a rendering technique used in computer graphics to simulate the way light interacts with objects in a scene to produce highly realistic images.

Material

- defines how a 3D object's surface interacts with light and gives it its color, texture, and reflective properties. If you change the specular shine on two different objects mapped to a single texture - that will require two **Materials**.

Draw Calls

- for our purposes - it's every time we need to load a texture sheet to the video card - which is a resource intensive process - **Draw Calls** significantly impact performance - so you want to minimize them. Each **Material** you use - will - at a minimum - be a single **Draw Call**.

Ambient Occlusion or AO

- is a shading and rendering technique used to calculate how exposed each point in a scene is to

ambient lighting. It simulates the soft shadows and subtle shading that occur in corners, creases, and near intersections of objects, where light is less likely to reach. This effect adds a sense of depth and realism to the scene by enhancing the perception of surface details and contact points between objects.

PBR Materials

- Physically Based Rendering (PBR) uses multiple texture maps to describe how a surface reacts to light. For traditional `.S` models, you may still rely on baked lighting and a single main texture. For glTF models, PBR maps can provide more realistic reflections, depth, and surface response.

glTF

- A model source format supported by Open Rails that can use PBR materials. The `.S` file format is still supported and is not going away.

Baking ambient occlusion layers requires objects to occupy unique texture real estate. If you want to apply details or baked textures, make sure no UV island overlaps another. If you have used the "Project From View" option while UV mapping, AO baking will be troublesome unless care is taken to deal with overlaps and hidden faces. Baking also has an option to adjust the margin size; keep the margin as small as practical while still preventing texture bleeding.

Scott's Example

I'll be using the RS3 as a reference model. I typically start out with two fairly large texture sheets - one for parts that have a Specular Shine and another for Flat or no shine parts. Typically - the upper painted metal body gets shine - the lower body does not. Some parts don't look good with shine - like handrails - so experiment and see what works best for you. I'll be focusing on the main body of the model for our discussion - as the trucks/bogies are a child - forcing a **Draw Call** - so it makes sense to have a separate texture just for them.

On the following texture images - a couple things to note:

The first two images are of the same flat texture - one with **Ambient Occlusion** and one without. It clearly shows how much you gain by taking the time to **Bake AO** into your textures. It's practically required. On the first two images, you can see the **Swatches** I use on various parts of the model; the hinges are probably the most identifiable. Multiple parts overlap in these small squares, so baked Ambient Occlusion can make a mess if it is included there. In your paint program, place the **Swatch** layer above your Ambient Occlusion layer to avoid the problem.

Figure 4. Flat texture without baked ambient occlusion

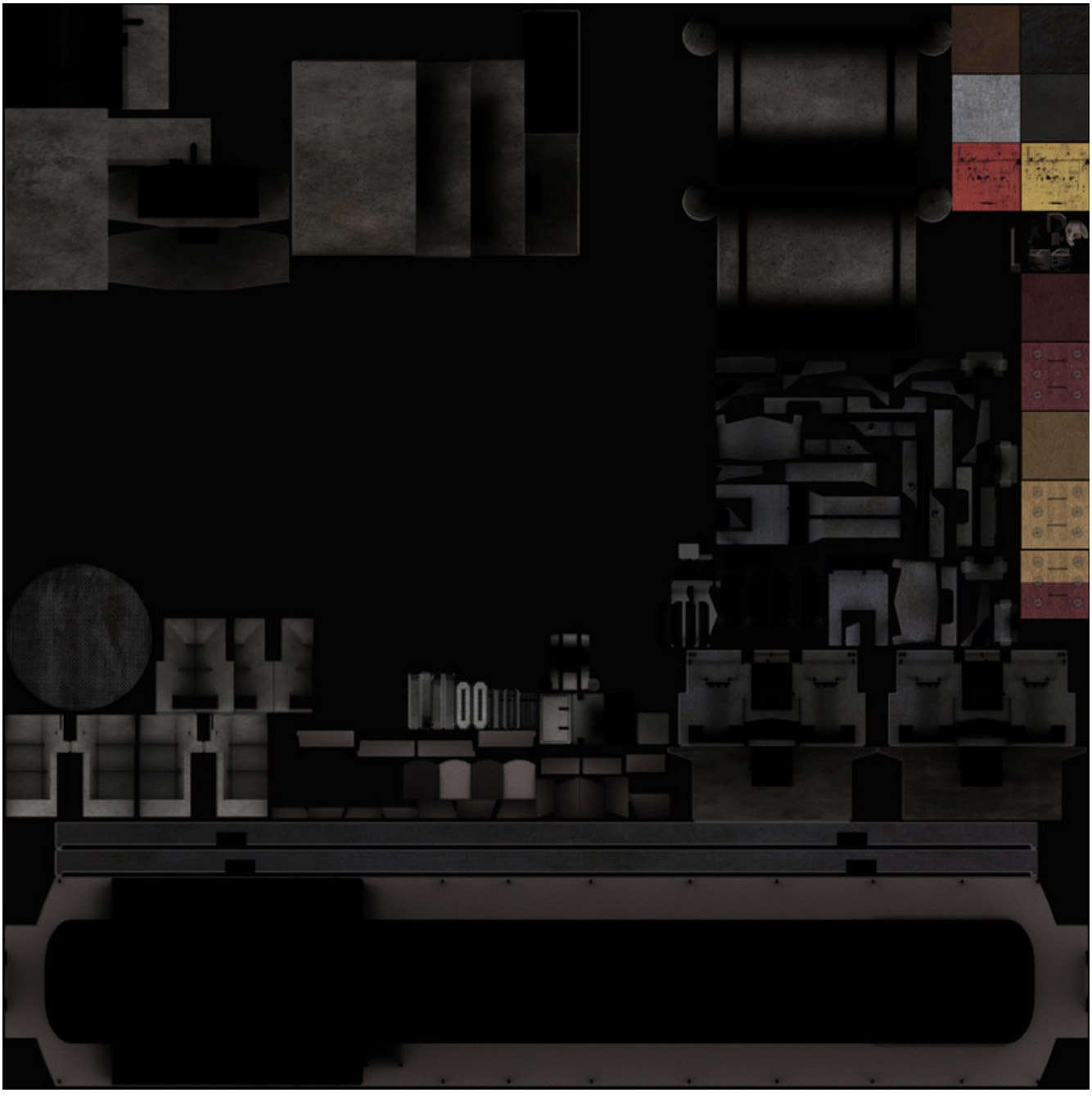


Figure 5. Flat texture with baked ambient occlusion

Figure 6. Finished texture with additional detail

Using SEAMS to Unwrap the Model

To create a seam - you must be in edit mode - select the desired edges - then **Edge › Mark Seam**.

Applicable Commands:

`L` - Select all parts by Texture or Seam.

Seams Supported Unwraps:

- **Angle Based**
- **Conformal**
- **Minimal Stretch**

Purpose of Seams

Unwrapping the Model

Seams allow you to "cut" your 3D model so it can be laid out flat in 2D space, similar to how you might cut and unfold a cardboard box. This process is known as unwrapping.

Reducing Distortion

By strategically placing seams, you can reduce the amount of distortion that occurs when the 3D model is flattened. This helps to ensure that textures are applied accurately and without stretching.

Isolating UV Islands

Seams help define UV islands, which are separate, contiguous areas of the UV map. These islands can be textured more precisely, making it easier to apply detailed textures to specific parts of the model.

Texture Alignment

Well-placed seams help align textures correctly, minimizing visible mismatches and ensuring a seamless appearance on the final model.

How to Place Seams

Strategic Placement

Place seams in less visible areas of the model, such as along edges or in natural creases, to minimize their appearance in the final texture.

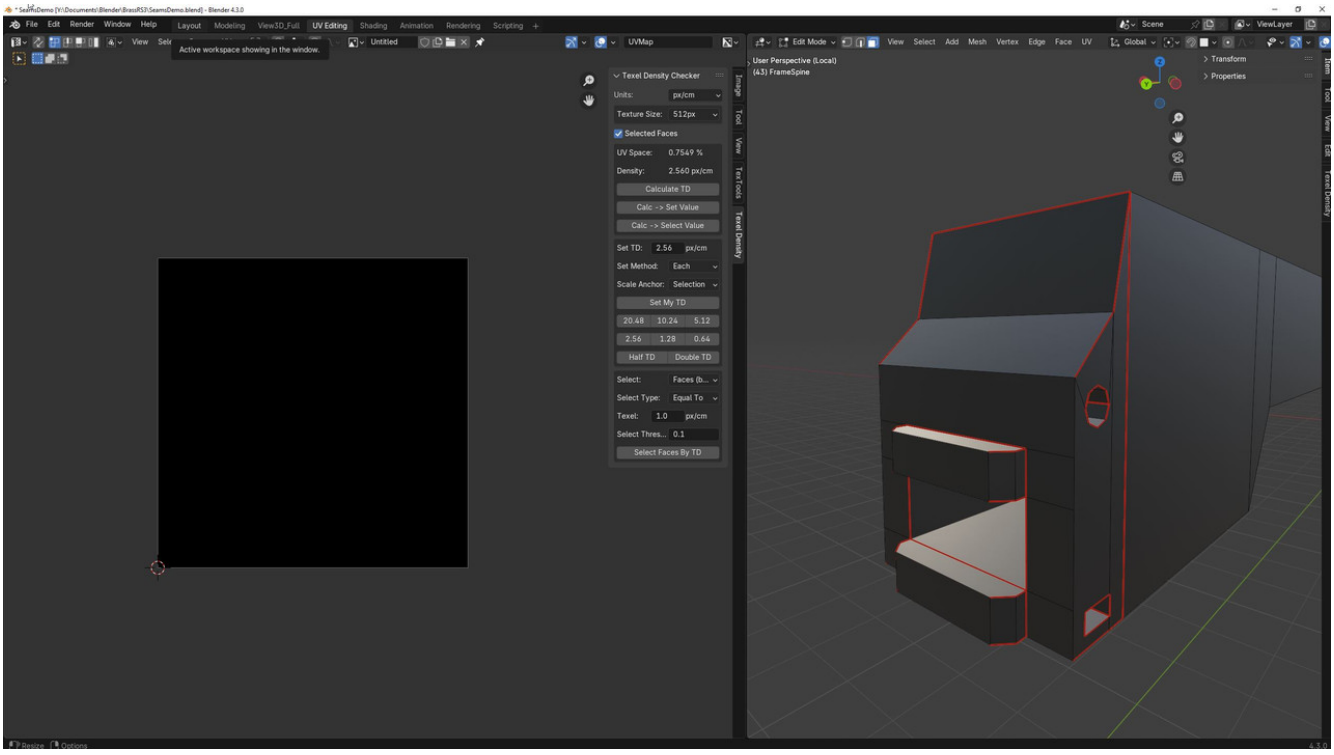
Testing Unwraps

Experiment with different seam placements and unwraps to find the best configuration that minimizes distortion and maximizes texture quality.

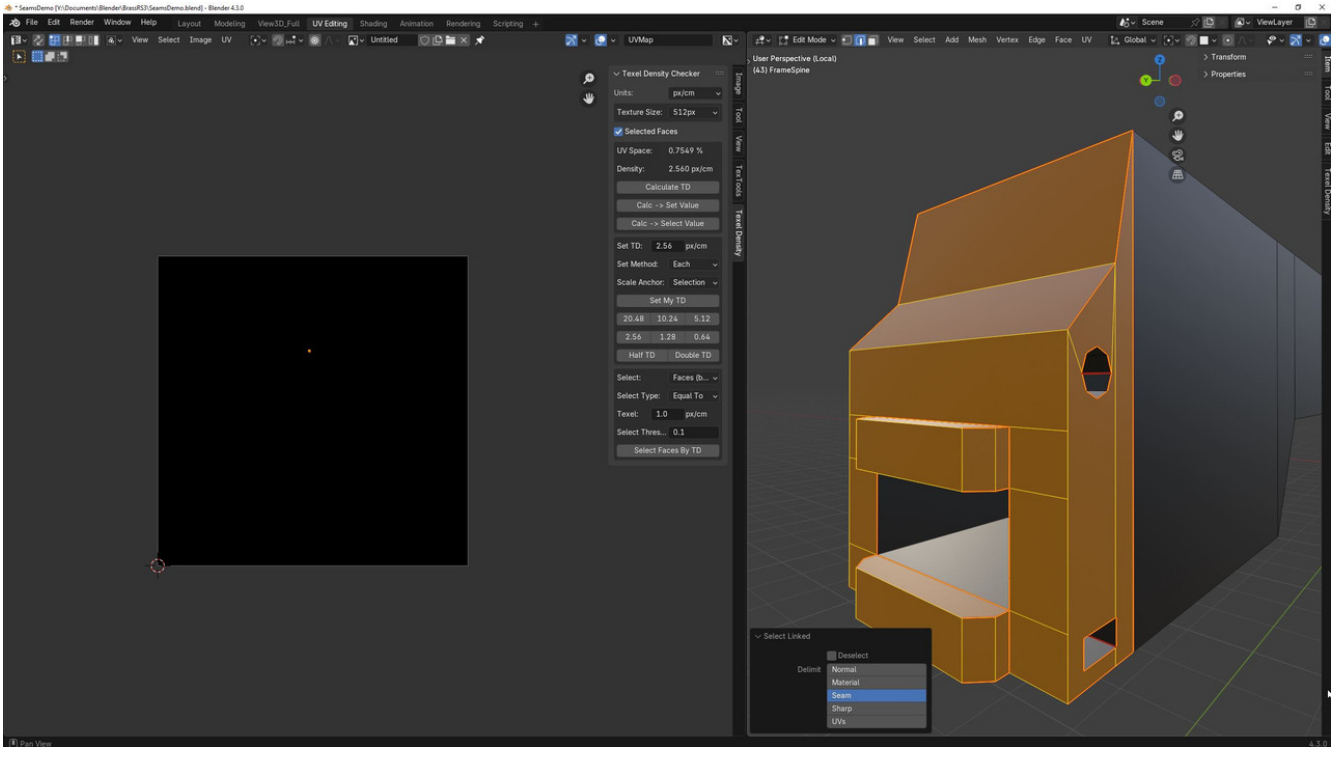
Consistency

Ensure that the seams create manageable UV islands that make texturing easier and more efficient.

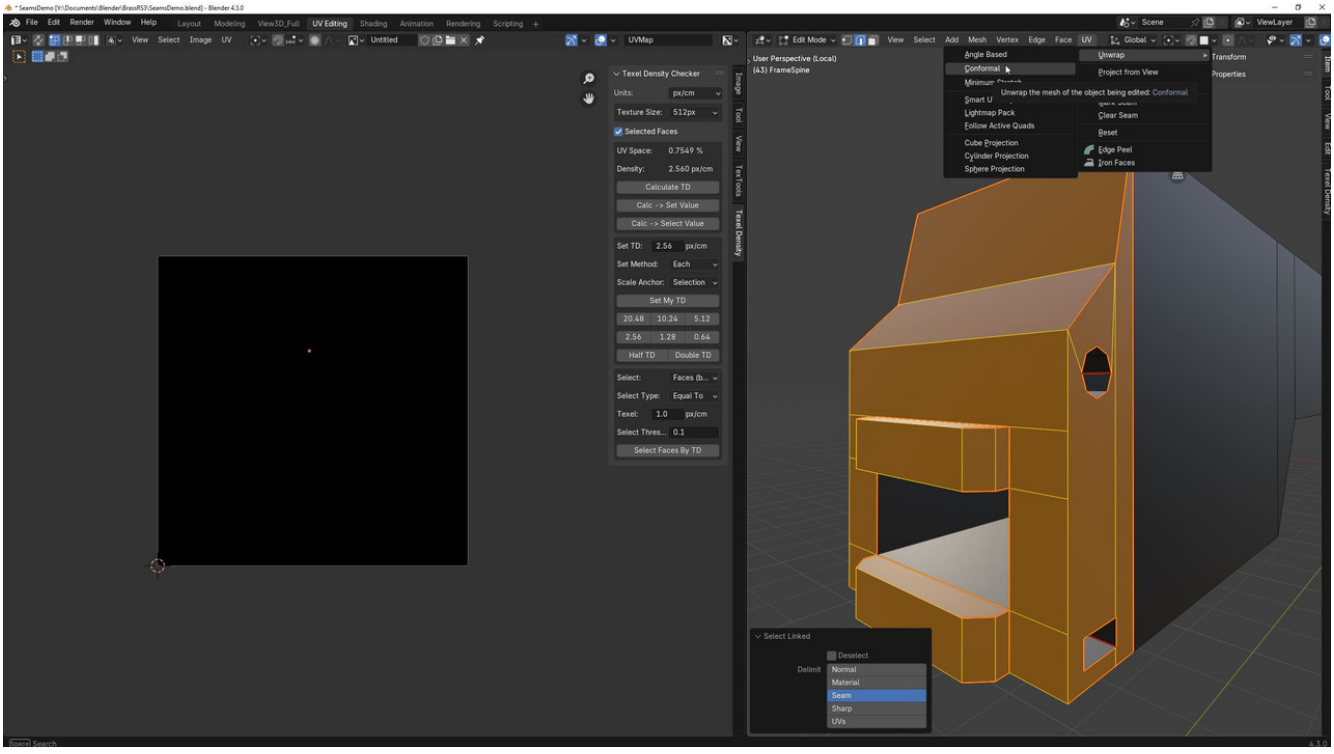
Marked Seams in Red



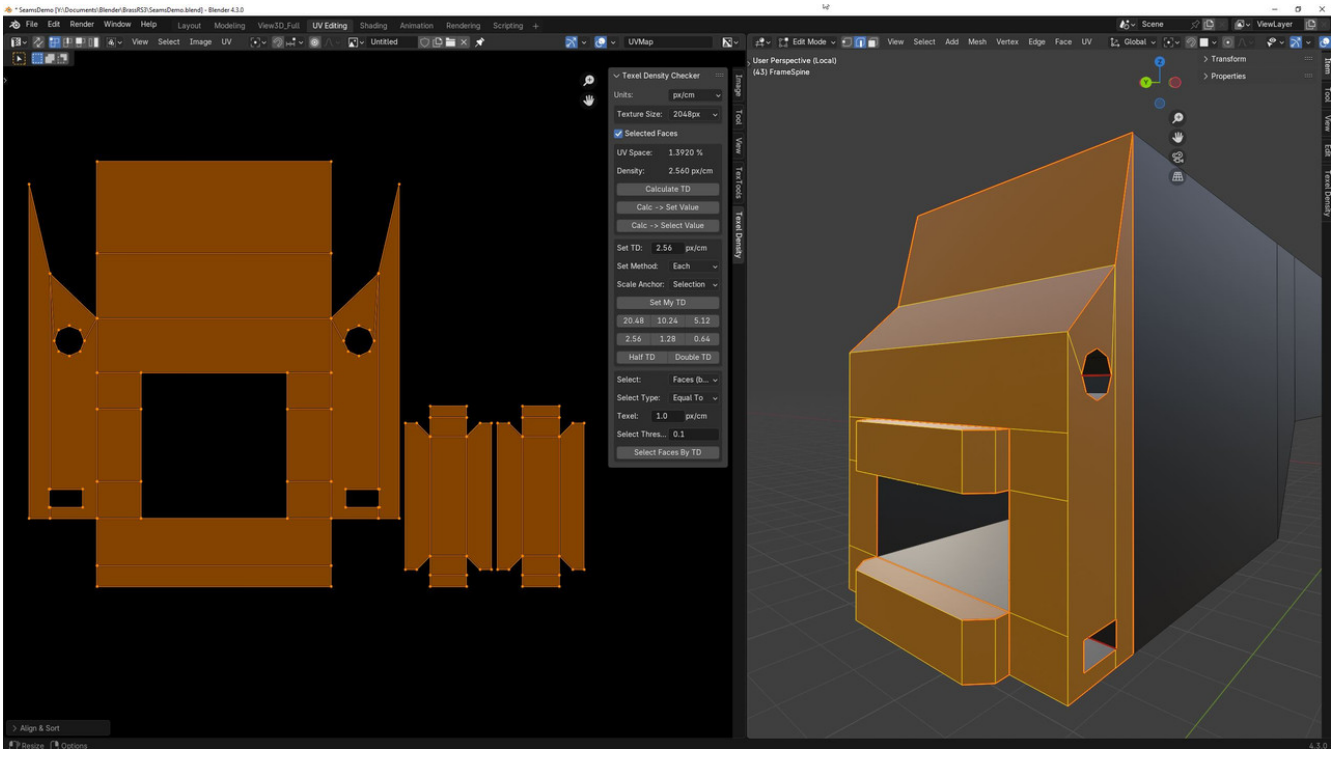
Select Desired Faces by Seam - Pressing L - Faces Must Be Fully Enclosed by Seams for Selection to Work



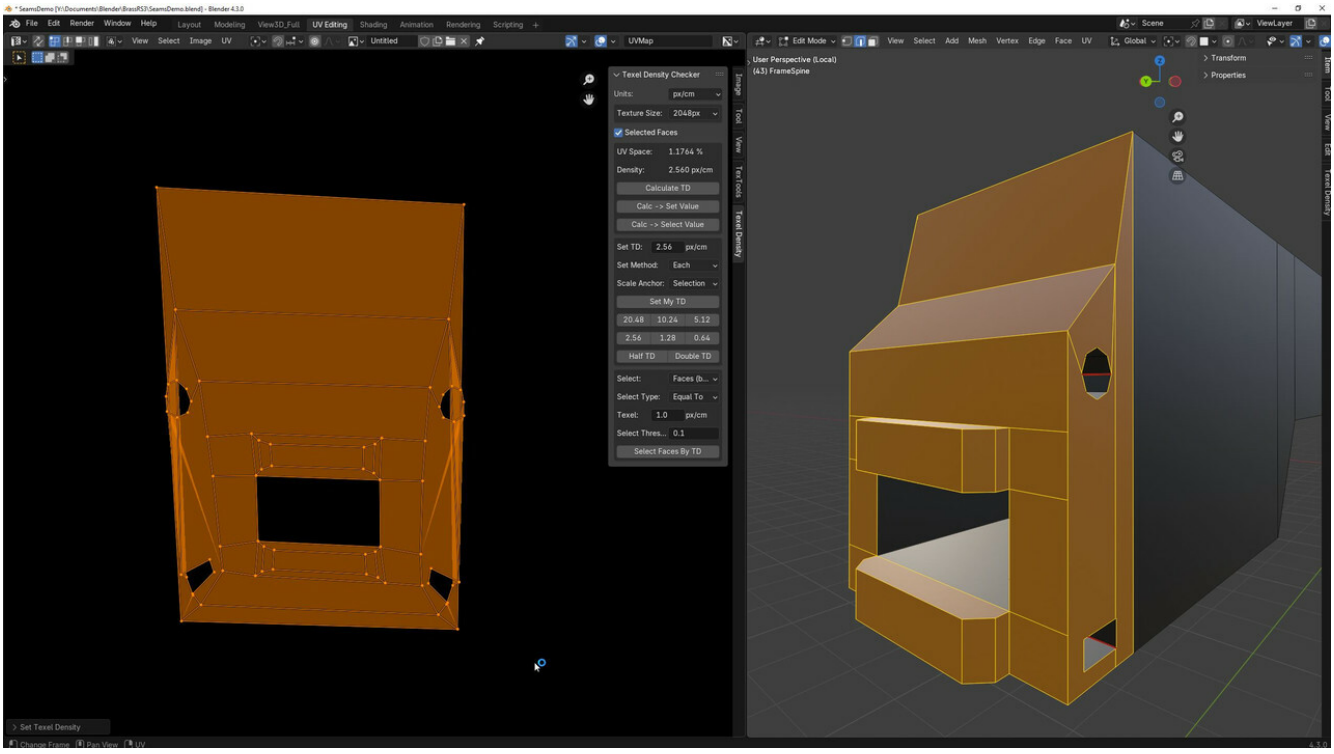
Conformal Unwrap



Results in (3) Perfectly Unwrapped Islands



Same Conformal Unwrap Without the Use of Seams - Completely Unusable

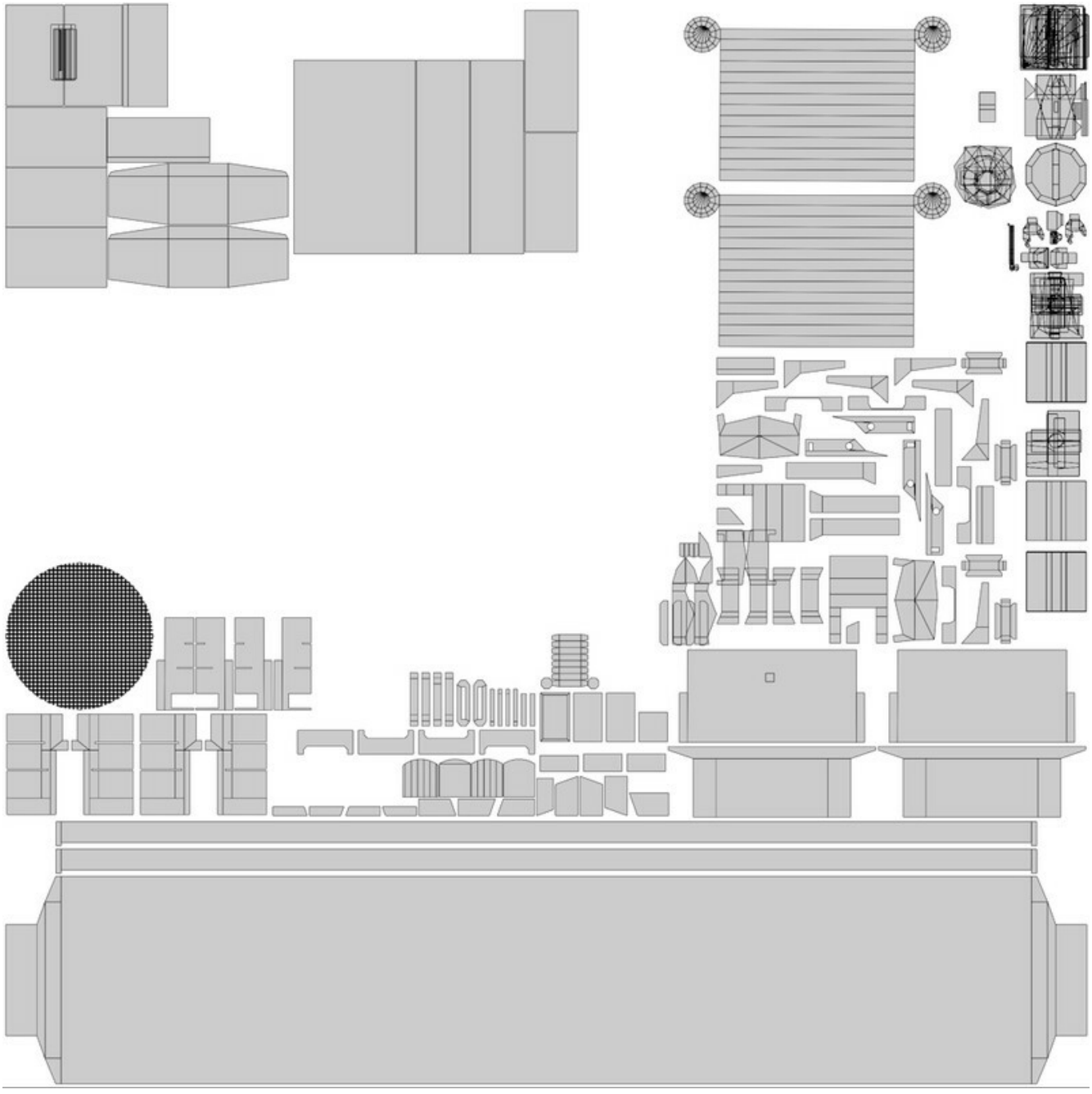


Once you have your **UV Islands** - you can set their **Texel Density** using **Texel Density Checker** - make sure you set your texture size appropriately on the top of the **Texel Density Checker** tool panel. Then just move them to where you want them placed on your texture.

You can save your **UV Map** to a texture file in the **UV Mapping** window by selecting **UV Export UV Map**. I would suggest setting the **Fill Opacity** to 1 for best results. It basically gives you a page out of a coloring book to apply your textures to.

One other important tip in the **UV Mapping** window - on the top left toolbar - there's a tiny little icon that has two arrows - one up the other down - at a 45 degree angle. This synchronizes the **UV Map** to your mesh. Where this comes in handy - you can select a **UV Island** and you may not know what it's mapped to - then move your mouse over to the **3D Viewport** and press `.` - it focuses and zooms in on the object that is mapped to your **UV Island**. Extremely helpful.

So easy a caveman could do it.



Packing Islands

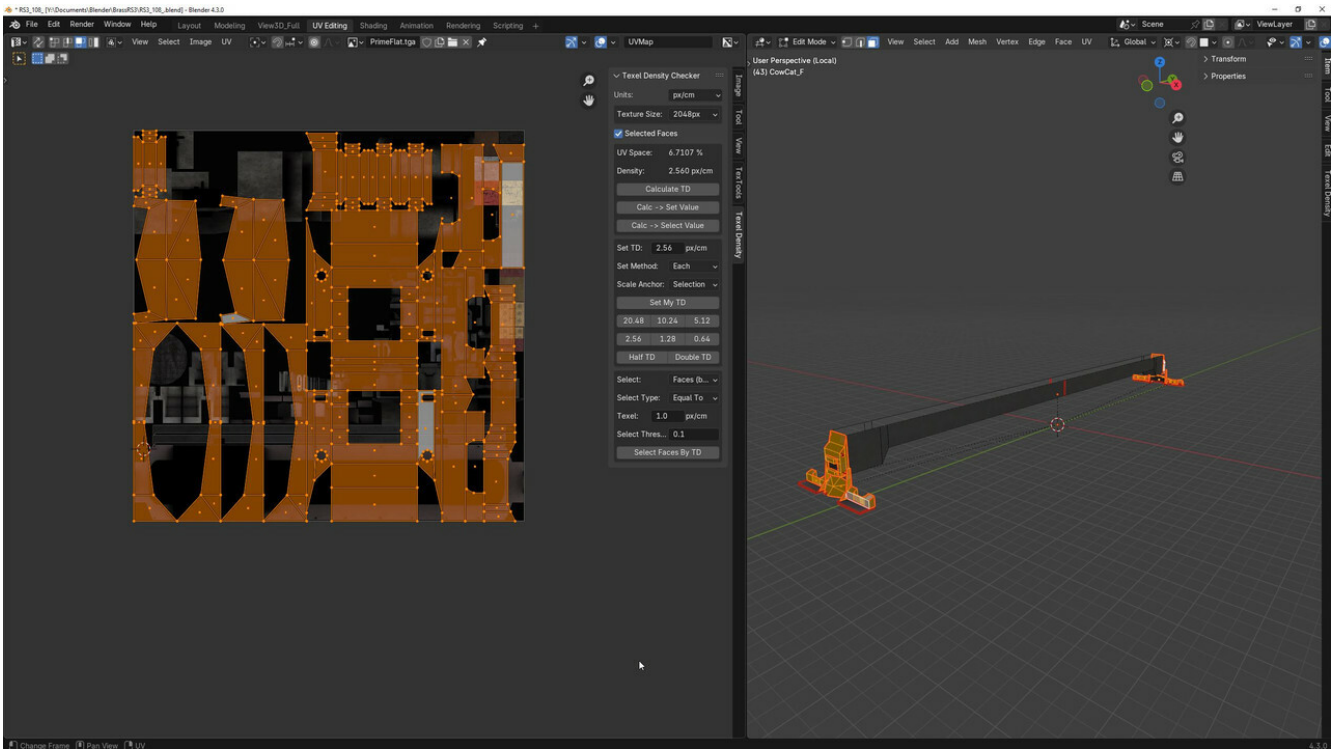
Pack Islands

- in **UV mapping** is the process of organizing and arranging **UV Islands** efficiently within the UV space to maximize the use of texture space. You can do this with entire texture sheets or any subset therein. You can define a Margin to set how much space is kept between Islands.

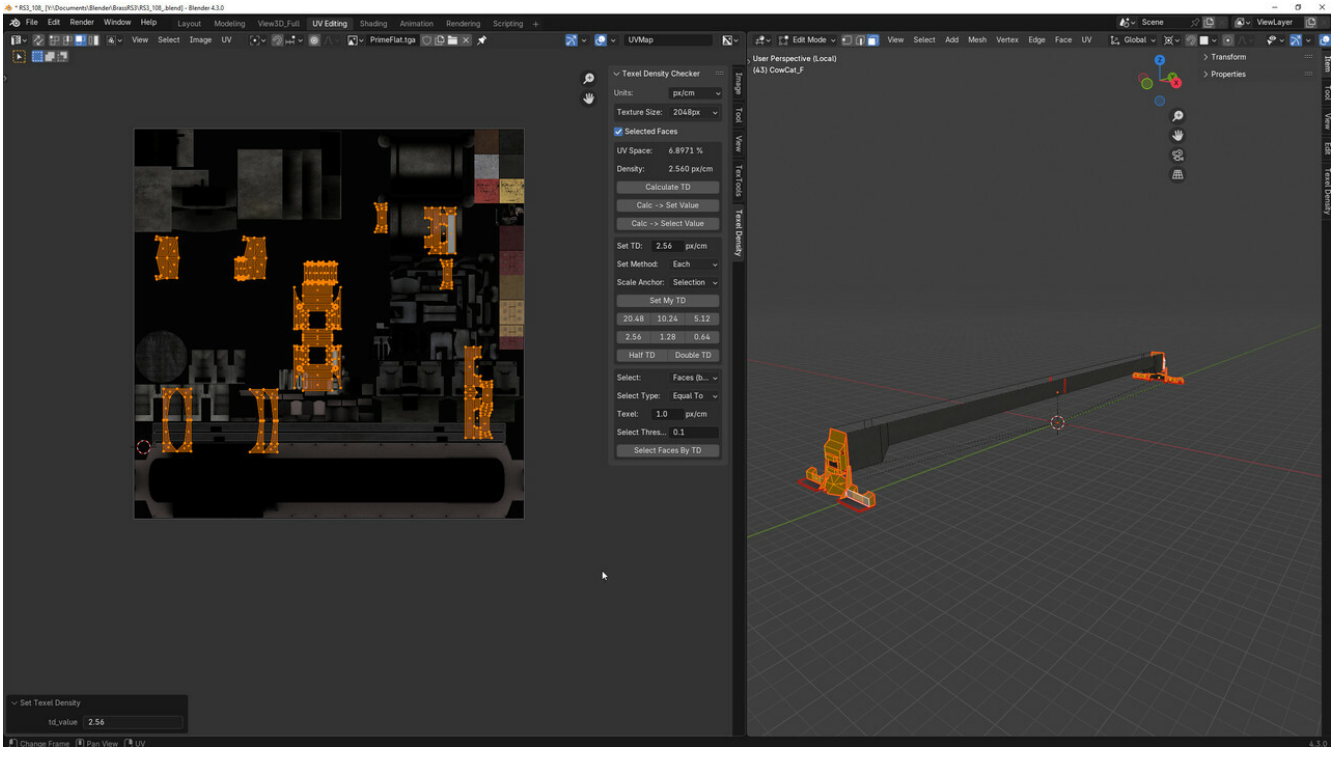
In practical use - you're probably going to be unwrapping far more faces than I did in my example above. You probably want to group them by texture used and perhaps proximity to each other. Instead of manually trying to juggle your **UV Islands** - Blender has included a great tool called **Pack**

Islands.

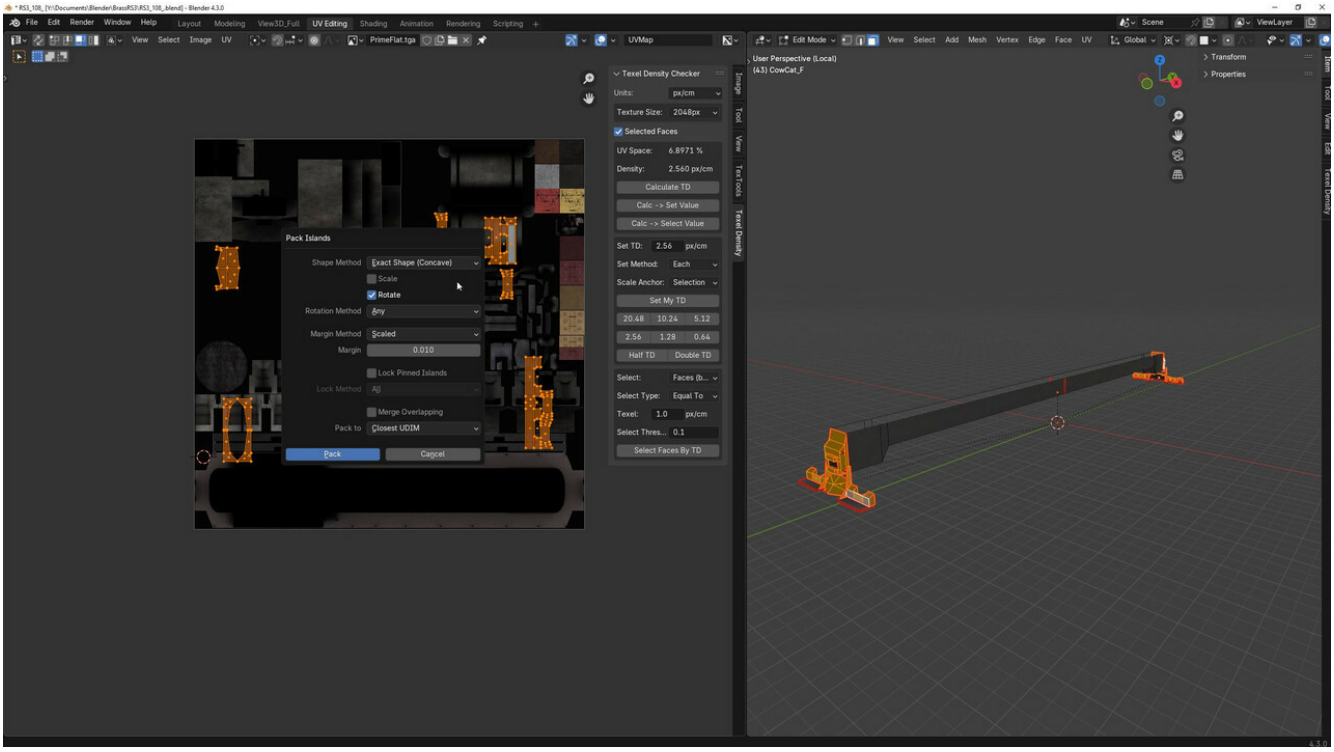
Realistic Unwrapping



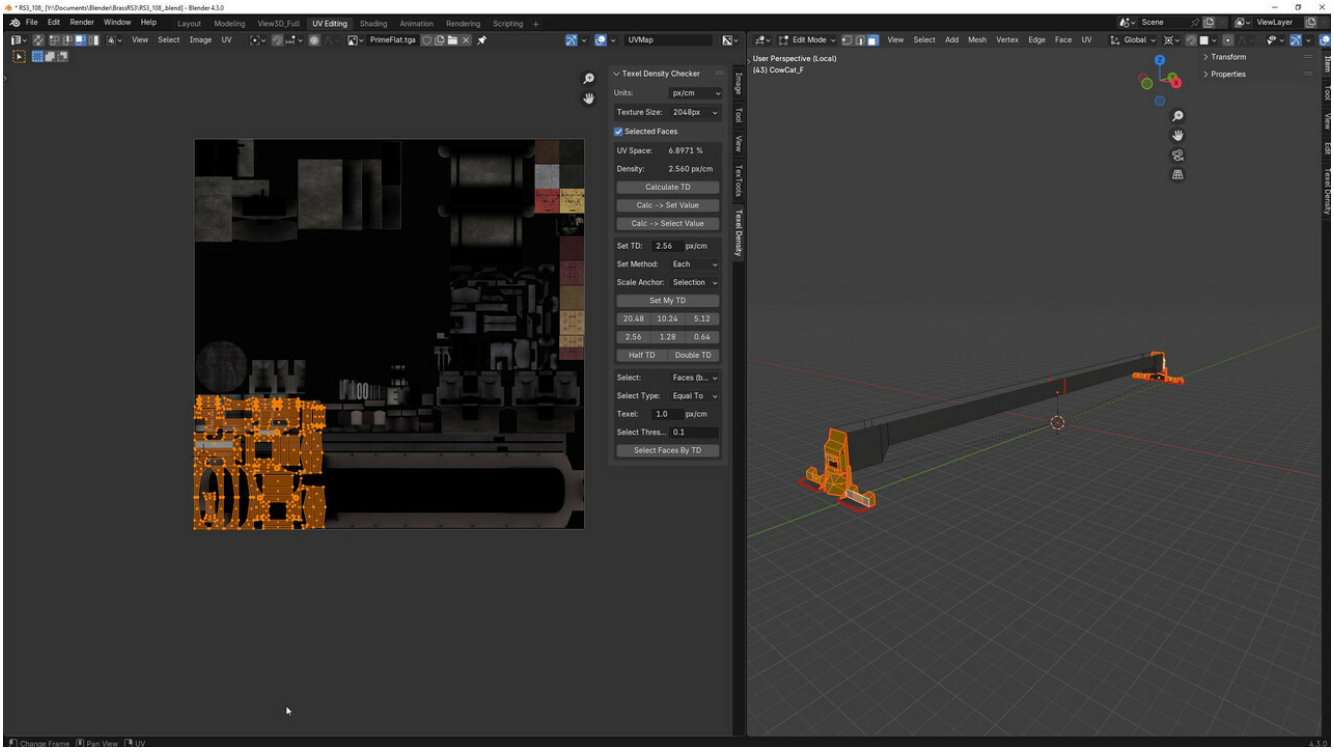
Set Your Texel Density as Required



Pack Islands - UV › Pack Islands - Uncheck Scale - Set Your Margin



Result - An Efficiently Packed Group of UV Islands



The packed group of UV Islands - is ready to be moved where desired on your texture sheet.

Windows

- Separated the glass faces into its own object.
- Apply a greyish blue texture partially translucent - `RGBA = 25, 28, 32, 220`
- In the MSTS Materials panel, set the Transparency to `Alpha Sorted, Lighting = Normal`.

Other Blender users have had success with using `alpha blended transparency`

Don't use DXT compression when you make your .ace file.

Texture Nodes

Texture nodes are a powerful tool in Blender that allow you to create complex and dynamic textures. They allow you to create procedural textures, blend multiple textures together, and apply filters and transformations to your textures. The problem is, that while these are powerful tools, the methods for creating them can be a bit daunting and not as useful as they might seem in our texture creation scenario. You would likely be better off with using the texture baking process to create your textures.

Lastly, the NODE system in Blender is a new-ish feature and is somewhat hard to explain. No additional mention of it will be made.

Decals

A newer *concept*, and one championed by the NAVS technique, is to use a separate bitmap or multiple bitmaps to generate various car numbers without having the numbers be part of the main bitmap. This gives added flexibility to car rosters as custom car numbers are easily generated without resorting to difficult post-release editing of `.ACE` or `.DDS` files as many have done in the past. Changing numbers that are part of a final bitmap are particularly difficult.

A DECAL using this method is a small section of the main model that has a smaller `3d plane` object floated just above the surface of the model. This plane is assigned a set of UV coordinates that map to a specific number or numbers desired on a particular car from its assigned texture. Using the Open Rails `INCLUDE` statement in a WAG or ENG file, you can specify decal mapping using the `FreightAnim` keyword to locate the related decal `S` file(s).

I'll share a decal creation technique here using Python code for the so inclined in the programming section of this documentation. The benefit of a code-based approach is that you have a nearly exact idea of how the texture is laid out and can easily adjust the layout to your needs. The disadvantage is that you have to have the Python programming language installed on your computer along with required graphic packages the code depends on. If you are not comfortable with Python, then you can use the manual method to lay out your decals, it's just a bit more time consuming.

Notice the outline. That is the shape of the 3D Plane object. A plane is a single sided shape that we can butt up against the car body. We rely on the Alpha Mask channel so only the lettering is visible. The UV coordinates are set to map to the lettering we want to add. The plnae object is saved as a separate `.s` file and used as `freightAnim` to customize the reporting marks.

Layering Basics

When working on a texture for a model, you are going to want to add bitmap layers that help to achieve the desired final result. This means being able to adjust the parameters and effects that each layer brings to the final result. These include making layers with more opacity so the details of lower layers are not obscured, or adding upper layers that contain effects for Grime, Rust, Dirt, Dust, as explained in the now lost "Painting Guide" that once existed on the 3DTrains.com website.

Since the 3DTrains website is now off the Internet, some of the things that I

> learned from that website will be shared here. Hopefully, 3dTrains folks don't mind that I've tried to share that information here as a sort of archive of what was at the website.

There was an explanation there that you would have you add four NEW layers to your base image and name them Grime, Rust, Dirt and Dust.

Grime Layer

- Set the airbrush tool to a width between 150 and 200
- Select a black color, lets say RGB 10,10,10, for example
- Spray all over the layer making sure it looks uneven and spotty
- Now **hide** this layer from view

Rust Layer

- With the same airbrush settings, choose a rust color like RGB 136,57,4
- Make sure its sprayed on so you can still see through it.
- Now **hide** this layer from view

Dirt Layer

- With the same airbrush settings, choose a yellowish/brown color like RGB 126,113,38
- Make sure its sprayed on so you can still see through it.
- Now **hide** this layer from view

Dust Layer

- With the same airbrush settings, choose a light color like RGB 192,192,192
- Make sure its sprayed on so you can still see through it.
- Un-hide all the layers

Finalization of effects

- Set the transparency/opacity properties of each of the new layers to somewhere between 10 and 30 percent
- Adjust the percentages of each weathering layer to get the best effects
- Save the file in the layered format

- Save a copy in the PNG format for use with Blender

Applying Fonts and Lettering

With the layered format file open, create a new layer just above the BASE layer in the document. This will make sure that the lettering being added is below the weathering effects.

While it might seem like a shameless plug for my website, you will find a number of railroad related fonts at http://www.railsimstuff.com to help with adding lettering and logo details to your textures. In case you are wondering, the fonts are all free and I make no money from this web site, in fact it only costs me money to keep it running so it is a labor of love that I have provided for nearly 20 years.

As mentioned above, the opacity of the DECAL layer should also be adjusted so it does not hide underlying details. The effect for decals though needs to resemble having been painted on, so the opacity will mbe much closer to 75% versus a lower value.

Layering Tips from Erick

Erick Cantu on Weathering

> When I'm walking around, I often find myself taking photographs of dirt, gravel, grass, concrete, and the like. You might think that I take these photos to use as textures. You're sort or right, but mostly wrong. I take these photos primarily to create layer masks used in weathering cars.
>
> — Erick Cantu

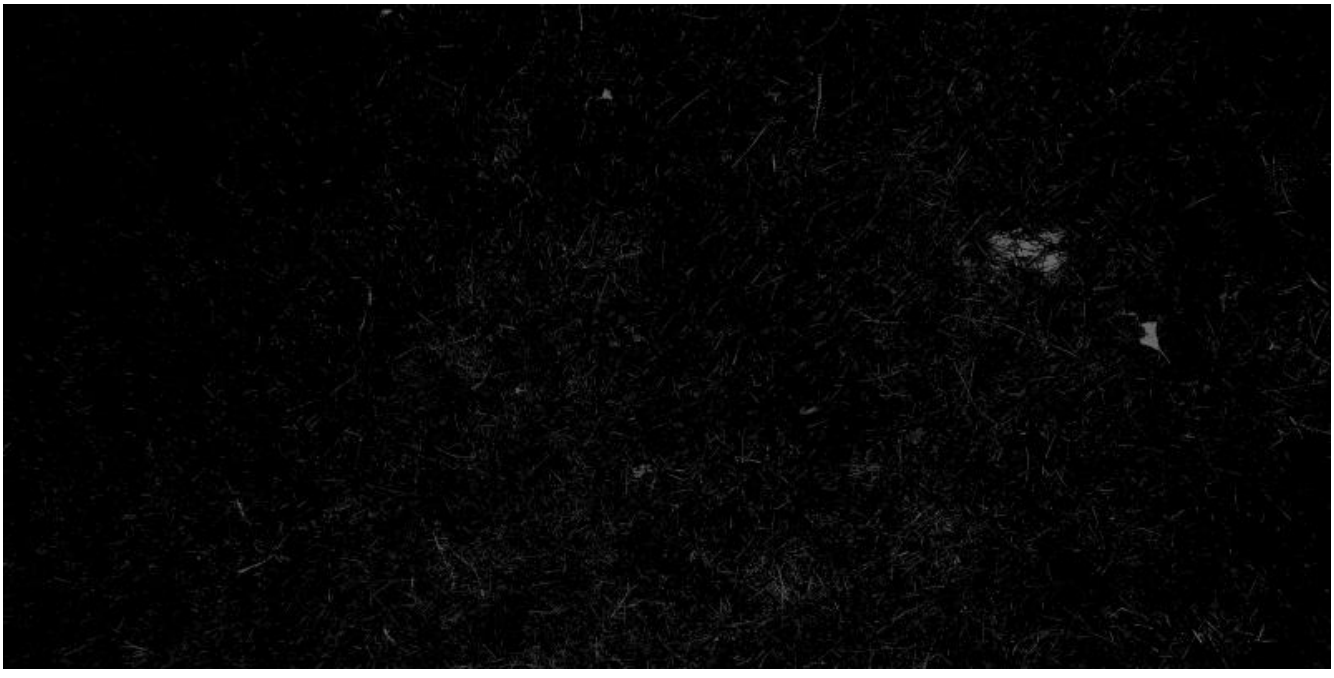
The master textures for all of my cars are always set up like this, from top to bottom:

1. A top mask to keep the overall image tidy
2. Any standalone parts that need to not be affected by the shadow layer
3. A highlight layer which adds a little bit of highlight to selected areas
4. A shadow map which contains the bulk of the detail, including panel lines, ribs, and so on
5. Several weathering layers
6. Car markings
7. The base color layer

8. A wireframe layer for reference (I never look at my cars in shape viewer as I'm painting - first, it wouldn't work, second, the wireframe layer makes it unnecessary)

Here's an example of a simple weathering technique. Freight cars get beaten up pretty severely on the road. They often end up with dents, gouges, and scratches. We can easily create textures for gouges and scratches with photographs of grass. I start with this photo:

I then turn it to gray-scale and darken it significantly while bumping up the contrast:



When you use a gray-scale image as a layer mask, pure white areas will be opaque, while pure black areas will be transparent, with values in between being semi-transparent to varying degrees. It's an opacity map. I can then take a photograph or dirt, or really any image of the right size with

some dark color, apply this image as a layer mask, set the properties to "multiply," and end up with dark, scratchy areas all over the car-body. But I can milk that image some more. If you're trying to maximize your output while minimizing your time, it pays to get the most out of all of your resources. I copy the layer, rotate it 180 degrees, invert the colors, and set the properties to "addition." I decrease the opacity to 30%. The end result is this:

Because the textures for the car are not perfectly symmetrical, it's hard to tell that the light, additive layer is the same image as the dark layer, but rotated 180 degrees. Similarly, I can rotate both images 180 degrees, change the opacity values slightly, and add perhaps another layer of spotty dirt, and those same layers easily create a car-body that looks totally different. You could go back to the original image and flip the scratch layers horizontally for a third car-body, or vertically for a fourth.

I am always on the lookout for walls with streaks of dirt from the rain, rusty metal, or anything that looks patchy. You can get so many great layer masks from those things, and most of us are carrying a perfectly-adequate camera in our pockets these days anyway. I used to hate weathering. Now it's quite easy and enjoyable, taking very little of my time.

Working with Alpha Channels

Historically, Microsoft Train Simulator has relied heavily on image transparency, the alpha channel in an image, to show intricate details on a model by applying a texture you can see through instead of having to model the actual shapes, thus replacing many object faces with a surface. It doesn't always work well, but has been used heavily by creators wanting to keep poly counts to a minimum.

In addition, you would use Alpha Channels in an image to create the translucency you see when looking at/through glass such as a window.

Both of these techniques require that you start with an image format that understands the concept of a transparency mask. In the case of "image cutting", you would rely on a 1 bit Alpha channel that is either see through or opaque, however, in the case of a Window, which is semi opaque, you would rely on a 8 bit Alpha channel with 255 shades of gray between Full Black (transparent) and Full white (opaque).

The tricky part is that every bitmap manipulation tool, GIMP, Paint.NET, PaintShop Pro, etc has their own idea about how to manipulate this Alpha Masking.

- You can't just use ANY bitmap editor to edit files for transparency manipulation… you need one that actually understands files (like TGA) with an alpha channel embedded.
- If you use TGA tools, it can separate the MAIN texture from the ALPHA channel and allow you to edit them separately (but they need to come back together at some point, as in - with a TGA FILE.)
- A .bmp FILE "technically" cannot contain and Alpha Channel… so if you are using a .BMP file… the alpha channel will just be **gone** and it would be something you need to fix/recover.

There are many formats of BMP bitmap file, including those with alpha channel support. The catch is that those such formats are not commonly encountered on Windows, and most bitmap editors won't edit or deal with them correctly. Therefore, its best to just consider that `.BMP` has no Alpha channel support… so just use `.TGA` or `.PNG`.

Creating Alpha Channels with Affinity Studio

Affinity Studio, formerly Affinity Photo and Designers Suite, is a popular photo editing software that has a built-in alpha channel editor. It allows users to create alpha channels for transparency in images. You can use formats like PNG or TIFF that support alpha channels when working with images in Affinity applications.

To begin working with alpha channels in Affinity, select the PIXEL mode and choose the layer you want to work with in the Layers panel. You would then add a MASK layer by choosing the MASK tool at the bottom of the layers panel. When you select the brush now, you will be able manipulate the ALPHA mask for the image using the brush options. White foreground brush with remove alpha mask and the Black foreground brush will add alpha mask.

When you are done creating the mask, using the brush or selection tools, you can refine it using the "Pixel" mode "Refine Mask" to visualize and refine the edges of the mask using the available 'sliders'. Apply to complete the "Refine Selection" option.

Creating Alpha Channels with GIMP

This document is a brief tutorial on Alpha Channel usage in GIMP (GNU Image Manipulation Program). It covers the basics of creating and editing alpha channels (transparency) in GIMP, including how to use the tools, how to select and refine the selection, and how to save your work.

Step 1: Start GIMP

Open GIMP (GNU Image Manipulation Program).

Step 2: Open your image

Go to `File → Open` and select the image you want to edit. TIP: GIMP supports many formats, such as `.png`, `.jpg`, `.bmp`, and `.tif`.

For this example, we'll use a `.png` image that has a sky background that we want to make transparent and a cityscape in the foreground we cant to keep.

Step 3: Add an alpha channel

To allow transparency, add an alpha channel if one doesn't already exist: `Layer → Transparency → Add Alpha Channel` (If this option is grayed out, the layer already has an alpha channel.)

Step 4: Select the image area

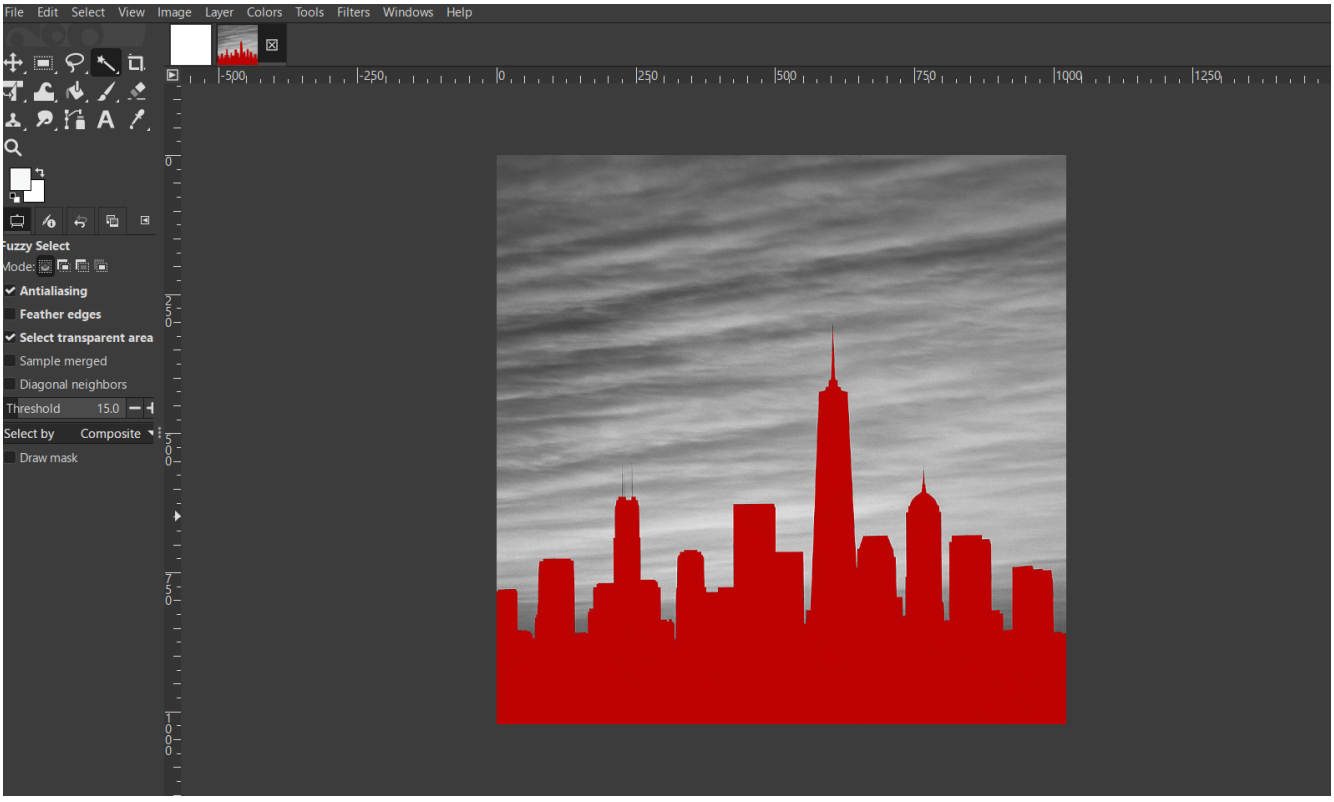
Choose the **Fuzzy Select Tool** (the "Magic Wand" equivalent). Click on the part of the image you want transparent. Adjust the **Threshold** in the Tool Options to expand or limit the selection.

It might take a few tries to get the selection just right. Or maybe even invert the selection if you

selected the wrong part or selection is easier. To do this, choose `Select → Invert` or press `Ctrl+I`.

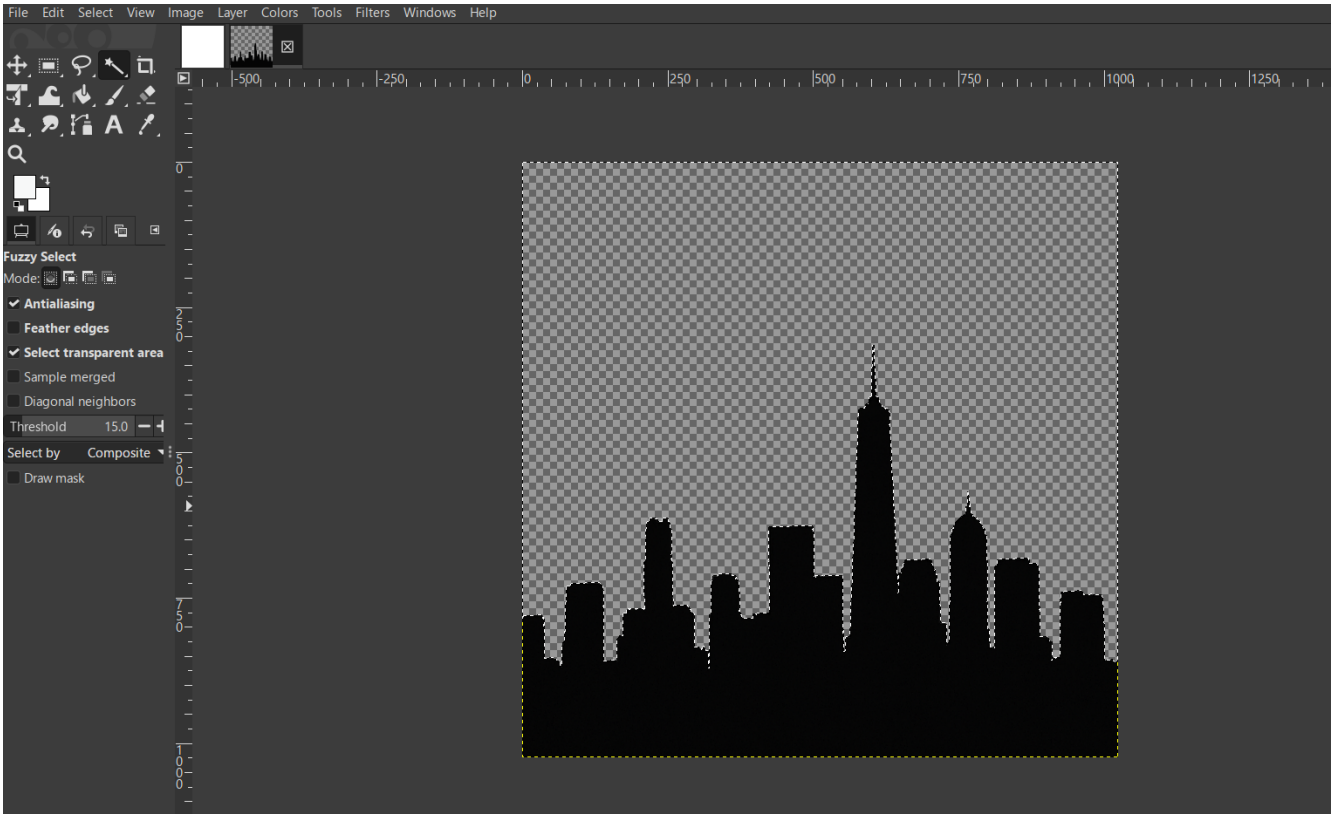
Step 5: Refine the selection

Use the **Select → Grow** or **Select → Shrink** commands to fine-tune edges. You can also use **Quick Mask** (`Shift+Q`) for precise painting.



Step 6: Clear or fill the selection

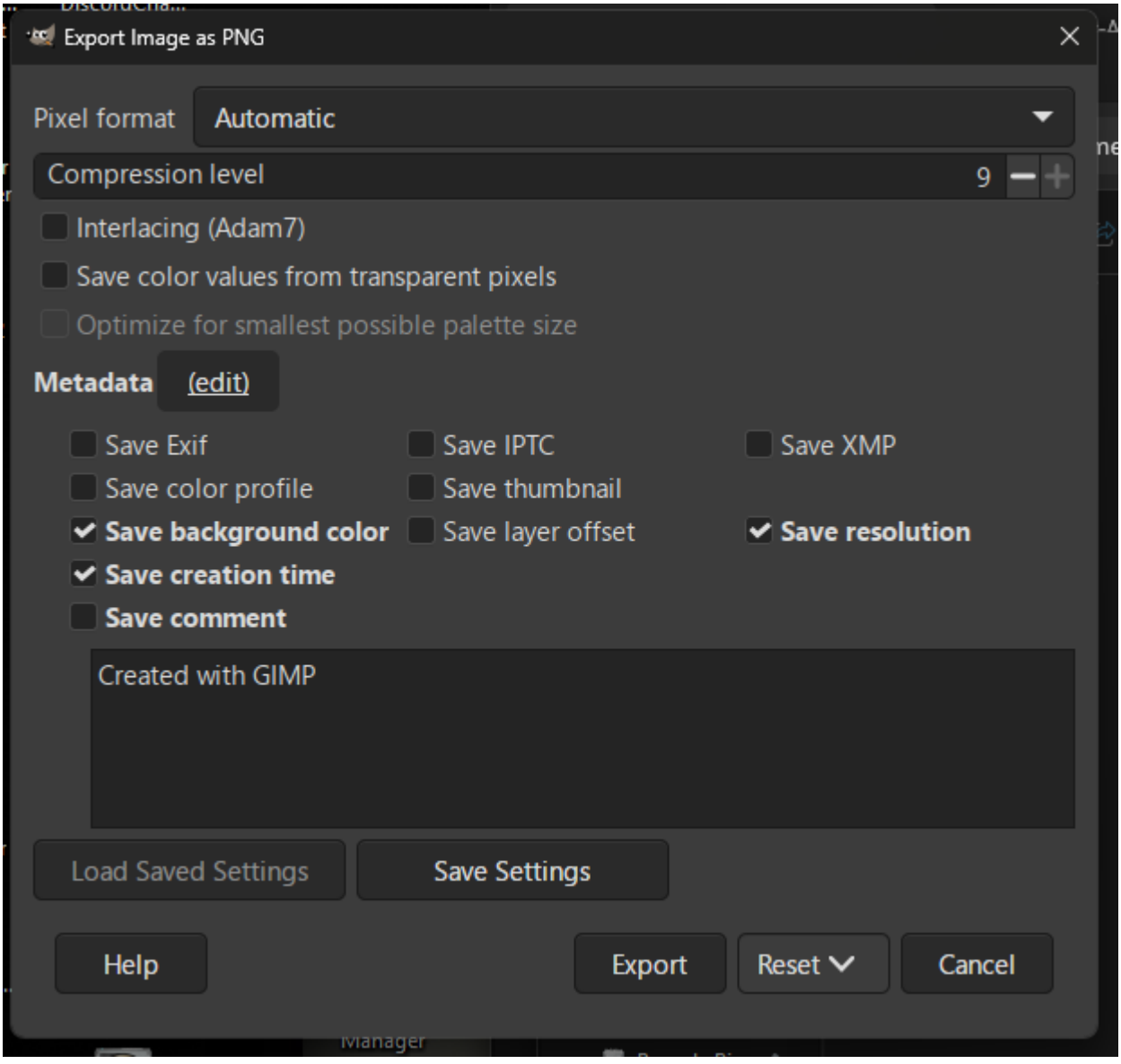
With the selection active: * Press **Delete** → removes the selected pixels (they become transparent). * Or go to `Edit → Fill with FG/BG Color` (set FG or BG to black/white for alpha control).



In GIMP's alpha channel, * Black = fully transparent * White = fully opaque * Gray = semi-transparent

Step 7: Save your work

To preserve alpha transparency, export to a format that supports it: * `.png` (recommended for web and most modern uses) * `.tga` (if specifically required) * `.xcf` (GIMP's native format, for keeping layers and masks editable)



Use `File → Export As …` and select your preferred format. For PNG, ensure **Save color values from transparent pixels** is enabled. For TGA, disable **RLE compression** if compatibility is an issue.

Step 8: Optional: Use a layer mask

Instead of directly editing transparency, you can create a layer mask: `Layer → Mask → Add Layer Mask` → **Initialize to Selection**. This gives you non-destructive control over transparency and can be applied later.

Gimp Alpha Wrap Up

Remember to save your project frequently in GIMP's native `.xcf` format, and keep backups of original images.

TIP: Using a **layer mask** is usually safer than directly erasing pixels. You can always refine the mask with brushes, gradients, or filters — and convert it back into an alpha channel at export.

Image Tools

This section lists some of the "extra" tools and functions that are used to create or work with textures.

Measuring Tools

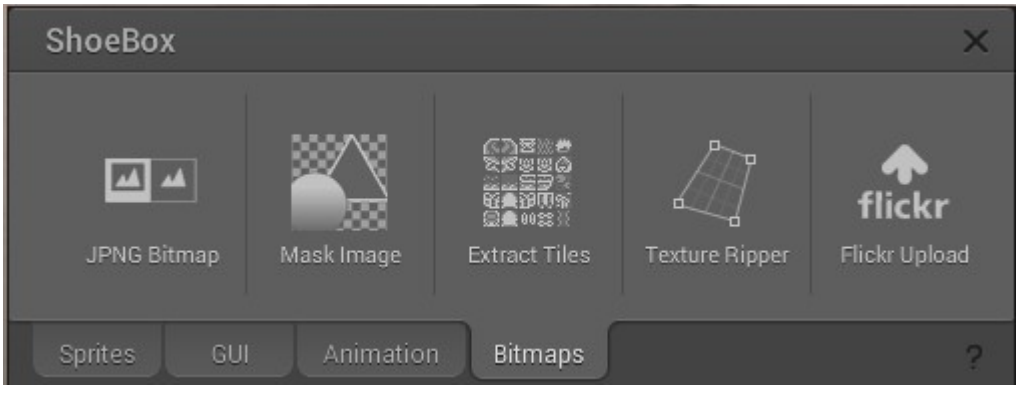
Measurements from photos. Not perfect… but it helps.

https://eleif.net/photomeasure

Dealing with perspective distortion

For texture pre-processing, particularly for photo-based texturing, use a perspective-correction tool when the photo was not taken square-on to the object. The older tool many content creators used for this was Shoebox: https://renderhjs.net/shoebox/. Shoebox still works for some users, but it requires the Adobe AIR runtime, which is now maintained by Harman rather than Adobe: https://airsdk.harman.com/download

For current work, try TextureRipper first. It provides a modern replacement for the texture-ripping workflow without depending on the abandoned Adobe AIR runtime.



The "texture ripping" option in Shoebox allows you to clip pieces from a master image and adjust the alignment while you do it. This is a useful way to get a good start on photo-based textures if you already have Shoebox installed.

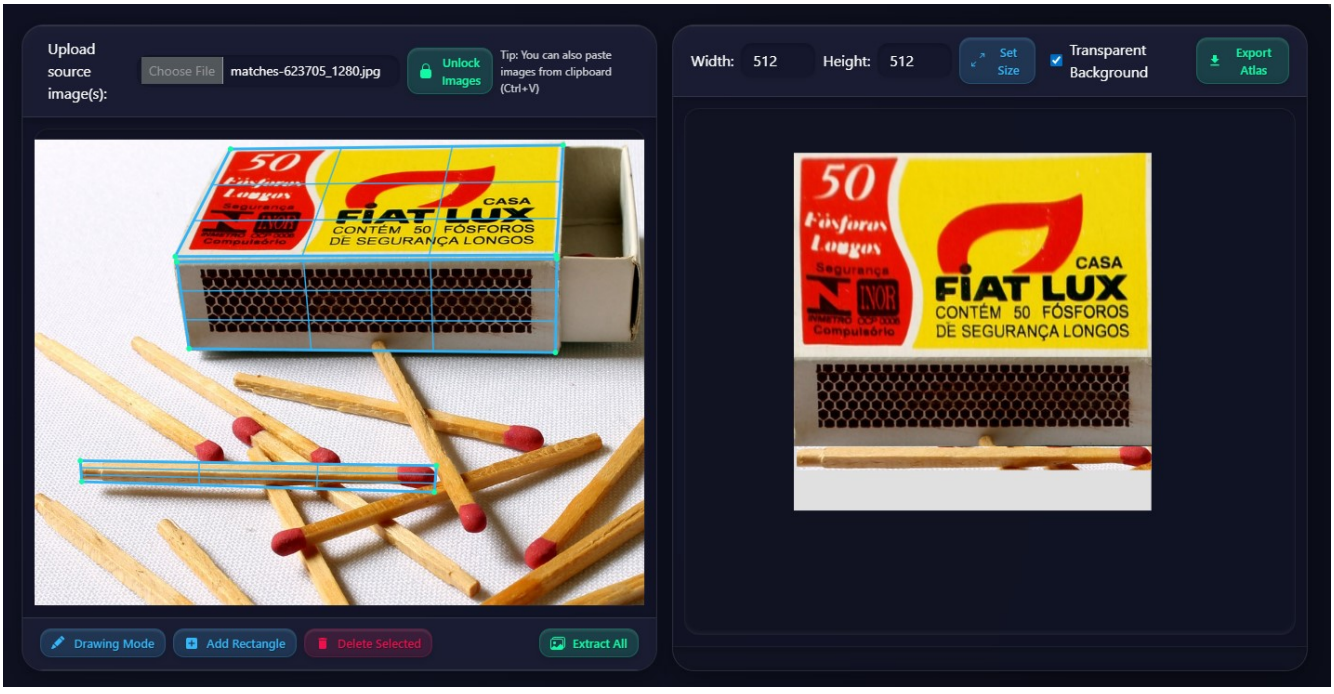
The image to be manipulated should be in your copy/paste buffer before starting the texture-ripping process. You then click four adjustment points on the image, and the program adjusts the image to fit those four points.

You can then save the image to your drive as a texture file you can import into your bitmap editor.

TextureRipper

Raycastly TextureRipper is a free, open-source tool for fixing perspective distortion in photos used for texture creation. It lets you extract usable textures from angled source photos so they become easier to turn into accurate textures for 3D models. Download it from its GitHub repository: https://github.com/raycastly/texture-ripper

TextureRipper was inspired by the Shoebox Texture Ripper, but it uses a newer interface and avoids the deprecated runtime dependency. If you are starting fresh, this is the better first choice.



ACE2BMP

ACE2BMP is a command line tool - to use in scripts - to covert ACE texture files to BMP texture files. It was created by David Webb and tested by me. Usage: ACE2BMP.exe [Source Ace] [Destination BMP]. (issued under GNU General Public License) By Scott Brunner

It is available at ElvasTower Downloads section. For the technical types, the original author's source code is available at https://github.com/djw-zr/ace2bmp

ACEIT

AceIt is a program designed to produce Kuju ACE format texture files from many common image formats. It is intended to be used as a replacement for the Microsoft Train Simulator-supplied tools `MAKEACE` and `MAKEACEwin` tools.

AceIt contains the newest and best methods for working with .ace files.

- You start with a .bmp file which you can generate using MS Paint or other drawing program. (Unless your texture has an alpha channel (transparency) then you would generate a .tga file).
- Then run Aceit.
- Enter as the input the .bmp file you created then enter where the output will go (usually your routes texture folder)
- Then finally, save it.

Aceit itself is very old and it may have difficulty being installed ot get running on modern Windows systems. You may need to run it in compatibility mode for Windows XP SP3 or consider using ACE2BMP utility.

TGATOOL2 and MAKEACE tools

If you want to continue to work with `.ace` file format defined by Microsoft Train Simulator, then these 2 tools, TGATool2 and MAKEACE were pretty much the starting point for texture manipulation. These tools are quite old.

TGATool2

a standalone program for working with the `.bmp`. Targa `.TGA` and `.Ace` files used by Trainsim. It works in conjuction with the MAKEACE utility. There is an updated version in ACEIT.

MAKEACE

The MAKEACE utility converts industry-standard image file formats into the Kuju image file format known as ACE (extension .ACE) which can be used with Microsoft Train Simulator. MAKEACE supports Windows Bitmap files (extension .BMP) which contain 24-bit RGB encoded images and Truevision Targa files (extension .TGA) which contain 24-bit RGB encoded or 32-bit RGBA (Red, Green, Blue + alpha "translucency" channel) encoded images.

The default MAKEACE utility that was supplied with Microsoft Train Simulator has limitations and it is highly recommended that you replace it with the ACEIT utility. At the very least, get the MSTS TOOLS 1.4 update.

I no longer recommend using TGAtools2 from MGGrapphics unless you are just trying to recover images from existing ACE files.

When you need to modify a texture that has been saved in the .ace file format, TGATools2A is typically the software of choice for opening the .ACE file. You can obtain TGATools2A through an Ace-It utility installation. Once you have the .ace file open in TGATools2A, it's advisable to promptly save it as a .tga file.

While the .ace format is primarily used within MSTS (Microsoft Train Simulator), the Targa format (.tga) is more versatile and compatible with a wider range of software applications. TGATools2A offers the capability to independently edit the color and alpha components of the texture. Alternatively, you can also open the .tga file in software like Photoshop, PaintShop Pro, etc and make direct edits.

It's important to understand that simply "applying alpha" to any texture and expecting it to function seamlessly on any model is not always feasible. While it's possible to "transparently remove" portions of a model by adding an alpha channel to the texture, it may also be necessary to modify the shape file to properly utilize the alpha component of the texture. However, if your goal is to merely correct the existing alpha, and the shape is already set up for it, these considerations may not be applicable in this context.

DXTBMP

This is a tool that is also available from MWGFX website, as is TGATOOLS2. Images can be passed to any paint program for editing in 24 bit and then re-imported and saved in any of the 16/24/32 bit formats. Transparency (Alpha) channel of textures can be viewed and edited separately from the main image. It supports `.DDS` files.

Example Session:

- Open the "Original " ace with the alpha included.
- Under the "Alpha " setting go to extract the alpha.
- Click `yes` , `ok`
- Name the "alpha" you extracted and place in the `ace` you created.
- Open your image with DXTBMP and import the "Alpha" into your image.
- At the bottom of the "Alpha" setting, Click `Apply Alpha to Image` and you should be done.

This applies too both `ace` and `dds` files

It’s really just easier to use a tool like Paint.net for creating the `.DDS` file.

ACE2DDS

ACE → PNG → DDS Conversion Tools - These utilities simplify working with Microsoft Train Simulator (MSTS) and Open Rails legacy ACE texture formats. https://github.com/pwillard/Ace2DDS/releases

ACE2DDS went through a start-over release in version 1.2. The older Microsoft DirectX `tex2dds.exe` / `texconv` path has been replaced because it did not generate reliable DDS output for this workflow. Current PNG to DDS conversion is handled by the PowerShell `png2dds.ps1` script, which uses ImageMagick.

The download contains these main tools:

- `ace2png.exe` - This tool converts the ACE texture(s) file to a PNG file.
- `png2dds.ps1` - This PowerShell script converts PNG file(s) to DDS texture files using ImageMagick.

Both tools are command-line tools, so you can run them from a command prompt or terminal window. *Items in brackets are optional arguments.*

To use the `ace2png.exe` tool, run it with the following command-line arguments:

```
usage: ace2png.exe [-h] [-o OUT] source
```

Replace `input.ace` with the path to the ACE texture file you want to convert, and replace `output.png` with the path where you want to save the PNG file.

To use the `png2dds.ps1` tool, run it from PowerShell 7 or newer with the following command-line arguments:

```
png2dds.ps1 [[-Source] <path>] [[-Destination] <path>] [-Recursive]
```

`Source` can be a folder of PNG files, `Destination` defaults to a `dds` folder, and `-Recursive` processes PNG files in subfolders. The script checks the image channels with ImageMagick and selects DXT1 for images without alpha and DXT5 for images with alpha.

You can process entire folders of ACE texture files at once with `ace2png` if you supply a folder name as the source. For PNG to DDS conversion, run `png2dds.ps1` on a folder and let it write the converted files to the destination folder.

Alpha materials in Blender

If you want to just keep your hair in your head, versus pulling it out… here is a recommendation.

- Use Paint.Net or GIMP or Affinity Studio
- Use the NATIVE format of your selected Bitmap Editor as your master file. (They will support "layers")
- Export your bitmaps in TGA or PNG format for use as Blender Texture files.

Affinity products do not support exporting to DDS directly, but you can use Paint.NET, GIMP, or command-line tools to convert the file used by Blender into the DDS format usable with Open Rails.

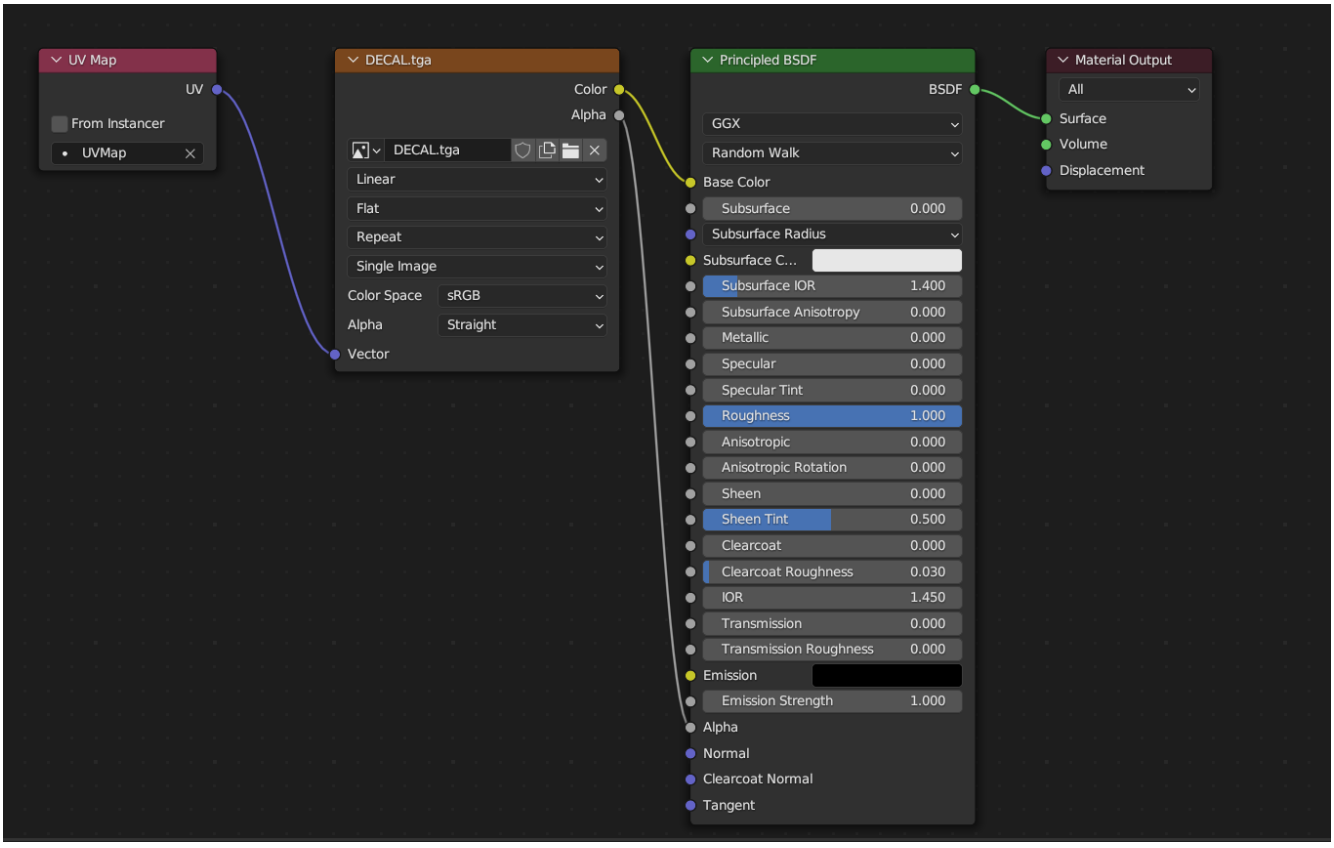
When you export your model to the S file format using the MSTS/Open Rails exporter script, export the DDS texture from Paint.NET, GIMP, or a command-line converter as a follow-up step.

Remember that having fewer texture files for your model reduces *draw calls* at run time. A larger texture uses more video card memory, but modern cards are designed for this.

The trick with Alpha channels in Blender is to remember that the hard work is done for you. You don't need to put things like **window glass** in a separate object collection.

The Shader Editor

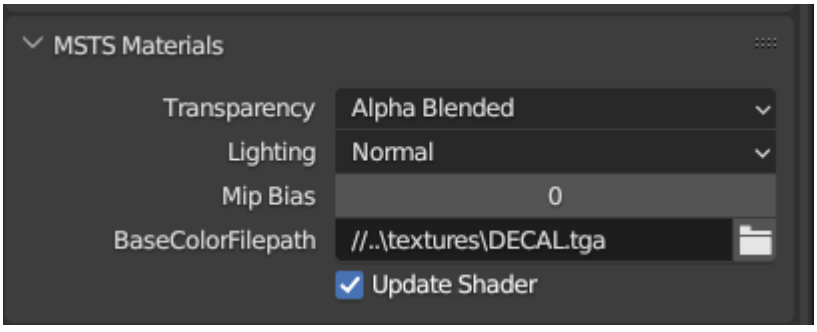
If you look at a typical setup in the Shader Editor for a texture that contains an alpha channel, you would see this.



It looks straightforward, but heed this warning: this is related to what you SEE, not necessarily what you export.

The MSTS Materials Settings

For exporting to Open Rails and Microsoft Train Simulator, you need to also have the MSTS Materials settings adjusted. See below:



For most scenarios where an alpha channel is used, this `Alpha Blended` setting is good enough and it works OK in my experience, so try it first. It is the grayscale alpha channel used for glass, while `Transparency ON/OFF` is the one-bit mask where only black RGB (0,0,0) is transparent in a mask.

Making PBR Textures

When using the new glTF model source format, it is possible to make use of PBR materials. By using PBR materials, we get a more dynamic result in how lighting and shadows work on a completed 3D model. The result is added realism, making objects look less flat and unrealistic in certain situations. The glTF format is an open standard supported by a multitude of existing products, unlike the proprietary `.S` and `.ACE` file formats inherited from Microsoft Train Simulator.

You will now be able to modify the shine or dullness of a surface as well as roughness and smoothness on surfaces. You can even infer a certain level of depth by using a form of a height map.

The terms used to describe these additional image maps are:

1) Base Color - The primary texture, sometimes referred to as "Albedo" or "Diffuse" 2) Normal - The Height Map, sometimes referred to as the Bump Map. 3) Emissive - Light Emission 4) Roughness - Wood Dull, Metal Shiny 5) Metallic - Metal or non metal amount 6) Ambient Occlusion - Shadows

PBR Materials still support Alpha Channels, don’t worry.

PBR Workflow

If you decide to use 3rd party tools to help with your PBR image workflow, remember the alternative terms mentioned, (Albedo and Diffuse) as they are often used to refer to the Base Color of your texture. There are some free PBR image generation tools as well as some PBR related add-ons for Blender. These will be discussed later.

If you have used Blender for a while, you have probably already seen the image adjustments for PBR in the "Principled BSDF" shader node as Metallic, Roughness, and Normal inputs.

Your first step is to generate the Base Color texture. When using Microsoft Train Simulator or traditional Open Rails models, this is actually where you stop in texture generation. With PBR textures, it’s just the beginning. Base Color is the name of the input used on the "Principled BSDF" shader node for this type of image.

A metallic map is next in the list and is used to simulate a metal surface, which often reacts with light in a specific way. The Metallic Map is often a binary color image that is made of just black and white pixels (0 and 255) color values. The metallic setting in the shader can be adjusted from 0 to 1. Color Space is non-color for this image.

The Roughness Map shows how rough or smooth the surface is. This affects the sharpness of

reflected light, in essence, how much light is scattered by the surface to produce gloss or matte surfaces. The Roughness setting in the shader can be adjusted from 0 to 1. (0 = gloss 1 = matte) Color Space is non-color for this image

The next step in PBR texture generation is generating the Normal Map. You might recall from other sections in this document that a Normal is related to the Facing Direction of a 3D surface. Whenever light hits this specific surface, the normal setting would be able to allow for the light reflection of the "outside" surface. The NORMAL is also a determination of the inside and outside facing direction of a surface. An improperly facing normal will cause incorrect lighting of a surface to occur.

So, how does the Normal PBR texture work? The Normal Map is used to develop a sense of depth in a surface or in other words, the bumpiness. The RGB values in the image translate to XYZ coordinates without changing the underlying geometry. A normal map can be used where extra geometry in the underlying model would be costly. (Think: doors on the long hood of a diesel engine) Normal Maps often look very purple, as a side effect of using the RGB color space for x,y,z displacement data. Color Space is non-color for this image.

When using the Shader Editor in Blender with Normal Map images, you would feed the image node into a "Normal Map" node before feeding it to the "Principled BSDF" shader "Normal" input.

The Emission Map can be used to simulate a source of light. It uses RGB data to perform this task. At this time, this is untested.

There is a potentially useful **paid** Blender Add-on available from SuperHive called **PBR PAINTER 3** that will assist with the PBR image workflow. There are also a series of tutorial videos on how to use **PBR Painter 3** available on Youtube. https://youtu.be/ntdlvndv1tE?list=PLMN9uGgVtpSt9hTaiDAKYJ1wIS1nQvTf9

Additionally, there are online tools such as PBR FORGE that will assign with PBR image generation. For offline installable software, there is a program called **Materialize** from BoundingBox Software.

How to Make Night Textures

The Microsoft way.

Much of this is content is a summary of KUJU supplied documentation along with some other notes from other sources.

Night textures on shapes are created by editing the original texture, darkening it and perhaps adding a couple of touches for effect.

The night textures take effect at a predetermined time within the game environment, so as to replicate a real world environment. The `extshape.dat` file must be updated with the correct parameters so that the shape is declared as having night textures (see the “How to write a .ref file” document for further clarification).

Once created, the night and day textures must have the same filename so that the code can pick up the correct texture. This means that the daytime / normal texture must be entered into the normal route textures directory and the night version into the route night textures directory.

Below are two textures taken from the Orient Express level. These were manipulated in a two dimensional drawing package:

Figure 7. Daytime source texture

Figure 8. Night texture counterpart

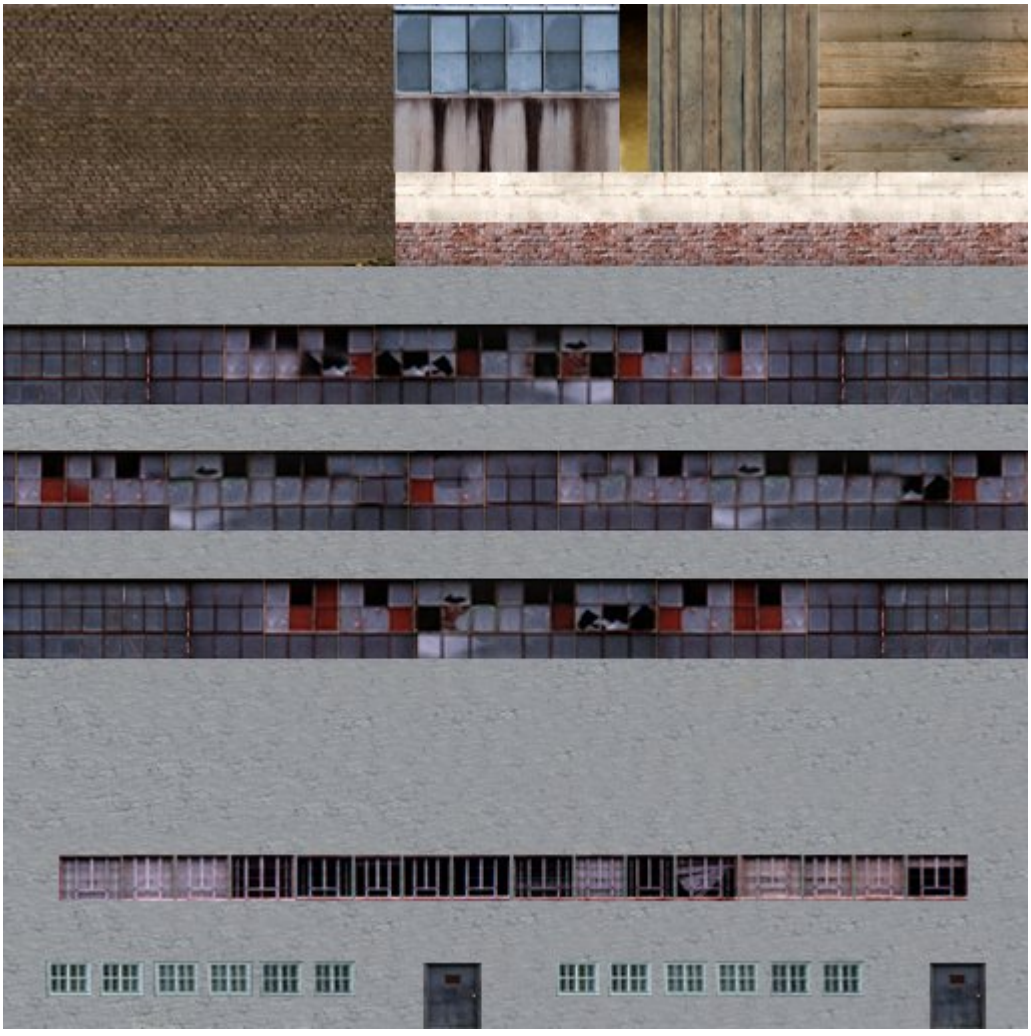


Figure 9. Second daytime source texture

Figure 10. Second night texture counterpart

This means that the texture on the left will be replaced with the texture on the right once the correct night timing has been switched on.

How to Make Night Textures with Back Lighting

Example:

Create the image for the building you want to texture, making sure that anything that requires a backlight is a separate objects and textures. Items such as windows should be separate objects, often created by just using a PLANE object.

The basic building shape should not have modeled windows. The window parts of the texture will be modified to use an alpha channel. The image can then be saved as a working PNG file if your editor allows it, or you could use a PSD format as you just need a format that retains layers.

If you are also working on the 3D MODEL while texturing, you would not model the windows doors or outdoor lights. You would work on these items as the last steps in the process. In essence, you would completely texture the model before proceeding to work on the lighting effects.

How to Make Snow Textures

Snow textures must also be created, so that your buildings and other objects will look correct when there is snow lying on the ground. These, too, are created using the original, daytime textures. The `extshape.dat` file must be updated in relation to this (see the “How to write a .ref file” document for further clarification) and must be placed in the route’s snow textures directory accordingly. The snow texture will only be used when the snow environment settings have been switched on through the Drive a Train User Interface.

Below are the snow versions of the textures above.

Figure 11. Daytime source texture

Figure 12. Snow texture counterpart

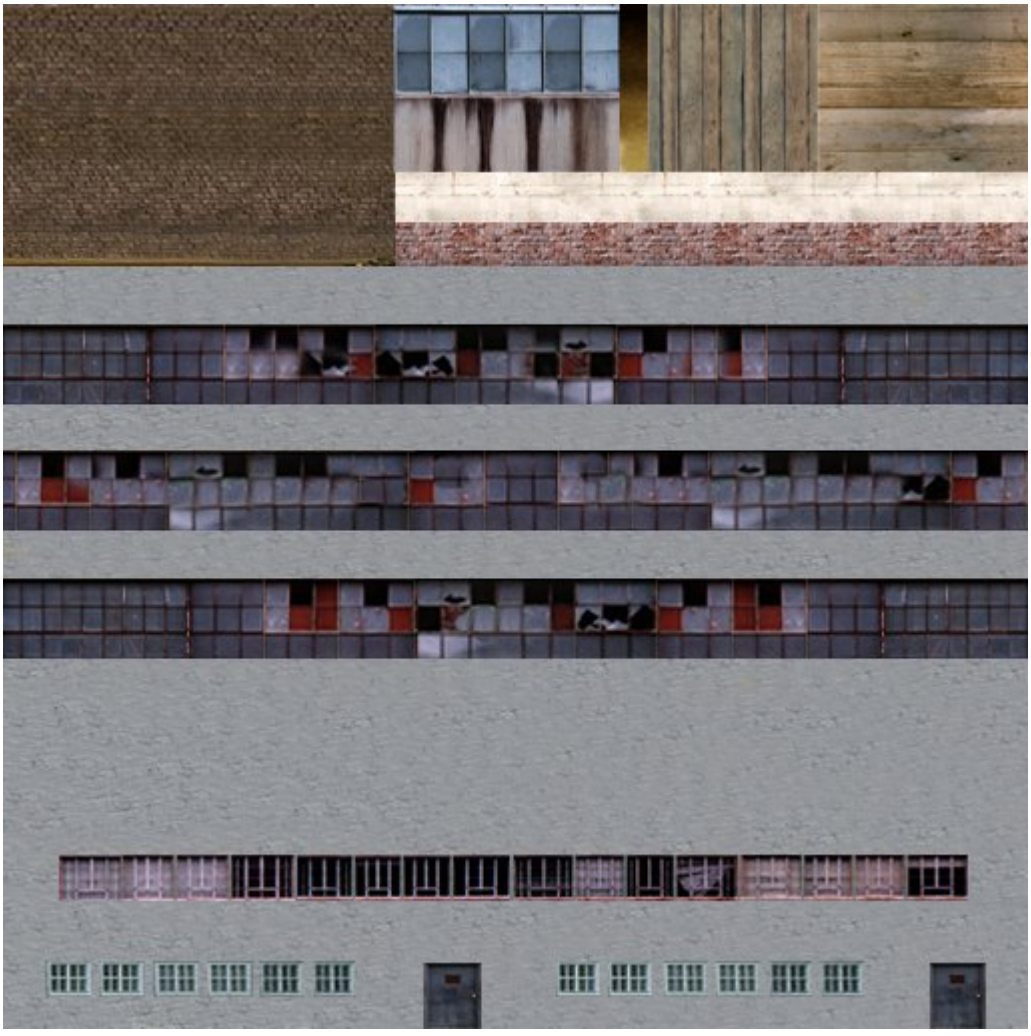


Figure 13. Second daytime source texture

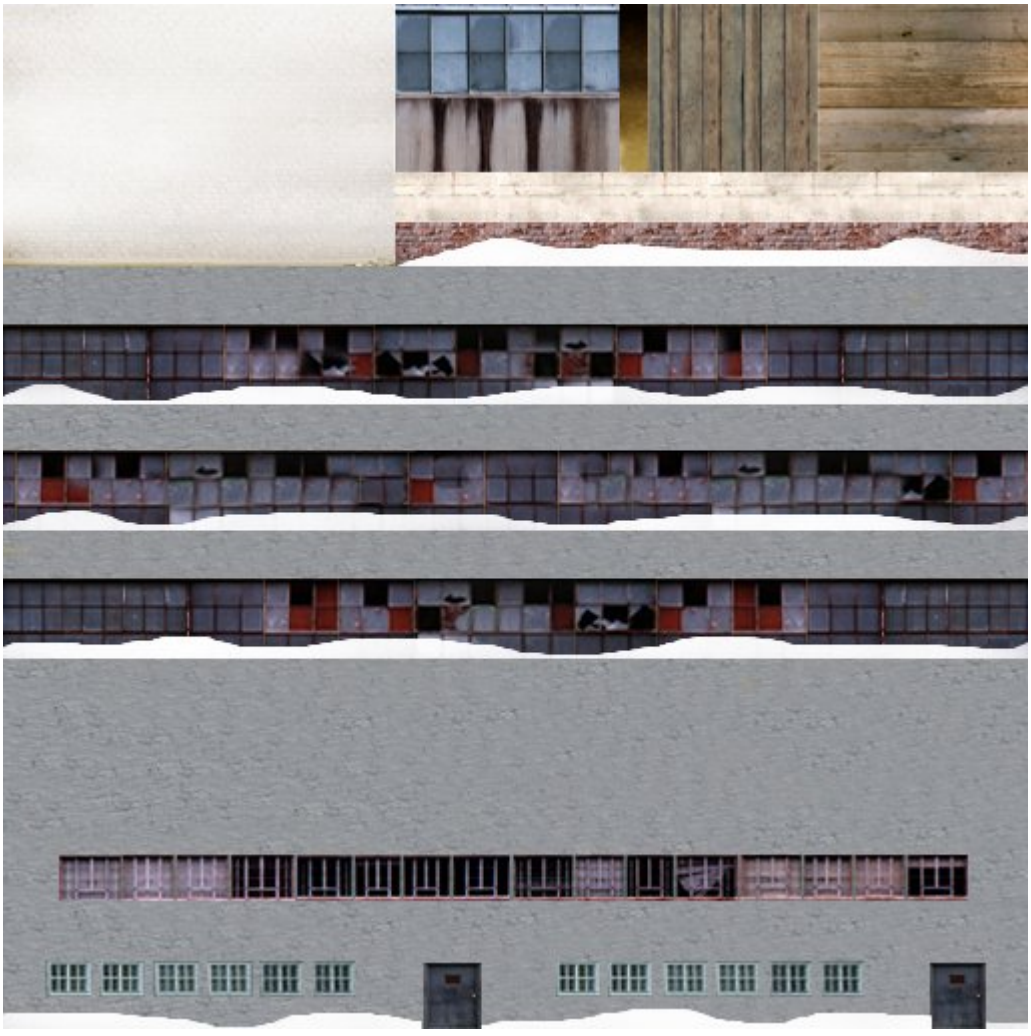


Figure 14. Second snow texture counterpart

Highlights and Shadows

A layer, or layers, used for "hard coded" shadows and possibly highlights, which could otherwise be known as the Ambient Occlusion layer, should reside near the DECAL layer and the BASE layer for a decent effect, though you could migrate the highlights layer higher in the stack. How to create an Ambient Occlusion layer using the Blender render engine is covered in another section of this document, but if you do create this layer using Blender, this is where it would be inserted. The color of this layer will essentially be only black and white. You can always choose to create and edit this layer manually as well.

The blend mode for the [ao] layer could also be "multiply" instead of "normal"

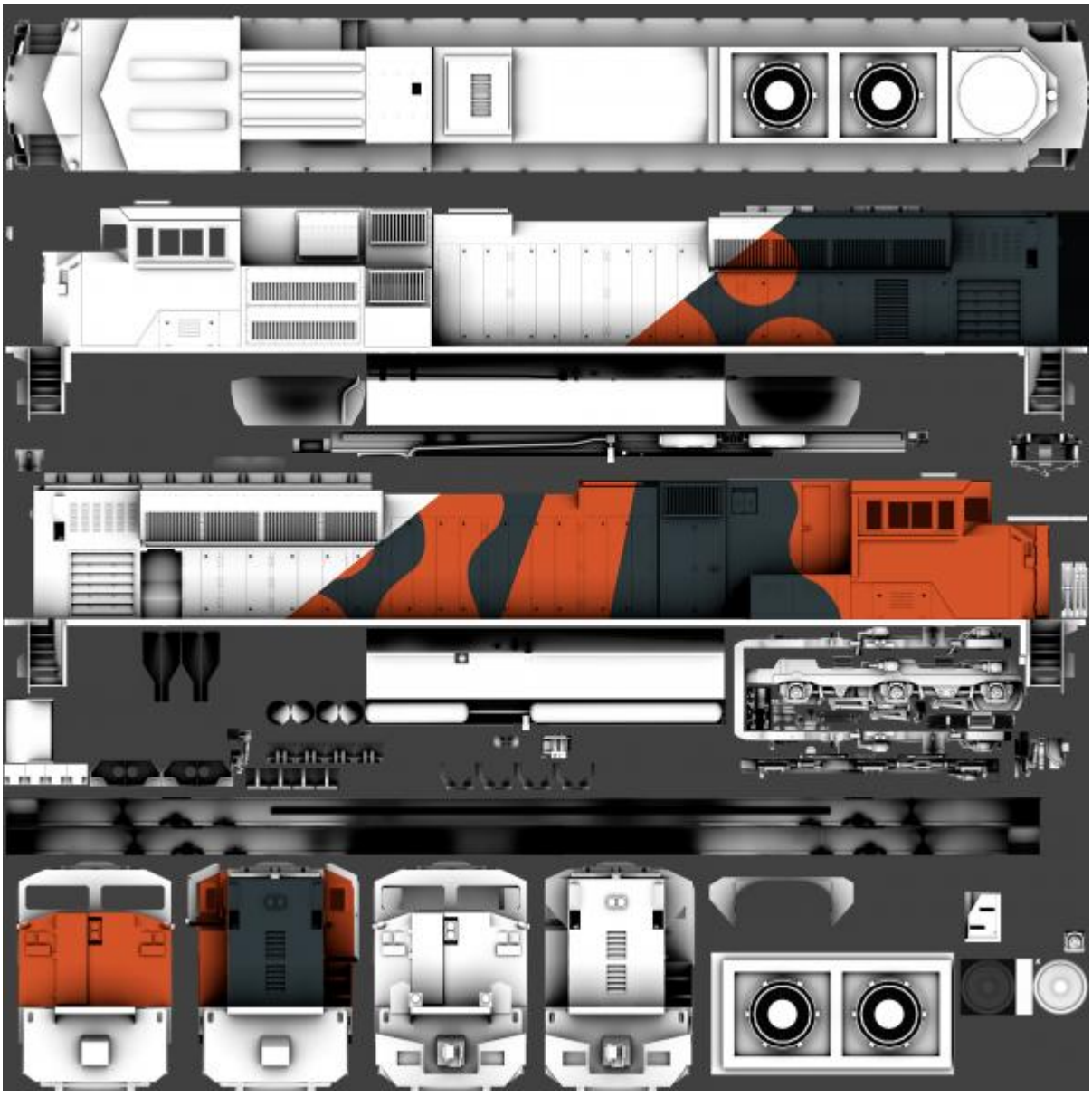
Generated Shadows - Ambient Occlusion

Ambient Occlusion is the generation of hard-coded or `baked` shadows instead of relying in dynamic lighting to generate shadows for an in-game asset. It provides extra depth to an asset that would not be achieved otherwise.

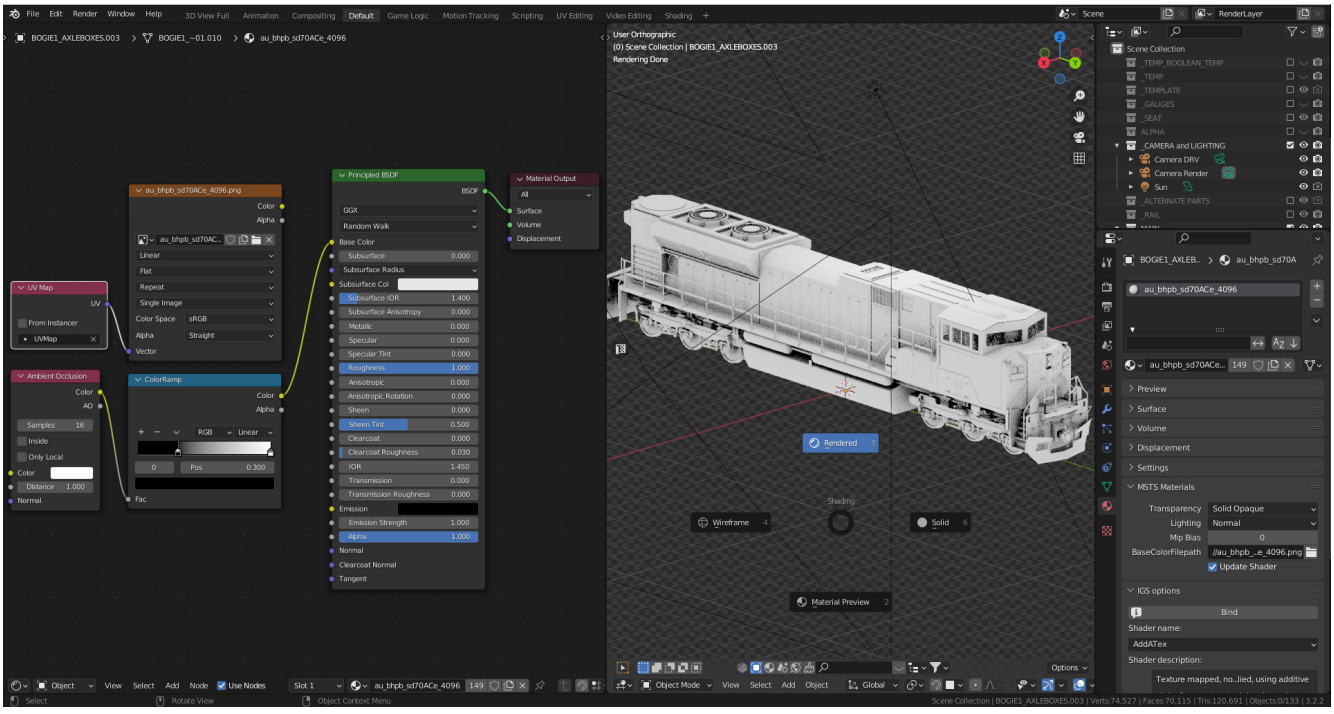
Marek on Elvas Tower shared how he achieves Ambient Occlusion on his models and it is shared here.

There are many ways to achieve Ambient Occlusion in Blender, so you will probably need to experiment with methods to see which one works best for your model or your sanity.

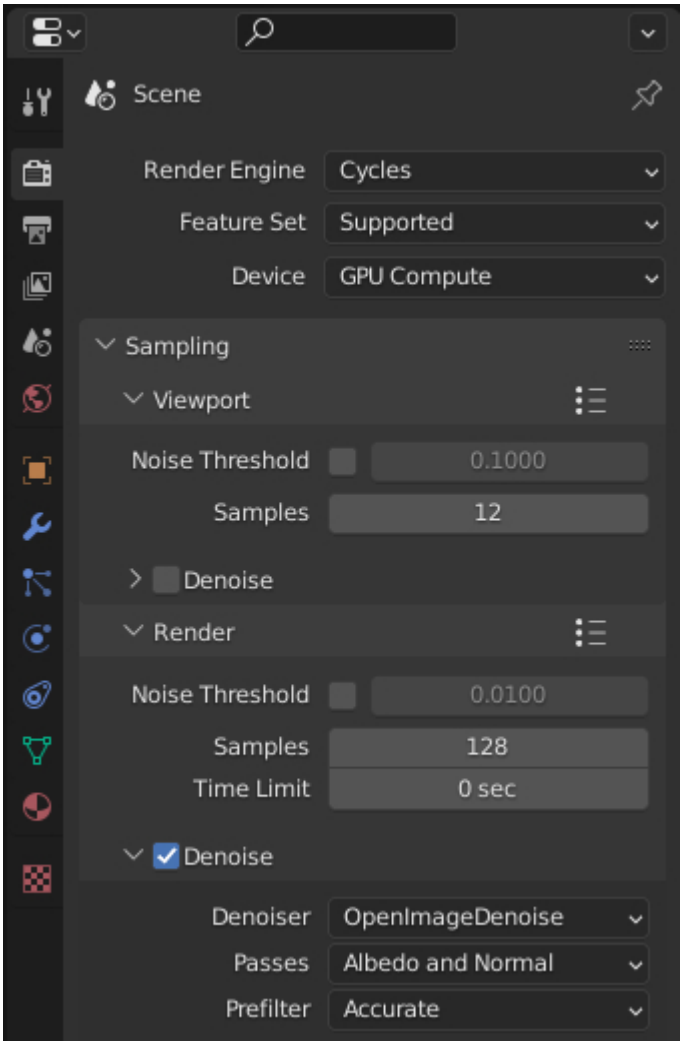
Your model needs to be UV UNWRAPPED prior to baking out an Ambient Occlusion image as described here.



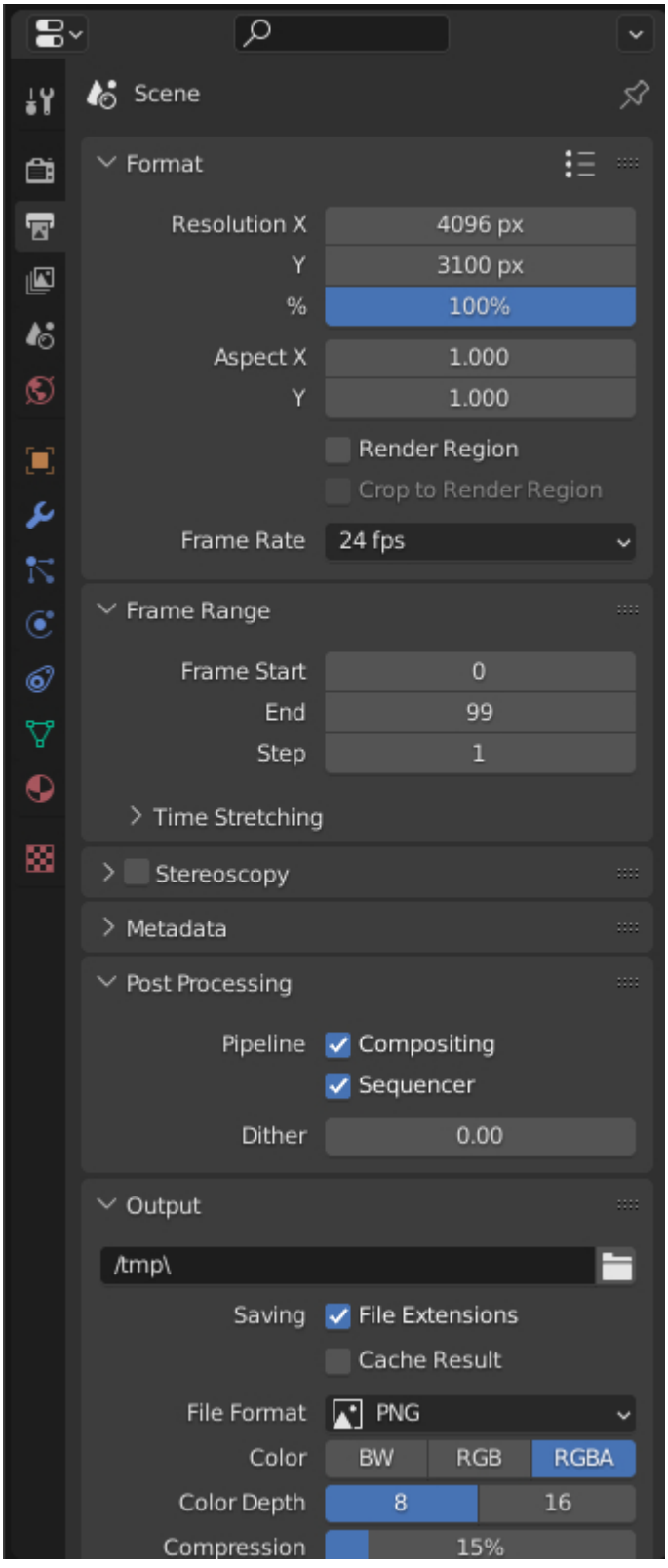
An orthographic camera can be used to render the Ambient Occlusion with all projection planes. Clipping planes can then be used on the camera to make certain parts invisible to the render camera, which helps exclude details that are not desired. The renders can then be imported into GIMP (or any other editor that supports layers) and used as the base for the final texture layout. The model can then be unwrapped to that. Livery colors can then be added in layers above the Ambient Occlusion layers and layer blend modes can be used to create the desired image. This method is more time-consuming than other methods, but it produces results that are far better than what can be achieved by hand in 2D alone.



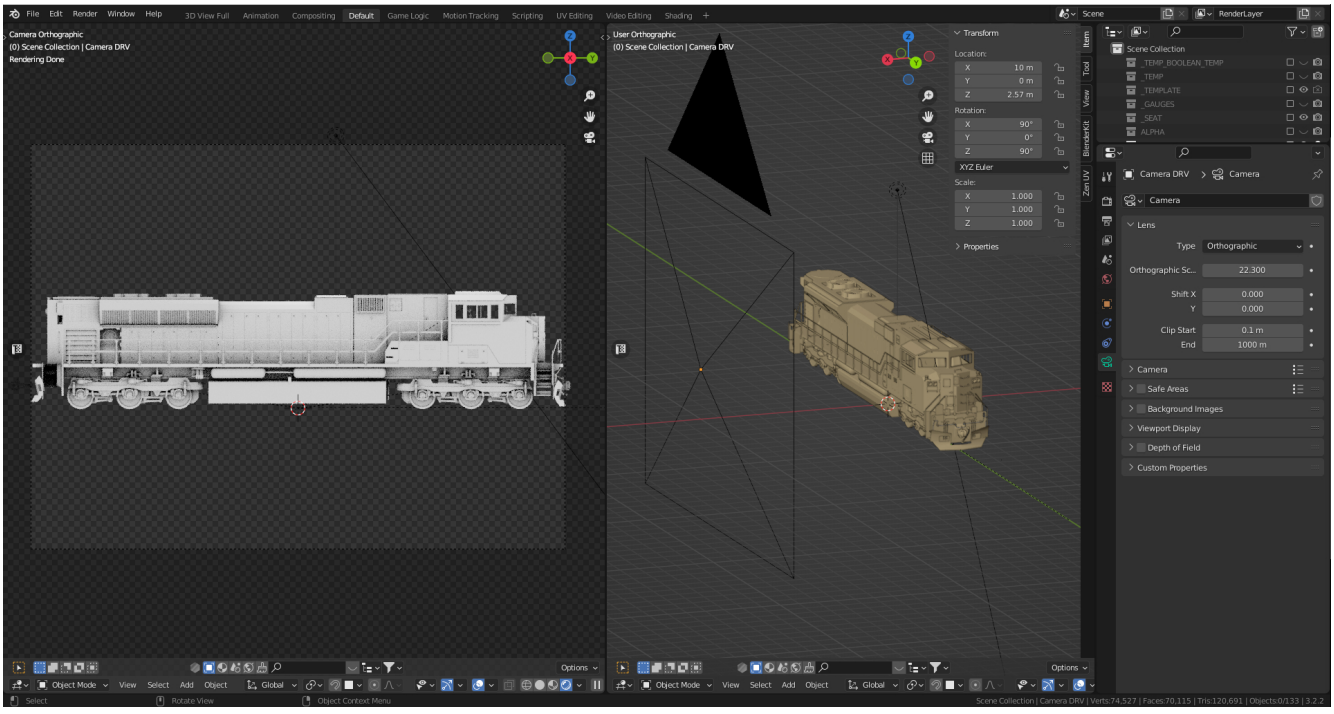
The image above shows a model with a single material applied to it called `au_bhpb_sd70ace_4096`. In the shader editor, two inputs can be chosen for the Base Color of the `Principled BSDF:` either an image file applied to the model at the top or an Ambient Occlusion shader via a Color Ramp node below it. When exporting to `.S` or wanting to view the texture in Blender, the `texture node` needs to be plugged in, and when rendering the Ambient Occlusion, the Ambient Occlusion Shader needs to be plugged in. By setting the 3D viewer to `Rendered`, one can get an idea of what the output will look like and can adjust the shadow effect via the sliders in the ColorRamp.



In the Render properties tab, switch your Render Engine to Cycles and your Device to GPU Compute for faster rendering. With a Render sample setting of 128, rendering can take a while on a PC. To get a faster render time when testing, try using a lower number initially. Don't forget to turn on `Denoise` to give you a cleanly rendered image.



In the Output properties tab, the `Format Resolution X = 4096 px` is used because there is a 4K texture being used on the model. You can adjust the Resolution Y value later when you know how much vertical space the render will take (see below).



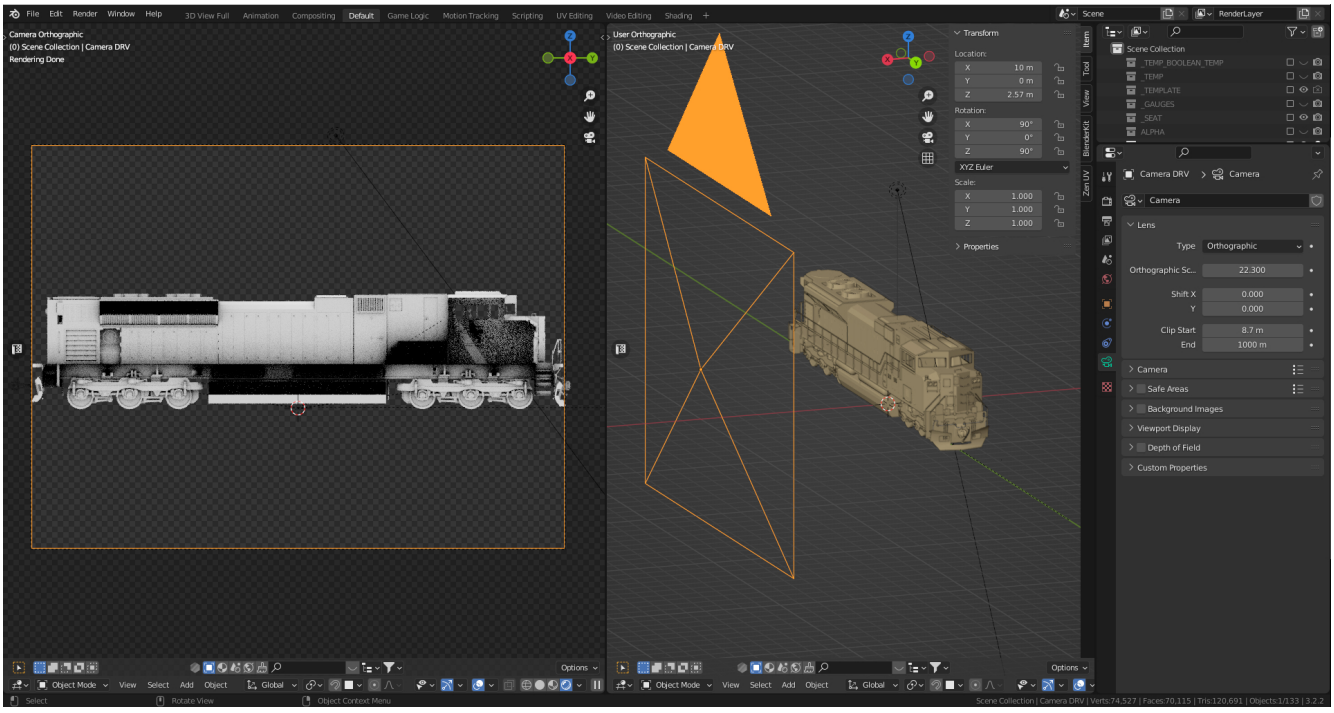
A camera should be added to the scene and moved to the side of the locomotive. The Rotation values in the Properties tab should be used to ensure that it is perpendicular to the locomotive. On the Object Data Properties tab, the Type of Camera should be changed to Orthographic. A separate 3D viewport should be opened and the Camera should be selected and `CTRL` + `Num 0` should be pressed to get a side view of the locomotive from the camera's point of view. If it is not already in Rendered view, pressing `Z` should allow for Rendered view to be selected; the Ambient Occlusion shader should be visible in the camera view.

The Orthographic Scale should be adjusted for the camera so that the entire length of the locomotive body fills the camera view (for this locomotive, the scale is 22.300). The camera should be moved in the `Y` and `Z` axes to ensure that it is in frame. Then, the `Format Resolution Y = value` should be adjusted so that no empty space is rendered above and below the locomotive. When the entire locomotive is framed in the camera view, `F12` should be pressed and Blender will render the Ambient Occlusion into a new window.

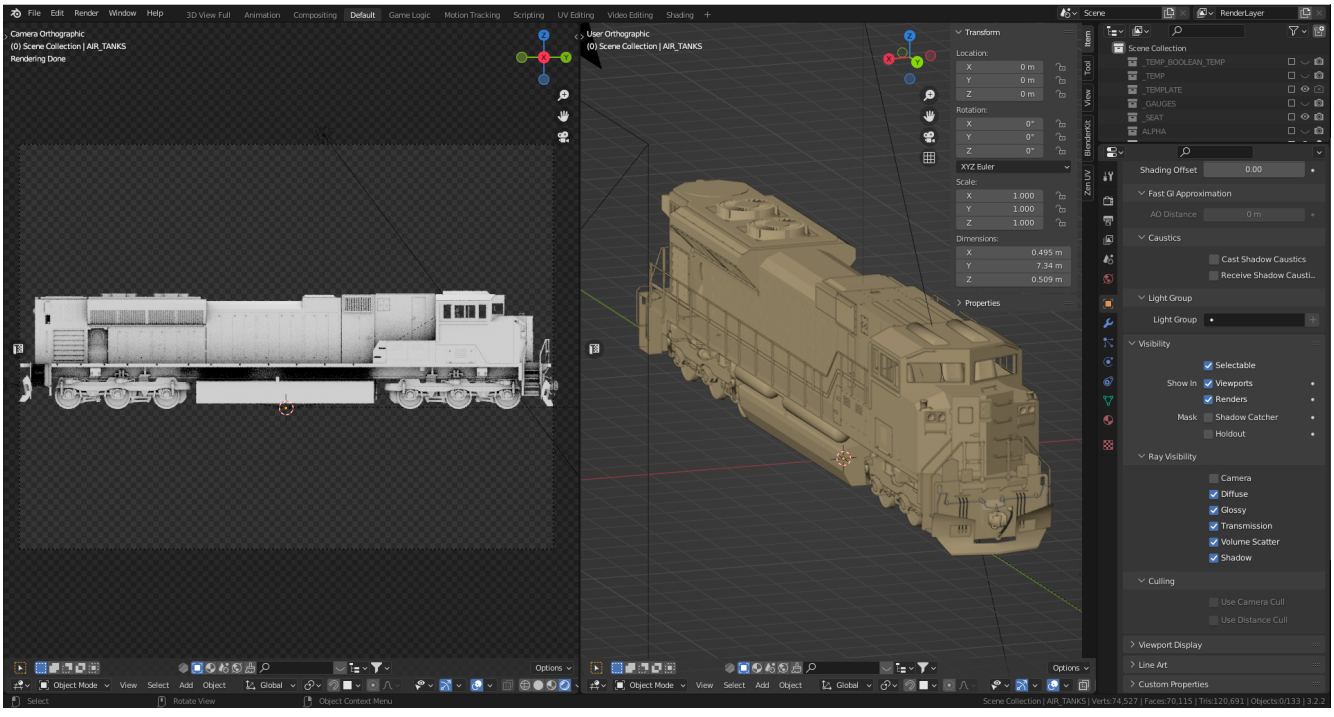
If denoise is enabled, the rendering may appear to hang, but it will complete after a while. Making multiple copies of the camera and moving them to the sides, ends, top and bottom is recommended in order to render the Ambient Occlusion from the different projections. Keeping the `Orthographic Scale` the same on all the cameras will result in the render being at the same textural density. Additionally, unlike a perspective camera, the distance of the camera from the object does not change the size of the resultant render.

Once you have finished rendering the image, you can save it to your computer and import it into your 2D graphics program to use as a base for your texture. To make sure the texture has the same textural density, you should make multiple copies of the camera and move them to the sides, ends, top and bottom. Again, unlike with a perspective camera, the distance of the camera from the object will not affect the size of the render. However, the hand rails may obstruct the details on the

body behind. To fix this, you have multiple options.



In the image above the `X` location of the camera is 10m to the side of the locomotive. The `Clip Start` distance has been changed from its default value of 0.1m to 8.7m. When rendered, the view no longer includes the handrails but displays the side of the body as the camera is drawing what it can see from 8.7m to 1000m, beyond the hand rail closest to the camera. This clipping results in the cab side, fuel tank and air tanks being cut off. The same outcome can be achieved by keeping the `Clip Start` and `End` at their default settings and moving the camera in the `X` direction, allowing for 'slices' of the scene similar to an MRI machine.

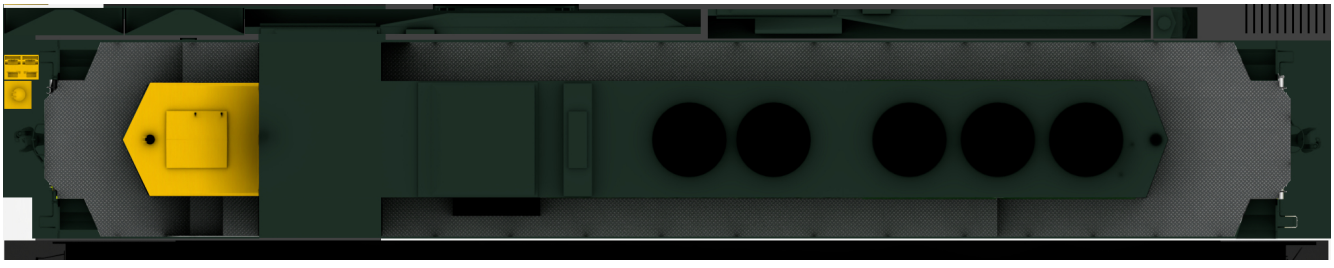


It is possible to make parts invisible to the render camera. In the image, the `Camera` checkbox in the Object Properties tab for the hand rails and air tank objects has been unchecked. This results in the entire side of the locomotive body being visible in the render view, but the handrails and air tanks not being rendered. These methods can be used to exclude objects in the foreground that are not required for the desired image.

The Ambient Occlusion can be rendered for all the different views and then mashed together in a 2D graphics program to export as the `au_bhpb_sd70ace_4096` texture to use for unwrapping and in Open Rails.

This technique for Baking Ambient Occlusion is a good way to get the added shadow details on a texture but the method described here is not the only way to accomplish Ambient Occlusion shadows. It might not be the most effcient way either, but it seems to work.

It is important to be aware that any lights in the scene, including environment lighting, will impact the rendered Ambient Occlusion. Lights are not used when baking an Ambient Occlusion, so if the rendered Ambient Occlusion image is too dark, the environment brightness can be increased or area lights can be added, typically the length of the locomotive, placed on either side to minimize shadow casting. If the lighting is changed later on, this will impact subsequent renders and will not match earlier renders. It is essential to be aware of this.



Ambient Occlusion Steps - Summary

To Recap... Blender Ambient Occlusion (AO) is a rendering technique that simulates the subtle shadows that occur where surfaces meet other surfaces or when they are blocked from direct light. AO can be used to add realism and depth to your renders, and it can be especially effective for close-up shots or scenes with complex geometry.

There are a few ways to add AO to your renders in Blender, these include:

1. Render settings: Go to Render Properties > Render Settings > World and enable the Ambient Occlusion checkbox. You can then adjust the Distance and Strength settings to control the intensity of the AO effect.
2. Shader node: Add an Ambient Occlusion node to your material shader and connect it to the

Surface output. You can then adjust the Distance and Strength settings directly on the node.

Tips for using AO in Blender:

- AO can be computationally expensive, so it is important to use it sparingly. If you are rendering a scene with a lot of geometry, consider using a lower sample count or baking the AO to a texture.
- AO can be used to add subtle details to your renders, but it is important not to overdo it. Too much AO can make your renders look dark and muddy.
- AO can be used to create a variety of effects, such as weathering, dirt, and grime. Experiment with different settings to achieve the desired look.

Here are some examples of how AO can be used to improve the realism and depth of your renders:

- Close-up shots: AO can be used to add subtle shadows and details to close-up shots of characters, objects, and environments. This can help to make your renders look more realistic and believable.
- Scenes with complex geometry: AO can be used to add depth and definition to scenes with complex geometry, such as city scapes, forests, and interiors. This can help to make your renders look less flat and more immersive.
- Weathering and dirt effects: AO can be used to create a variety of weathering and dirt effects, such as dust on a car or moss on a tree trunk. This can help to make your renders look more realistic and worn-in.

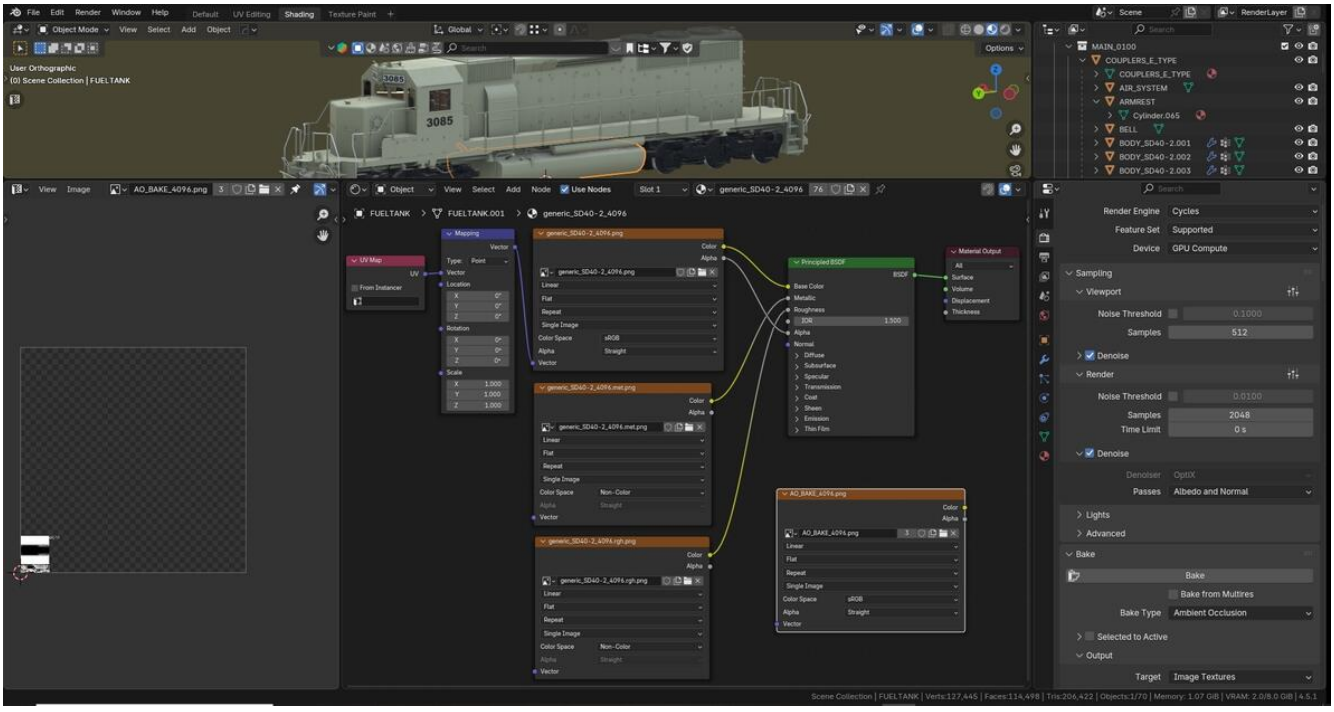
Overall, AO is a powerful rendering technique that can be used to add realism and depth to your renders. By understanding the basics of AO and experimenting with different settings, you can achieve a variety of effects to enhance your renders.

Success with Ambient Occlusion is strongly related to how the underlying model is created and textured. A surefire way to have issues is due to lazy texture mapping. For example, if you relied on "Project From View" to generate the UV assignment on an object and did not MOVE the UV coords of the "back side" of the object, the generated AO shadow will be placed at the same location as the front side, ruining the effect. You can also correct this issue by removing the backside faces from the object if they will never actually be seen.

Ambient Occlusion Baking with GIMP

Tips from Marek

Here are the settings I use in the shading menu when I perform an AO bake.



Firstly, ignore the Metallic and Roughness texture nodes connected to the Principled BSDF shader. These are used for glTF exports and will not impact on AO baking or .S export.

This model uses a single 4K texture for the external model called generic_sd40-2_4096.png. Each object on the model has had its faces UV Mapped to a unique location where I want it on the texture file.

When ready to bake, I create a new 4K image texture node called AO_BAKE_4096.png. You can see this node below the principled BSDF but it is not connected to anything. I leave it below and to the right of the Principled BSDF so that it is easy to find and to remind me not to connect it to anything.

In the Render Properties tab I have set the samples to 2048, which is high but gives a fine high-resolution AO bake. Denoise is enabled because it improves the quality of the bake. I use the OptiX Denoiser because it is faster on my graphics card. Bake type is set to Ambient Occlusion. Device is set to GPU Compute because my Nvidia 2070 is considerably faster at baking than the CPU. Current Blender versions can use the Vulkan display backend, which may be faster than OpenGL on some systems. Test both to see which is faster for you using Edit > Preferences > System > Display Graphics > Backend > Vulkan or OpenGL.

I always bake at least 2 objects at the same time. If I bake a single object I get a black background for some reason. If I have 2 or more objects selected when baking, I get a transparent background which is what I want when importing into my 2D graphics program.

Also before baking, I unplug the Alpha noodle between the Base Colour and Principled BSDF. If the alpha noodle is connected, and the UV Map of the object being baked is over an untextured/alpha area of the base colour, the resultant bake will be all black. Took me ages to work this one out.

When the objects I want to bake are UV Mapped over a unique area of the texture file, I again

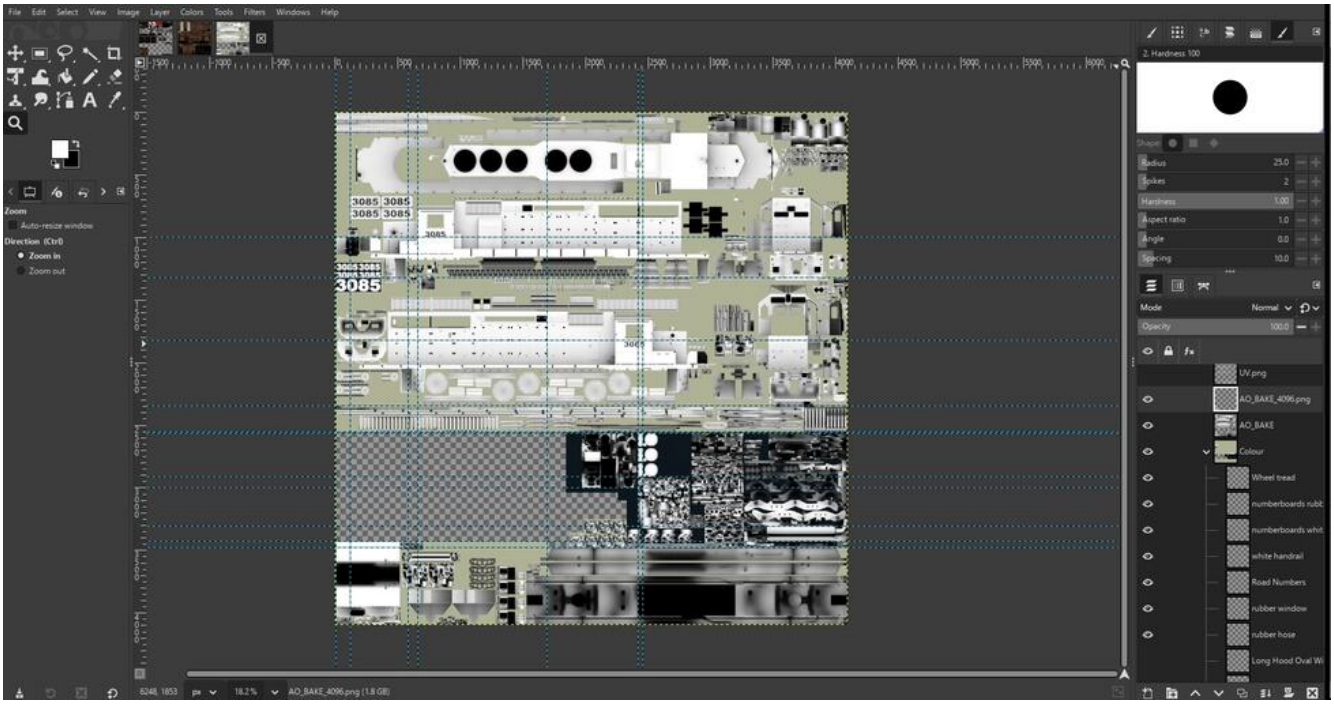
ensure that the alpha noodle is not connected. I confirm I have 2 or more objects selected. I then left click to highlight the AO_BAKE_4096.png texture in the Shader Editor. This is important as Blender will now render the bake to the selected texture. In the UV editor on the left hand side I have the target AO_BAKE_4096.png texture selected so I can see the progress of the baking process.

I then click on the 'Bake' button in the 'Render Properties' tab. The progress bar along the bottom of the screen shows how slowly it is going. The denoising does not show in the progress bar and makes Blender look like it has frozen. Baking objects with a small number of faces does not take long. For large parts with many triangles 10-15 minutes at 2048 samples is the norm with my models on my computer.

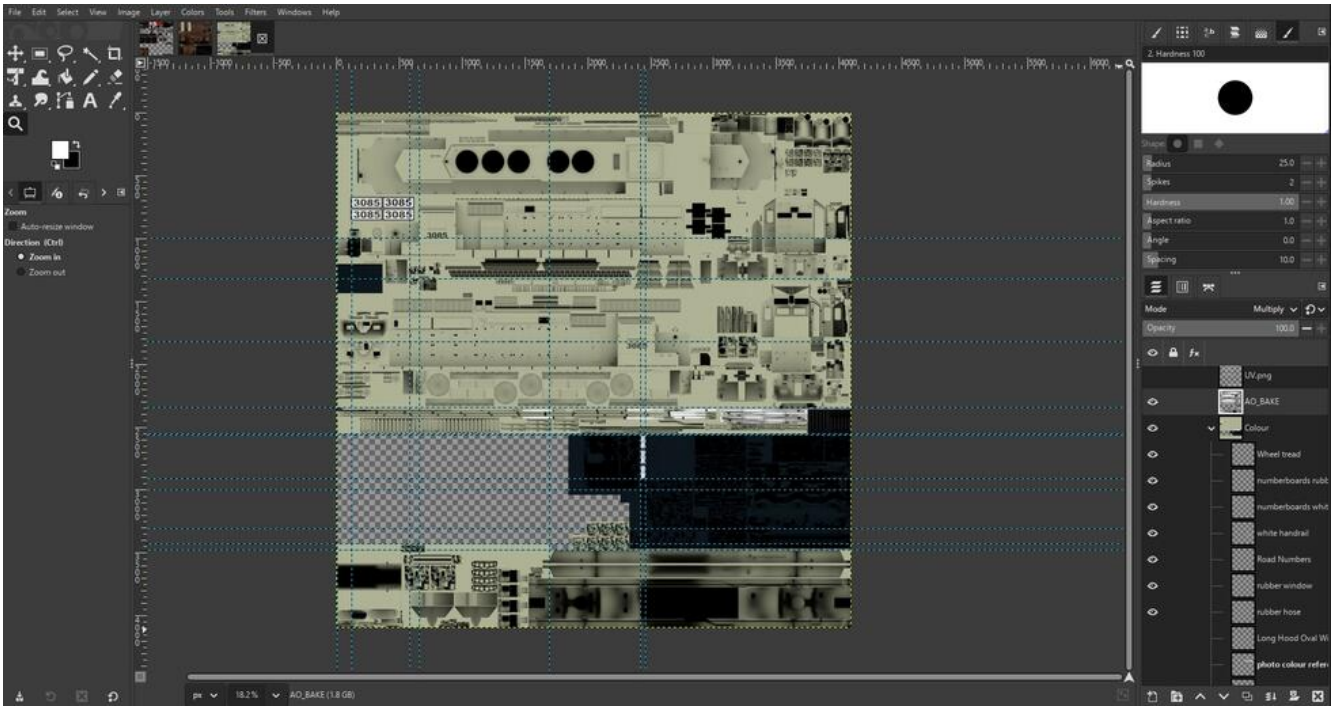
Once the bake has completed, I use the Image > Save As drop down menu in the UV Editor and I save the file as AO_BAKE_4096.png.

In the Blender screen shot you can see the AO for the fuel tank on the bottom left of the texture with the rest of the area being transparent/alpha.

I then go into GIMP and then import the AO_BAKE_4096.png file as a layer on top of my already present AO layer which is a combination of all the other objects that have baked the AO for over time.



I then merge that imported AO layer into the existing AO layer so I have a single AO layer. Below this AO layer I have the diffuse colours. I then apply the 'Multiply' mode to the AO layer that blends it with the underlying coloured layers.



I then export this from GIMP as the texture file that is applied to the model in Blender, in this model it is generic_SD40-2_4096.png and export it as generic_SD40-2_4096.dds for use directly in Open Rails.

The next object/s that need to have their AO baked, are unwrapped to another unique location on the texture file, selected and baked to the AO_BAKE_4096.png file as described above. This same file is used to transfer the AO bake from Blender to be combined with the AO layer in GIMP.

If you are using texture files with different resolutions then make the texture file you are going to bake the AO to the same size.

If you get unexpected black areas in your bake then that can be caused by different things. Having the alpha noodle connected when baking has already been mentioned above. Flipped normals are another common cause. Other geometry overlapping the baked surface is another common cause. These can be hard to find, especially if you have hidden some objects, they still mask the baking process.

More About Baking Ambient Occlusion

Ambient Occlusion Baking is potentially a confusing task in Blender. Here is an alternate method for baking ambient occlusion if you want a bit more control over the process.

When baking occlusion, here are some general tips adapted from Wayne Campbell's notes:

1. Gather parts that will receive Ambient Occlusion into a collection to make them easy to select.

2. Ensure the UV Maps for those parts do not wrap off the edge of the texture sheet.
3. Coplanar surfaces are a problem, including two-sided surfaces; they may bake black.
4. Set 'Render Engine' to Cycles.
5. Set Render Max Samples low, such as 8, for testing. Use higher values such as 128 or 256 for better final quality.
6. If your model has secondary LODs, ensure they do not cast a shadow on your primary model. Use 'Disable In Render' in the outliner.

When baking Ambient Occlusion, you should select at least 2 objects at the same time. If you select only one object, the bake will be black. If you select more than 2 objects, the bake will be properly transparent.

Ambient Occlusion Steps

To bake Ambient Occlusion in Blender, you can use the Bake tool in the Render tab of the Properties panel. Here's a step-by-step guide:

- Select the object or objects that you want to bake the Ambient Occlusion for.
- In the Properties panel, go to the Render tab and then click on the Bake tab.
- In the Bake tab, set the Bake Mode to Ambient Occlusion.
- Set the Samples value to the number of samples you want to use for the Ambient Occlusion bake. The higher the value, the more accurate the Ambient Occlusion will be, but the longer the bake will take.
- Set the Margin value to add a margin around the baked texture to prevent texture bleeding.
- Set the Space to either "Object" or "World" depending on whether you want the Ambient Occlusion to be baked in object space or world space.
- Check the "Clear" box to clear the image before baking.
- Check the "Normalized" box to normalize the Ambient Occlusion values, which can help with artifacts and banding.
- Click the "Bake" button to start the Ambient Occlusion bake.

The Ambient Occlusion bake can take some time, depending on the complexity of the objects and the number of samples you are using. Once the bake is complete, the Ambient Occlusion map will be saved as an image in the UV/Image Editor. You can then use this image as a texture to apply the Ambient Occlusion effect to your object.

Introduction to Blender Scripting

Some of Blender's true potential is unlocked through scripting. Blender scripting allows users to automate repetitive tasks, create custom tools, and extend the software's capabilities to suit their specific needs.

Blender provides a Python API (Application Programming Interface) that enables users to interact with its internal functionalities and control almost every aspect of the software programmatically. Whether you're a beginner or an experienced programmer, Blender scripting offers a flexible and efficient way to work with 3D objects, materials, animations, and much more.

With scripting, you can perform complex operations that would be tedious or time-consuming to achieve manually. Mastering Blender scripting opens up many possibilities for efficiency in your workflow.

In this section, we will explore the fundamentals of Blender scripting, starting with some basics and gradually diving into more advanced concepts. We will cover essential topics such as accessing and manipulating objects and materials, as well as exploring scripting techniques for creating custom tools and add-ons.

Before diving into Blender scripting, it's recommended to have a basic understanding of the Python programming language. However, even if you're new to programming, this guide will try to provide you with the necessary foundations to get started.

Whether you're looking to automate repetitive tasks or wish to extend Blender's functionality, Blender scripting is a valuable skill that can enhance your workflow.

Actually Doing It

So we now consider diving into Python. The Python that you find embedded in Blender is the same Python you find everywhere else, with one exception. That is, it includes the Python Blender Library (or API) for accessing the internal features of Blender. The `Library` is called "BPY". You will often see at the very top of Python code written for Blender the following line of code: `import bpy`. This is where you add all of the Blender functionality to your code.

To get started with Python, though, you should probably set up a development environment for Python. This way you can begin to learn how to code in Python and practice and develop ideas without actually having to do it inside Blender.

Getting started with Python is relatively easy and straightforward. Here's a step-by-step guide to help you begin your Python journey:

Install Python

Visit the official Python website (https://www.python.org) and download the latest version of Python for your operating system. Follow the installation instructions provided by the Python installer.

Set up a development environment

You'll need a code editor or an integrated development environment (IDE) to write and run your Python code. Some popular choices include Visual Studio Code, PyCharm, and Atom. Choose the one that suits your preferences and install it.

Learn the basics

Familiarize yourself with the fundamentals of Python programming. There are numerous online resources, tutorials, and books available to help you learn Python. Some recommended beginner-friendly resources include:

- "Python.org" - The official Python website provides a comprehensive tutorial (https://docs.python.org/3/tutorial/) to get you started.
- Codecademy's Python course (https://www.codecademy.com/learn/learn-python)
- "Automate the Boring Stuff with Python" by Al Sweigart (https://automatetheboringstuff.com/)
- "Python Crash Course" by Eric Matthes

Practice coding

As you learn the basics, it's essential to practice writing Python code. You can solve coding challenges on platforms like HackerRank (https://www.hackerrank.com/domains/tutorials/10-days-of-statistics) or LeetCode (https://leetcode.com/problemset/all/).

Explore Python libraries and frameworks

Python offers a vast ecosystem of libraries and frameworks that extend its capabilities. Depending on your interests and goals, you can explore libraries such as NumPy for numerical computations, Pandas for data manipulation, Matplotlib for data visualization, or Django for web development. Look for resources specific to the libraries or frameworks you wish to explore.

Join the Python community

Engaging with the Python community is a great way to learn, ask questions, and collaborate with others. Participate in online forums like the Python subreddit (https://www.reddit.com/r/Python/) or join Python-related communities on platforms like Discord or Slack.

Build projects

To solidify your Python skills, start working on small projects. Projects provide practical experience and help you apply what you've learned. Think of something you'd like to create—a web scraper, a calculator, a data analysis tool—and work towards building it using Python.

Remember, learning programming is an iterative process. Start small, be patient, and gradually tackle more complex concepts. With practice and persistence, you'll gain confidence and proficiency in Python.

Embarcadero has a Python IDE that is quite handy and much better than the one that comes with Python by default. You can get it here: https://www.embarcadero.com/free-tools/pyscripter/free-download

As mentioned, Blender has a built-in Python API and Python interpreter that allows you to control its functionality programmatically.

There is a Blender Add-On called **SERPENS** that will help you develop Blender GUI based add-ons from your code. It's a paid add-on, but it's really neat.

Blender and AI Coding Assistants

AI coding assistants such as ChatGPT, Claude, Gemini, and others can help explain Blender Python examples, draft small scripts, and diagnose error messages. They are useful learning aids, but they do not replace testing inside Blender because generated code may use the wrong Blender API version or make assumptions about your scene.

Running a Blender Python Script Inside Blender

To run a Blender script, follow these steps:

- Open Blender: Launch Blender. The current stable release gives you the newest features, while the current Long Term Support (LTS) release is usually the safest choice for exporter and add-on compatibility.
- Open the Text Editor: Once Blender is open, locate the Text Editor. You can find it by selecting the "Scripting" workspace from the top bar or by navigating to the "Editor Type" dropdown at the top left of any editor area and choosing "Text Editor".
- Create a New Text Block: In the Text Editor, click on the "New" button to create a new text block. This will provide you with a clean space to write or paste your script.
- Write or Import your Script: You have a few options here:
    1. Write the script: If you're familiar with Python scripting, you can directly write your script in the Text Editor. Ensure that your script follows the correct syntax and indentation rules

of the Python language.

2. Import an existing script: If you have a pre-existing script saved as a file on your computer, you can open it in the Text Editor by clicking on the "Open" button and navigating to the script's location.
3. COPY/PASTE the script code from somewhere. For example, copy a script from an AI assistant or tutorial and paste it into Blender's text editor.

- Run the Script: Once you have your script ready in the Text Editor, you can run it by clicking on the "Run Script" button or by using the keyboard shortcut `Alt` + `P`. Blender will execute the script, and its functionality will take effect within the Blender environment.
- Observe the Results: Depending on the script you've written or imported, you will see the desired outcome within the Blender interface. This could involve creating or modifying objects, materials, animations, or any other aspect of the 3D scene.

It's important to ensure that your script interacts with Blender's API correctly and that it doesn't contain any errors that might cause unexpected behavior or crashes. Always double-check your code for syntax errors or logic mistakes before running it.

By following these steps, you can easily run a Blender script and leverage the power of scripting to automate tasks, create custom tools, or extend Blender's functionality to suit your specific needs.

A Collection of Scripts

The following scripts are tested examples of very simple scripts to help get you started with scripting. They are only a few lines each so you can take the time to digest them and learn how they work.

Fixing Material Problems

Let's say you have completed some steps with your project that involved using multiple copies of the same object to be placed around your scene. Let's also say that these parts actually came from another model so you ended up using a `COPY/PASTE` operation to get them into your scene. With this as our background setup, what is a possible issue that we might encounter? Do you know?

Well, one thing that happens when you `COPY/PASTE`, is that your parts get renamed to be unique, which is normal for a `DUPLICATE` operation as well, but your material reference gets replicated the same way, in other words, it gets renamed. So if your project has been using a Material named "MAIN"... you will likely end up with a material named "MAIN01" and so on. This will happen with any objects that were duplicated using `COPY/PASTE` instead of a `DUPLICATE` operation.

Duplicate objects are created by using `SHIFT` + `D` and the process will not try to

> create a duplicated unique material reference.

The goal of the script then is to fix this issue so you don't end up with 20 materials that are all actually 100% identical. You can run the following script:

```
import bpy

# Get the currently active material or create a new one
material_name = "main"  # Replace with your desired material name
material = bpy.data.materials.get(material_name)
if material is None:
    material = bpy.data.materials.new(name=material_name)

# Assign the material to each selected object
selected_objects = bpy.context.selected_objects
for obj in selected_objects:
    if obj.type == 'MESH':
        if obj.data.materials:
            # If the object already has materials, replace the first one
            obj.data.materials[0] = material
        else:
            # If the object has no materials, assign the new material
            obj.data.materials.append(material)
```

Now selected materials will just have the one assignment of your choosing in slot 0. It doesn't completely eliminate the issue of material mismanagement, but its a start.

3DC Export/Import Issues

If you are exporting a 3D model from 3D Crafter/3D Canvas and importing it into Blender, you may encounter various issues with the UV mapping and materials.

Below is a more complete example of how to cope with these issues, submitted by Scott Brunner.

This script below contains a scale factor for converting FEET to METERS. If you are not using FEET, you can remove (comment out) the scale factor.

```
import bpy

for mesh in bpy.data.meshes :           # for every mesh in the .blend file
    if len( mesh.uv_layers ) == 0:      #   if it doesn't have any uv maps
        mesh.uv_layers.new()            #     create one
    if not mesh.uv_layers.get('UVMap'): #   if it doesn't have one with the default
name
        firstmap = mesh.uv_layers[0]    #     rename the first map
        firstmap.name = 'UVMap'         #     to the default name

# Remove "S_" prefix from object names
for obj in bpy.data.objects:
    if obj.name.startswith("S_"):
        obj.name = obj.name[2:]

# Remove "_mat" suffix from object names
for obj in bpy.data.objects:
    if "_mat" in obj.name:
        obj.name = obj.name.split("_mat")[0]

scale_factor = 0.3047999902464

for obj in bpy.data.objects:
    obj.scale = (scale_factor, scale_factor, scale_factor)

textures = set()

# get all textures used in current scene
for obj in bpy.data.objects:
    for slot in obj.material_slots:
        mat = slot.material
        if mat.use_nodes:
            for node in mat.node_tree.nodes:
                if node.type == 'TEX_IMAGE':
                    textures.add(node.image.name)
```

```
# create new materials
for texture in textures:
    material_name = texture.split(".")[0]
    new_material = bpy.data.materials.new(material_name)
    new_material.use_nodes = True
    bsdf = new_material.node_tree.nodes["Principled BSDF"]
    # create new texture node and set its image
    texture_node = new_material.node_tree.nodes.new("ShaderNodeTexImage")
    texture_node.image = bpy.data.images[texture]
    # link texture node to the material output
    new_material.node_tree.links.new(texture_node.outputs[0], bsdf.inputs[0])

# replace materials for objects
for obj in bpy.data.objects:
    for slot in obj.material_slots:
        mat = slot.material
        if mat.use_nodes:
            for node in mat.node_tree.nodes:
                if node.type == 'TEX_IMAGE':
                    texture_name = node.image.name
                    new_material_name = texture_name.split(".")[0]
                    obj.material_slots[slot.name].material = bpy.data.materials
[new_material_name]
```

Making Copies

While making a copy of an object is easy enough with `SHIFT` + `D`, sometimes you want to make a copy and offset it by a specific amount. This next ChatGPT generated script will do that.

To use this script, follow these steps:

- Open Blender and make sure the object you want to duplicate is selected.
- Open the "Scripting" layout.
- Create a new text block in the text editor and paste the script.
- Run the script by clicking the "Run Script" button or pressing Alt+P.

- The selected object will be duplicated, and the duplicate will be offset by 3 meters in the -Y axis.

Make sure you have the desired unit settings in Blender before running the script to ensure the correct offset distance. Using the Metric units is assumed since it is the default setting.

```
import bpy

# Get the selected object
selected_obj = bpy.context.object

# Duplicate the selected object
duplicated_obj = selected_obj.copy()
bpy.context.collection.objects.link(duplicated_obj)

# Offset the duplicated object
duplicated_obj.location.y -= 3.0
```

Make Selected Objects Become ASSETS

The Asset Browser is a Blender feature introduced in version 3.0, therefore, this script will not work on older versions of Blender as the feature is not available.

To use this script, follow these steps:

- Open Blender and switch to the scripting layout.
- Create or open a new Blender file.
- Select the objects that you want to mark as assets.
- Open the "Text Editor" panel and create a new text block.
- Copy and paste the script into the text block.
- Click the "Run Script" button or press "Alt+P" to execute the script.

The selected objects will now be marked as assets.

Please note that this script assumes you have the necessary objects selected in Blender before running it. Also, make sure to save your work before running the script, as it will modify the objects in your scene.

```
import bpy

def mark_objects_as_asset():
    # Get the currently selected objects
    selected_objects = bpy.context.selected_objects

    # Iterate over each selected object
    for obj in selected_objects:
        # Mark the object as an asset
        obj.asset_mark()

# Call the function to mark selected objects as assets
mark_objects_as_asset()
```

Automating the Boring Stuff with Python

"Hey, isn't that the title of a book from Al Sweigert?"

"Why yes, it is and it is the inspiration for the following Blender Python code"

The book "Automate the Boring Stuff with Python" is available from NoStarch Press or Amazon but it is also free to read. https://automatetheboringstuff.com/#toc I highly recommend it.

So what are we going to do next?

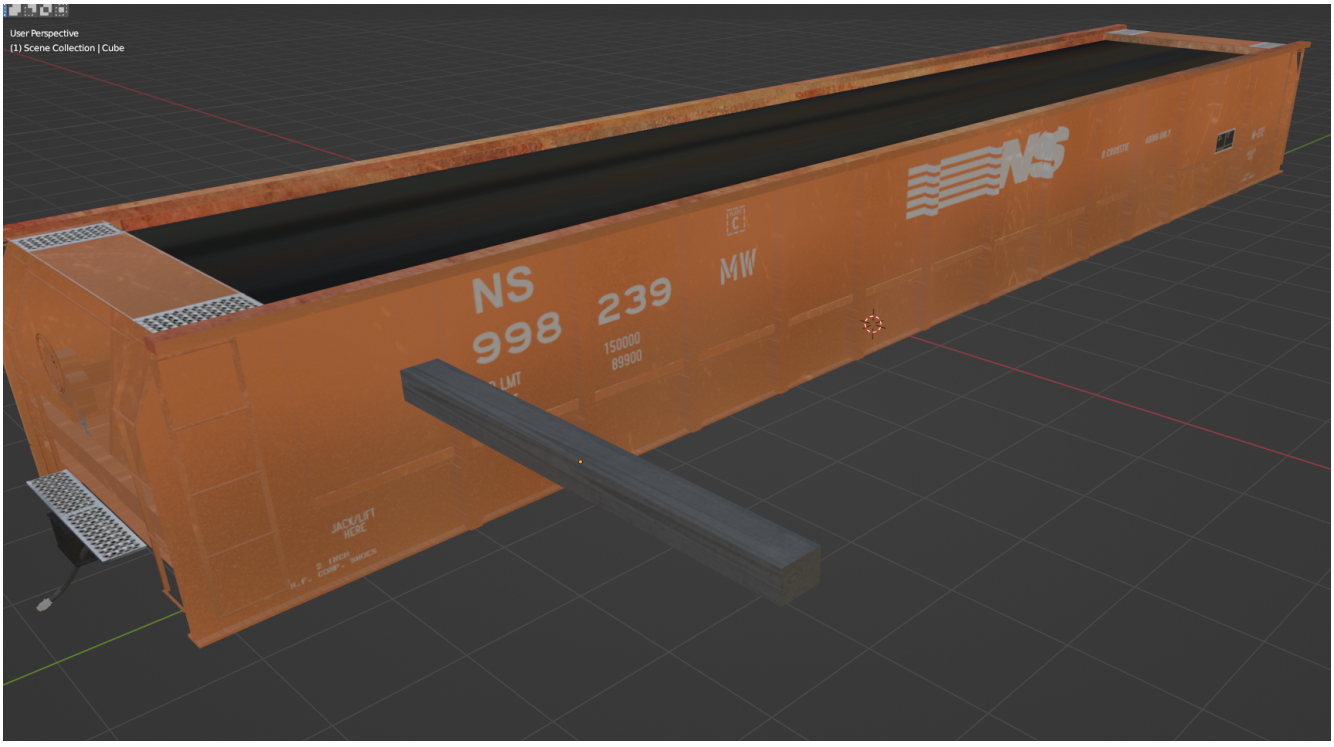
When working on a Blender model, you often encounter tasks that require repetitive steps. These tasks can become tedious, especially when you need to repeat them a large number of times, such as 56 times. This is how it all began. Now, let's discuss what led up to this.

While working on the crosstie load for my Norfolk Southern Crosstie Gondola, it was realized that we only needed to create a single crosstie, apply a texture to it, and then use an ARRAY MODIFIER to duplicate it until the objects filled the gondola load area. Below is the final result.

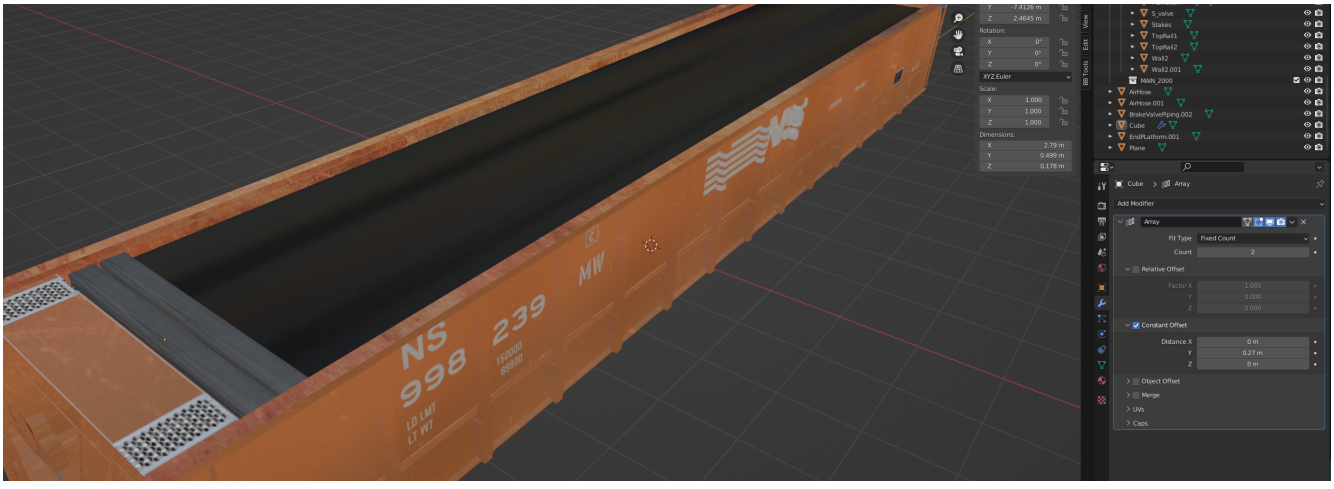
But how did I get there?

The initial crosstie shape is created using a cube primitive. The dimensions used are 7" x 9" x 9' or in Blender → Metric (Z = 0.1778m Y = .2286m X = 2.7492m). The next steps are to Apply Rotation and

Scale and then to texture it.

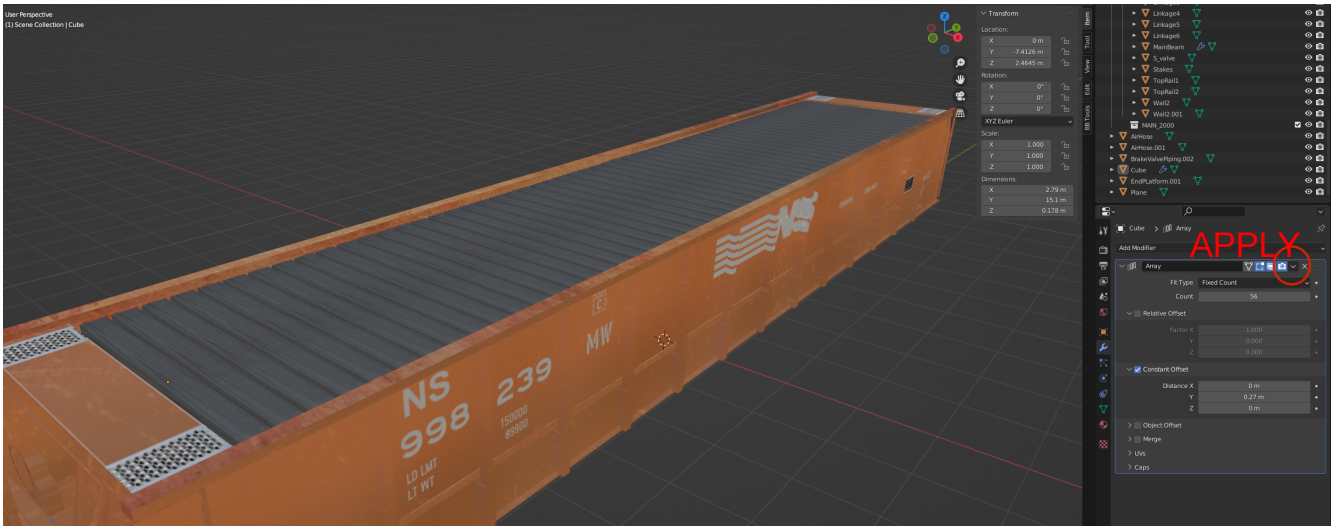


So now that we have a crosstie built and textured, it needs to be replicated to fill the gondola load area. This is made easier by using the **ARRAY MODIFIER** under the Wrench icon.

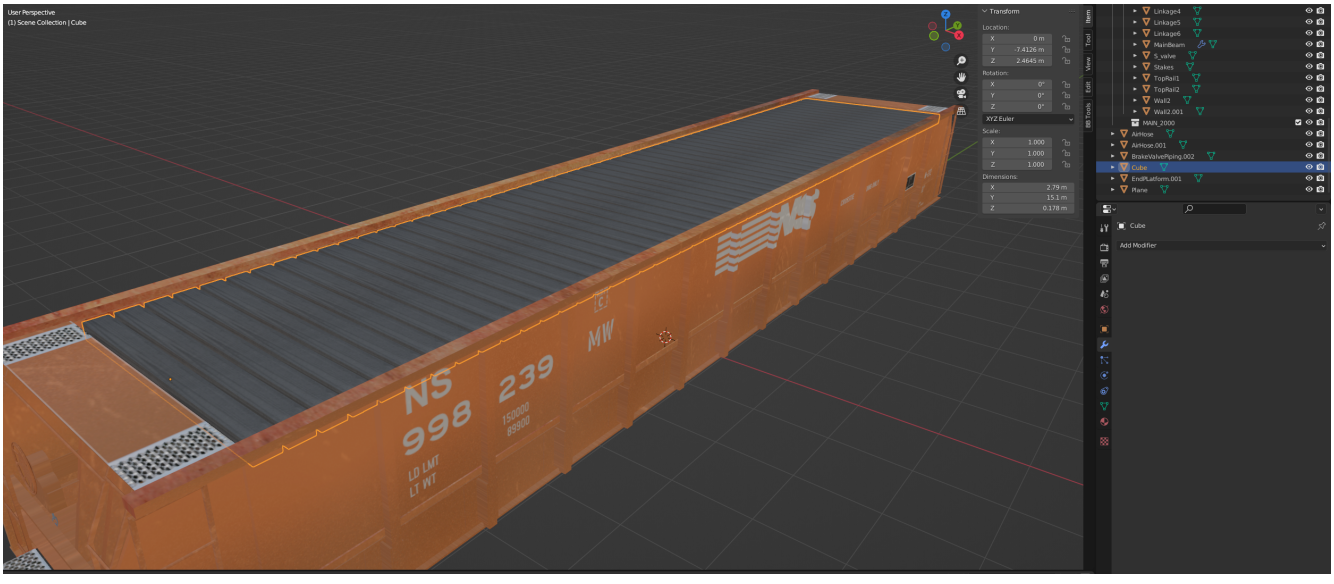


The settings used are: **CONSTANT OFFSET** (Not default) Y Axis OFFSET VALUE: 0.027 (or so).

Uncheck Relative Offset. Zero Out the `X` Field value. But we need to FILL the area... so the initial `Count` field is "2" and it needs to be increased to fit the area (in this scene, it needed 56 arrayed items to make it fit)



Great! We have a FILLED gondola with an array of crossties and we still only needed to make one crosstie to start with. BUT! They are all way too uniform and look unnatural. The problem is, these objects (as they are now) will all continue to imitate any changes we make to the original cube since the array modifier is still **active**. So now we need to **APPLY** the **ARRAY** modifier while in OBJECT mode.



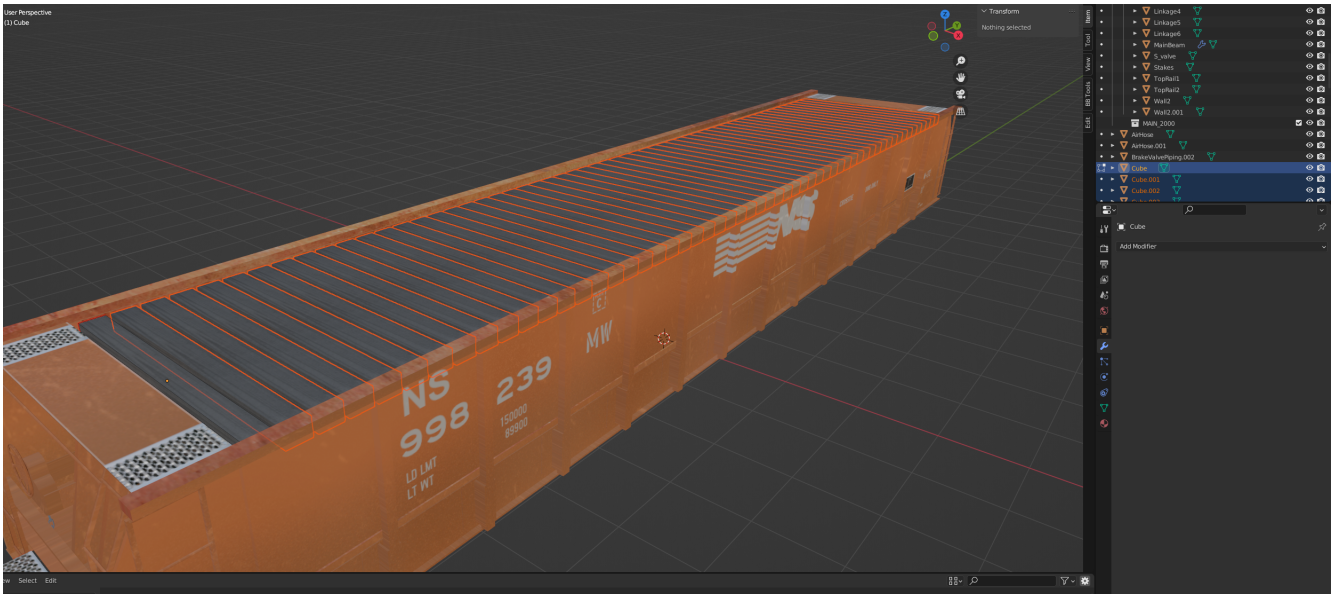
Our next problem is that these arrayed objects are all still basically the same grouped object. All of these objects will also share the SAME origin point as well. So the next step to do we to break them apart.

- Select the crosstie object (cube)
- Press `TAB` for EDIT MODE
- Press `P` - to Separate. Now choose: "By loose parts" from the menu.

Now all of the individual items are **created**, having been separated from the original. (Check your collection list as in this case we now have 56 CUBES that make up the crossties)

All the crossties share the same origin location, making individual adjustments to them is currently rather challenging. Therefore, it becomes necessary to assign each crosstie its own origin point based on its specific geometry. This will allow for easier customization and modifications to each crosstie.

Normally in this case, we would need to make 56 individual edits! Oh the DRUDGE! So, let's make use of Blender's scripting TAB.



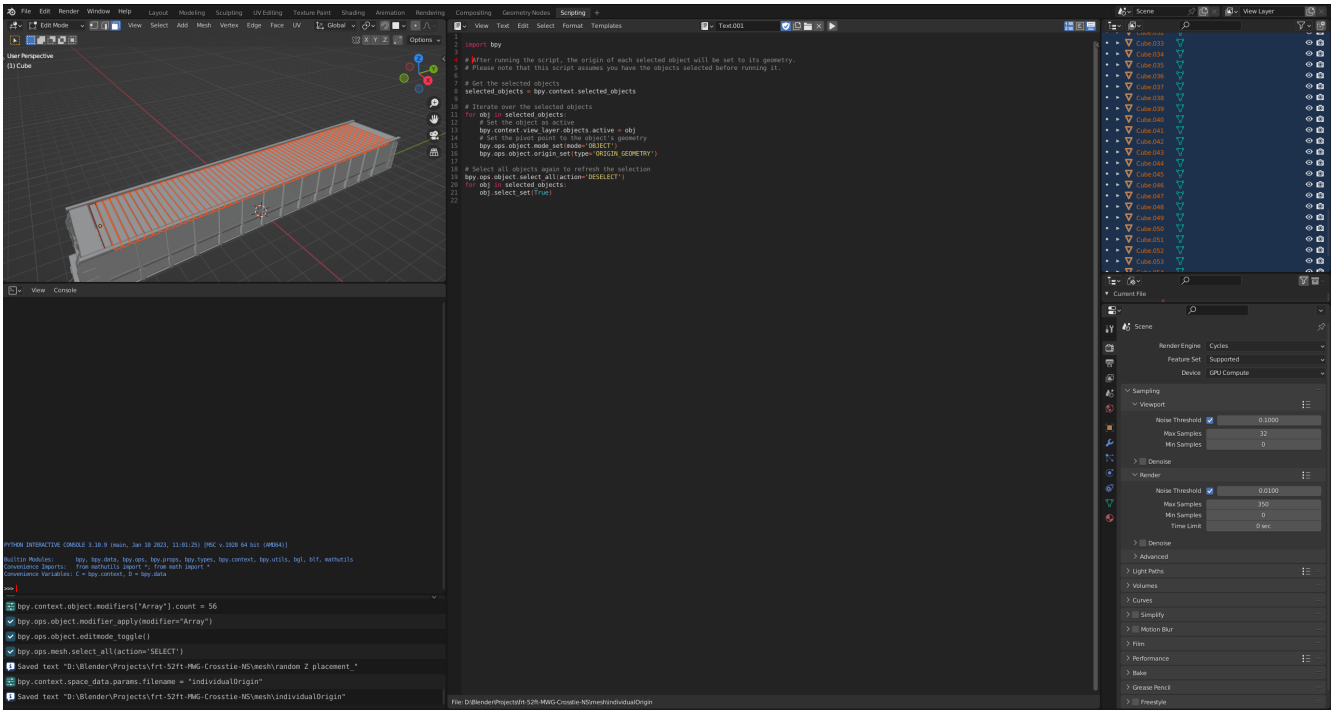
Make sure all of your objects (crossties) are currently selected. (You might want to just select them all from the Collection list.)

```
# After running the script, the origin of each selected object will be set
# to its geometry. Please note that this script assumes you have the
# objects selected before running it.
import bpy

# Get the selected objects
selected_objects = bpy.context.selected_objects

# Iterate over the selected objects
for obj in selected_objects:
    # Set the object as active
    bpy.context.view_layer.objects.active = obj
    # Set the pivot point to the object's geometry
    bpy.ops.object.mode_set(mode='OBJECT')
    bpy.ops.object.origin_set(type='ORIGIN_GEOMETRY')

# Select all objects again to refresh the selection
bpy.ops.object.select_all(action='DESELECT')
for obj in selected_objects:
```

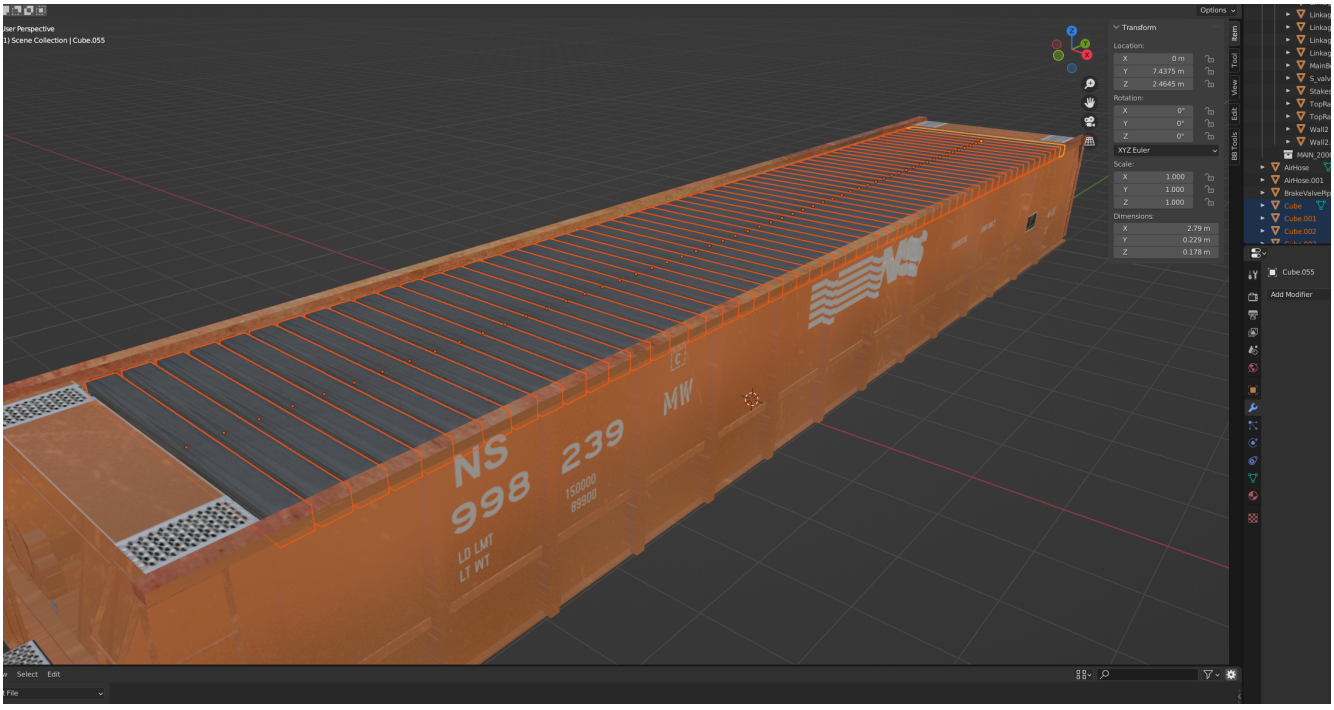
```
    obj.select_set(True)
```

Open the scripting tab in Blender, follow these steps:

1. In Blender and navigate to the top header of the application.
2. Locate and click on the "Scripting" tab.
3. Within the "Scripting" tab, go to the "Text" tab.
4. Choose "Create New" to create a new file.
5. Enter or paste the provided code into the Blender Python Editor window.



Press the RUN icon to run the code. (You will see that it did something in the windows on the left). You can switch back to LAYOUT from the Scripting tab for a better view.

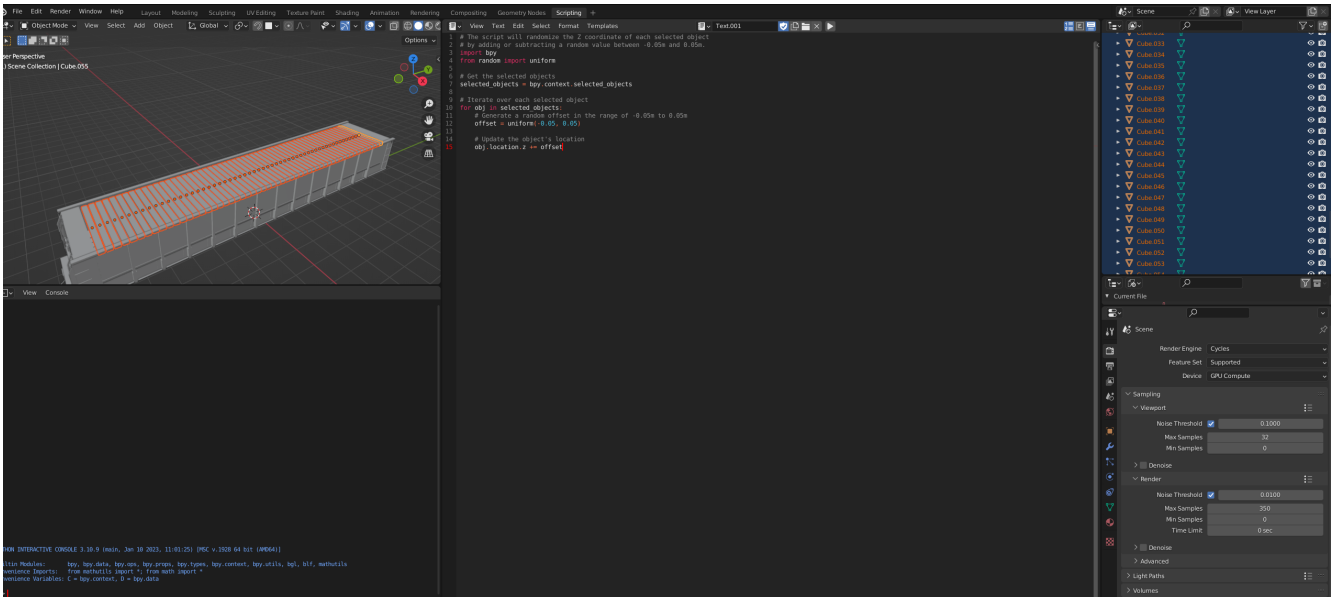


Now all of the objects have their own origin based on their specific geometry. (See all the orange origin dots on each crosstie?)

The next step is to randomize the `Z` axis location of these objects (crossties) so they look less uniform.

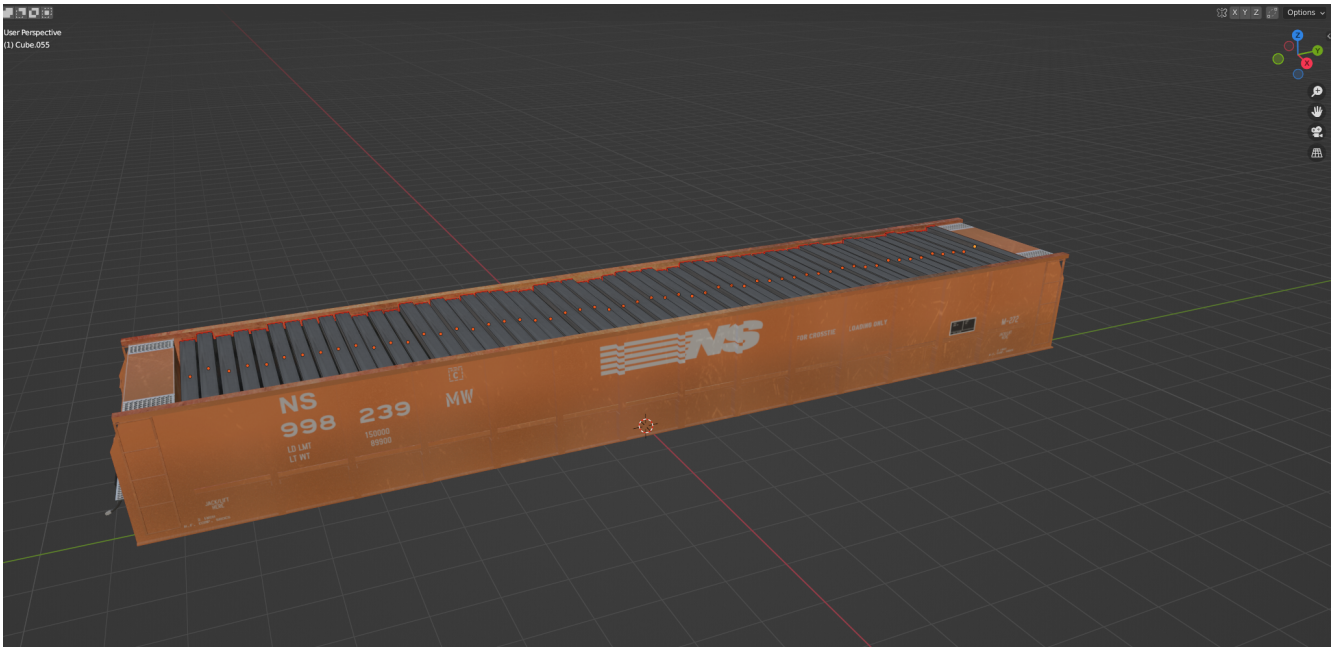
In the Blender Python Editor (Scripting Tab), create a NEW text document and enter/paste-in the following code:

```
import bpy
from random import uniform

# Get the selected objects
selected_objects = bpy.context.selected_objects

# Iterate over each selected object
for obj in selected_objects:
    # Generate a random offset in the range of -0.05m to 0.05m
    offset = uniform(-0.05, 0.05)

    # Update the object's location
    obj.location.z += offset
```



Press the RUN icon to run the code.

You will end up with the following...



and finally...

To summarize:

We encountered a situation where the crosstie load needed to be placed non-uniformly, but it was realized that creating just a single textured cube object and performing minimal additional steps, such as replication with offsets would suffice. Furthermore, as a result of this process, you now have two more scripts in your collection that can assist with future object manipulation in Blender.

Using a script from the console

You can execute the following code in the python console to execute an external script without opening it up in the text editor:

```
filename = "/full/path/to/myscript.py"
exec(compile(open(filename).read(), filename, 'exec'))
```

or run the following code in the python console to execute another script:

```
import bpy

script = bpy.data.texts["script_name.py"]
exec(script.as_string())
```

Using a script from the command line

Using a terminal window (command shell) in your operating system, you can execute the following code in the python console to execute an external script without opening it up in the text editor:

`blender yourblendfilenameorpath --python myscript.py `

A Few Setup Script Examples

So lets try some more scripting examples. Let's say we want to create a new vehicle in Blender. We can use the following code to create the beginnings anew vehicle in Blender if we know some of the key measurements

Lets say we know that the distance from the center of the vehicle to the center of the bogie is 3.5 meters. We also know that the vehicle is 10 meters long so we have an idea where the couplers are. We would also know that the vehicle is 2.9 meters wide.

So what if we could place some marker references using the AXIS Plane object in Blender to visualize the locations of the bogies and couplers. Here is some sample code to do that:

```
import bpy

# Define the locations where objects will be placed (in this case, a list of tuples).
object_locations = [
    (0.0, 3.5, 0.89), # Bogie location 1
    (0.0, -3.5, 0.89), # Bogie location 2
    (0.0, 5.0, 0.89), # Coupler location 1
    (0.0, -5.0, 0.89), # Coupler location 2
    # Add more locations as needed
]

# Create a new mesh object and place it at each location
for i, location in enumerate(object_locations):
    # Create a new cube mesh (you can use any object type)
    bpy.ops.object.empty_add(type='PLAIN_AXES', align='WORLD', location=location)

    # Optionally, give it a unique name (e.g., Cube_1, Cube_2, etc.)
    obj = bpy.context.object
    obj.name = f"axis_{i+1}"

    # You can also adjust other object properties like scale or rotation if needed
    # obj.scale = (1.0, 1.0, 1.0)
    # obj.rotation_euler = (0.0, 0.0, 0.0)
```

If we are following along with the MSTS/OPEN RAILS Exporter documentation, we would want to create a new collection called "MAIN" and then create a sub-collection called "MAIN_700" under "MAIN". We can use the following code to do that:

```
import bpy
```

```
# Function to create a new collection if it doesn't exist
def create_collection(name, parent_collection=None):
    if name not in bpy.data.collections:
        new_collection = bpy.data.collections.new(name)
        if parent_collection:
            parent_collection.children.link(new_collection)
        else:
            bpy.context.scene.collection.children.link(new_collection)
        return new_collection
    else:
        return bpy.data.collections[name]

# Create the main collection "MAIN"
main_collection = create_collection("MAIN")

# Create the sub-collection "MAIN_700" under "MAIN"
sub_collection = create_collection("MAIN_700", parent_collection=main_collection)

print(f"Created collections: {main_collection.name}, {sub_collection.name}")

sub_collection = create_collection("MAIN_2000", parent_collection=main_collection)

print(f"Created collections: {main_collection.name}, {sub_collection.name}")
```

Creating a basic UV Map layout

On of the things that Erick Cantu recommends is to get the right proportions for the UV map layout. Here is an example script to create a basic UV map layout for a 40 ft Boxcar with reasonably correct proportions.

Quick Start:

- pip install Pillow
- get the source code from github: https://github.com/pwillard/blender-uv-layout
- If you downloaded the ZIP file, Save it to a folder on your computer and extract it.
- You will have a folder called "blender-uv-layout" on your computer.
- Inside the folder will be the files: `blender-uv-layout.py` and `uv_meta.json`

The uv-meta.json file contains the metadata for the UV layout. You can edit this file to change the layout.

Rather than post the code here, I will provide a link to the Github repository. You can download the files and follow the instructions in the README.md file to create your own UV layout.

https://github.com/pwillard/blender-uv-layout

Creating a DECAL sheet

Here is an example script to create a decal sheet in Blender. The resulting image will be a set of reporting marks on an Alpha background that can use used with the freightAnim decal method discussed in the *making textures* section of this documentation.

Rather than post the code here, I will provide a link to the Github repository.

https://github.com/pwillard/repmark

Creating Rolling Stock and Locomotives

In order to ensure the precise functioning of locomotives and other rolling stock, it is crucial to acquire as much authentic data as possible and comprehend the expected behavior of the locomotive or rolling stock. These bits of collected data will be used to flesh out the `ENG` and `WAG` files that accompany the 3D Model and textures that make up a particular content item that can be added to Open Rails.

You would need to gather enough prototypical data to accurately complete the physics descriptions in the `WAG` or `ENG` file.

Please note that there have been many improvements and additions to the simulator that go well beyond the capabilities and options of the original Microsoft Train Simulator. Many of the options listed in this section contain values and parameters that are unique to Open Rails. It is assumed that the reader is NOT trying to retain compatibility with legacy Microsoft Train Simulator and doing so is beyond the scope of this section. If you wish to retain Microsoft Train Simulator compatibility, please refer to the technical documents that come with Microsoft Train Simulator.

The process of creating content and config files is often described as a complicated mess, and that description is not entirely inaccurate. However, there is a side project underway to develop a set of replacement files for MSTS 3D models, config files, and textures that are free to use and do not violate anyone's copyrights. These replacement files will serve as examples of the standards for creating 3D models, textures, and config files for Open Rails, which can be shared with everyone.

General Configuration File Format

The formatting of the configuration files are based on a standard known as S-Expression. This method, known as "symbolic expression format" pre-dates a lot of the more modern programming language solutions for managing data, like JSON or XML. It was originally devised to build tree-structured data notation and was a commonly used with languages like LISP. Like LISP, it is heavily dependent on parentheses. You could even say that XML has a somewhat S-Expression like structure if you are familiar with it.

The SIMIS Game Studio which eventually became KUJU, used this format to store the configuration data for the Microsoft Train Simulator. Many of the files that form part of the sim are in one of a number of formats bearing the "SIMISA" signature at the top of the file. The SIMIS file format is identified a 16 byte or longer signature and header, with the remainder of the file being an inner format/data. For example; you might see `SIMISA@@@@@@@@@@JINX0D0t` at the top of a file. This header section does not get compressed with the rest of the file, and is only used to identify the file type. The remaining data is compressed using the Zlib's DEFLATE algorithm.

Open Rails "Units of Measure"

Source: http://www.coalstonewcastle.com.au/physics/or-parameter-uom/

Aim - this section describes the standard units of measure (UoM) available in Open Rails Train Simulator. These are the UoM used to input values from the ENG or WAG files.

Note:

- The UoM values shown in the "Default" column is the UoM value that OR assumes if no UoM value is included in the parameter statement. **(However it is always good practice to include the relevant UoM)**
- The "Alternative UoMs" column shows alternate UoM currently supported by OR.
- The UoM should directly follow the numerical value in the parameter with any spacing, otherwise it will not be correctly interpreted.

Unit of Measure	Description	Default UoM	Alternative UoMs	Typical Examples
Mass	Describes the weight of an object	**kg**	**lb**, **t** (metric tonne), **t-uk** (imperial ton), **t-us** (us ton)	Mass (106t-us)
Distance	Describes the length or distance between two points or locations.	**m**	**cm**, **mm**, **km**, **ft**, **in**	WheelRadius (21.5in)
Area	Describes the area of an object.	**m^2**	**ft^2**	ORTSSuperheatArea (295ft^2)
Volume (of a space)	Describes the volume of an object.	**m^3**	**ft^3**, **in^3**	BoilerVolume (340ft^3)
Volume (fluid quantity)	Describes the fluid quantity contained by an object.	**l**	**g-uk** (gallons - uk), **g-us** (gals - us)	MaxDieselLevel (800g-uk)
Time	Describes the elapsed time between two events.	**s**	**m**, **h**	

Unit of Measure	Description	Default UoM	Alternative UoMs	Typical Examples
Current	Describes the current flow in a circuit.	**A**		MaxCurrent (2700A)
Voltage	Describes the voltage of a circuit.	**V**	**kV**	
Speed	Describes the speed of an object.	**m/s**	**mph**, **km/h**	Sanding (20mph)
Mass Rate of Change	Describes the rate of change in mass for an object.	**lb/h**	**kg/h**	
Frequency	Describes the frequency of movement of an object.	**hz**	**rpm**, **rps**	
Force	Describes the force applied to an object.	**N**	**kN**, **lbf**	MaxForce (227kN)
Power	Describes the power of an object.	**W**	**kW**, **hp**	MaxPower (2650hp)
Pressure	Describes the pressure of an object.	**psi**	**bar**, **kpa**, **inhg**	MaxBoilerPressure (250psi)
Pressure rate of change	Describes the rate of change in pressure of an object.	**psi/s**	**bar/s**, **kpa/s**, **inhg/s**	TrainPipeLeakRate (0.01inhg/s)
Energy Density	Describes the energy density of an object.	**kj/kg**	**btu/lb**	ORTSFuelCalorific (13700btu/lb)
Temperature	Describes the temperature of an object.	**degc**	**degf**	
Davis B Coefficient	Describes the UoM for the Davis B value.	**ns/m**	**lbf/mph**	

Unit of Measure	Description	Default UoM	Alternative UoMs	Typical Examples
Davis C Coefficient	Describes the UoM for the Davis C value.	**Nm/s^2**	**lbf/mph^2**	
Stiffness	Describes the stiffness.	**n/m**		Stiffness (1e6N/m 2e6N/m)
Rotational Inertia	Describes the inertia due to the rotation of an object.	**kgm^2**		
Angle	Describes the angle of an object.	**rad**	deg	ORTSMaximumWheelFlangeAngle (75deg)

The contents of an ENG or WAG file use an S-expression type of notation for grouping of related items. Token definitions are delimited by opening (and closing) parenthesis. All strings are quoted using the double quote character (") and are UTF-8 encoded. Tokens can have zero or more attributes. Human readability is a design goal.

With Open Rails it is very common that things like coupler and brake settings are consistent between many different pieces of rolling stock. For this reason, it is common to see content releases that share these common settings in a specific folder on the hard drive and make call-outs to the common settings using the Open Rails `INCLUDE` token in the `ENG` or `WAG` file.

Using *INCLUDE* files

The INCLUDE token is new to Open Rails and did not exist in Microsoft Train Simulator. Using INCLUDE looks like the following example, where this NAVS content `WAG` FILE makes a call-out to FreightAnims for car body and numbers and then coupler and then the remainder of the specific car details:

```
SIMISA@@@@@@@@@@JINX0D0t______

Wagon ( NAVX_105007_MT

    ORTSFreightAnims (
        WagonEmptyWeight ( 64400lb )
```

```
        FreightAnimStatic (
            Shape( ../NAVS_COMMON/Flatcar-GS-Type-53-6/Flatcar-GS-Type-53-
6_BULK_Carbody-4.s ) ①
            Offset( 0, 0, 0 )
            FreightWeight( 0lb )
        )
            FreightAnimStatic (
            Shape( ../NAVS_COMMON/Flatcar-GS-Type-53-6/Decals/Flatcar-GS-Type-53-
6_Decals_08.s ) ②
            Offset( 0, 0, 0 )
            FreightWeight( 0lb )
        )
    )

    Include( "../NAVS_COMMON/INCLUDE/AAR_Type-E.inc" ) ③
    Include( "../NAVS_COMMON/Flatcar-GS-Type-53-6/Flatcar-GS-Type-53-
6_S2_Roller_Bulkhead_MT.inc" ) ④

        name ( "NAVX_105007_MT" )
```

① Get the 3D MODEL of this vehicle

② Get the additional added modeled items, in this case, the DECALS

③ Include the common Coupler Type

④ Include the physics details that define this particular vehicle

While you **can** continue to place all of your physics data in a single `WAG` or `ENG` file, you might find that this method of including other files helps out when you create many variants of the same basic model. It remains totally optional to make use of this technique but it will save you some typing and allow you to have smaller files.

The INCLUDE token is a new token that did not exist in Microsoft Train Simulator. So using it means that you are using a feature that is not supported by the original Microsoft Train Simulator making the content Open Rails specific.

Erik Cantu has come up with a winning set of options here that would even allow swapping of trucks on an existing model because the trucks are defined in a separate file.

Basic ENG and WAG file Details

The goal of this section is to assist the reader when creating new vehicles or when upgrading their old MSTS ENG and WAG files to current ORTS standards.

The list of Open Rails ENG/WAG file parameters is very likely to be a constantly moving target as new tokens/parameters are being added regularly.

Should there be a ENG/WAG standard layout?

So far, we have not been required to follow a rigid standard for ENG and WAG file layout and there is no indication that this will ever change. While this offers freedom of choice to the developer of content, it has created truly horrific examples of how the s-expression format does not lend itself to good results. Much uploaded content has suffered from the extra or missing parenthesis due opening and closing pairs not being very visible when spanning many lines of text.

There **should** also be an attempt by the reader to adopt a standardized or consistent way they create their own ENG/WAG files. The goal of making these files more easily readable and validated should be attempted.

One recommendation made by Peter Newell is to separate the contents of these files into sections with comment headers, for example:

```
Comment ( ****************************** General Information
******************************
          Included in this section - Type, Shape, Size, Mass, etc

******************************************************************************** )
```

It's not a bad idea, though you actually seldom see it being used.

There are essentially 2 sets of parameter types in Open Rails. These are BASIC and ADVANCED settings. For the most part, BASIC settings are compatible with legacy MSTS while ADVANCED settings are specifically used only in Open Rails.

For example:

Couplers Basic

- CouplingHasRigidConnection
- Coupling - Automatic Bar Chain
- Type
- Spring
- Break
- r0

Couplers Advanced

- ORTSTensionStiffness
- ORTSTensionR0
- ORTSTensionSlack
- ORTSCompressionStiffness
- ORTSCompressionR0
- ORTSCompressionSlack
- ORTSBreak

Universal Settings

These settings are found in nearly every type of rolling stock or locomotive content and while some are optional, most are really not. Many of the new ORTS tokens are related to Derailment Coefficient determination.

Token	MSTS	ORTS	Parameters
Type()	x	x	Steam Diesel Electric Freight Carriage Tender
Wagon()	x	x	Section contains all related wagon() tokens
WagonShape()	x	x	Defines the file path to locate the shape ".S" file
Size()	x	x	3 Dimensions: X Y Z, in meters by default unless a Unit is supplied
WheelRadius()	x	x	Wheel Radius in meters unless a Unit is supplied. Example: "33in"
ORTSLengthCouplerFace ()		x	DC - Length between coupler faces (Is this the same a 'length over strikers' + about 24"?)
ORTSLengthCarBody ()		x	DC - Actual length of car

Token	MSTS	ORTS	Parameters
ORTSLengthBogieCentre ()		x	DC - Actual length of distance between center of bogies
ORTSRigidWheelBase ()		x	DC - The distance between immovable axles
ORTSNumberBogies ()		x	DC - Number of Bogies on the car
ORTSNumberAxles ()		x	DC = Number of total axles for all bogies
ORTSWheelFlangeLength ()		x	DC - Wheel Flange length
ORTSMaximumWheelFlangeAngle (72deg)		x	DC - (Degrees or Radians) default is radians
include		x	Defines the file and path if external file to inserted into the current file

DC indicated that the token is related to the new Derailment Coefficient

See Appendix A for a more complete list of tokens/parameters. Note: this list is still incomplete.

Example WAG file with INCLUDE

Some of the best examples of how to use the Open Rails ENG/WAG design goals come from Erick Cantu's NAVS rolling stock and engines. Erick heavily makes use of the `INCLUDE` token and new `Freight Anim` options. I would recommend downloading and looking at one of Erick's SDK files for a good background on his techniques.

An `INCLUDE` related Excerpt from Elvas Tower Forum is below, where it is pointed out that you can redefine existing sections of a WAG/ENG file with replacement entries that come after the original definitions.

Let's start with the original WVOGrain.wag:

```
SIMISA@@@@@@@@@@JINX0D0t______
```

```
Wagon ( WVOGrain
        comment( WVO Covered hopper, hi-hip version)
        comment( Physics by Chris Lee)
        Type ( Freight )
        WagonShape ( WVOGrain.s )
        Size ( 3.243m 4.728m 17.503m)
        comment( 23.776t empty, 106.747t full )
        Mass ( 73.78t )
        WheelRadius ( 36in/2 )
        InertiaTensor ( Box (3m 3.6m 13.5m) )
        Coupling (
                Type ( Automatic )
                Spring (
                        Stiffness ( 1e6N/m 5e6N/m )
                        Damping ( 1e6N/m 1e6N/m )
                        Break ( 5.1e8N 5.1e8N )
                        r0 ( 20cm 30cm )
                )
                comment( CouplingHasRigidConnection () )
                Velocity ( 0.1m/s )
        )
        Buffers (
                Spring (
                        Stiffness ( 1e6N/m 5e6N/m )
                        Damping ( 1e6N/m/s 1e6N/m/s )
                        r0 ( 0m 1e9 )
                )
                Centre ( 0.5 )
                Radius ( 1 )
                Angle ( 0.5deg )
        )

        Adheasion ( 0.2 0.4 2 0 )
        DerailRailHeight ( 4cm )
        DerailRailForce ( 2.5N/kg*23t )
        DerailBufferForce ( 400kN )
        NumWheels (     8 )
        Friction (
                871N/m/s                0               1mph            3.26N/m/s
1.8
                5.1N/rad/s              1               -1rad/s         0
1
        )
        Lights  (       2

                Light   (
```

```
                comment( Rear red light flashing dim )
                Type            ( 0 )
                Conditions      (
                        Headlight ( 2 )
                        Unit ( 3 )
                )
                FadeIn  ( 0.5 )
                FadeOut ( 0.5 )
                Cycle           ( 0 )
                States  (       2
                        State   (
                                Duration ( 0.35 )
                                LightColour ( 80ff0000 )
                                Position ( 0.0 1.2 -8.955 )
                                Azimuth ( -180 -180 -180 )
                                Transition ( 0 )
                                Radius ( 1.0 )
                        )
                        State   (
                                Duration ( 0.35 )
                                LightColour ( 00000000 )
                                Position ( 0.0 1.2 -8.955 )
                                Azimuth ( -180 -180 -180 )
                                Transition ( 0 )
                                Radius ( 1.0 )
                        )
                )
        )
        Light   (
                comment( Rear red light flashing bright )
                Type            ( 0 )
                Conditions      (
                        Headlight ( 3 )
                        Unit ( 3 )
                )
                FadeIn  ( 0.5 )
                FadeOut ( 0.5 )
                Cycle           ( 0 )
                States  (       2
                        State   (
                                Duration ( 0.35 )
                                LightColour ( 80ff0000 )
                                Position ( 0.0 1.2 -8.955 )
                                Azimuth ( -180 -180 -180 )
                                Transition ( 0 )
                                Radius ( 1.0 )
                        )
```

```
                                State   (
                                        Duration ( 0.35 )
                                        LightColour ( 00000000 )
                                        Position ( 0.0 1.2 -8.955 )
                                        Azimuth ( -180 -180 -180 )
                                        Transition ( 0 )
                                        Radius ( 1.0 )
                                )
                        )
                )
        )

        BrakeEquipmentType( "Handbrake, Triple_valve, Auxilary_reservoir,
Emergency_brake_reservoir" )    <----- THIS SECTION HAS SOME SORT OF ISSUE THAT I'M
GOING TO CHECK
        BrakeSystemType( "Air_single_pipe" )
<----- IN OPENRAILS AS THE BRAKES AREN'T RECOGNIZED
        MaxBrakeForce( 22.13kN )
!

        MaxHandbrakeForce( 22.13kN )
!
        NumberOfHandbrakeLeverSteps( 100 )

!
        EmergencyBrakeResMaxPressure( 110 )
        TripleValveRatio( 2.5 )
!
        MaxReleaseRate( 2.27 )
        MaxApplicationRate( 1.717 )
!
        MaxAuxilaryChargingRate( 1 )
        EmergencyResCapacity( 2.604 )
        EmergencyResChargingRate( 1 )
!
        BrakeCylinderPressureForMaxBrakeBrakeForce( 64 )
<-----------------------------------------------------------------------

        Sound ( "GenFreightWag1.sms" )
)
```

Make a new brake system section and typed it in a STD_Wag_Brake.inc:

```
        BrakeEquipmentType ( "Handbrake, Triple_valve, Auxilary_reservoir,
Emergency_brake_reservoir" )
        BrakeSystemType( "Air_single_pipe" )
```

```
        MaxBrakeForce( 78kN )
        MaxHandbrakeForce ( 57.2kN )
        NumberOfHandbrakeLeverSteps( 100 )
        EmergencyBrakeResMaxPressure( 110 )
        TripleValveRatio( 2.5 )
        EmergencyResVolumeMultiplier ( 1.461 )
        MaxReleaseRate( 22.2 )
        MaxApplicationRate( 13.9 )
        MaxAuxilaryChargingRate( 20 )
        EmergencyResCapacity( 2.025ft^3 )
        EmergencyResChargingRate( 20 )
        BrakePipeVolume ( 0.307ft^3 )
        BrakeCylinderPressureForMaxBrakeBrakeForce( 90 )
```

Make an `Open Rails` folder inside the WVOGrain folder, where I placed a text file with the following text called `WVOGrain.wag` (the well known include method, with comments)

```
                                                                        <-----
REQUIRED EMPTY LINE
include ( "..\\WVOGrain.wag" )                                          <-----
CALLS THE ORIGINAL WAG FILE
Wagon (                                                                 <----- THIS
MEANS: INSERT THE NEXT LINES IN THE WAGON SECTION
        include ( "..\\..\\Common.inc\\Wagons\\Std_Wag_Brakes.inc" )    <-----
CALLS THE .INC FILE WITH THE NEW PARAMETERS
)                                                                       <-----
CLOSE THE WAGON SECTION
```

This trick is in the manual, where it explains how to add instructions to remove the trees on rails in a route without touching the original .trk definition file.

The final result, to Open Rails, would be:

```
SIMISA@@@@@@@@@@JINX0D0t______
<---- START OF THE ORIGINAL WVOGRAIN.WAG

Wagon ( WVOGrain
        comment( WVO Covered hopper, hi-hip version)
        comment( Physics by Chris Lee)
        Type ( Freight )
        WagonShape ( WVOGrain.s )
        Size ( 3.243m 4.728m 17.503m)
        comment( 23.776t empty, 106.747t full )
        Mass ( 73.78t )
```

```
        WheelRadius ( 36in/2 )
        InertiaTensor ( Box (3m 3.6m 13.5m) )
        Coupling (
                Type ( Automatic )
                Spring (
                        Stiffness ( 1e6N/m 5e6N/m )
                        Damping ( 1e6N/m 1e6N/m )
                        Break ( 5.1e8N 5.1e8N )
                        r0 ( 20cm 30cm )
                )
                comment( CouplingHasRigidConnection () )
                Velocity ( 0.1m/s )
        )
        Buffers (
                Spring (
                        Stiffness ( 1e6N/m 5e6N/m )
                        Damping ( 1e6N/m/s 1e6N/m/s )
                        r0 ( 0m 1e9 )
                )
                Centre ( 0.5 )
                Radius ( 1 )
                Angle ( 0.5deg )
        )

        Adheasion ( 0.2 0.4 2 0 )
        DerailRailHeight ( 4cm )
        DerailRailForce ( 2.5N/kg*23t )
        DerailBufferForce ( 400kN )
        NumWheels (     8 )
        Friction (
                871N/m/s                0               1mph            3.26N/m/s
1.8
                5.1N/rad/s              1               -1rad/s         0
1
        )
        Lights  (       2

                Light   (
                        comment( Rear red light flashing dim )
                        Type            ( 0 )
                        Conditions      (
                                Headlight ( 2 )
                                Unit ( 3 )
                        )
                        FadeIn  ( 0.5 )
                        FadeOut ( 0.5 )
                        Cycle           ( 0 )
```

```
                States  (       2
                        State   (
                                Duration ( 0.35 )
                                LightColour ( 80ff0000 )
                                Position ( 0.0 1.2 -8.955 )
                                Azimuth ( -180 -180 -180 )
                                Transition ( 0 )
                                Radius ( 1.0 )
                        )
                        State   (
                                Duration ( 0.35 )
                                LightColour ( 00000000 )
                                Position ( 0.0 1.2 -8.955 )
                                Azimuth ( -180 -180 -180 )
                                Transition ( 0 )
                                Radius ( 1.0 )
                        )
                )
        )
        Light   (
                comment( Rear red light flashing bright )
                Type            ( 0 )
                Conditions      (
                        Headlight ( 3 )
                        Unit ( 3 )
                )
                FadeIn  ( 0.5 )
                FadeOut ( 0.5 )
                Cycle           ( 0 )
                States  (       2
                        State   (
                                Duration ( 0.35 )
                                LightColour ( 80ff0000 )
                                Position ( 0.0 1.2 -8.955 )
                                Azimuth ( -180 -180 -180 )
                                Transition ( 0 )
                                Radius ( 1.0 )
                        )
                        State   (
                                Duration ( 0.35 )
                                LightColour ( 00000000 )
                                Position ( 0.0 1.2 -8.955 )
                                Azimuth ( -180 -180 -180 )
                                Transition ( 0 )
                                Radius ( 1.0 )
                        )
                )
```

```
                )
        )

        BrakeEquipmentType( "Handbrake, Triple_valve, Auxilary_reservoir,
Emergency_brake_reservoir" )
        BrakeSystemType( "Air_single_pipe" )
        MaxBrakeForce( 22.13kN )

        MaxHandbrakeForce( 22.13kN )
        NumberOfHandbrakeLeverSteps( 100 )

        EmergencyBrakeResMaxPressure( 110 )
        TripleValveRatio( 2.5 )
        MaxReleaseRate( 2.27 )
        MaxApplicationRate( 1.717 )
        MaxAuxilaryChargingRate( 1 )
        EmergencyResCapacity( 2.604 )
        EmergencyResChargingRate( 1 )
        BrakeCylinderPressureForMaxBrakeBrakeForce( 64 )

        Sound ( "GenFreightWag1.sms" )

        BrakeEquipmentType ( "Handbrake, Triple_valve, Auxilary_reservoir,
Emergency_brake_reservoir" )   <---- THIS IS THE ADDED SNIPPET FROM THE .INC
        BrakeSystemType( "Air_single_pipe" )
        MaxBrakeForce( 78kN )
        MaxHandbrakeForce ( 57.2kN )
        NumberOfHandbrakeLeverSteps( 100 )
        EmergencyBrakeResMaxPressure( 110 )
        TripleValveRatio( 2.5 )
        EmergencyResVolumeMultiplier ( 1.461 )
        MaxReleaseRate( 22.2 )
        MaxApplicationRate( 13.9 )
        MaxAuxilaryChargingRate( 20 )
        EmergencyResCapacity( 2.025ft^3 )
        EmergencyResChargingRate( 20 )
        BrakePipeVolume ( 0.307ft^3 )
        BrakeCylinderPressureForMaxBrakeBrakeForce( 90 )
<---- ...TO HERE
)
<----- END OF ORIGINAL FILE
```

Clear are mud still, I suppose... more examples are needed.

Option 1 - The Benefits of Common Folders for Re-skinners

Lets say you were going to repaint one of Erick's models. If this is the case, all you really need to do is create a new folder for your content, develop a new SKIN and adjust the existing SHAPE details to refer to the new TEXTURE. Many of the NAVS releases also include a paint kit release that offer much in the way of guidance for re-skinning, making these the best choice for newcomers that want to give it a try.

For example, if you used the NAVS 40 ft Boxcar kit as a base model, your WAG files would basically look like this (Taken from CP 40 foot Grain Boxcars by JW Mercer and based on NAVS_BOXCAR_40_PS-TYPE car by Erick Cantu):

```
SIMISA@@@@@@@@@@JINX0D0t______

Wagon ( CP_123541_LD ①

        Name ( "CPR_Grain_Boxcar_123541_LD" ) ②

    ORTSFreightAnims (
        MSTSFreightAnimEnabled ( 0 )
        FreightAnimStatic (
            Shape( ../NAVS_COMMON/Boxcar_40_PS-Type/Boxcar_40_PS-Type_carbody_2.s ) ③
            Offset( 0, 0, 0 )
            FreightWeight( 0lb )
        )
    )

    Include( "../NAVS_COMMON/INCLUDE/AAR_Type-E.inc" ) ④
    Include( "../NAVS_COMMON/Boxcar_40_PS-Type/PS-Type_40_LD_A3_Conv.inc" )

)
```

① The updated name of the new content, Note: Its is common to use ReportingMark_CarNumber_LD or MT (Loaded or Empty) format for naming.

② The Updated Descriptive name that will be used.

③ The updated .S file with the reference to the new texture being applied

④ The default includes for common data

If you did not have the NAVS 40 Ft Box car installed, you would need to install `NAVS_Boxcar_40_PS-Type.zip` (Elvas Tower or Trainsim) and `navsfr10.zip` (Trainsim).

Option 2 - Completely New Content (with possible help from NAVS)

When creating new content, the amount of work involved can seem daunting at first. Not only do you need to consider the 3D Model and Texturing details, you need to research the Physics and Sounds for the model.

For Sounds

It would be a good recommendation to consider employing the file: `navsfr10.zip` aka "Open Rails NAVS Car Sounds v2.5" available at trainsim.com. Current Link: https://www.trainsim.com/forums/downloadnew.php?fid=37487 If you are working on an ENG file, consider `navscf10.zip`, Current Link: https://www.trainsim.com/forums/downloadnew.php?fid=34168

For WAG file

It would be a good recommendation to use Erick's examples as guidance for how a model Open Rails WAG or ENG file could be formatted.

If you are not a legacy Microsoft Train Simulator user, you may not have access to some of the track and weather sounds used by Microsoft's simulator. If this is the case, you will also want `navs_env.zip` which is a public domain replacement for potentially missing environmental sounds.

If you do want to follow the **NAVS** guidelines, consider indicating this in the naming of your content by adding the `NAVS` prefix to your file names. So what does using this approach mean? Well, it is a well thought out plan for implementing rolling stock that differs greatly from legacy Microsoft Train Simulator. It is a framework that fully embraces Open Rails and it's feature set.

Erick Cantu is very particular about how to model a 3D Model freight car. Using his work as reference material is probably the best place to start. So let's take a deep dive into a model.

Gathering Details

Things that you already know would be Car Dimensions, Coupler Type, Brake Type but you might still need to gather details related to Car Empty Weight and Car Loaded Weight.

So, if we look at a typical NAVS Example again as a guide, what will we find? Once again we see the minimal MAIN File that references a specific shape .S file and freightAnim references for Reporting Mark decals.

```
SIMISA@@@@@@@@@@JINX0D0t______

Wagon ( NAVS_Boxcar_NAVX_103100_LD
```

```
    ORTSFreightAnims (
        FreightAnimStatic (
            Shape( ../NAVS_COMMON/Boxcar_50_PS-Type/Boxcar_50_PS-Type_carbody_1.s )
            Offset( 0, 0, 0 )
            FreightWeight( 0lb )
        )
        FreightAnimStatic (
            Shape( ../NAVS_COMMON/Boxcar_50_PS-Type/DECALS/A1.s )
            Offset( 0, 0, 0 )
            FreightWeight( 0lb )
        )
    )

    Include( "../NAVS_COMMON/INCLUDE/AAR_Type-E.inc" )
    Include( "../NAVS_COMMON/Boxcar_50_PS-Type/PS-Type_50_LD_A3_Plain.inc" )

)
```

The rest is coming from INCLUDE references... so let's look at those as well. In the folder reference `../NAVS_COMMON/INCLUDE/AAR_Type-E.inc` we find the Coupler Type details and these values can be shared among many vehicles.

```
Comment ( //////////AAR Type E Couplers\\\\\\\\\\ )

    Coupling (
        Type ( Automatic )
        Spring (
            Stiffness ( 1.1e6N/m 4.8e6N/m )
            Damping ( 1.1e6N/m/s 1.1e6N/m/s )
            Break ( 390000lbf 390000lbf )
            r0 ( 20cm 30cm )
        )
        Velocity ( 0.1m/s )
    )
    Coupling (
        Type ( Automatic )
        Spring (
            Stiffness ( 1.1e6N/m 4.8e6N/m )
            Damping ( 1.1e6N/m/s 1.1e6N/m/s )
            Break ( 390000lbf 390000lbf )
            r0 ( 20cm 30cm )
        )
        Velocity ( 0.1m/s )
    )
    Buffers (
        Spring (
```

```
            Stiffness ( 1e6N/m 5e6N/m )
            Damping ( 1.1e6N/m/s 1.1e6N/m/s )
            r0 ( 0m 1e9 )
        )
        Centre ( 0.5 )
        Radius ( 1 )
        Angle ( 0.5deg )
    )
```

And then we have the references to the specifics of this type of car in `../NAVS_COMMON/Boxcar_50_PS-Type/PS-Type_50_LD_A3_Plain.inc`. This folder would could many alternatives configurations of the main model as well.

```
    WagonShape ( ../NAVS_COMMON/Boxcar_50_PS-Type/Truck_50T_S2_Plain_50_PS-Type.s )

    Type ( Freight )
    Size ( 122.00in 180.00in 652.00in )
    Mass ( 169000lb )
    WheelRadius ( 16.5in )
    NumWheels ( 4 )
    AntiSlip ( 0 )

Comment ( //////////Friction\\\\\\\\\\ )

    ORTSBearingType ( Friction )
    ORTSDavis_A ( 225.84lb )
    ORTSDavis_B ( 3.8025lbf/mph )
    ORTSDavis_C ( 0.061998lbf/mph^2 )

Comment ( //////////Brakes - general\\\\\\\\\\ )

    BrakeEquipmentType( "Handbrake, Triple_valve, Auxilary_reservoir,
Emergency_brake_reservoir, Retainer_4_Position" )
    BrakeSystemType( "Air_single_pipe" )

    MaxBrakeForce ( 18928lbf )
    Comment ( 169,000-lb loaded weight x 35% loaded cast iron shoe brake ratio x 32%
brake shoe friction coefficient )

    MaxHandbrakeForce( 5408lbf )
    Comment ( AAR minimum 10% handbrake ratio x 169,000-lb Gross Rail Load x 32% brake
shoe friction coefficient )
    NumberOfHandbrakeLeverSteps( 100 )

    ORTSBrakeShoeFriction   (
                            000.00      0.32
```

```
                            016.09      0.15
                            032.19      0.10
                            048.28      0.08
                            064.37      0.07
                            096.56      0.05
                            128.75      0.05
                            160.93      0.05
                            )

    EmergencyBrakeResMaxPressure( 90 )
    EmergencyResVolumeMultiplier ( 1.50 )
    EmergencyResCapacity( 1.86ft^3 )
    EmergencyResChargingRate( 10 )

    TripleValveRatio( 2.5 )
    MaxReleaseRate( 2.50 )
    MaxApplicationRate( 10.00 )
    MaxAuxilaryChargingRate( 11 )
    BrakeCylinderPressureForMaxBrakeBrakeForce( 70 )

    Sound ( "freight_33_LD_castiron.sms" )
```

Notice how Erick has carefully laid out the design of the overall `WAG` file and the related INCLUDES to allow for sharing of common settings and to allow for a refinement of specific details. Also note that the CAR BODY is actually a Freight Anim entry and the MAIN BODY of the vehicle is actually based on the type of BOGEY used. A novel idea that is worth imitating in our own designs. The TRUCK texture file (shared among all variants) is located in the same folder as `../NAVS_COMMON/Boxcar_50_PS-Type/PS-Type_50_LD_A3_Plain.inc` while the car body texture and decal texture is located in the same location as the main WAG file.

While you can still use a **default** MSTS-like WAG file with Open Rails, after seeing what Erick has done, you might be able to see why embracing the new way could be beneficial. This NAVS method is very re-painter-friendly and the fact that Erick also supplies a layered BITMAP file is the ultimate in "being nice to re-painters".

My recommendation is to use these settings as a guide for making new USA Based Rolling stock. If you would like guidance on other types of vehicles, I would recommend using the resources at Peter Newell's Coals to Newcastle website: https://www.coalstonewcastle.com.au/physics/

It's up to you how you want to handle your ENG and WAG files. You can even go with the location reference of \common\

Another Example of Rolling Stock

Here is an example of a traditional MSTS style WAG file for USA style rolling stock. This is a BLLW release. It contains NONE of the new tokens related to Open Rails specifically so should work reliably in MSTS as well. It does utilize `FreightAnim` though the WAG for this car is to an EMPTY car, so the freight anim setting is commented out. In Open Rails, only one Coupling entry is used, so Open Rails will only need to see one in the WAG file.

The `SIMISA@@@@@@@@@@JINX0D0t` entry is a header requirement of Microsoft Train Simulator and is not required for Open Rails.

```
SIMISA@@@@@@@@@@JINX0D0t______

Wagon ( BLLW-H21a-PRR01
    Type ( Freight )
    Comment ( BLLW H21a 2.0 Model by Thomas J Pearce )
    Comment ( Models setup for Empty Coal or Ore Loads )
    Comment ( Freightanim ( BLLW-Coal.s 1 1 ) )
    Comment ( Freightanim ( BLLW-Ore.s 1 1 ) )
    Name ( "BLLW H21a PRR 137788 CK" )
    WagonShape ( BLLW-H21a-PRR01.s )
    Size ( 2.965933m 3.463495m 13.28m )
    CentreOfGravity ( 0m 1.771784m 0m )
    Mass       ( 22.90641t )
    WheelRadius ( 0.4604005m )
    InertiaTensor ( Box (2.965933m 3.463495m 13.28m) )
    Coupling (
        Type ( Automatic )
        Spring (
            Stiffness ( 9.8e5N/m 1.76e6N/m )
            Damping ( 1.77e6N/m/s 1.77e6N/m/s )
            Break ( 2.45e7N 1.78e6N )
            r0 ( 0cm 4cm )
        )
        Comment ( CouplingHasRigidConnection ( 1 ) )
        Velocity ( 0.1m/s )
    )
    Coupling (
        Type ( Automatic )
        Spring (
            Stiffness ( 9.8e5N/m 1.76e6N/m )
            Damping ( 1.77e6N/m/s 1.77e6N/m/s )
            Break ( 2.45e6N 1.78e6N )
            r0 ( 0cm 4cm )
```

```
        )
        CouplingHasRigidConnection ( 1 )
        Velocity ( -0.1m/s )
    )
    Buffers (
        Spring (
            Stiffness ( 9.8e5N/m 9.8e5N/m )
            Damping ( 1.7e6N/m/s 1.7e6N/m/s )
            r0 ( 20cm 30cm )
        )
        Centre ( 0.5 )
        Radius ( 1 )
        Angle ( 0.5deg )
    )
    Adheasion ( 0.2 0.4 2 0 )
    DerailRailHeight ( 4cm )
    DerailRailForce ( 2.5*27t )
    DerailBufferForce ( 400kN )
    NumWheels ( 8 )
    Friction (
        100N/m/s        1        -1mph        0        1
        5.1N/rad/s        1        -1rad/s        0        1
    )
    BrakeEquipmentType( "Handbrake, Triple_valve, Auxilary_reservoir,
Emergency_brake_reservoir" )
    BrakeSystemType( "Air_single_pipe" )
    MaxBrakeForce( 50kN )

    MaxHandbrakeForce( 35kN )
    NumberOfHandbrakeLeverSteps( 100 )

    TripleValveRatio( 2.5 )
    MaxReleaseRate( 15 )
    MaxApplicationRate( 25 )
    MaxAuxilaryChargingRate( 5 )
    EmergencyResCapacity( 7 )
    EmergencyResChargingRate( 5 )
    EmergencyBrakeResMaxPressure( 90 )
    BrakeCylinderPressureForMaxBrakeBrakeForce( 50 )

      Sound (    "GenFreightWag2.sms" )
)
```

If you keep up with changes in Open Rails, you will see that some items have been made more logical. For instance, if you supply the newer DERAIL COEFFICIENT tokens then you no longer need to supply `NUMWHEELS` as `ORTSNumAxles` from the Derail values makes it redundant. Additionally, you

only need to supply one `COUPLING` token, not two. Refer to the Derail Coefficient section of the Open Rails manual for further details.

Specifying Lights on Locomotives and Wagons

Introduction

In Microsoft Train Simulator, you can use the built-in lighting system to create a variety of lights for locomotives and wagons, including headlights, warning lights, and ditch lights. These lights can be used on both computer-controlled and player-drivable trains. The lights can be set to be steady or flashing, and can also be programmed to change based on certain parameters, such as a penalty brake application, or to turn on automatically based on the time of day or weather.

Lights are specified as part of a locomotive/wagon's .eng file (the same file that defines the physical characteristics of the unit, such as its height and weight, and its cab controls). Essentially, a set of conditions is defined and then lights that should light in each case are listed. Like all of the hand-editable files in Train Simulator, the .eng file can be edited in any Unicode text editor.

Available Tokens for Lights

- Type Glow Cone
- FadeIn
- FadeOut
- Cycle
- Headlight
- Unit
- Penalty
- Control
- Service
- TimeOfDay
- Weather
- Coupling
- Duration
- LightColour
- Position
- Radius

- Azimuth
- Elevation
- Transition
- Angle

Setting requirements

The first thing you must specify is when you want the light or lights in question to come on. This is done by setting a number of flags to define the requirements for the lights concerned to light – if no requirements are listed, the light will always be on.

Possible Requirements

- Headlight (0) (ignore headlight control status)
- Headlight (1) (the headlight control is at “Off”)
- Headlight (2) (the headlight control is at “Dim”)
- Headlight (3) (the headlight control is at “Bright”)
- -----
- Unit (0) (ignore unit status)
- Unit (1) (the unit is in the train but isn’t at the front or back)
- Unit (2) (the unit is the frontmost unit of the train)
- Unit (3) (the unit is the rearmost unit of the train)
- -----
- Penalty (0) (ignore penalty status)
- Penalty (1) (the train has not “stopped due to a penalty brake application”)
- Penalty (2) (penalty) (the train has stopped due to a penalty brake application” (useful for specifying an emergency warning light)
- -----
- Control (0) (ignore control status)
- Control (1) (the player is not control of the unit [i.e. it is a helper locomotive or wagon)
- Control (2) (the player is in control of the unit)
- -----
- Service (0) (ignore service status)
- Service (1) (the unit is not in service e.g. a failed train lying in a siding)

- Service (2) (the unit is in service)
- -----
- TimeOfDay (0) (ignore time of day)
- TimeOfDay (1) (only come on if it's daytime)
- TimeOfDay (2) (only come on if it's night
- -----
- Weather (0) (ignore weather)
- Weather (1) (only come on if it's fine)
- Weather (2) (only come on if it's raining)
- Weather (3) (only come on if it's snowing)
- -----
- Coupling (0) (ignore coupling status)
- Coupling (1) (original front of wagon is coupled)
- Coupling (2) (original rear of wagon is coupled)
- Coupling (3) (both ends of wagon are coupled)

Defining lights

Once you have defined a set of requirements, you must list all the lights which you want to light when those requirements are fulfilled.

Type of light

Two types of lights can be defined in Train Simulator.

Glow

Two textured triangles. Glows are relatively inexpensive in terms of processing time and are perfect for use when you merely wish to show that a light is on rather than off. They do not affect polygons around them (i.e. they do not "light things up").

Cone

A "real" light which has an effect on the polygons around it. Again, this takes more processing time than a glow and should only be used if you wish to "light something up" as opposed to merely showing a light is on. Headlights are a suitable use for cone lights – one cone can simulate the light from a bank of headlights, with a glow being used to highlight each individual light if desired.

```
Type(n) (where n = 0 is glow, n = 1 is cone)
```

Fade-in and Fade-out times

These values simply let you define a number of seconds that the light will take to brighten to the brightness of its current state when it is lit, and the number of seconds it will take to die away when it is switched off. If you specify 0 for both values, the light will light and darken instantaneously.

`FadeIn(t) (where t is the time in seconds)` `FadeOut(t) (where t is the time in seconds)`

Cycle

If this value is set to 1, the light's states will be cycled through first forwards, then backwards and so on (e.g. 1, 2, 3, 4, 5, 4, 3, 2, 1, 2, 3, 4...). If this value is left as 0, the states will be played through in order repeatedly (e.g. 1, 2, 3, 4, 5, 1, 2, 3, 4, 5, 1, 2, 3, 4...)

`Cycle(n) (where n = 0 is play in order repeatedly, n = 1 is cycle)`

States

A light has a number of states – a state is simply the form of the light at a given time. States can be played through in order repeatedly or made to cycle backwards and forwards (see above). For example, to define a red flashing light, you would define one state that had the light bright red for 1 second and another that had it off for one second. When the two states were cycled between, the light would appear to flash. By altering colours, coordinates and durations of states, you can make lights appear to change colour, move around or flash.

A state is made up of the following values:

- Duration(t) Specifies the duration of this light state where t is the time in seconds.
- Colour(aarrggbb) A 32-bit hexadecimal value specifying the colour and translucency of the light. aa, rr, gg, bb are 8-bit hexadecimal values specifying the translucency (how "solid" the light is - this only affects glows) and red, green and blue colour components of the light colour respectively. (If you are not familiar with hexadecimal, a brief explanation is included in the appendices of this document).
- Position(x y z) The position offset in Cartesian coordinates from the centre of the unit.
- Radius(r) This is the radius of the light sphere for cone lights and the radius of the polygon "disc" for glow lights.
- Azimuth(min centre max) This specifies the heading (rotation about the Y-axis) of glow lights – in the simplest case min, centre and max can all be set to the same angle (in degrees). For more complex situations where you want to define the arc of the light you can specify for example -45, 0 and 45 degrees respectively to produce a light with a 90 degree arc. Essentially glow lights attempt to always face the camera but are limited by the azimuth and elevation settings.

The best way to see what the values do is to experiment - set up a light with a set of values and then move the camera around it to see what happens.

- Elevation(min centre max) This specifies the elevation (rotation about the X-axis) of glow lights and (apart from the axis of effect) works identically to the azimuth.
- Transition(n) This specifies how the transition from one state to the next occurs - if n equals zero then the transition is instantaneous, if it is one then the light parameters are interpolated from this state to the next.
- Angle(n) (Cone lights only) the radius of the cone, in degrees.

Examples

Here are a few examples, taken from the Dash 9 .eng file.

Example 1 – the Dash 9's front headlights

```
Lights  (   10
        Light   (
            comment( Sphere of light )
            Type        ( 1 )
            Conditions  (
                Headlight ( 3 )
                Unit ( 2 )
                )
            Cycle   ( 0 )
            FadeIn  ( 0.5 )
            FadeOut ( 0.5 )
            States  (   1
                State   (
                    Duration ( 0.0 )
                    LightColour ( ffffffff )
                    Position ( 0.0 3.5 18 )
                    Transition ( 0 )
                    Radius ( 400.0 )
                    Angle ( 15.0 )
                    )
                )
            )
        Light   (
            comment( Head light dim )
            Type        ( 0 )
            Conditions  (
                Headlight ( 2 )
                Unit ( 2 )
```

```
                )
            FadeIn  ( 0.5 )
            FadeOut ( 0.5 )
            Cycle   ( 0 )
            States  (   1
                State   (
                    Duration ( 0.0 )
                    LightColour ( 80ffffff )
                    Position ( 0.15 3.20 9.679 )
                    Azimuth ( 0.0 -5.0 5.0 )
                    Transition ( 0 )
                    Radius ( 1.0 )
                    )
                )
            )
        Light   (
            comment( Head light bright )
            Type        ( 0 )
            Conditions  (
                Headlight ( 3 )
                Unit ( 2 )
                )
            FadeIn  ( 0.5 )
            FadeOut ( 0.5 )
            Cycle   ( 0 )
            States  (   1
                State   (
                    Duration ( 0.0 )
                    LightColour ( ffffffff )
                    Position ( 0.15 3.20 9.679 )
                    Azimuth ( 0.0 -5.0 5.0 )
                    Transition ( 0 )
                    Radius ( 1.0 )
                    )
                )
            )

Light   (
            comment( Head light bright )
            Type        ( 0 )
            Conditions  (
                Headlight ( 3 )
                Unit ( 2 )
                )
            FadeIn  ( 0.5 )
            FadeOut ( 0.5 )
            Cycle       ( 0 )
```

```
            States  (   1
                State   (
                    Duration ( 0.0 )
                    LightColour ( ffffffff )
                    Position ( 0.15 3.20 9.679 )
                    Azimuth ( 0.0 -5.0 5.0 )
                    Transition ( 0 )
                    Radius ( 1.0 )
                    )
                )
            )
```

Example 2 – the Dash 9's front right flashing (ditch) light.

```
Light   (
            comment( Front right flashing light )
            Type        ( 0 )
            Conditions  (
                Headlight ( 3 )
                Unit ( 2 )
                )
            Cycle   ( 0 )
            FadeIn  ( 0.5 )
            FadeOut ( 0.5 )
            States  (   2
                State   (
                    Duration ( 0.3 )
                    LightColour ( ffffffff )
                    Position ( -0.60932 1.98713 10.600 )
                    Azimuth ( 0 -17.0 -17.0 )
                    Transition ( 0 )
                    Radius ( 1.5 )
                    )
                State   (
                    Duration ( 0.3 )
                    LightColour ( 00000000 )
                    Position ( -0.60932 1.98713 10.600 )
                    Azimuth ( 0 -17.0 -17.0 )
                    Transition ( 0 )
                    Radius ( 1.5 )
                    )

                )
            )
```

Example 3 – the Dash 9's rear red light

```
        Light   (
            comment( Rear red light )
            Type        ( 0 )
            Conditions  (
                Headlight ( 3 )
                Unit ( 2 )
                )
            FadeIn  ( 0.5 )
            FadeOut ( 0.5 )
            Cycle   ( 0 )
            States  (   1
                State   (
                    Duration ( 0.0 )
                    LightColour ( 80ff0000 )
                    Position ( 0.98651 2.12618 -10.677706 )
                    Azimuth ( -180 -180 -180 )
                    Transition ( 0 )
                    Radius ( 0.5 )
                    )
                )
            )

        )
```

A brief explanation of hexadecimal numbers

Hexadecimal is a base 16 counting system used by computers. Humans generally use base 10. In base 10, we have a units column, a tens column, a hundreds (ten squared) column, etc. and numbers can be described by specifying a value for each column.

For example, 57 has 5 in the tens column and 7 in the units column and (5 * 10) + 7 * 1 = 57.

In base 16 you have a units column, a sixteens column, a 256s (16 squared) column, etc. The extra digits are A to F so you count in hex like this : 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A (10 in decimal), B (11 in decimal), C (12), D (13), E (14), F (15), 10 (remember this is hex so that's one in the 16s column and zero in the units column giving you 16).

Another example the hex number 4D is 4 in the 16s column and D in the units column, but D is 13 so that's (4 * 16) + (13 * 1) =64 + 13 = 77.

In the case of lights, hexadecimal values are used to set an alpha, red, green and blue value for a

light. Each value is set by a two-digit hexadecimal number, so can be anything between 00 (zero) and FF (F in the 16s column and F in the units column, so since F=15 that's (15*16) + (15*1) = 240 + 15 = 255). So a two-digit hexadecimal number can be anything between 0 and 255, i.e. there are 256 possible values.

Therefore the following setting:

```
0000FF00
```

...would result in a light that has no alpha (i.e. it's completely "transparent"), no red, full green and no blue. The result would be a transparent green light.

A more realistic example is:

```
80ff0000
```

The first two values (80) control the alpha. 80 = 8*16 = 128, so this light will be half-alphaed, or semi-transparent. The two red values are set to ff (255, as explained above) so the light's red value is at maximum. Since the green and blue values are set to 00, the light will be bright red (if they were set to ff as well, the colours would combine together to form a bright white light).

One final example. The value:

```
60808000
```

Would result in a yellow, slightly more than half transparent (i.e. more transparent than the red light above) yellow light.

Effects

Available tokens for Effects

- StackFX
- CylindersFX
- WhistleFX
- SafetyValvesFX
- Exhaust
- Cylinders2FX Open Rails
- CompressorFX Open Rails
- GeneratorFX Open Rails
- Injectors1FX Open Rails
- Injectors2FX Open Rails
- HeatingSteamBoilerFX Open Rails

- WagonGeneratorFX Open Rails
- WagonSmokeFX Open Rails
- HeatingHoseFX Open Rails
- WaterScoopFX Open Rails
- TenderWaterOverflowFX Open Rails
- BearingHotboxFX Open Rails

Freight Animations

Available tokens for Freight Animations

- FreightAnim
- IntakePoint FuelWater FuelCoal FuelWood FuelSand FuelDiesel FreightGeneral FreightLivestock FreightFuel FreightGrain FreightCoal FreightGrvel FreightSand
- ORTSFreightAnims Open Rails
- FreightAnimContinuous Open Rails
- WagonEmptyWeight Open Rails
- EmptyMaxBrakeForce Open Rails
- EmptyMaxHandbrakeForce Open Rails
- EmptyORTSDavis_A Open Rails
- EmptyORTSDavis_B Open Rails
- EmptyORTSDavis_C Open Rails
- EmptyCentreOfGravity_Y Open Rails
- EmptyORTSWagonFrontalArea Open Rails
- EmptyORTSDavisDragConstant Open Rails
- FreightWeightWhenFull Open Rails
- FullMaxBrakeForce Open Rails
- FullMaxHandbrakeForce Open Rails
- FullORTSDavis_A Open Rails
- FullORTSDavis_B Open Rails
- FullORTSDavis_C Open Rails
- FullCentreOfGravity_Y Open Rails
- FullORTSWagonFrontalArea Open Rails
- FullORTSDavisDragConstant Open Rails

- MSTSFreightAnimEnabled Open Rails
- IsGondola Open Rails
- UnloadingStartDelay Open Rails
- FullAtStart Open Rails
- Shape Open Rails
- MaxHeight Open Rails
- MinHeight Open Rails
- ORTSWaterScoopFillElevation Open Rails
- ORTSWaterScoopDepth Open Rails
- ORTSWaterScoopWidth Open Rails

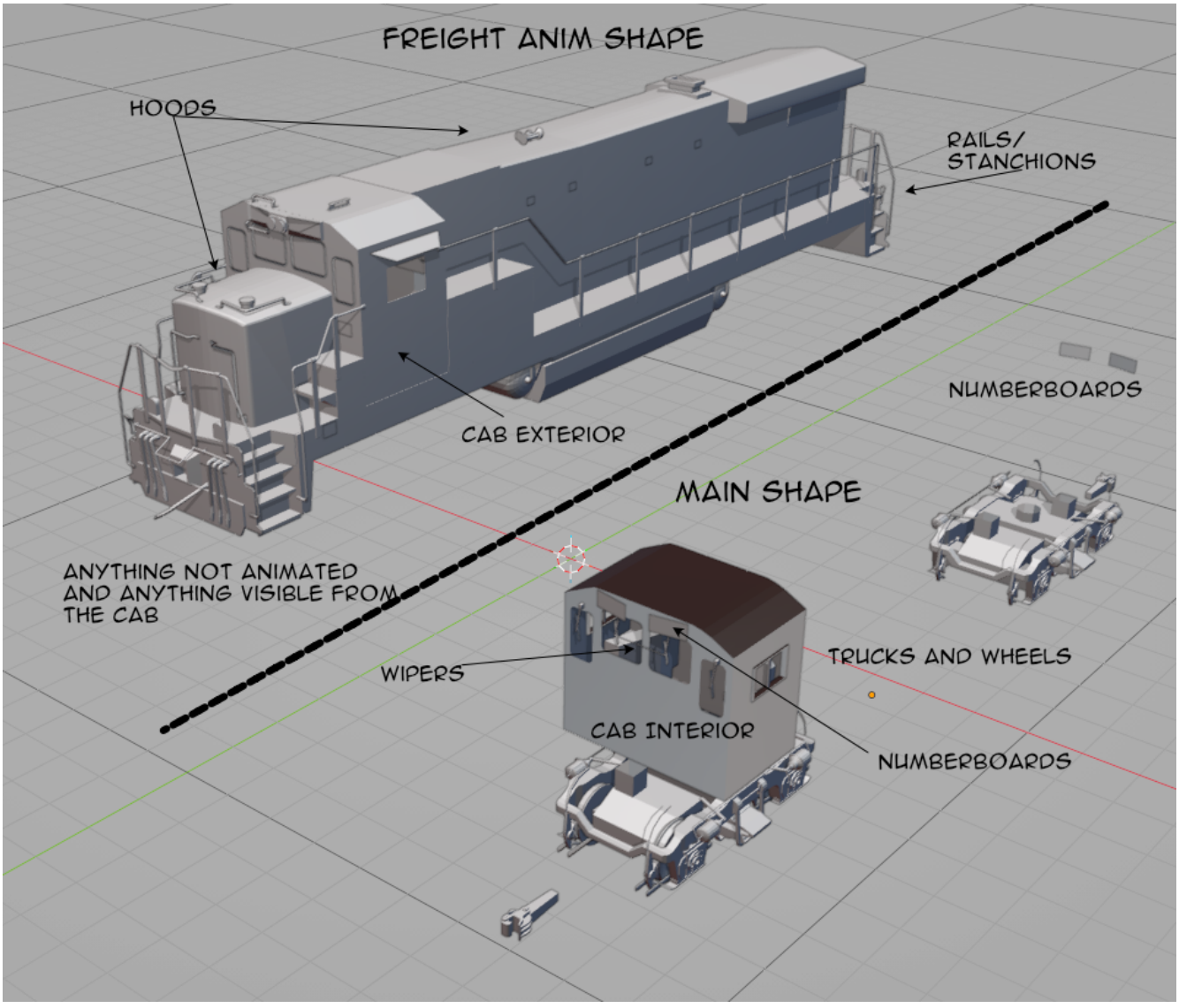
Creating 3D Cabs

Planning Ahead

(Eric Cantu)

Here's the secret sauce to making a locomotive compatible with a 3D cab, whether it's your own cab or a hypothetical future one. The problem with using a standalone 3D cab model is any locomotive you build, unless it's one of those Soviet or British units with a flat face and no nose, is going to have parts of the body visible out the cab windows, even an F-unit will have the nose visible. Think of all the rails, stanchions, and grabs you can see from the cab. You could include all this stuff in your cab, but then you need a separate cab for each and every individual locomotive that's the same but for the textures. This is, in a word, a really dumb way to do things.

The thing is that Open Rails has, for quite some time, supported visibility parameters for freight shapes that allow you to control when a model is visible. This means that, if you design your locomotive right, you can include everything that will be visible from the cab in the freight shape. This, obviously, excludes anything animated, but most of the animated parts aren't going to be visible from the cab anyway. You can then build a single 3D cab model, place it in a common folder, edit the shape file to hard-code the texture paths, and use a single cab for many locomotives. Here's a quick-and-dirty sketch of a B23-7 that shows how a model should be set up:



Notice that anything that might interfere with the 3D cab goes in the main model, along with the animated parts (note that I animate my numberboards, bell clappers, and cab windows on my current models, so they're included here in this drawing). The main thing you don't want in your freight shape is the external model's cab interior. This is going to step on your 3D cab, so obviously it should go in the main model where it won't be visible from interior views.

Now, look at the freight shape. Notice that anything visible from the cab interior (I included the cab exterior shell because this often interfaces with the interior in a way where the edges of things like gaskets and window ledges will be visible from the windows) is in this model. For this model, you set the visibility parameters such that the model is visible in both interior and exterior views. Here's an example from the 70-tonner:

```
ORTSFreightAnims (
     MSTSFreightAnimEnabled ( 0 )
     FreightAnimStatic (
         Shape( ../NAVS_COMMON/GE_70T/GE_70T_Carbody_1.s )
```

```
        Visibility ( "Outside,Cab3D" )
        Offset( 0.000, 0.000, 0.000 )
        FreightWeight( 0lb )
    )
    FreightAnimStatic (
        Shape( ../NAVS_COMMON/GE_70T/GE_70T_Figures_1.s )
        Visibility ( "Outside" )
        Offset( 0.000, 0.000, 0.000 )
        FreightWeight( 0lb )
    )
    FreightAnimStatic (
        Shape( ../NAVS_COMMON/GE_70T/Horn_Single_Chime_1.s )
        Visibility ( "Outside,Cab3D" )
        Offset( 0.000, 4.140, 3.264 )
        FreightWeight( 0lb )
    )
    FreightAnimStatic (
        Shape( ../NAVS_COMMON/GE_70T/Decals/GE_70T_Decal_1.s )
        Visibility ( "Outside,Cab3D" )
        Offset( 0.000, 0.000, 0.000 )
        FreightWeight( 0lb )
    )
)
```

Notice these models all have their visibility set to outside and cab3D. These will show up in the 3D cab.

The 3D cab should have nothing in it but the cab walls and everything within them, or you won't be able to use the same cab with multiple locomotives. Anything visible outside the 3D cab should be placed as a freight shape visible in both external and internal views. Here's a practical example with the GP20 from the CTN test stock I made for the track audio project:

MAIN MODEL

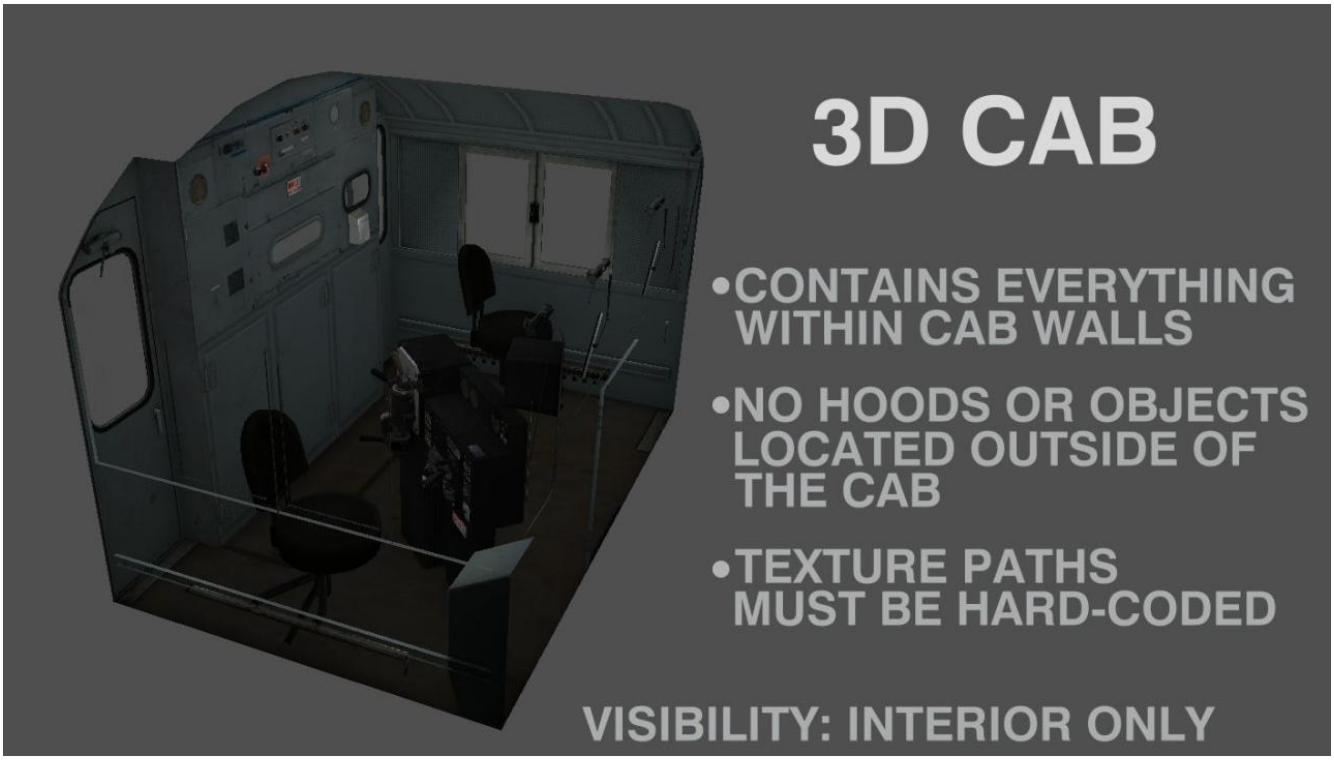
- CONTAINS ALL ANIMATED PARTS
- CONTAINS EXTERNAL MODEL CAB INTERIOR

VISIBILITY: EXTERIOR ONLY

HOOD MODEL

- CONTAINS PARTS VISIBLE IN ALL VIEWS

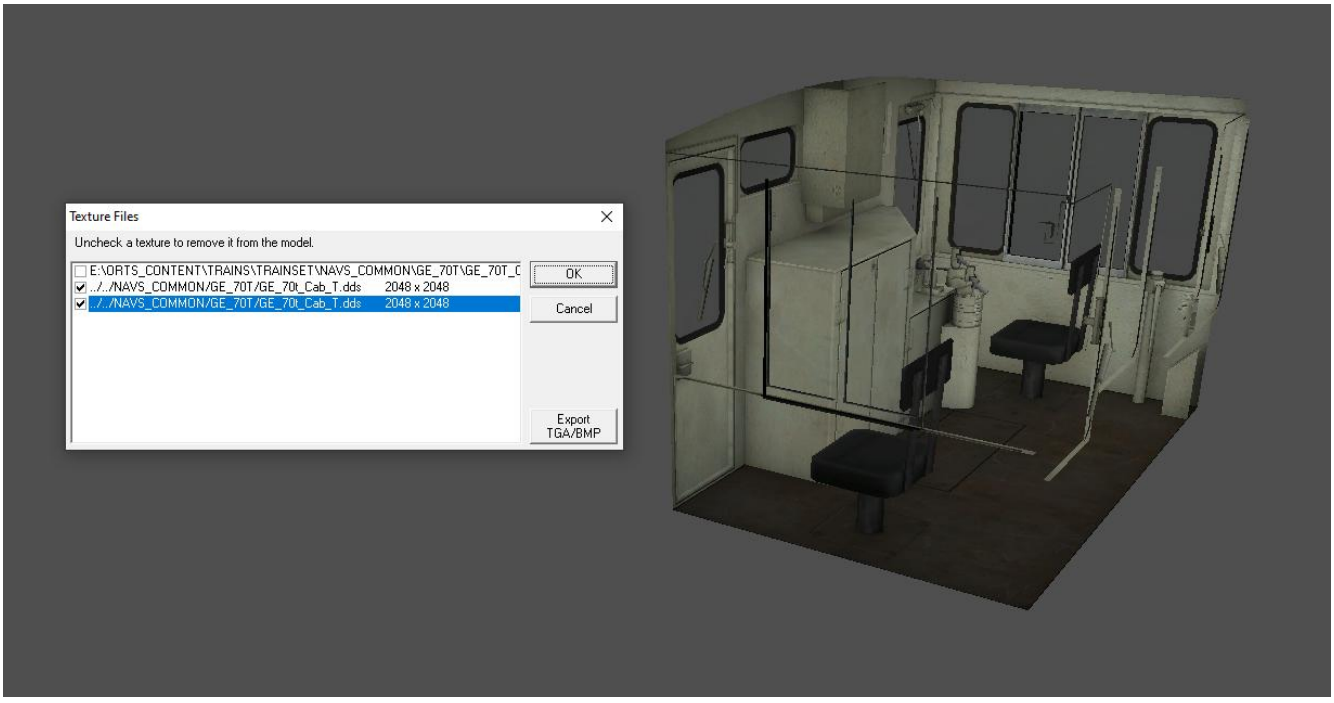
VISIBILITY: ALL VIEWS



If you attempt to load a 3D cab from a remote folder, OR will, by default, look for the textures in the nonexistent `cabview3D` folder of the locomotive calling it. Thus, to ensure the cab textures load properly, the paths must be hard-coded by decompressing the shape, searching for "images," and adding an appropriate path. Open Rails Train Simulator will start two levels down from the `TRAINSET` folder by default, since it expects textures in the locomotive's `Cabview3D` folder (which a locomotive calling a remote 3D cab will not have), so we must first tell it to go up two levels. Here's an example texture path from the 70-tonner cab:

```
../../NAVS_COMMON/GE_70T/GE_70T_Cab_T.dds0
```

And, to go with that example, here is one of the two 70-ton cab options along with the textures dialog showing the hard-coded paths. You can call this cab from any locomotive and the cab textures will load from the folder the shape and CVF are physically located in.


Texture Files
Uncheck a texture to remove it from the model.
E:\ORTS_CONTENT\TRAINS\TRAINSET\NAVS_COMMON\GE_70T\GE_70T_C
../../NAVS_COMMON/GE_70T/GE_70t_Cab_T.dds 2048 x 2048
../../NAVS_COMMON/GE_70T/GE_70t_Cab_T.dds 2048 x 2048
OK
Cancel
Export TGA/BMP

Appendix A

Parameter	Program	Notes
Universal		
Type	MSTS	Steam Diesel Electric Freight Carriage Tender
WagonShape	MSTS	
Wagon	MSTS	
Size	MSTS	
Mass	MSTS	
WheelRadius	MSTS	
NumWheels	MSTS	
Sanding	MSTS	
Sound	MSTS	
CabView	MSTS	
HeadOut	MSTS	
Name	MSTS	
Description	MSTS	
Include	ORTS	
Steam Basic		
Type	MSTS	Steam Diesel Electric
WheelRadius	MSTS	
NumWheels	MSTS	
Sanding	MSTS	
IsTenderRequired	MSTS	
BoilerVolume	MSTS	
MaxBoilerPressure	MSTS	
MaxTenderWaterMass	MSTS	
MaxTenderCoalMass	MSTS	
SteamFiremanIsMechanicalStoker	MSTS	
NumCylinders	MSTS	

Parameter	Program	Notes
CylinderStroke	MSTS	
CylinderDiameter	MSTS	
ORTSSteamLocomotiveType	ORTS	Simple Compound Geared
ORTSDriveWheelWeight	ORTS	
ORTSSteamBoilerType	ORTS	Saturated Superheated
ORTSEvaporationArea	ORTS	
ORTSSuperheatArea	ORTS	
MaxSteamHeatingPressure	ORTS	
ORTSGrateArea	ORTS	
ORTSFuelCalorific	ORTS	
ORTSSteamFiremanMaxPossibleFiringRate	ORTS	
LPNumCylinders	ORTS	
LPCylinderStroke	ORTS	
LPCylinderDiameter	ORTS	
ORTSSteamGearRatio	ORTS	
ORTSSteamMaxGearPistonRate	ORTS	
ORTSSteamGearType	ORTS	
Steam Advanced		
ORTSBoilerEvaporationRate	ORTS	
ORTSMaxIndicatedHorsepower	ORTS	
ORTSMaxSuperheatTemperature	ORTS	
ORTSSuperheatCutoffPressureFactor	ORTS	
ORTSCylinderPortOpening	ORTS	
ORTSCylinderExhaustOpen	ORTS	
ORTSBoilerEfficiency	ORTS	
ORTSBurnRate	ORTS	
ORTSCylinderInitialPressureDrop	ORTS	
ORTSCylinderBackPressure	ORTS	

Parameter	Program	Notes
Diesel Basic		
WheelRadius	MSTS	
NumWheels	MSTS	
ORTSDriveWheelWeight	MSTS	
Sanding	MSTS	
AntiSlip	MSTS	
MaxDieselLevel	MSTS	
MaxTemperature	MSTS	
MaxOilPressure	MSTS	
DieselUsedPerHourAtMaxPower	MSTS	
DieselUsedPerHourAtIdle	MSTS	
DieselEngineIdleRPM	MSTS	
DieselEngineMaxRPM	MSTS	
DieselEngineMaxRPMChangeRate	MSTS	
MaxPower	MSTS	
MaxForce	MSTS	
MaxVelocity	MSTS	
MaxContinuousForce	MSTS	
MaxCurrent	MSTS	
DieselSmokeEffectInitialMagnitude	MSTS	
DieselSmokeEffectMaxMagnitude	MSTS	
DieselSmokeEffectInitialSmokeRate	MSTS	
DieselSmokeEffectMaxSmokeRate	MSTS	
ORTSWheelSlipCausesThrottleDown	ORTS	
ORTSDieselEngineMaxPower	ORTS	
Diesel Advanced		

Parameter	Program	Notes
ORTSMaxTractiveForceCurves	ORTS	
ORTSCurtius_Kniffler	ORTS	
ORTSInertia	ORTS	
Couplers Basic		
CouplingHasRigidConnection	MSTS	
Coupling	MSTS	Automatic Bar Chain
Type	MSTS	
Spring	MSTS	
Break	MSTS	
r0	MSTS	
Couplers Advanced		
ORTSTensionStiffness	ORTS	
ORTSTensionR0	ORTS	
ORTSTensionSlack	ORTS	
ORTSCompressionStiffness	ORTS	
ORTSCompressionR0	ORTS	
ORTSCompressionSlack	ORTS	
ORTSBreak	ORTS	
Brakes Air		
BrakeEquipmentType	MSTS	handbrake retainer_3_position retainer_4_position vacuum_brake triple_valve graduated_release_triple_valve ep_brake ecp_brake auxilary_reservoir emergency_brake_reservoir distributor
BrakeSystemType	MSTS	Air_single_pipe Air_twin_pipe Vacuum_single_pipe Vacuum_twin_pipe ECP EP Air_piped Vacuum_piped
MaxBrakeForce	MSTS	
MaxHandbrakeForce	MSTS	

Parameter	Program	Notes
EmergencyResVolumeMultiplier	MSTS	
TripleValveRatio	MSTS	
MaxReleaseRate	MSTS	
MaxApplicationRate	MSTS	
MaxAuxilaryChargingRate	MSTS	
EmergencyResChargingRate	MSTS	
EmergencyResCapacity	MSTS	
BrakeCylinderPressureForMaxBrakeBrakeForce	MSTS	
AirBrakesMainMaxAirPressure	MSTS	
AirBrakesCompressorRestartPressure	MSTS	
AirBrakesMainResVolume	MSTS	
TrainPipeLeakRate	MSTS	
TrainBrakesControllerMaxSystemPressure	MSTS	
TrainBrakesControllerMaxReleaseRate	MSTS	
TrainBrakesControllerMaxQuickReleaseRate	MSTS	
TrainBrakesControllerMaxApplicationRate	MSTS	
TrainBrakesControllerEmergencyApplicationRate	MSTS	
TrainBrakesControllerFullServicePressureDrop	MSTS	
TrainBrakesControllerMinPressureReduction	MSTS	
ControlName	MSTS	
ORTSBrakeShoeFriction	ORTS	
BrakePipeVolume	ORTS	
ORTSBrakeShoeFriction	ORTS	

Parameter	Program	Notes
ORTSMainResChargingRate	ORTS	
ORTSBrakePipeChargingRate	ORTS	
ORTSBrakePipeTimeFactor	ORTS	
ORTSBrakeServiceTimeFactor	ORTS	
ORTSBrakeEmergencyTimeFactor	ORTS	
Brakes Vacuum		
BrakeEquipmentType	MSTS	handbrake vacuum_brake
BrakeSystemType	MSTS	Vacuum_single_pipe Vacuum_twin_pipe Vacuum_piped
MaxBrakeForce	MSTS	
MaxHandbrakeForce	MSTS	
MaxReleaseRate	MSTS	
MaxApplicationRate	MSTS	
ORTSNumberBrakeCylinders	MSTS	
VacuumBrakesMinBoilerPressureMaxVacuum	MSTS	
TrainPipeLeakRate	MSTS	
BrakesTrainBrakeType	MSTS	vacuum_single_pipe_eq vacuum_single_pipe
VacuumBrakesSmallEjectorUsageRate	MSTS	
VacuumBrakesLargeEjectorUsageRate	MSTS	
VacuumBrakesHasVacuumPump	MSTS	
TrainBrakesControllerMaxSystemPressure	MSTS	
TrainBrakesControllerMaxReleaseRate	MSTS	
TrainBrakesControllerMaxApplicationRate	MSTS	

Parameter	Program	Notes
TrainBrakesControllerEmergencyApplicationRate	MSTS	
TrainBrakesControllerFullServicePressureDrop	MSTS	
ControlName	MSTS	
ORTSBrakeShoeFriction	ORTS	
BrakePipeVolume	ORTS	
ORTSDirectAdmissionValve	ORTS	
ORTSAuxilaryResCapacity	ORTS	
ORTSBrakeCylinderSize	ORTS	
ORTSBrakePipeChargingRate	ORTS	
ORTSBrakePipeTimeFactor	ORTS	
ORTSBrakeServiceTimeFactor	ORTS	
ORTSBrakeEmergencyTimeFactor	ORTS	
ORTSBrakePipeDischargeTimeMult	ORTS	
ORTSVacuumBrakesMainResVolume	ORTS	
ORTSVacuumBrakesMainResMaxVacuum	ORTS	
ORTSVacuumBrakesExhausterRestartVacuum	ORTS	
ORTSVacuumBrakesMainResChargingRate	ORTS	
Resistance		
CentreOfGravity	MSTS	
ORTSBearingType	ORTS	Roller Low Friction
ORTSDavis_A	ORTS	
ORTSDavis_B	ORTS	
ORTSDavis_C	ORTS	
ORTSBearingType	ORTS	
ORTSDriveWheelWeight	ORTS	

Parameter	Program	Notes
ORTSTrackGauge	ORTS	
ORTSRigidWheelbase	ORTS	
ORTSUnbalancedSuperelevation	ORTS	
ORTSTrackSuperElevation	ORTS	
ORTSSingleTunnelArea	ORTS	
ORTSDoubleTunnelArea	ORTS	
ORTSSingleTunnelPerimeter	ORTS	
ORTSDoubleTunnelPerimeter	ORTS	
ORTSWagonFrontalArea	ORTS	
ORTSDavisDragConstant	ORTS	
ORTSTrailLocomotiveResistanceFactor	ORTS	
Lights		
Type	MSTS	Glow Cone
FadeIn	MSTS	
FadeOut	MSTS	
Cycle	MSTS	
Headlight	MSTS	
Unit	MSTS	
Penalty	MSTS	
Control	MSTS	
Service	MSTS	
TimeOfDay	MSTS	
Weather	MSTS	
Coupling	MSTS	
Duration	MSTS	
LightColour	MSTS	
Position	MSTS	
Radius	MSTS	
Azimuth	MSTS	

Parameter	Program	Notes
Elevation	MSTS	
Transition	MSTS	
Angle	MSTS	
Effects		
StackFX	MSTS	
CylindersFX	MSTS	
WhistleFX	MSTS	
SafetyValvesFX	MSTS	
Exhaust	MSTS	
Cylinders2FX	ORTS	
CompressorFX	ORTS	
GeneratorFX	ORTS	
Injectors1FX	ORTS	
Injectors2FX	ORTS	
HeatingSteamBoilerFX	ORTS	
WagonGeneratorFX	ORTS	
WagonSmokeFX	ORTS	
HeatingHoseFX	ORTS	
WaterScoopFX	ORTS	
TenderWaterOverflowFX	ORTS	
BearingHotboxFX	ORTS	
Freight Animations		
FreightAnim	MSTS	
IntakePoint	MSTS	FuelWater FuelCoal FuelWood FuelSand FuelDiesel FreightGeneral FreightLivestock FreightFuel FreightGrain FreightCoal FreightGrvel FreightSand
ORTSFreightAnims	ORTS	
FreightAnimContinuous	ORTS	
WagonEmptyWeight	ORTS	

Parameter	Program	Notes
EmptyMaxBrakeForce	ORTS	
EmptyMaxHandbrakeForce	ORTS	
EmptyORTSDavis_A	ORTS	
EmptyORTSDavis_B	ORTS	
EmptyORTSDavis_C	ORTS	
EmptyCentreOfGravity_Y	ORTS	
EmptyORTSWagonFrontalArea	ORTS	
EmptyORTSDavisDragConstant	ORTS	
FreightWeightWhenFull	ORTS	
FullMaxBrakeForce	ORTS	
FullMaxHandbrakeForce	ORTS	
FullORTSDavis_A	ORTS	
FullORTSDavis_B	ORTS	
FullORTSDavis_C	ORTS	
FullCentreOfGravity_Y	ORTS	
FullORTSWagonFrontalArea	ORTS	
FullORTSDavisDragConstant	ORTS	
MSTSFreightAnimEnabled	ORTS	
IsGondola	ORTS	
UnloadingStartDelay	ORTS	
FullAtStart	ORTS	
Shape	ORTS	
MaxHeight	ORTS	
MinHeight	ORTS	
ORTSWaterScoopFillElevation	ORTS	
ORTSWaterScoopDepth	ORTS	
ORTSWaterScoopWidth	ORTS	
Passenger Views		
Inside	MSTS	
PassengerCabinFile	MSTS	
PassengerCabinHeadPos	MSTS	

Parameter	Program	Notes
RotationLimit (No Longer Used ?)	MSTS	
StartDirection	MSTS	
Sound	MSTS	
ORTSAlternatePassengerViewPoints	ORTS	
ORTSAlternatePassengerViewPoint	ORTS	
Controllers		
EngineControllers	MSTS	
Throttle	MSTS	
Brake_Engine	MSTS	
Brake_Train	MSTS	
VacuumBrakesHasVacuumPump	MSTS	
NumNotches	MSTS	Dummy TrainBrakesControllerReleaseStart TrainBrakesControllerGraduatedSelfLapLimitedHoldingStart TrainBrakesControllerSuppressionStart TrainBrakesControllerContinuousServiceStart TrainBrakesControllerEmergencyStart
Notch	MSTS	
DirControl	MSTS	
BellToggle	MSTS	
AWS	MSTS	
Sanding	MSTS	
Horn	MSTS	
Wipers	MSTS	
EmergencyStopResetToggle	MSTS	
EmergencyStopToggle	MSTS	

Parameter	Program	Notes
Vigilance	MSTS	
BailOffButton	MSTS	
Regulator	MSTS	
Cutoff	MSTS	
FireDoor	MSTS	
Blower	MSTS	
HeatingTap	MSTS	
Shovel	MSTS	
DampersFront	MSTS	
DampersBack	MSTS	
Injector1Steam	MSTS	
Injector2Steam	MSTS	
Injector1Water	MSTS	
Injector2Water	MSTS	
CylinderCocks	MSTS	
Whistle	MSTS	
BellToggle	MSTS	
SmallEjectorOrCompressor	MSTS	
HeadLights	MSTS	

Appendix II

Consider Transitioning away from Microsoft Train Simulator to Open Rails

By 2026, Microsoft Train Simulator is roughly 25 years old and has long since been abandoned by its creators. The fact that it has lived on this long is a testament to its community, not to the completeness of the original program. Many years after Microsoft Train Simulator was introduced, and after its planned sequel was cancelled, a team of talented programmers began building an open-source replacement. This is how we got Open Rails.

At some point, most Microsoft Train Simulator users will want to transition from Microsoft Train Simulator to Open Rails. By design, Open Rails has an important advantage: it can use a vast amount of legacy Microsoft Train Simulator content created over the years. Most content will work without major issues, although some items may need small adjustments to run correctly in Open Rails.

The Open Rails Program

The Open Rails program installs in a location that will remain totally isolated from simulator content, which is a fundamental difference from the way Microsoft Train Simulator works. The benefits of this approach are numerous but the primary benefit is that you will never need to reinstall content when you perform an update or re-installation of Open Rails. You can completely delete and reinstall Open Rails and then inform it about where your content is and you are ready to go.

You can also isolate content, which helps when one content installation affects another. Since content locations can be switched within Open Rails, you can select isolated route installations as needed. This eliminates the need for legacy solutions like *Train Store* with Microsoft Train Simulator.

Technically, you do not need any Microsoft Train Simulator content to use Open Rails. However, if you have an old Microsoft Train Simulator installation installed or saved, the beauty of the Open Rails solution is that only the following folders need to be copied from the old Microsoft Train Simulator installation:

- GLOBAL folder
- SOUND folder
- ROUTES folder
- TRAINS folder

All other Microsoft Train Simulator files are not required and are redundant under Open Rails.

As a content developer, you will still need a set of utilities like Aceit, TGATOOLS2 and FFEDIT_SUB to actually access and manipulate content without the Microsoft Train Simulator **UTILS** folder.

A typical Open Rails solution that does not depend on a complete Microsoft Train Simulator installation could be set up like this:

Installation

Program location	**Comment**
C:\Program Files (x86)\Open Rails	Where the PROGRAM gets installed

Content

Program location	**Comment**
C:\ORTS\TrainSimulations\	Any TrainSimulations Freeware/Payware content (Global, Routes, Sound, Trains)
C:\ORTSWORLD\	Copied over legacy MSTS installation content folders (Global, Routes, Sound, Trains)
C:\ORTS\GNRR	Any custom or 3rd party installations that don't have MSTS dependencies
C:\ORTS_xxx_	Where *XXX* is any other route that will share (Global, Routes, Sound,Trains) from the OR folder
C:\ORTS\developer	Any **work in progress routes**
C:\ORTS\	(Global, Routes, Sound,Trains)

Index

@

3D Canvas
    Nice Things, 70

A

ACE2BMP
    Tools, 199
ACE2DDS
    Tools, 202
AceIt
    Tools, 199
Assets
    General Texture Mapping, 62

B

Blender
    Add-ons, 29
    Asset Libraries, 28
    Background Images, 66
    Bevel, 87
    Blender, SHADING, 43
    Community Add-ons, 34
    Considering Level Of Detail, 115
    DDS, 156
    Edit Mode, 82
    Exercise 1, 94
    Extrude, 83
    File Paths, 27
    General Settings, 38
    Getting Started, 13
    High to Low Poly Baking, 65
    Importing, 125
    Importing Background Images, 130
    Inset, 85
    Installation, 16, 16
    Installation Video, 18
    Installed via Steam, 16
    Interface, 76
    Interface Preferences, 20
    Introduction, 64
    Level Of Detail, 149
    Long Term release, LTS, 17
    Loop Cut, 88
    Making a vehicle, 140
    MATCAP, 40
    Materials with Alpha Layer, 203
    Mirror and Flip, 90
    Modeling, 81
    Modeling Primer, 74
    Modeling Tips, 65
    Move selected objects, 147
    Object Mode, 81
    Overview, 64
    Polygon Count and Drawcalls, 66
    Save & Load Preferences, 26
    Setting up to create a model, 66
    Setup, 19
    Shading Settings, 40
    Shortcuts, 78
    System Preferences, 24
    TABS, 37
    Texturing, 107
    Texturing a model, 107
    Units of Measure, 41
    Units, Feet, Meters, Metric, U.S. customary units, 41
    Unwrapping a mesh for texturing, 111
    View Control, 79
    Viewport, 75
    Viewport Preferences, 22
    Why Blender?, 14
    Workflow, 119
    Workspace, 91
    X-Ray Mode, 99

C

Configuration
    3rd Party Tools, 59
    Deconstruction, 56
    Files, 53
    Include, 53
    Include, Structure, 54

relative reference, 55
Sample Layout, 58
Content
Configuration Files, 53
Content Creation
Basic Overview, 48
Engine/Wagon Model Hierarchy, 48
Orientation, 48
Standard 2 Axle Freight Bogies, 50
Content Design
Choices, 11
Context
An old but useful text editor, 16

D

DXTBMP
Tools, 200

F

File Formats with Layer Support, 153

G

General Modeling Standards
continued, 59
GIMP
Alpha Channels, 190
Graphics
Texture
Alpha Channels, 189
Layers, 186
Using Fonts, 186
Using Layers, 184
Graphics Basics, 159

H

Hexadecimal
Numbering, 282

L

Lights
Examples, 279
Fade-in and Fade-out times, 278
Setting requirements, 276

M

Making Contentm TERMS. DEFINITIONS, 9
Measurements, 195
Modeling
Reference Material, 141

O

Open Rails
The Open Rails Train Simulator, free and open source project, 8
ORTS
Another Example of Rolling Stock, 272
Basic ENG and WAG file Details, 256
Completely New Content, 268
Creating Rolling Stock and Locomotives, 251
Example WAG File, 259
General Configuration File Format, 252
Specifying Lights on Locomotives and Wagons, 275
The Benefits of Common Folders for Re-skinners, 267
Units, 253
Universal Settings, 258
Using INCLUDE, 255

P

Perspective Distortion, 196
Project
Folder Layout, 47
Folders, 46
PYTHON
FIX UV MAP, 127
Python, DECAL Layout, 250
Python
Running A Script, 232

R

Reference File, 207

S

Scripting
AI Assistants, 232
helping you setup, 248
Inside Blender, 232

Scripts, 233
Using Python, 230

T

Texture
About DDS, 114
ACE, 158
ambient occlusion, 217
Ambient Occlusion Baking, 224
Ambient Occlusion Baking, GIMP, 225
Baking Ambient Occlusion, additional, 228
DDS FILES, 155, 155
Decals, 182
DXT COMPRESSION, 156
Emissive Lighting, 114
JPG Format, 161
LAYERING TECHNIQUE, 185
Layering Tips, 186
PBR Textures, 205
PNG Format, 161
Preparing a texture, 165
Seasonal, 207
Shader Properties, 113
Shadows, 216
Snow, 213
Texture Nodes, 182
Tools Overview, 152
Upscaling, 161
TextureRipper
Tools, 198
textures
Windows, 181
Texturing
Common Terms, 169
Setting up Blender for Texture Mapping and Baking, 167
UV Mapping Basics, 167
TGATool2
MAKEACE
Tools, 199

U

UV Mapping
Packing Islands, 179
Seams, 174

V

Vector Graphics, 162
Font Tricks, 162

W

Windows 7
No longer supported, 16